BACKEND SOFTWARE JOURNEY

The First 150 Days

A practical ASP.NET Core backend build using Clean Architecture, SQL Server, Entity Framework Core, authentication, testing, and continuous integration.

150 focused lessons. One evolving production-style API.

Contents

The journey is organized into five 30-day phases. Review days consolidate the work before the next phase begins.

Month 1

Day 1 - Backend Setup

Day 2 - User Entity

Day 3 - First Task Feature

Day 4 - Cleanup And Validation

Day 5 - Get Task By ID

Day 6 - Update Task Endpoint

Day 7 - Delete Task Endpoint

Day 8 - Clean Task CRUD

Day 9 - Improve API Responses

Day 10 - Complete Task Endpoint

Day 11 - Reopen Task Endpoint

Day 12 - Task Completion Filtering

Day 13 - Task Search By Title

Day 14 - Review Task API

Day 15 - User Service Structure

Day 16 - First User API Endpoints

Day 17 - Get User By ID Endpoint

Day 18 - Update User Endpoint

Day 19 - Delete User Endpoint

Day 20 - Connect Tasks To Users

Day 21 - Review User And Task APIs

Day 22 - Prepare Infrastructure Layer

Day 23 - Add EF Core Database Context

Day 24 - Register SQL Server DbContext

Day 25 - Create First EF Core Migration

Day 26 - Apply EF Core Migration

Day 27 - Move User Storage To SQL Server

Day 28 - Move Task Storage To SQL Server

Day 29 - Test Database Persistence

Day 30 - First Backend Month Review

Month 2

Day 31 - Review Database CRUD

Day 32 - Add Task User Relationship

Day 33 - Get Tasks By User Endpoint

Day 34 - Add UserId Filter To Task List

Day 35 - Add Optional Due Date To Tasks

Day 36 - Add Simple Task Priority

Day 37 - Review Database Task Changes

Day 38 - Add Data Annotation Validation

Day 39 - Review ASP.NET Core Validation Responses

Day 40 - Centralize Common API Messages

Day 41 - Add Basic Global Exception Handling

Day 42 - Review HTTP Status Codes

Day 43 - Review Response DTO Consistency

Day 44 - Weekly Validation Review

Day 45 - Review Application Layer Boundaries

Day 46 - Review Infrastructure Layer

Day 47 - Review Service Interfaces

Day 48 - Add Async Suffix To Service Methods

Day 49 - Remove Dead Code And Starter Files

Day 50 - Architecture Review Day

Day 51 - Plan Authentication Flow

Day 52 - Create Auth DTOs

Day 53 - Add Password Hashing Service

Day 54 - Add Register Use Case

Day 55 - Add Login Use Case

Day 56 - Review Auth Controller Endpoints

Day 57 - Add JWT Configuration

Day 58 - Generate JWT Token On Login

Day 59 - Protect Task Endpoints

Day 60 - Month 2 Review

Month 3

Day 61 - Review Authentication

Day 62 - Add User Id Claim Usage

Day 63 - Make Tasks User-Owned

Day 64 - Restrict Get All Tasks

Day 65 - Restrict Get Task By Id

Day 66 - Restrict Update Task

Day 67 - Restrict Delete Task

Day 68 - Review User Ownership

Day 69 - Add Basic Unit Test Project

Day 70 - Test Task Creation

Day 71 - Test Task Get By Id

Day 72 - Test Task Update

Day 73 - Test Task Delete

Day 74 - Test User Service

Day 75 - Test Auth Service

Day 76 - Test Build And CI Readiness

Day 77 - Add GitHub Actions Build

Day 78 - Add GitHub Actions Tests

Day 79 - Review README For Testing

Day 80 - Add Basic Request Logging

Day 81 - Review Structured Error Responses

Day 82 - Add ProblemDetails Error Responses

Day 83 - Add A Health Check Endpoint

Day 84 - Review Security Basics

Day 85 - Clean Up Application Settings

Day 86 - Document The Local Environment

Day 87 - Manual API Regression Test

Day 88 - Cleanup Before The Month Review

Day 89 - Update README And Progress

Day 90 - Month 3 Review

Month 4

Day 91 - Plan Task Query Improvements

Day 92 - Add Pagination Request Parameters

Day 93 - Implement Task Pagination

Day 94 - Add Paged Response DTO

Day 95 - Sort Tasks By Created Date

Day 96 - Sort Tasks By Due Date

Day 97 - Verify Filtering, Sorting, And Pagination Together

Day 98 - Task Query Review

Day 99 - Add Current User Profile Endpoint

Day 100 - Update Current User Profile

Day 101 - Plan The Change Password Flow

Day 102 - Implement Change Password

Day 103 - Add Change Password Tests

Day 104 - Strengthen Task Authorization Tests

Day 105 - API Contract Review

Day 106 - Add XML Comments to Swagger

Day 107 - Configure JWT Authentication in Swagger

Day 108 - Update the README Setup Guide

Day 109 - Add Optional Development Seed Data

Day 110 - Review EF Core Migrations

Day 111 - Document a Safe Local Database Reset

Day 112 - Plan API Integration Testing

Day 113 - Add the First API Integration Test

Day 114 - Add an Authentication Integration Test

Day 115 - Add an Authenticated Task Integration Test

Day 116 - Add a Task Delete Integration Test

Day 117 - Review Integration Tests in GitHub Actions

Day 118 - Clean Up Task Query and Integration Tests

Day 119 - Add Authentication and Task Query Examples to README

Day 120 - Month 4 Review

Month 5

Day 121 - Architecture Responsibility Review

Day 122 - Make Service Method Names Consistent

Day 123 - Review DTO Folder Organization

Day 124 - Complete Basic Request Validation

Day 125 - Decide Whether JobTrackr Needs FluentValidation

Day 126 - Review Repeated DTO Mapping

Day 127 - Clean Task Response Mapping

Day 128 - Clean User Response Mapping

Day 129 - Clean Auth Response Mapping

Day 130 - Review Domain Entity Behavior

Day 131 - Define Task Completion Rules

Day 132 - Validate Task Due Dates

Day 133 - Validate Task Priority

Day 134 - Validate User Email

Day 135 - Add Password Validation

Day 136 - Review Error Response Shapes

Day 137 - Decide Whether Services Need a Result Pattern

Day 138 - Confirm the Result Pattern Decision

Day 139 - Review Controller Thinness

Day 140 - Review Service Size

Day 141 - Plan Task Comments

Day 142 - Add the Task Comment Entity

Day 143 - Create a Comment on an Owned Task

Day 144 - List Comments for an Owned Task

Day 145 - Delete a Comment from an Owned Task

Day 146 - Test Task Comment Workflows and Ownership

Day 147 - Document Task Comments in the README

Day 148 - Focused Cleanup and Release Verification

Day 149 - Review Git and Confirm Publication

Day 150 - Month 5 Review

How to Use This Book

Each day has one focused goal. Read the concept, examine the implementation shape, complete the exercise, and verify the result before moving forward.

The project grows incrementally. Later lessons intentionally build on decisions made earlier, while review days pause feature work to validate the system as a whole.

Code samples are learning snapshots. Adapt configuration, secrets, database settings, and deployment details to your own environment.

MONTH 1

FOUNDATIONS

Build the API, shape task and user features, and introduce database persistence.

Day 1 - Backend Setup

Lesson Goal

Create the first working backend solution skeleton for JobTrackr.

By the end of this lesson, the project has:

- one solution
- one ASP.NET Core Web API project
- three support class libraries
- project references
- a successful API run

What Changed

The backend project starts with a clean solution structure.

Important project folders:

- `src/JobTrackr.Api`
- `src/JobTrackr.Application`
- `src/JobTrackr.Domain`
- `src/JobTrackr.Infrastructure`

Core Concept

Before adding features, a backend needs a stable project structure.

The Day 1 structure introduces the main layers:

- API layer for HTTP endpoints
- Application layer for use cases
- Domain layer for business models
- Infrastructure layer for database and external systems later

Commands

```
dotnet new sln -n JobTrackr
dotnet new webapi -n JobTrackr.Api -o src/JobTrackr.Api --framework net8.0
dotnet new classlib -n JobTrackr.Application -o src/JobTrackr.Application --framework net8.0
dotnet new classlib -n JobTrackr.Domain -o src/JobTrackr.Domain --framework net8.0
dotnet new classlib -n JobTrackr.Infrastructure -o src/JobTrackr.Infrastructure --framework
net8.0
dotnet build
dotnet run --project src/JobTrackr.Api
```

Why This Matters

The first day is not about business features. It is about proving the environment works.

If the solution builds and the API runs, future lessons can focus on backend concepts instead of setup problems.

Lesson Explanation

The first step in building JobTrackr was creating a backend solution that could grow over time.

Instead of putting everything into one project, the solution starts with four projects. This gives each part of the backend a clear responsibility from the beginning.

The API project is the entry point. It receives HTTP requests and returns HTTP responses.

The Application project will hold the use cases. These are the actions the system supports, such as creating a task or updating a task.

The Domain project will hold the core business objects. These objects should describe the problem we are solving, not the technology we are using.

The Infrastructure project is reserved for database and external system code later.

On Day 1, most of these projects are empty. That is intentional. The goal is not to build everything immediately. The goal is to create a structure that is simple enough to understand and strong enough to grow.

Reader Exercise

Create the same solution structure from scratch and run the API once.

Then explain what each project is responsible for in one sentence.

Future Improvement

Later, this structure will support real features, persistence, authentication, testing, and deployment.

Next Lesson

After the backend skeleton runs, the next step is adding the first real domain model: `User`.

Day 2 - User Entity

Lesson Goal

Create the first real domain model in JobTrackr.

By the end of this lesson, the Domain project has:

- `Entities` folder
- `User.cs`
- basic user properties
- a successful build

What Changed

Important file:

- `src/JobTrackr.Domain/Entities/User.cs`

The first business entity is added to the Domain layer.

Core Concept

A backend starts with business meaning.

The `User` entity is not an HTTP endpoint and not a database table yet. It is the first model that describes something important in the system.

Code Shape

```
namespace JobTrackr.Domain.Entities;

public class User
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
}
```

Why This Matters

The Domain layer should describe the core objects of the application.

Starting with a simple entity keeps the project understandable while introducing the habit of separating business models from API code.

Lesson Explanation

After the solution structure was created, the next step was to add meaning to the backend.

Day 2 introduced the first domain entity: `User`.

A domain entity represents something the system cares about. In JobTrackr, users will eventually own tasks, apply for jobs, and interact with the application. Even though the user feature is not complete yet, adding the entity starts shaping the business model.

The `User` class stays intentionally simple:

- `Id` identifies the user.
- `FullName` stores the user's name.
- `Email` stores contact identity.
- `CreatedAtUtc` records when the user was created.

At this stage, there is no database and no controller. That is deliberate. The goal is to understand the Domain layer before connecting it to HTTP or persistence.

Reader Exercise

Add one more property to the `User` entity, such as:

```
public bool IsActive { get; set; } = true;
```

Then explain whether that property belongs in the Domain layer or somewhere else.

Future Improvement

Later, the `User` entity can be connected to authentication, persistence, and task ownership.

Next Lesson

After creating the first domain entity, the next step is creating the first task feature that connects API, Application, and Domain layers.

Day 3 - First Task Feature

Lesson Goal

Create the first simple task feature.

By the end of this lesson, JobTrackr supports:

```
GET /api/tasks
POST /api/tasks
```

What Changed

Important files:

- `src/JobTrackr.Domain/Entities/JobTask.cs`
- `src/JobTrackr.Application/Tasks/CreateTaskRequest.cs`
- `src/JobTrackr.Application/Tasks/TaskResponse.cs`
- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Api/Program.cs`

Core Concept

Day 3 connects the layers for the first time:

```
Api -> Application -> Domain
```

The controller receives HTTP requests. The service handles application logic. The domain entity represents the business object.

API Shape

Create task:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Create first task endpoint",
  "description": "Practice controller to service flow"
}
```

Get all tasks:

```
GET /api/tasks
```

Why This Matters

This is the first real backend workflow in the project.

It shows how an API request travels from the controller into the application layer and returns a clean response to the client.

Lesson Explanation

Day 3 is where JobTrackr starts behaving like a real backend API.

The project already had separate layers, but they were not doing much together. The task feature connects them.

The Domain layer gets a new entity called `JobTask`. The name avoids conflict with C#'s built-in `Task` type and clearly describes the business object.

The Application layer gets request and response DTOs. `CreateTaskRequest` describes the data the API user sends when creating a task. `TaskResponse` describes the data returned by the API.

The Application layer also gets `ITaskService` and `TaskService`. The interface defines what the task feature can do. The service implements the logic.

For now, tasks are stored in memory. This is not production persistence, but it is useful for learning the flow without introducing SQL Server too early.

The API layer gets `TasksController`. The controller exposes `GET /api/tasks` and `POST /api/tasks`. It does not manage the task list directly. It calls the service.

This is the first clean architecture habit in practice: the controller should handle HTTP, while application logic belongs in the application layer.

Reader Exercise

Create a task through Swagger, then call `GET /api/tasks`.

Explain the flow in this order:

```
HTTP request -> controller -> service -> domain entity -> response DTO -> HTTP response
```

Future Improvement

The task list is currently in memory. Later, it should be saved in SQL Server through the Infrastructure layer.

Next Lesson

After the first feature works, the next step is cleanup and validation.

Day 4 - Cleanup And Validation

Lesson Goal

Clean up and stabilize the Day 3 task feature.

By the end of this lesson:

- default WeatherForecast code is removed
- `Program.cs` is cleaner
- task creation rejects an empty title
- existing task endpoints still work

What Changed

Important files:

- `src/JobTrackr.Api/Program.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`

Core Concept

Backend development is not only adding new features.

A normal workflow is:

```
Build small feature -> test it -> clean it -> continue
```

Day 4 makes the first task workflow more professional before adding more endpoints.

API Shape

Valid create request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Clean Day 3 task API",
  "description": "Remove template code and add validation"
}
```

Invalid create request:

```
{
  "title": "",
  "description": "This should fail"
}
```

Expected result:

```
400 Bad Request
```

Why This Matters

APIs need clear behavior when input is invalid.

Returning `400 Bad Request` for an empty task title makes the API easier to use and easier to test.

Lesson Explanation

Day 4 is a cleanup day.

That may sound less exciting than adding a new endpoint, but cleanup is a real part of backend engineering. If a project only grows and never gets cleaned, the code becomes harder to understand.

The first cleanup removes the default WeatherForecast code from the ASP.NET Core template. That code was useful when the project was created, but it does not belong in JobTrackr.

The second improvement is validation.

Before this lesson, the API could create a task with an empty title. That is not useful behavior. A task needs a title because the title is the main thing the user sees.

The validation rule belongs in the application service because it is part of the task use case:

```
if (string.IsNullOrWhiteSpace(request.Title))
{
    throw new ArgumentException("Task title is required.");
}
```

The controller catches that validation failure and converts it into an HTTP response:

```
400 Bad Request
```

This keeps the responsibilities clear. The service protects the business rule. The controller translates the result into HTTP.

Reader Exercise

Add validation for a maximum title length.

Example rule:

```
Task title must be 100 characters or fewer.
```

Then return `400 Bad Request` when the title is too long.

Future Improvement

Later, validation can move to a more structured approach, such as validation classes or middleware.

Next Lesson

After create and get-all work cleanly, the next step is reading one specific task by id.

Day 5 - Get Task By ID

Lesson Goal

Add an endpoint to get one task by id.

By the end of this lesson, JobTrackr supports:

```
GET /api/tasks/{id}
```

The endpoint returns a task when it exists and `404 Not Found` when it does not.

What Changed

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`

Core Concept

Getting one task by id is a search operation.

The service searches the task list for a matching id. The controller turns the result into the correct HTTP response.

API Shape

Create a task first:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Test get by id",
  "description": "Create a task then fetch it by id"
}
```

Get existing task:

```
GET /api/tasks/1
```

Expected:

```
200 OK
```

Get missing task:

```
GET /api/tasks/999
```

Expected:

```
404 Not Found
```

Why This Matters

Most APIs need a way to fetch one specific record.

List endpoints are useful, but real clients often need details for one selected item.

Lesson Explanation

Day 5 adds the next read pattern: get one task by id.

The API already supports creating tasks and returning all tasks. That is useful, but incomplete. A client usually needs to open one task, inspect its details, or prepare it for editing.

The route describes the resource:

```
GET /api/tasks/{id}
```

The `{id}` part comes from the URL. If the client calls `/api/tasks/1`, the controller receives `1` as the task id.

The controller asks the service for that task:

```
var task = await _taskService.GetById(id);
```

The service searches the current task list. If it finds a matching task, it returns the response. If it does not, it returns `null`.

The controller then converts `null` into:

```
404 Not Found
```

This is an important backend pattern. The application service can represent "not found" as `null`, while the API layer decides that `null` means HTTP 404.

Reader Exercise

Write the logic for searching an array of numbers for a target value.

Then compare it with searching the task list by id.

Future Improvement

When SQL Server is added, this search will become a database query instead of an in-memory list search.

Next Lesson

After the API can create tasks, list tasks, and fetch one task, the next CRUD step is updating an existing task.

Day 6 - Update Task Endpoint

Lesson Goal

Add an update endpoint for tasks so an existing task can be modified through the API.

By the end of this lesson, the project supports:

```
PUT /api/tasks/{id}
```

The endpoint updates the task title and description for an existing task.

What Changed

The Day 6 work extends the task feature from read/create behavior into update behavior.

Important files:

- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs`

Core Concept

An update endpoint usually needs three pieces of information:

- The resource id from the route
- The new data from the request body
- A service method that applies the change

For this project, the route id identifies which task should be updated, while the request body carries the new title and description.

API Shape

Example request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "Apply for backend developer role",
  "description": "Update resume and send application"
}
```

Expected behavior:

- If the task exists, update it and return the updated task.
- If the task does not exist, return `404 Not Found`.
- If the title is empty, return `400 Bad Request`.

Why This Matters

Create, read, update, and delete are the foundation of most backend APIs.

This lesson moves JobTrackr closer to a real task management backend because users need to correct, rename, or refine tasks after creating them.

Lesson Explanation

When we added the update task endpoint, we introduced the first real modification workflow in the API.

Creating a task adds a new item to the system. Getting a task reads existing state. Updating is different because it changes an object that already exists. That means the code has to answer two questions before making the change:

1. Does this task exist?
2. Is the new data valid?

The route handles identity:

```
PUT /api/tasks/{id}
```

The request body handles the new values:

```
{
  "title": "Updated title",
  "description": "Updated description"
}
```

This keeps the API clear. The URL says which task we are changing. The body says what we want to change it to.

The controller should stay focused on HTTP behavior. It receives the route id and request body, calls the application service, and converts the result into the right HTTP response. The application service owns the use-case logic, including finding the task and applying validation.

That separation is one of the early clean architecture habits in this project: controllers should not become the place where all business decisions live.

Reader Exercise

After completing this lesson, try adding one more field to the task update request.

Example:

```
{
  "title": "Updated title",
  "description": "Updated description",
  "isCompleted": true
}
```

Then update the service and response so the API can mark a task as completed.

Future Improvement

The current project is still using simple in-memory behavior. Later, this update flow should move toward persistence with SQL Server and Entity Framework Core.

When that happens, the same API concept remains:

- Route id identifies the task
- Request body provides the update data
- Service/application layer validates and applies the change
- Infrastructure layer saves the change

Next Lesson

The next natural feature is deleting a task:

```
DELETE /api/tasks/{id}
```

After create, get, get by id, and update, delete will complete the basic CRUD workflow.

Day 7 - Delete Task Endpoint

Lesson Goal

Add an endpoint to delete an existing task.

By the end of this lesson, JobTrackr supports:

```
DELETE /api/tasks/{id}
```

The endpoint deletes the task when it exists and returns `404 Not Found` when it does not.

What Changed

Day 7 completes the basic in-memory CRUD flow for tasks.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

Core Concept

Deleting a resource is usually a find-and-remove operation.

The service searches for the task by id. If the task exists, it removes it from the list. If the task does not exist, the service reports that deletion failed.

The controller converts that result into HTTP:

- `204 No Content` when deletion succeeds
- `404 Not Found` when the task does not exist

API Shape

Create a task first:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Task to delete",
  "description": "This task should be deleted"
}
```

Delete existing task:

```
DELETE /api/tasks/1
```

Expected:

```
204 No Content
```

Try to get the deleted task:

```
GET /api/tasks/1
```

Expected:

```
404 Not Found
```

Delete missing task:

```
DELETE /api/tasks/999
```

Expected:

```
404 Not Found
```

Why This Matters

After Day 7, the task API has the basic CRUD workflow:

```
Create -> Read all -> Read one -> Update -> Delete
```

This is a major early checkpoint. Even though the data is still in memory, the API now has the same basic shape used by many real backend systems.

Lesson Explanation

Day 7 adds the delete task endpoint.

At this point, JobTrackr can create tasks, return all tasks, return one task by id, and update an existing task. The missing CRUD operation is delete.

The route is:

```
DELETE /api/tasks/{id}
```

The id comes from the URL. The controller does not need a request body because deleting a task only requires knowing which task should be removed.

The application service exposes a method such as:

```
Task<bool> DeleteTask(int id);
```

The `bool` return value keeps the result simple:

- `true` means the task was found and deleted.
- `false` means the task was not found.

Inside the service, the logic is straightforward:

```
var task = _tasks.Find(x => x.Id == id);

if (task is null)
{
    return false;
}

_tasks.Remove(task);
return true;
```

The controller turns that service result into the correct HTTP response:

```
if (!isDeleted)
{
    return NotFound("Task is not found");
}

return NoContent();
```

The important detail is the `204 No Content` response. A successful delete does not need to return the deleted task. The client only needs to know the operation succeeded.

This lesson also updates the README so the public repository matches the current API. Documentation should move with the code, especially when the project is being used as a learning archive.

Reader Exercise

Add a second delete-style method that clears all completed tasks.

Example route idea:

```
DELETE /api/tasks/completed
```

Think through what status code should be returned when there are no completed tasks to delete.

Future Improvement

The delete logic currently removes tasks from an in-memory list. Later, when SQL Server and Entity Framework Core are added, this operation will become a database delete.

The API behavior should stay the same even when the storage implementation changes.

Next Lesson

With basic CRUD complete, the next step is to improve the project structure and prepare for persistence.

The future lessons should move toward:

- SQL Server
- Entity Framework Core
- repositories
- better validation
- tests

Day 8 - Clean Task CRUD

Lesson Goal

Clean and stabilize the completed Task CRUD feature before starting the next module.

Day 8 does not add a new endpoint. The goal is to make the existing task code easier to read, remove starter noise, and confirm the project still builds after cleanup.

What Changed

Day 8 turns the first CRUD milestone into cleaner book-quality code.

Important files:

- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Application/Tasks/CreateTaskRequest.cs`
- `src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs`
- `src/JobTrackr.Application/Tasks/TaskResponse.cs`
- `src/JobTrackr.Domain/Entities/JobTask.cs`
- `src/JobTrackr.Domain/Entities/User.cs`
- `src/JobTrackr.Application/Class1.cs`
- `src/JobTrackr.Domain/Class1.cs`
- `src/JobTrackr.Infrastructure/Class1.cs`

The default `Class1.cs` starter files were removed because they were not part of the real project.

Core Concept

Professional backend work is not only adding features.

After a feature reaches a working checkpoint, the next step is cleanup:

```
Build feature -> verify behavior -> clean code -> document -> continue
```

This prevents the project from becoming harder to change with every new lesson.

API Shape

The API shape stays the same after cleanup:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

The expected behavior also stays the same:

- `200 OK` for successful reads and updates
- `201 Created` for task creation

- `204 No Content` for successful delete
- `400 Bad Request` for invalid task title
- `404 Not Found` for missing task id

Why This Matters

Day 8 protects the project from accidental complexity.

At the end of Day 7, the API could create, read, update, and delete tasks. That was enough to prove the feature worked, but working code still needs review. Small cleanup steps make the next feature easier to add.

This is especially important in a learning project that will become a public repository and a book. Readers should see the habit of stopping at checkpoints, removing noise, and verifying the code before moving forward.

Lesson Explanation

Day 8 is a cleanup day.

The task CRUD workflow is now complete, so this lesson pauses feature development and reviews the code that already exists. This is a normal professional workflow. A backend project should not move from feature to feature without checking whether the code is still readable.

The first cleanup target is starter code. New class library projects often include default files such as:

```
Class1.cs
```

These files are useful only as placeholders. Once the real project has domain entities, application services, request models, and controllers, the starter files become noise. Removing them makes the solution easier to scan.

The next cleanup target is naming and formatting. The code should explain itself through clear method names, request model names, and response model names. The task service should expose only the operations the API needs:

```
Task<IEnumerable<TaskResponse>> GetAllTasks();
Task<TaskResponse> CreateTask(CreateTaskRequest request);
Task<TaskResponse?> GetTaskById(int id);
Task<TaskResponse?> UpdateTask(int id, UpdateTaskRequest request);
Task<bool> DeleteTask(int id);
```

The controller remains responsible for HTTP responses. The service remains responsible for task behavior. This separation matters because it keeps the API layer thin and makes the application logic easier to move later when persistence is added.

The lesson also reinforces a useful rule: cleanup should not secretly become redesign.

Day 8 does not add SQL Server, Entity Framework Core, authentication, or a new User API. Those may come later, but the point of this lesson is to stabilize the current milestone.

After cleanup, the build should still pass. The final result should feel boring in a good way: same behavior, cleaner code, less noise.

Reader Exercise

Open a small project you have worked on before and find three pieces of starter or unused code.

For each one, decide:

- Is this file still used?

- Does it help explain the project?
- Would removing it make the project easier to understand?

Only delete code when you can build and verify the project afterward.

Future Improvement

The task CRUD feature still uses in-memory storage. Later lessons should replace this with real persistence using SQL Server and Entity Framework Core.

When that happens, the external API behavior should stay stable even though the internal storage implementation changes.

Next Lesson

With the task CRUD code cleaned, the project is ready for the next module.

The next lessons can move toward a new feature area or start preparing the architecture for persistence.

Day 9 - Improve API Responses

Lesson Goal

Improve the consistency and correctness of the existing Task API responses without adding a new feature module.

Day 9 keeps the API focused on the completed task CRUD workflow. The work is small, but it teaches an important backend habit: once an API works, the next step is making its responses predictable for clients.

What Changed

Day 9 improves response behavior in the current task endpoints.

Important files:

- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `README.md`

The main changes are:

- Missing tasks now use the same not-found message everywhere.
- Title validation keeps one consistent validation message.
- Task creation returns `201 Created` instead of `200 OK`.
- The README documents the current API behavior more clearly.

Core Concept

An API should not only return data. It should communicate clearly through status codes and response messages.

For a create endpoint, this distinction matters:

```
200 OK      -> the request succeeded
201 Created -> the request succeeded and created a new resource
```

Both can look similar during manual testing, but `201 Created` gives API clients a more accurate contract.

API Shape

The task endpoints stay the same:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

The expected behavior after Day 9:

- `GET /api/tasks` returns `200 OK`
- `POST /api/tasks` returns `201 Created`
- `GET /api/tasks/{id}` returns `200 OK` or `404 Not Found`
- `PUT /api/tasks/{id}` returns `200 OK`, `400 Bad Request`, or `404 Not Found`

- `DELETE /api/tasks/{id}` returns `204 No Content` or `404 Not Found`

Response Consistency

Before this cleanup, the controller could drift into slightly different messages for the same problem.

Day 9 standardizes missing task responses:

```
Task not found.
```

It also keeps validation consistent:

```
Task title is required.
```

This may feel minor, but inconsistent messages become a problem when frontends, tests, documentation, and API consumers start depending on responses.

CreatedAtAction

The create endpoint now uses:

```
return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
```

This does three useful things:

- returns `201 Created`
- includes the created task in the response body
- points to the endpoint where the new task can be retrieved

This is a better HTTP response for a successful `POST`.

Why This Matters

Day 9 is about treating the API like a real contract.

When the project is small, it is easy to accept rough responses because everything is still visible in one controller. But the project is going to grow. It will later include persistence, more modules, authentication, tests, and probably a frontend.

Small consistency decisions now prevent confusion later.

A backend project should not wait until it is large before acting professionally. The habit starts while the codebase is still small enough to clean easily.

Lesson Explanation

At this point in the project, the task API can create, read, update, and delete tasks. That means the first CRUD milestone is working.

But working is not the same as polished.

The next improvement is response consistency. API clients do not only care about whether an endpoint returns something. They care about what the status code means, what message appears when something fails, and whether similar failures behave the same way.

The create endpoint is the first example. A task is a resource. When the API creates a resource, the correct response is not just:

```
200 OK
```

A better response is:

```
201 Created
```

In ASP.NET Core, `CreatedAtAction` is a clean way to express this. It returns the created object and connects the response to the endpoint that can retrieve the new resource.

The second improvement is consistency in error messages. If one endpoint says:

```
Task is not found
```

and another says:

```
Task not found.
```

the user understands both, but the API contract becomes messy. A frontend developer or test writer should not need to handle multiple versions of the same message.

Day 9 fixes this by using one message everywhere:

```
Task not found.
```

The validation message also stays consistent:

```
Task title is required.
```

The lesson is simple: small API response details matter because they become part of the system's public behavior.

Reader Exercise

Pick one endpoint from your own API and write down its expected responses.

Include:

- success status code
- validation failure status code
- not-found status code
- exact error message text

Then check the code and confirm whether the endpoint really behaves that way.

Future Improvement

The API still returns plain string messages for some errors.

Later, the project can introduce a standard error response shape, such as:

```
{
  "message": "Task not found."
}
```

or a fuller problem-details response.

That should wait until the project needs a broader error-handling pattern.

Next Lesson

The task CRUD API now has cleaner response behavior.

The next lesson can continue improving backend quality without rushing into a new module too early.

Day 10 - Complete Task Endpoint

Lesson Goal

Add a focused workflow endpoint that marks a task as completed.

Day 10 adds:

```
PATCH /api/tasks/{id}/complete
```

This endpoint does not replace the existing update endpoint. It represents a specific action in the task workflow: completing a task.

What Changed

Day 10 adds completion behavior to the task API.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

The main changes are:

- `ITaskService` now exposes a `CompleteTask` method.
- `TaskService` can find a task and set `IsCompleted` to `true`.
- `TasksController` exposes `PATCH /api/tasks/{id}/complete`.
- Missing tasks still return the consistent `Task not found.` message.
- The README documents the new endpoint.

Core Concept

Not every change to a resource needs to be a full update.

The existing endpoint:

```
PUT /api/tasks/{id}
```

updates editable task fields such as title and description.

The new endpoint:

```
PATCH /api/tasks/{id}/complete
```

changes one specific piece of state:

```
IsCompleted = true
```

This makes the API more expressive because the URL describes the user action directly.

API Shape

After Day 10, the task API includes:

```
GET /api/tasks
```

```
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
```

The new endpoint behavior:

- `200 OK` when the task exists and is completed
- `404 Not Found` when the task does not exist

The not-found response remains:

```
Task not found.
```

Service Method

The application service contract gains:

```
Task<TaskResponse?> CompleteTask(int id);
```

The nullable return type matters.

It means the service can return either:

- an updated `TaskResponse`
- `null` when the task is missing

The controller can then translate `null` into `404 Not Found`.

Completion Logic

The in-memory implementation is intentionally small:

```
public async Task<TaskResponse?> CompleteTask(int id)
{
    await Task.Yield();

    var task = _tasks.Find(x => x.Id == id);

    if (task is null)
    {
        return null;
    }

    task.IsCompleted = true;

    return task;
}
```

This code follows the current architecture:

- the service handles task behavior
- the controller handles HTTP responses
- storage remains in memory for now

Controller Endpoint

The controller adds:

```
[HttpPatch("{id}/complete")]
public async Task<IActionResult> Complete(int id)
{
    var response = await _taskService.CompleteTask(id);

    if (response is null)
    {
```

```
            return NotFound("Task not found.");
        }

        return Ok(response);
    }
```

This keeps the API behavior clear:

- call the service
- return `404` if missing
- return `200` with the updated task if successful

Why This Matters

Day 10 moves the API from generic CRUD into task workflow behavior.

CRUD endpoints are useful, but real applications often need actions that describe what the user is trying to do. Completing a task is a good example. The user is not trying to replace the whole task. The user is performing a domain action.

This is why a focused endpoint can be cleaner than forcing every state change through `PUT`.

The lesson also reinforces discipline. The endpoint is useful, but the project still avoids adding too much at once. There is no reopen endpoint, no User API, no SQL Server, and no Entity Framework Core today.

Lesson Explanation

The task API already has complete CRUD behavior. A user can create tasks, read tasks, update tasks, and delete tasks. Day 10 adds the first small workflow action on top of that foundation.

The action is completing a task.

A simple way to do this would be to use the existing update endpoint and send a full update request. But that makes the client responsible for changing fields it may not care about. It also hides the meaning of the action inside a generic update call.

A more expressive endpoint is:

```
PATCH /api/tasks/{id}/complete
```

This URL tells the reader exactly what will happen. It will complete the task with the given id.

The service method returns a nullable response:

```
Task<TaskResponse?> CompleteTask(int id);
```

This matches the same pattern already used by read and update operations. If the task exists, the service returns the updated task. If the task does not exist, the service returns `null`.

The controller converts that service result into HTTP behavior:

```
updated task -> 200 OK
null         -> 404 Not Found
```

The response message also stays consistent with the previous cleanup:

```
Task not found.
```

This is the important part of the lesson: adding a new endpoint should not break the API habits already established. New behavior should fit into the existing style.

Reader Exercise

Think about a resource in one of your own projects.

Find one action that is more specific than a full update.

Examples:

- mark an order as paid
- archive a note
- complete a task
- approve a request
- cancel a booking

Design one focused endpoint for that action and write its expected status codes.

Future Improvement

The API can later add the opposite workflow:

```
PATCH /api/tasks/{id}/reopen
```

That should wait until a future lesson so the project continues to grow in small, clear steps.

Later, persistence will also change the implementation. The external endpoint can stay the same while the internal storage moves from an in-memory list to a database.

Next Lesson

The task API now supports both CRUD and a workflow action.

The next lesson can continue improving the task module or prepare the project for a more durable architecture.

Day 11 - Reopen Task Endpoint

Lesson Goal

Add the opposite workflow to the complete task endpoint.

Day 10 added:

```
PATCH /api/tasks/{id}/complete
```

Day 11 adds:

```
PATCH /api/tasks/{id}/reopen
```

The goal is to let a completed task move back into an active state without using the full update endpoint.

What Changed

Day 11 adds reopen behavior to the task API.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

The main changes are:

- `ITaskService` now exposes a `ReopenTask` method.
- `TaskService` can find a task and set `IsCompleted` to `false`.
- `TasksController` exposes `PATCH /api/tasks/{id}/reopen`.
- Missing tasks still return the consistent `Task not found.` message.
- The README documents the new endpoint.

Core Concept

Completion status is now a small workflow instead of a one-way flag.

The API has two explicit actions:

```
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

This keeps the intent clear. A client does not need to send a full task update just to change whether the task is completed.

API Shape

After Day 11, the task API includes:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The task status workflow is now:

```
active -> complete -> reopen
```

Service Method

The application service contract now includes:

```
Task<TaskResponse?> ReopenTask(int id);
```

The nullable return type means the service may not find a task for the supplied id.

Reopen Logic

The reopen operation follows the same pattern as complete:

```
var task = _tasks.FirstOrDefault(task => task.Id == id);

if (task is null)
{
    return null;
}

task.IsCompleted = false;

return MapToResponse(task);
```

The important part is that reopen changes only one field:

```
IsCompleted = false
```

This keeps the action focused.

Controller Endpoint

The controller exposes the workflow as:

```
[HttpPatch("{id}/reopen")]
public async Task<IActionResult> ReopenTask(int id)
{
    var task = await _taskService.ReopenTask(id);

    if (task is null)
    {
        return NotFound("Task not found.");
    }

    return Ok(task);
}
```

The endpoint returns:

- `200 OK` with the updated task when the task exists
- `404 Not Found` when the task does not exist

Why This Matters

This is a small feature, but it improves the API design.

Without a reopen endpoint, a client would need to call the general update endpoint and send a request body just to set one boolean value. That works, but it makes the client responsible for more details than necessary.

With a reopen endpoint, the API describes the business action directly.

```
PATCH /api/tasks/5/reopen
```

That is easier to read, easier to test, and easier to explain.

Lesson Explanation

At this point, the task API has moved beyond basic CRUD.

CRUD gives us create, read, update, and delete operations. Real applications also need workflow actions. Completing a task is one workflow action. Reopening a task is another.

The important design choice is to avoid hiding every action inside a generic update endpoint. When an operation has a clear meaning in the product, it can deserve its own endpoint.

The `reopen` endpoint does not edit the title, description, or any other task field. It only changes the completion state. That makes the endpoint predictable and keeps the controller method small.

Reader Exercise

Try the endpoint in Swagger:

```
PATCH /api/tasks/{id}/reopen
```

Then verify:

- Reopening an existing completed task returns `200 OK`.
- The returned task has `isCompleted` set to `false`.
- Reopening a missing task returns `404 Not Found`.
- The error message remains `Task not found.`

Future Improvement

Right now, tasks are still stored in memory.

When persistence is added later, the same service method should update the task in the database instead of only changing the in-memory list.

The API shape should stay stable even when the storage implementation changes.

Next Lesson

Day 11 completes the basic task completion workflow.

The next step should move the project closer to real backend architecture by improving persistence, separating concerns further, or preparing the application layer for database-backed behavior.

Day 12 - Task Completion Filtering

Lesson Goal

Add optional filtering to the task list endpoint.

Before Day 12, this endpoint returned all tasks:

```
GET /api/tasks
```

Day 12 keeps that behavior, but also allows the client to filter by completion status:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

What Changed

Day 12 adds query parameter support to the task API.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

The main changes are:

- `ITaskService.GetAll` now accepts `bool? isCompleted`.
- `TaskService.GetAll` filters tasks when a value is provided.
- `TasksController.GetAll` accepts the optional query parameter.
- Swagger can show and test the optional `isCompleted` query parameter.
- The README documents the new filtering examples.

Core Concept

Query parameters are useful when the endpoint still represents the same resource collection, but the client wants a filtered view.

The resource is still:

```
/api/tasks
```

The filter is:

```
?isCompleted=true
```

This is cleaner than creating separate endpoints such as:

```
GET /api/tasks/completed
GET /api/tasks/incomplete
```

The API keeps one list endpoint and lets the query string describe the requested subset.

API Shape

After Day 12, the task API includes:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The three list behaviors are:

- no query parameter: return all tasks
- `isCompleted=true`: return completed tasks
- `isCompleted=false`: return active/incomplete tasks

Nullable Boolean

The service uses:

```
bool? isCompleted
```

The question mark matters.

`bool` can only represent:

```
true
false
```

`bool?` can represent:

```
true
false
null
```

That gives the API three states:

- `true`: completed tasks
- `false`: incomplete tasks
- `null`: all tasks

Service Contract

The application service changes from:

```
Task<List<TaskResponse>> GetAll();
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted);
```

This keeps filtering inside the application service instead of pushing it into the controller.

Filtering Logic

The service method checks whether a filter was provided:

```
if (isCompleted is null)
{
    return _tasks.ToList();
}
```

If no filter exists, all tasks are returned.

When a filter is provided, the service returns only matching tasks:

```
return _tasks
```

```
    .Where(task => task.IsCompleted == isCompleted)
    .ToList();
```

The key idea is that the same endpoint can serve both general and filtered list requests.

Controller Endpoint

The controller accepts the optional query parameter:

```
public async Task<IActionResult> GetAll(bool? isCompleted)
{
    var tasks = await _taskService.GetAll(isCompleted);

    return Ok(tasks);
}
```

ASP.NET Core automatically binds `isCompleted` from the query string.

For example:

```
GET /api/tasks?isCompleted=true
```

sets:

```
isCompleted == true
```

Why This Matters

Filtering is one of the first steps from a basic CRUD API toward a useful application API.

Real clients rarely want every record all the time. A task screen may need:

- all tasks
- completed tasks
- incomplete tasks

Instead of making the frontend fetch everything and filter locally, the backend can return the correct subset.

This keeps the API more useful and prepares the project for future database-backed queries.

Lesson Explanation

Day 12 introduces query parameters through a practical example.

The task API already supports completing and reopening tasks. Once tasks can move between active and completed states, users need a way to view those states separately.

The solution is not another command endpoint. Completing and reopening are actions, so they use `PATCH`. Filtering is a read operation, so it belongs on `GET /api/tasks`.

The optional `isCompleted` parameter gives the list endpoint three behaviors without changing the route. If the parameter is missing, the API returns everything. If it is present, the API returns only matching tasks.

This is a small change in code, but an important API design habit.

Reader Exercise

Try these requests in Swagger:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

Then verify:

- all tasks are returned when the query parameter is missing
- only completed tasks are returned when `isCompleted=true`
- only incomplete tasks are returned when `isCompleted=false`
- completing and reopening a task changes which filtered list it appears in

Future Improvement

The project still uses in-memory storage.

When Entity Framework Core is added, this filtering should move naturally into the database query:

```
query = query.Where(task => task.IsCompleted == isCompleted);
```

The API shape can stay the same while the storage implementation becomes real.

Next Lesson

Day 12 adds the first list filtering behavior.

The next lessons can continue moving toward real backend behavior by adding more query options, persistence, or stronger application-layer boundaries.

Day 13 - Task Search By Title

Lesson Goal

Add basic task search by title to the existing task list endpoint.

Day 12 added completion filtering:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

Day 13 adds title search:

```
GET /api/tasks?search=resume
```

The endpoint should also support filtering and searching together:

```
GET /api/tasks?isCompleted=true&search=resume
```

What Changed

Day 13 extends the existing list endpoint instead of creating a new route.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

The main changes are:

- `ITaskService.GetAll` now accepts `bool? isCompleted` and `string? search`.
- `TaskService.GetAll` starts with all tasks and applies optional filters.
- `TasksController.GetAll` accepts both query parameters.
- Existing completion filtering still works.
- The README documents search examples.

Core Concept

Search is another form of querying a collection.

The resource is still:

```
/api/tasks
```

The query string describes what the client wants:

```
?search=resume
```

This keeps the API focused. The endpoint is not asking for a single task by id. It is asking for a list of tasks that match a title search.

API Shape

After Day 13, the task list endpoint supports:

```
GET /api/tasks
```

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

The full task API now includes:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The route list did not grow, but the list endpoint became more useful.

Service Contract

The service changes from:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted);
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search);
```

The nullable search value means the client can omit the search parameter.

Query Pipeline

The service starts with all tasks:

```
var query = _tasks.AsEnumerable();
```

Then it applies completion filtering if requested:

```
if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}
```

Then it applies title search if requested:

```
if (!string.IsNullOrWhiteSpace(search))
{
    query = query.Where(task =>
        task.Title.Contains(search, StringComparison.OrdinalIgnoreCase));
}
```

Finally, it returns the result:

```
return query.ToList();
```

This creates a simple query pipeline:

```
all tasks -> optional completion filter -> optional title search -> final list
```

Case-Insensitive Search

The search uses:

```
StringComparison.OrdinalIgnoreCase
```

That means a search for:

```
resume
```

can still match:

```
Update Resume
```

This is important because users should not need to match the exact casing of a task title.

Controller Endpoint

The controller accepts both query parameters:

```
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
{
    var tasks = await _taskService.GetAll(isCompleted, search);

    return Ok(tasks);
}
```

ASP.NET Core binds both values from the query string.

Example:

```
GET /api/tasks?isCompleted=true&search=resume
```

sets:

```
isCompleted == true
search == "resume"
```

Why This Matters

Day 13 teaches how small query options make an API more practical.

A real task list needs more than CRUD. Users need to find tasks quickly. Search by title is a simple first version of that behavior.

This also prepares the project for later database work. In memory, the filtering uses LINQ over a list. With Entity Framework Core later, the same idea can become a database query.

Lesson Explanation

Day 13 builds directly on Day 12.

Day 12 introduced filtering by completion state. Day 13 adds searching by title. Together, these features show how one list endpoint can support multiple query options without becoming messy.

The important implementation detail is to avoid hard-coding every combination. Instead of writing separate branches for every possible query, the service starts with all tasks and then applies each optional condition only when it exists.

This pattern scales better. Later, if the API needs more query options such as priority, due date, or assigned user, the same pipeline style can be extended.

Reader Exercise

Try these requests in Swagger:

```
GET /api/tasks
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?isCompleted=true&search=resume
```

Then verify:

- search returns matching titles
- search is case-insensitive
- empty or missing search returns normal list behavior

- completion filtering still works after adding search
- filtering and search can work together

Future Improvement

The current search checks whether the title contains the search text.

Future improvements could include:

- trimming the search value before filtering
- searching description as well as title
- adding pagination
- adding sorting
- moving the query into Entity Framework Core when persistence is added

Next Lesson

Day 13 adds the first search behavior.

The next useful step is to continue shaping the list endpoint into a real production-style query API with pagination, sorting, or persistence-backed querying.

Day 14 - Review Task API

Lesson Goal

Review, test, and stabilize the completed task API before starting the next backend area.

Day 14 does not add a new endpoint.

The goal is to confirm that the current task API works as expected:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Why This Day Matters

Not every productive development day needs a new feature.

After several days of adding endpoints and query behavior, the project needs a checkpoint. Review days prevent the codebase from becoming a pile of partially tested features.

Day 14 closes the current task API phase before moving into the next module.

What To Review

Important files:

- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `README.md`
- `Day/PROGRESS.md`

The review should confirm:

- the project builds successfully
- Swagger shows the expected endpoints
- the README matches the current API
- all task endpoints behave consistently
- GitHub has the latest clean code

Current Task API

The task API now supports the full in-memory task workflow:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
```

```
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The list endpoint also supports optional query behavior:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

This is no longer only a basic CRUD endpoint. It now has simple workflow and query behavior.

Build Check

Run the build from the repository folder:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

The build check matters because it gives a fast signal that the solution still compiles after the recent endpoint changes.

Create Task Test

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Review task API",
  "description": "Day 14 endpoint testing"
}
```

Expected result:

```
201 Created
```

The response should include the created task data, and the endpoint should still behave consistently with the earlier API response improvements.

Get All Tasks Test

Request:

```
GET /api/tasks
```

Expected result:

```
200 OK
```

This should return the full task list.

Get Task By Id Test

Request:

```
GET /api/tasks/1
```

Expected result:

```
200 OK
```

Missing task request:

```
GET /api/tasks/999
```

Expected result:

```
404 Not Found
```

Expected message:

```
Task not found.
```

Update Task Test

Request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "Updated review task",
  "description": "Updated from Day 14"
}
```

Expected result:

```
200 OK
```

Invalid title request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "",
  "description": "Invalid"
}
```

Expected result:

```
400 Bad Request
```

Expected message:

```
Task title is required.
```

Complete And Reopen Tests

Complete task:

```
PATCH /api/tasks/1/complete
```

Expected result:

```
200 OK
```

The returned task should have:

```
{
  "isCompleted": true
}
```

Reopen task:

```
PATCH /api/tasks/1/reopen
```

Expected result:

```
200 OK
```

The returned task should have:

```
{
  "isCompleted": false
}
```

Filtering Tests

Completed tasks:

```
GET /api/tasks?isCompleted=true
```

Incomplete tasks:

```
GET /api/tasks?isCompleted=false
```

Expected result:

```
200 OK
```

The completed filter should return only completed tasks. The incomplete filter should return only active tasks.

Search Tests

Search by title:

```
GET /api/tasks?search=review
```

Search with completion filter:

```
GET /api/tasks?isCompleted=false&search=review
```

Expected result:

```
200 OK
```

Search should be case-insensitive and should continue working with completion filtering.

Lesson Explanation

Day 14 is a checkpoint day.

This is an important habit in backend development. After adding several features, it is easy to keep moving forward without checking whether the earlier pieces still work together. A review day slows the project down just enough to protect quality.

The task API now includes CRUD operations, workflow endpoints, filtering, and search. Before adding persistence or a new module, the current behavior should be tested from the outside through Swagger or HTTP requests.

This day also teaches that documentation is part of the product. If the README does not match the API, future readers and users will be confused even if the code works.

Reader Exercise

Use Swagger or an HTTP client to test:

- create task
- get all tasks
- get task by id
- update task
- delete task
- complete task
- reopen task
- filter completed tasks

- filter incomplete tasks
- search tasks by title
- combine search and completion filtering

For each request, write down:

- request URL
- expected status code
- actual status code
- response body
- any mismatch found

Future Improvement

This kind of manual review is useful, but it should not stay manual forever.

Future improvements should include automated tests for:

- task creation
- validation failures
- missing task behavior
- complete and reopen workflow
- filtering
- search

Automated tests will make later refactoring safer when the project moves from in-memory storage to a real database.

Next Lesson

Day 14 closes the first task API phase.

The next phase can move toward more realistic backend work: persistence, repositories, database queries, or automated tests.

Day 15 - User Service Structure

Lesson Goal

Start the next backend phase by preparing the User API application layer.

Day 15 does not add a database and does not need to add a controller.

The goal is to create the first user service structure:

```
request DTO -> response DTO -> service interface -> service implementation
```

Why This Is Day 15

The task API is stable enough for the current in-memory phase.

It already supports:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

After reviewing that work on Day 14, Day 15 starts a new module: users.

What Changed

Day 15 adds the user application-layer files.

Important files:

- `src/JobTrackr.Application/Users/CreateUserRequest.cs`
- `src/JobTrackr.Application/Users/UserResponse.cs`
- `src/JobTrackr.Application/Users/IUserService.cs`
- `src/JobTrackr.Application/Users/UserService.cs`

The main changes are:

- A `Users` folder now exists in the Application project.
- `CreateUserRequest` defines the input for creating a user.
- `UserResponse` defines the output returned from user operations.
- `IUserService` defines the user service contract.
- `UserService` stores users in memory and validates required fields.

Core Concept

Before exposing an API endpoint, build the application-layer shape.

That means creating:

- the input model

- the output model
- the service contract
- the service implementation

This keeps the controller simple when it is added later.

Folder Structure

The new user files live here:

```
src/JobTrackr.Application/Users
```

This keeps user behavior separate from task behavior.

The project now has a clearer application layout:

```
Application/
  Tasks/
  Users/
```

That separation matters as the project grows.

CreateUserRequest

The request DTO receives the data needed to create a user:

```
public class CreateUserRequest
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;
}
```

This model does not include `Id`.

The client should send the user details, but the application should generate the identifier.

UserResponse

The response DTO returns user data from the application:

```
public class UserResponse
{
    public int Id { get; set; }

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public DateTime CreatedAtUtc { get; set; }
}
```

The response includes:

- generated `Id`
- `FullName`
- `Email`
- creation timestamp

This separates what the client sends from what the API returns.

IUserService

The service interface defines the first user actions:

```
public interface IUserService
{
    Task<UserResponse> CreateUser(CreateUserRequest request);

    Task<List<UserResponse>> GetAll();
}
```

This follows the same application service style already used for tasks.

The interface gives the future controller a clear dependency.

UserService

The service implementation stores users in memory for now:

```
private readonly List<UserResponse> _users = [];
private int _nextId = 1;
```

This is not permanent persistence. It is a temporary implementation while the project continues to build structure.

The create method validates required fields:

```
if (string.IsNullOrWhiteSpace(request.FullName))
{
    throw new ArgumentException("User full name is required.");
}

if (string.IsNullOrWhiteSpace(request.Email))
{
    throw new ArgumentException("User email is required.");
}
```

Then it creates a response:

```
UserResponse response = new UserResponse()
{
    Id = _nextId,
    FullName = request.FullName,
    Email = request.Email,
    CreatedAtUtc = DateTime.UtcNow
};
```

Finally, it stores and returns the user:

```
_nextId += 1;
_users.Add(response);

return response;
```

Why No Controller Yet

Day 15 intentionally stops at the application layer.

That is a useful discipline. The controller should come after the request, response, interface, and service are ready.

This avoids building an endpoint first and then forcing the rest of the structure around it.

Lesson Explanation

Day 15 starts the user module without adding an HTTP endpoint.

This is an important shift. Earlier days focused heavily on the task API surface. Today focuses on application structure. Before users can be exposed through a controller, the application needs to know what a user creation request looks like, what a user response looks like, and what user operations exist.

The request DTO and response DTO keep input and output separate. The service interface defines the behavior. The service implementation gives the project a working in-memory version that can later be replaced or extended when persistence is added.

This is how a backend grows without everything collapsing into controllers.

Reader Exercise

Open the new user application files and answer:

- What data does `CreateUserRequest` accept?
- What extra data does `UserResponse` return?
- Why does the request model not include `Id`?
- What methods does `IUserService` expose?
- What validation does `UserService` perform?
- Why is user data still stored in memory?

Then run:

```
dotnet build
```

Expected:

```
0 warnings
0 errors
```

Future Improvement

The next steps can add:

- dependency injection registration
- `UsersController`
- `POST /api/users`
- `GET /api/users`
- better validation
- email format checks
- database persistence

The service structure created today makes those steps easier.

Next Lesson

Day 15 creates the User application layer.

The next lesson can expose that work through API endpoints by registering `UserService` and adding a `UsersController`.

Day 16 - First User API Endpoints

Lesson Goal

Expose the User application layer through the first User API endpoints.

Day 15 created the user service structure. Day 16 connects that structure to HTTP.

The new endpoints are:

```
GET /api/users
POST /api/users
```

Why This Is Day 16

The user module already has:

- `CreateUserRequest`
- `UserResponse`
- `IUserService`
- `UserService`

Those files live in the Application layer. Day 16 adds the API layer entry point.

This follows the same direction used by the task feature:

```
Controller -> Service -> Response DTO
```

What Changed

Day 16 adds the first user controller and registers the user service.

Important files:

- `src/JobTrackr.Api/Controllers/UsersController.cs`
- `src/JobTrackr.Api/Program.cs`
- `README.md`

The main changes are:

- `UserService` is registered with dependency injection.
- `UsersController` is created.
- `GET /api/users` returns all users.
- `POST /api/users` creates a user.
- Invalid full name returns `400 Bad Request`.
- Invalid email returns `400 Bad Request`.
- README documents the new user endpoints.

Dependency Injection

The API project needs to know how to provide `IUserService`.

In `Program.cs`, the user service is registered:

```
builder.Services.AddSingleton<IUserService, UserService>();
```

This allows ASP.NET Core to inject `IUserService` into `UsersController`.

The application now has service registrations for both task behavior and user behavior.

UsersController

The controller is created in:

```
src/JobTrackr.Api/Controllers/UsersController.cs
```

It uses the normal controller route pattern:

```
[Route("api/[controller]")]
[ApiController]
public class UsersController : ControllerBase
```

Because the controller is named `UsersController`, the route becomes:

```
/api/users
```

Get All Users

The first endpoint returns all users:

```
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var users = await _userService.GetAll();

    return Ok(users);
}
```

Request:

```
GET /api/users
```

Expected response:

```
200 OK
```

This matches the simple list behavior already used in the task API.

Create User

The second endpoint creates a user:

```
[HttpPost]
public async Task<IActionResult> Create(CreateUserRequest request)
{
    try
    {
        var response = await _userService.CreateUser(request);

        return Ok(response);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Request:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Expected response:

```
200 OK
```

For now, the endpoint returns `200 OK`. A future improvement can change this to `201 Created` after adding `GET /api/users/{id}`.

Validation Behavior

The service already validates required user fields.

Invalid full name:

```
{
  "fullName": "",
  "email": "test@example.com"
}
```

Expected response:

```
400 Bad Request
```

Invalid email:

```
{
  "fullName": "Test User",
  "email": ""
}
```

Expected response:

```
400 Bad Request
```

The controller catches `ArgumentException` and returns the message as a bad request.

API Shape

After Day 16, the project has task endpoints and user endpoints.

Task endpoints:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

User endpoints:

```
GET /api/users
POST /api/users
```

This is the first step toward a broader backend, not only a task feature.

Lesson Explanation

Day 16 turns the user application service into real HTTP endpoints.

This is where the separation between layers becomes visible. The controller does not store users itself. It does not decide how users are created. It receives HTTP requests and delegates the work to `IUserService`.

The service remains responsible for user creation and validation. The controller is responsible for translating that result into HTTP responses.

This is a small example of Clean Architecture direction: the API layer depends on the Application layer, while the Application layer does not depend on the API layer.

Reader Exercise

Open Swagger and test:

```
GET /api/users
POST /api/users
```

Create a valid user:

```
{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then test invalid requests:

```
{
  "fullName": "",
  "email": "test@example.com"
}
```

```
{
  "fullName": "Test User",
  "email": ""
}
```

Verify:

- valid create returns `200 OK`
- invalid full name returns `400 Bad Request`
- invalid email returns `400 Bad Request`
- `GET /api/users` returns created users

Future Improvement

The current create endpoint returns `200 OK`.

Later, the API can improve this by adding:

```
GET /api/users/{id}
```

Then `POST /api/users` can return:

```
201 Created
```

with a location header pointing to the created user.

Other future improvements:

- email format validation
- duplicate email checks
- user update endpoint
- user delete endpoint
- database persistence
- automated tests

Next Lesson

Day 16 exposes the first user endpoints.

The next lesson can improve the user API by adding `GET /api/users/{id}` or improving create behavior to return `201 Created`.

Day 17 - Get User By ID Endpoint

Lesson Goal

Add the endpoint to get one user by id.

Day 16 added:

```
GET /api/users
POST /api/users
```

Day 17 adds:

```
GET /api/users/{id}
```

The endpoint should return the user when it exists and `404 Not Found` when it does not.

Why This Is Day 17

The User API now has create and list behavior.

The next normal CRUD step is reading one user by identifier. This follows the same pattern already used in the Task API.

What Changed

Day 17 adds single-user lookup behavior.

Important files:

- `src/JobTrackr.Application/Users/IUserService.cs`
- `src/JobTrackr.Application/Users/UserService.cs`
- `src/JobTrackr.Api/Controllers/UsersController.cs`
- `README.md`

The main changes are:

- `IUserService` now exposes `GetById`.
- `UserService` can search the in-memory user list by id.
- `UsersController` exposes `GET /api/users/{id}`.
- Missing users return `404 Not Found`.
- README documents the new endpoint.

Service Contract

The user service interface adds:

```
Task<UserResponse?> GetById(int id);
```

The nullable return type matters.

`UserResponse?` means:

- return `UserResponse` when the user exists
- return `null` when no user is found

This lets the controller translate application behavior into HTTP behavior.

Service Implementation

The service searches the in-memory list:

```
public async Task<UserResponse?> GetById(int id)
{
    await Task.Yield();

    return _users.Find(user => user.Id == id);
}
```

This is simple for now because users are still stored in memory.

Later, when persistence is added, this same method can query the database instead.

Controller Endpoint

The controller adds:

```
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var response = await _userService.GetById(id);

    if (response is null)
    {
        return NotFound("User not found.");
    }

    return Ok(response);
}
```

The route is:

```
GET /api/users/{id}
```

Example:

```
GET /api/users/1
```

API Behavior

When the user exists:

```
200 OK
```

When the user does not exist:

```
404 Not Found
```

Expected missing-user message:

```
User not found.
```

This keeps the behavior clear and consistent with the task API pattern.

API Shape

After Day 17, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
```

The project now has:

- Task CRUD and workflow endpoints

- User create endpoint
- User list endpoint
- User get-by-id endpoint

Lesson Explanation

Day 17 adds the next basic read operation for users.

Creating users and listing all users is useful, but most APIs also need to fetch one item by id. The service method returns a nullable response because finding a user is not guaranteed. The controller then decides how to represent that result in HTTP.

If the user exists, the controller returns `200 OK`. If the service returns `null`, the controller returns `404 Not Found`.

This separation is important. The service describes application behavior. The controller describes HTTP behavior.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then fetch the user:

```
GET /api/users/1
```

Then test a missing user:

```
GET /api/users/999
```

Verify:

- existing user returns `200 OK`
- missing user returns `404 Not Found`
- missing-user message is `User not found.`
- `GET /api/users` still returns the user list
- `POST /api/users` still creates users

Future Improvement

Now that `GET /api/users/{id}` exists, `POST /api/users` can later be improved to return:

```
201 Created
```

with a location header pointing to:

```
GET /api/users/{id}
```

Other future improvements:

- update user endpoint
- delete user endpoint

- duplicate email validation
- email format validation
- database persistence
- automated user API tests

Next Lesson

Day 17 adds single-user lookup.

The next lesson can improve user creation behavior or continue building the user CRUD flow with update and delete endpoints.

Day 18 - Update User Endpoint

Lesson Goal

Add the endpoint to update an existing user.

Day 18 adds:

```
PUT /api/users/{id}
```

This endpoint should:

- update user full name
- update user email
- return `200 OK` when the user exists
- return `404 Not Found` when the user does not exist
- return `400 Bad Request` when full name or email is empty

Why This Is Day 18

The User API already supports:

```
GET /api/users
POST /api/users
GET /api/users/{id}
```

The next normal CRUD step is updating a user.

This continues the same pattern already used by the Task API: add the request model, update the service contract, implement the service logic, then expose it through the controller.

What Changed

Important files:

- `src/JobTrackr.Application/Users/UpdateUserRequest.cs`
- `src/JobTrackr.Application/Users/IUserService.cs`
- `src/JobTrackr.Application/Users/UserService.cs`
- `src/JobTrackr.Api/Controllers/UsersController.cs`
- `README.md`

The main changes are:

- `UpdateUserRequest` was added.
- `IUserService` now exposes `UpdateUser`.
- `UserService` can update an existing user.
- `UsersController` exposes `PUT /api/users/{id}`.
- Missing users return `User not found.`
- Invalid full name or email returns `400 Bad Request`.

UpdateUserRequest

The update request contains the editable user fields:

```
public class UpdateUserRequest
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;
}
```

The request does not include `Id`.

The id comes from the URL:

```
PUT /api/users/{id}
```

This keeps route identity separate from request body data.

Service Contract

The user service interface adds:

```
Task<UserResponse?> UpdateUser(int id, UpdateUserRequest request);
```

The nullable response means:

- return the updated user when found

- return `null` when the user does not exist

Validation errors are represented with `ArgumentException`, matching the current user service style.

Service Implementation

The service first finds the user:

```
var user = _users.Find(user => user.Id == id);

if (user is null)
{
    return null;
}
```

Then it validates input:

```
if (string.IsNullOrWhiteSpace(request.FullName))
{
    throw new ArgumentException("User full name is required.");
}

if (string.IsNullOrWhiteSpace(request.Email))
{
    throw new ArgumentException("User email is required.");
}
```

Then it updates the user:

```
user.FullName = request.FullName;
user.Email = request.Email;

return user;
```

This keeps update behavior in the Application layer.

Controller Endpoint

The controller exposes:

```
[HttpPut("{id}")]
```

```
public async Task<IActionResult> Update(int id, UpdateUserRequest request)
{
    try
    {
        var response = await _userService.UpdateUser(id, request);

        if (response is null)
        {
            return NotFound("User not found.");
        }

        return Ok(response);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(ex.Message);
    }
}
```

The controller handles three outcomes:

- updated user -> `200 OK`
- missing user -> `404 Not Found`
- invalid request -> `400 Bad Request`

API Behavior

Valid update:

```
PUT /api/users/1
Content-Type: application/json

{
  "fullName": "Updated User",
  "email": "updated@example.com"
}
```

Expected:

```
200 OK
```

Missing user:

```
PUT /api/users/999
```

Expected:

```
404 Not Found
```

Invalid input:

```
{
  "fullName": "",
  "email": "updated@example.com"
}
```

Expected:

```
400 Bad Request
```

API Shape

After Day 18, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
```

The project now has a partial User CRUD flow:

- create user
- list users
- get user by id
- update user

Lesson Explanation

Day 18 adds the update operation for users.

This is another standard CRUD step, but it also reinforces an important API design rule: the resource id belongs in the route, while the editable data belongs in the request body.

The service is responsible for finding and updating the user. The controller is responsible for translating service outcomes into HTTP responses. When the service returns `null`, the controller returns `404 Not Found`. When validation fails, the controller returns `400 Bad Request`.

This keeps the responsibilities clear and prevents controller code from becoming the place where all application behavior lives.

Reader Exercise

Create a user:

```
POST /api/users
```

Then update it:

```
PUT /api/users/1
```

Verify:

- valid update returns `200 OK`
- updated full name is returned
- updated email is returned
- missing user returns `404 Not Found`
- empty full name returns `400 Bad Request`
- empty email returns `400 Bad Request`
- `GET /api/users/1` returns the updated user

Future Improvement

The current validation checks only empty values.

Future improvements can add:

- email format validation
- duplicate email validation
- better error response objects
- automated tests for user updates
- persistence-backed user updates

Next Lesson

Day 18 adds update behavior to the User API.

The next logical step is to add delete behavior or improve create/update responses before moving toward persistence.

Day 19 - Delete User Endpoint

Lesson Goal

Add the endpoint to delete an existing user.

Day 19 adds:

```
DELETE /api/users/{id}
```

This endpoint should:

- delete the user when it exists
- return `204 No Content` when deletion succeeds
- return `404 Not Found` when the user does not exist

Why This Is Day 19

The User API already supports:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
```

The next normal CRUD step is deleting a user.

After Day 19, the basic in-memory User CRUD flow is complete.

What Changed

Important files:

- `src/JobTrackr.Application/Users/IUserService.cs`
- `src/JobTrackr.Application/Users/UserService.cs`
- `src/JobTrackr.Api/Controllers/UsersController.cs`
- `README.md`

The main changes are:

- `IUserService` now exposes `DeleteUser`.
- `UserService` can remove a user from the in-memory list.
- `UsersController` exposes `DELETE /api/users/{id}`.
- Successful deletion returns `204 No Content`.
- Missing users return `404 Not Found`.
- README documents the new endpoint.

Service Contract

The user service interface adds:

```
Task<bool> DeleteUser(int id);
```

The boolean return value gives the controller a simple result:

- `true`: user existed and was deleted
- `false`: user was not found

This is different from `GetById` and `UpdateUser`, which return nullable user responses.

Service Implementation

The service finds the user first:

```
var user = _users.Find(user => user.Id == id);
```

If the user does not exist, it returns `false`:

```
if (user is null)
{
    return false;
}
```

If the user exists, it removes the user:

```
_users.Remove(user);

return true;
```

This keeps deletion behavior inside the Application layer.

Controller Endpoint

The controller exposes:

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var isDeleted = await _userService.DeleteUser(id);

    if (!isDeleted)
    {
        return NotFound("User not found.");
    }

    return NoContent();
}
```

The route is:

```
DELETE /api/users/{id}
```

Example:

```
DELETE /api/users/1
```

API Behavior

When deletion succeeds:

```
204 No Content
```

When the user does not exist:

```
404 Not Found
```

Expected missing-user message:

```
User not found.
```

`204 No Content` is appropriate because the delete action succeeded and there is no response body to return.

API Shape

After Day 19, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

This completes the basic in-memory User CRUD phase.

The project now has:

- task CRUD and workflow endpoints
- task filtering and search
- user CRUD endpoints

Lesson Explanation

Day 19 completes the basic User CRUD flow by adding delete behavior.

The delete operation has a slightly different response shape from create, get, and update. When a user is deleted successfully, the API returns `204 No Content`. That tells the client the operation succeeded, but there is no response body.

If the user does not exist, the service returns `false`, and the controller translates that into `404 Not Found`.

This keeps the same separation used in previous lessons: the service handles application behavior, and the controller translates that behavior into HTTP responses.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Delete the user:

```
DELETE /api/users/1
```

Then try to fetch the deleted user:

```
GET /api/users/1
```

Verify:

- deleting an existing user returns `204 No Content`
- fetching the deleted user returns `404 Not Found`
- deleting a missing user returns `404 Not Found`
- `GET /api/users` no longer includes the deleted user

Future Improvement

The User API is still in-memory.

Future improvements should include:

- persistence with a database
- automated tests for delete behavior
- duplicate email validation
- better error response objects
- `201 Created` for user creation
- user-task relationship modeling

Next Lesson

Day 19 completes the basic User CRUD phase.

The next phase can focus on cleanup, review, testing, persistence, or connecting users and tasks in the domain model.

Day 20 - Connect Tasks To Users

Lesson Goal

Connect tasks to users.

Day 20 updates task creation so every new task belongs to an existing user.

The key new field is:

```
UserId
```

This starts the relationship between the Task API and the User API.

Why This Is Day 20

Tasks and users currently exist separately.

A real task tracking system needs this relationship:

```
One user can have many tasks.
One task belongs to one user.
```

For now, this relationship stays in memory. SQL Server and Entity Framework Core come later.

What Changed

Important files:

- `src/JobTrackr.Application/Tasks/CreateTaskRequest.cs`
- `src/JobTrackr.Application/Tasks/TaskResponse.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Program.cs`
- `README.md`

The main changes are:

- `CreateTaskRequest` now includes `UserId`.
- `TaskResponse` now includes `UserId`.
- `TaskService` receives `IUserService`.
- Task creation validates that the user exists.
- Creating a task for a missing user returns an error.
- README documents the new task creation behavior.

CreateTaskRequest

Task creation now requires a user id:

```
public class CreateTaskRequest
{
    public string Title { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public int UserId { get; set; }
}
```

The request now says:

```
Create this task for this user.
```

TaskResponse

The task response also includes `UserId`:

```
public int UserId { get; set; }
```

This lets the API response show which user owns the task.

That is important for clients because a task without ownership is not useful in a real task tracking system.

Service Dependency

`TaskService` needs to confirm that the user exists before creating the task.

It receives `IUserService` through dependency injection:

```
private readonly IUserService _userService;

public TaskService(IUserService userService)
{
    _userService = userService;
}
```

This lets the task service ask the user service for the user.

User Validation

Task creation now validates `UserId`:

```
if (request.UserId <= 0)
{
    throw new ArgumentException("UserId is required.");
}
```

Then it checks that the user exists:

```
var user = await _userService.GetById(request.UserId);

if (user is null)
{
    throw new ArgumentException("User not found.");
}
```

Only after that should the task be created.

API Behavior

Valid task creation now looks like this:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Prepare resume",
  "description": "Update backend project section",
  "userId": 1
}
```

Expected result:

```
201 Created
```

Missing or invalid user:

```
{
  "title": "Prepare resume",
```

```
  "description": "Update backend project section",
  "userId": 999
}
```

Expected result:

```
400 Bad Request
```

Expected message:

```
User not found.
```

Service Registration

Both services must be registered:

```
builder.Services.AddSingleton<IUserService, UserService>();
builder.Services.AddSingleton<ITaskService, TaskService>();
```

`TaskService` depends on `IUserService`, so the User service registration must exist.

Keeping user registration before task registration is clearer for readers.

API Shape

After Day 20, task creation is no longer isolated from users.

User API:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Task API:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The important Day 20 change is that `POST /api/tasks` now requires a valid user.

Lesson Explanation

Day 20 is the first relationship day.

Until now, tasks and users were separate pieces of the backend. That is useful for learning CRUD, but it is not realistic for a task tracking system. A task should belong to a user.

The project is still in memory, but the design starts moving toward the real database relationship that will exist later. `CreateTaskRequest` now includes `UserId`, and `TaskService` validates that the referenced user exists before creating the task.

This is a small change, but it introduces an important backend idea: one feature may need to depend on another feature's application service to enforce a business rule.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json
```

```
{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then create a task for that user:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Prepare resume",
  "description": "Update backend project section",
  "userId": 1
}
```

Then try creating a task for a missing user:

```
{
  "title": "Invalid task",
  "description": "This user does not exist",
  "userId": 999
}
```

Verify:

- creating a task for an existing user succeeds
- the response includes `userId`
- creating a task with `userId` less than or equal to zero returns an error
- creating a task for a missing user returns `User not found.`

Future Improvement

This relationship is still in memory.

Future improvements should include:

- database-backed users and tasks
- real foreign key relationship
- user-specific task queries
- `GET /api/users/{id}/tasks`
- stronger validation
- automated tests for task-user ownership

Next Lesson

Day 20 connects tasks to users.

The next phase can continue improving relationships, add user-specific task endpoints, or start preparing for persistence with Entity Framework Core.

Day 21 - Review User And Task APIs

Lesson Goal

Review and stabilize the User and Task APIs after connecting tasks to users.

Day 21 does not add a new feature.

The goal is to confirm:

- users can be created
- tasks can be created for existing users
- tasks cannot be created for missing users
- Task CRUD still works
- User CRUD still works
- the project is ready before moving toward Infrastructure, EF Core, and SQL Server

Why This Is Day 21

Day 20 connected tasks to users with `UserId`.

Before adding database work, the in-memory behavior should be correct. This review day checks that the relationship works and that older endpoints were not broken by the new user-task connection.

What To Review

Important files:

- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `src/JobTrackr.Api/Controllers/UsersController.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Application/Users/UserService.cs`
- `src/JobTrackr.Application/Tasks/CreateTaskRequest.cs`
- `src/JobTrackr.Application/Tasks/TaskResponse.cs`
- `README.md`

The review should confirm:

- the project builds successfully
- Swagger shows the expected endpoints
- task creation requires a valid user
- the README matches the current API behavior
- GitHub has the latest clean code

Current API Shape

Task endpoints:

```
GET /api/tasks
```

```
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Task query behavior:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

User endpoints:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

This confirms the project still compiles after the relationship work from Day 20.

User Create Test

Request:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Task Owner",
  "email": "owner@example.com"
}
```

Expected:

```
200 OK
```

Check that the response includes:

- `id`
- `fullName`
- `email`

Get User Tests

Get all users:

```
GET /api/users
```

Expected:

```
200 OK
```

Get one user:

```
GET /api/users/1
```

Expected:

```
200 OK
```

Missing user:

```
GET /api/users/999
```

Expected:

```
404 Not Found
```

Expected message:

```
User not found.
```

Create Task For Existing User

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "User-owned task",
  "description": "Task belongs to user 1",
  "userId": 1
}
```

Expected:

```
201 Created
```

Check:

- response includes `userId`
- `userId` is `1`

Create Task For Missing User

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Invalid task",
  "description": "User does not exist",
  "userId": 999
}
```

Expected:

```
400 Bad Request
```

Expected message:

```
User not found.
```

Invalid Task UserId

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Invalid user id",
  "description": "UserId is invalid",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

Expected message:

```
UserId is required.
```

Task Workflow Tests

After creating a valid task, verify:

```
GET /api/tasks
GET /api/tasks/1
PATCH /api/tasks/1/complete
PATCH /api/tasks/1/reopen
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=owned
```

These tests confirm that connecting tasks to users did not break the existing task workflow.

User CRUD Tests

Verify:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

These tests confirm that User CRUD still works after task creation began depending on users.

Lesson Explanation

Day 21 is a checkpoint day.

The project now has two connected areas: users and tasks. That makes review more important. A task can no longer be created in isolation; it must belong to a valid user. This relationship is still in memory, but it introduces the same kind of rule that will later be enforced by a database relationship.

The purpose of the review is to confirm that the new rule works without breaking existing behavior. Users should still be creatable, readable, updateable, and deleteable. Tasks should still support CRUD, complete/reopen, filtering, and search. Task creation should now reject invalid users.

This is the right point to stabilize before moving toward Infrastructure and persistence.

Reader Exercise

Test the API in this order:

1. Create a user.
2. Get all users.
3. Get the created user by id.
4. Create a task for that user.
5. Create a task for a missing user and confirm it fails.
6. Complete and reopen the valid task.
7. Search and filter tasks.
8. Update the user.

9. Delete the user.

For each request, write down:

- request URL
- request body if any
- expected status code
- actual status code
- response body

Future Improvement

Manual review is useful, but this flow should eventually become automated tests.

Future improvements should include:

- API tests for user CRUD
- API tests for task CRUD
- tests for task-user ownership
- tests for invalid `UserId`
- persistence with SQL Server
- EF Core relationships between users and tasks

Next Lesson

Day 21 stabilizes the in-memory User and Task APIs.

The next phase can begin moving toward Infrastructure, EF Core, SQL Server, and real persistence.

Day 22 - Prepare Infrastructure Layer

Lesson Goal

Prepare the Infrastructure layer for future database work.

Day 22 does not add EF Core.

Day 22 does not add SQL Server.

The goal is to review the project structure and make sure the solution is ready before persistence is introduced.

Why This Is Day 22

So far, data is stored in memory inside services.

The next major backend phase is persistence:

```
Infrastructure -> EF Core -> SQL Server
```

Before adding packages and database code, the project structure should be understandable and clean.

Current Architecture

The solution currently has these layers:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Expected responsibilities:

```
Api             -> controllers, Program.cs, Swagger, HTTP behavior
Application     -> DTOs, service interfaces, use-case logic
Domain          -> business entities
Infrastructure  -> database and external system code later
```

What To Review

Important files and folders:

- `src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj`
- `src/JobTrackr.Infrastructure`
- `src/JobTrackr.Api/JobTrackr.Api.csproj`
- `src/JobTrackr.Application/JobTrackr.Application.csproj`
- `README.md`

The review should confirm:

- Infrastructure project exists
- project references are understandable
- unused starter files are removed if present
- README explains Infrastructure responsibility clearly
- project builds successfully

Infrastructure Responsibility

The Infrastructure layer is reserved for code that talks to outside systems.

In this project, that will mainly mean database code later:

- EF Core `DbContext`
- database configuration
- persistence logic
- repository implementation if needed
- SQL Server integration

The important point is that database concerns should not be placed directly in controllers.

What Not To Do Today

Day 22 has strict boundaries.

Do not add:

- EF Core packages
- SQL Server configuration
- `AppDbContext`
- migrations
- repositories
- database connection strings

This day is about preparation, not persistence implementation.

Project References

The Infrastructure project should be reviewed for references.

Infrastructure will later need to understand:

- Application contracts
- Domain entities

The API currently uses the Application layer for controllers and service registration.

Later, the API may also reference Infrastructure when database-backed services are registered.

For today, the goal is to understand the references and avoid unnecessary changes.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

This confirms the solution is still healthy before persistence work begins.

Git Check

Run:

```
git status
```

If files changed, commit them.

If no files changed:

```
No commit needed.
```

Review days may or may not create commits. The important thing is that the working tree ends clean.

Lesson Explanation

Day 22 starts the persistence preparation phase.

The project already has working in-memory User and Task APIs. Tasks are connected to users through `UserId`, and the current APIs have been reviewed. That means the next major step is moving from in-memory storage to real persistence.

Before adding EF Core or SQL Server, the Infrastructure layer should be reviewed. In Clean Architecture, Infrastructure is the place for database and external system code. Controllers should not directly own database logic, and Application services should not be tightly coupled to SQL Server details.

This day is intentionally conservative. It protects the project from jumping into database work before the structure is ready.

Reader Exercise

Open the solution and answer:

- Does `JobTrackr.Infrastructure` exist?
- Does it contain unused starter files?
- What project references does Infrastructure currently have?
- What project references does API currently have?
- Does README explain the role of Infrastructure?
- Does the project build successfully?
- Is git status clean?

Then explain the layer responsibility map in your own words:

```
Api
Application
Domain
Infrastructure
```

Future Improvement

The next phase can add persistence gradually:

- install EF Core packages
- create `AppDbContext`
- configure SQL Server
- add database entities or mappings

- add migrations
- move in-memory data to database-backed storage

The key is to do this step by step without mixing database code into controllers.

Next Lesson

Day 22 prepares the Infrastructure layer.

The next lesson can begin the actual persistence work by adding EF Core packages or creating the first database context.

Day 23 - Add EF Core Database Context

Lesson Goal

Add the first Entity Framework Core foundation to the Infrastructure layer.

Day 23 does not move the application fully to SQL Server.

The goal is to prepare the project for persistence by adding EF Core packages, creating `AppDbContext`, and confirming the solution still builds.

Why This Is Day 23

Until now, JobTrackr stores users and tasks in memory.

That is useful for learning API behavior, but it is not permanent storage. When the API stops, the data disappears.

The next backend phase is persistence:

```
In-memory services -> EF Core DbContext -> SQL Server
```

Before replacing service logic, the project needs a database context inside the Infrastructure layer.

Files Changed

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

What Was Added

The Infrastructure project received EF Core package references:

```
Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Design
```

The API project now references Infrastructure so the API can later register database services from that layer.

A new persistence folder was created:

```
src/JobTrackr.Infrastructure/Persistence
```

Inside that folder, the project now has:

```
AppDbContext.cs
```

AppDbContext

`AppDbContext` is the EF Core class that represents the database session for the application.

It exposes tables through `DbSet` properties:

```
public DbSet<User> Users { get; set; }
public DbSet<JobTask> Tasks { get; set; }
```

This does not create the database by itself.

It defines the shape EF Core will use when the project later connects to SQL Server and adds migrations.

Architecture Decision

Database code belongs in Infrastructure.

The API project should not own EF Core models or database setup directly. The API can call Infrastructure during startup, but the database implementation should stay behind the Infrastructure boundary.

This keeps the Clean Architecture direction clear:

```
API -> Application -> Domain
API -> Infrastructure
Infrastructure -> Domain
```

The Domain layer still contains entities. Infrastructure uses those entities to map database tables.

What Not To Change Yet

Do not remove the in-memory services on Day 23.

Do not rewrite all controllers.

Do not add migrations yet unless the lesson specifically reaches that step.

Do not configure every database setting before the persistence flow is ready.

This lesson is only the database foundation.

Build Check

After adding EF Core and `AppDbContext`, the project should build successfully:

```
dotnet build
```

Expected result:

```
0 errors
```

Existing Swagger endpoints should still work the same as before because the runtime behavior has not been moved to the database yet.

Git Check

The related commit is:

```
8811257 Day 23: Add EF Core database context
```

Files changed in that commit:

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

Lesson Explanation

Day 23 is the first real step toward permanent storage.

The application already has user and task APIs, but the data still lives in memory. That means the API can demonstrate behavior, but it cannot yet behave like a real backend system.

Entity Framework Core gives the project a bridge between C# entities and SQL Server tables.

The important part of this lesson is restraint. The project does not need to replace every service in one step. A cleaner approach is to first install the correct EF Core packages, create the database context, confirm entity names, and make sure the solution still builds.

This keeps the learning path controlled. One lesson adds the foundation. Later lessons can add SQL Server configuration, migrations, repositories, and then replace in-memory behavior one feature at a time.

Reader Exercise

Open `AppDbContext.cs` and identify which domain entities are being prepared for database storage.

Then answer:

- Why should `AppDbContext` live in Infrastructure?
- Why is it better not to remove the in-memory services in the same lesson?
- What will EF Core need next before it can create SQL Server tables?

Future Improvement

The next persistence steps should include:

- registering `AppDbContext` in `Program.cs`
- adding a SQL Server connection string
- creating the first EF Core migration
- applying the migration to the database
- replacing in-memory services with database-backed behavior

Next Lesson

Day 23 prepares the database context.

The next lesson can connect the context to SQL Server and start turning the persistence foundation into a working database-backed backend.

Day 24 - Register SQL Server DbContext

Lesson Goal

Register `AppDbContext` in the API startup configuration and add a SQL Server connection string.

Day 23 created the EF Core database context.

Day 24 connects that context to the ASP.NET Core application so EF Core can be used by the API later.

Why This Is Day 24

Creating `AppDbContext` is not enough by itself.

ASP.NET Core needs to know how to create it through dependency injection.

That setup belongs in `Program.cs`, because `Program.cs` is where the application registers services before the API starts.

The progression is:

```
Day 23 -> create AppDbContext
Day 24 -> register AppDbContext with SQL Server
Later  -> add migrations and database-backed services
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/appsettings.json
```

Connection String

`appsettings.json` now contains the SQL Server connection string:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

This tells EF Core where the database will be created and used.

If SQL Server uses a named instance, the server value can be changed later, for example:

```
localhost\SQLEXPRESS
```

Registering AppDbContext

`Program.cs` now imports the Infrastructure persistence namespace and EF Core:

```
using JobTrackr.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
```

Then it registers `AppDbContext`:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

This line tells ASP.NET Core:

- create `AppDbContext` through dependency injection
- use SQL Server as the EF Core provider

- read the connection string from configuration

Important Architecture Point

The application is still using the existing in-memory services.

That is correct for Day 24.

Registering `AppDbContext` does not automatically move reads and writes to SQL Server.

The API will continue to behave the same until the services are changed to use EF Core.

What Not To Do Today

Do not rewrite `TaskService`.

Do not rewrite `UserService`.

Do not create migrations yet unless the lesson has reached that point.

Do not try to move all data access in one step.

The purpose of this lesson is configuration only.

Build Check

Run the build from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Existing Swagger endpoints should still appear:

```
GET /api/tasks
GET /api/users
```

Git Check

The related commit is:

```
eff9c2a Day 24: Register SQL Server DbContext
```

Files changed in that commit:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/appsettings.json
```

Lesson Explanation

Day 24 connects the EF Core foundation to the running API.

The previous lesson created `AppDbContext`, but the API still had no startup configuration for it. In ASP.NET Core, services are registered in `Program.cs`, so this lesson adds the database context to the dependency injection container.

The connection string is stored in `appsettings.json` under `ConnectionStrings`. `Program.cs` reads that value and passes it to EF Core with `UseSqlServer`.

This is a small change, but it is an important backend step. It separates setup from behavior. The project is now configured for SQL Server, but the actual user and task services still use in-memory storage. That keeps the migration toward persistence controlled and easier to debug.

Reader Exercise

Open `Program.cs` and find the `AddDbContext` registration.

Then answer:

- Where does the SQL Server connection string come from?
- Why is `AppDbContext` registered in the API project?
- Why do the existing services still behave the same after this lesson?

Future Improvement

The next database steps should include:

- adding the first EF Core migration
- creating the database
- checking generated tables
- moving one service from in-memory storage to EF Core
- testing API behavior after persistence is introduced

Next Lesson

Day 24 makes EF Core available to the API.

The next lesson can start using that setup to create migrations and move closer to real SQL Server persistence.

Day 25 - Create First EF Core Migration

Lesson Goal

Create the first Entity Framework Core migration for the JobTrackr backend.

Day 23 created `AppDbContext`.

Day 24 registered `AppDbContext` with SQL Server.

Day 25 creates migration files that describe the first database structure.

Why This Is Day 25

EF Core migrations track database structure over time.

A migration is a C# description of what tables, columns, keys, and constraints should exist in the database.

The project is moving through persistence in controlled steps:

```
Entities -> AppDbContext -> Migration files -> Database update -> EF-backed services
```

Day 25 reaches the migration files step.

Files Changed

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

EF Core Tooling

Before creating migrations, the EF Core CLI tool must be available:

```
dotnet ef --version
```

If the tool is missing, install the matching .NET 8 version:

```
dotnet tool install --global dotnet-ef --version 8.0.27
```

If it already exists but uses the wrong version, update it:

```
dotnet tool update --global dotnet-ef --version 8.0.27
```

Startup Project Design Package

The API project may need `Microsoft.EntityFrameworkCore.Design` because it is the startup project.

EF Core tooling loads `Program.cs` and configuration from the startup project when creating migrations.

Command:

```
dotnet add src/JobTrackr.Api package Microsoft.EntityFrameworkCore.Design --version 8.0.27
```

This is why `JobTrackr.Api.csproj` is part of the Day 25 changes.

Migration Command

The migration is created from the repository root:

```
dotnet ef migrations add InitialCreate --project src/JobTrackr.Infrastructure --startup-project
src/JobTrackr.Api --output-dir Persistence/Migrations
```

Meaning:

```
InitialCreate
```

The migration name.

```
--project src/JobTrackr.Infrastructure
```

Migration files belong to the Infrastructure project.

```
--startup-project src/JobTrackr.Api
```

The API project contains `Program.cs` and `appsettings.json`.

```
--output-dir Persistence/Migrations
```

Migration files are stored under the persistence area of Infrastructure.

Expected Migration Output

The migration folder should contain files like:

```
20260607060440_InitialCreate.cs
20260607060440_InitialCreate.Designer.cs
AppDbContextModelSnapshot.cs
```

The important file to review first is:

```
InitialCreate.cs
```

It should contain `migrationBuilder.CreateTable` calls for the current entities.

Expected Tables

Based on the current project, EF Core should prepare tables for:

```
Users
Tasks
```

Expected `Users` fields:

```
Id
FullName
Email
CreatedAtUtc
```

Expected `Tasks` fields:

```
Id
Title
Description
IsCompleted
CreatedAtUtc
UserId
```

Use the actual entity properties in the codebase. Do not add random columns just to make the database look bigger.

What Not To Do Today

Do not rewrite `TaskService`.

Do not rewrite `UserService`.

Do not move the API from memory to SQL Server yet.

Do not update the database unless the lesson plan explicitly reaches that step.

Day 25 creates migration files only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

If the project builds, the migration files are valid C# and the solution remains consistent.

Git Check

The related commit is:

```
cc3b907 Day 25: Add initial database migration
```

Files changed in that commit:

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

Lesson Explanation

Day 25 creates the first database migration.

At this point, the application has domain entities, an EF Core `AppDbContext`, and SQL Server registration in `Program.cs`. The next step is to ask EF Core to inspect the model and generate a migration.

The migration is not the database itself. It is a versioned set of instructions that tells EF Core what database structure should be created.

For JobTrackr, the first migration prepares the users and tasks tables based on the current domain entities. This is why reviewing the generated migration matters. It shows whether EF Core understood the current model correctly.

The key discipline in this lesson is to stop after creating and reviewing the migration. The application services still use memory. Database-backed behavior will come later after the database structure is confirmed.

Reader Exercise

Open the generated `InitialCreate.cs` migration file.

Find the `CreateTable` calls and answer:

- Which tables will EF Core create?
- Which columns belong to `Users`?
- Which columns belong to `Tasks`?
- Is `UserId` present on tasks?

- Why are migrations stored in Infrastructure instead of Domain?

Future Improvement

The next persistence steps should include:

- running `dotnet ef database update`
- confirming the database exists in SQL Server
- checking generated tables
- deciding which service should move to EF Core first
- testing API behavior after persistence is introduced

Next Lesson

Day 25 creates the migration plan.

The next lesson can apply that plan to SQL Server and create the actual database tables.

Day 26 - Apply EF Core Migration

Lesson Goal

Apply the first EF Core migration to SQL Server.

Day 25 created the migration files.

Day 26 uses those migration files to create the real database and tables.

Why This Is Day 26

A migration file is only a database change plan.

It does not create tables until it is applied to a database.

The persistence flow is:

```
Entities -> AppDbContext -> Migration files -> SQL Server database
```

Day 26 moves from migration files to an actual SQL Server database.

Expected Result

After applying the migration, SQL Server should contain:

```
JobTrackrDb
```

Inside that database, the expected tables are:

```
dbo.Users
dbo.Tasks
dbo.__EFMigrationsHistory
```

`__EFMigrationsHistory` is created by EF Core. It records which migrations have already been applied to the database.

Files To Check

```
src/JobTrackr.Api/appsettings.json
src/JobTrackr.Infrastructure/Persistence/Migrations
```

The connection string in `appsettings.json` should point to the SQL Server instance:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

If SQL Server uses a named instance, use that server name instead:

```
localhost\SQLEXPRESS
```

Migration Command

Run the database update command from the repository root:

```
dotnet ef database update --project src/JobTrackr.Infrastructure --startup-project src/
JobTrackr.Api
```

Meaning:

```
dotnet ef database update
```

Applies pending migrations to the database.

```
--project src/JobTrackr.Infrastructure
```

Tells EF Core where the migration files are stored.

```
--startup-project src/JobTrackr.Api
```

Tells EF Core where to load `Program.cs`, dependency injection setup, and `appsettings.json`.

SQL Server Verification

After the command succeeds, open SQL Server Management Studio and connect to the same server from the connection string.

Check:

```
Databases
  JobTrackrDb
    Tables
      dbo.Users
      dbo.Tasks
      dbo.__EFMigrationsHistory
```

Optional SQL check:

```
SELECT name
FROM sys.tables;
```

Expected result:

```
Users
Tasks
__EFMigrationsHistory
```

Migration history check:

```
SELECT *
FROM __EFMigrationsHistory;
```

Expected migration:

```
InitialCreate
```

Important Architecture Point

The API still uses the existing in-memory services.

That is correct for Day 26.

Applying the migration creates the database structure, but it does not automatically move API reads and writes to SQL Server.

The service logic must be changed separately in a later lesson.

What Not To Do Today

Do not rewrite `TaskService`.

Do not rewrite `UserService`.

Do not move API data storage to SQL Server yet.

Do not add extra tables manually in SQL Server.

Day 26 only applies the migration and verifies the database.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

If no code files changed during this lesson, no new commit is required.

Lesson Explanation

Day 26 is the point where the database stops being theoretical.

Day 25 generated migration files, but those files were only C# instructions. The database still did not exist until EF Core applied the migration.

The `dotnet ef database update` command reads the migrations from Infrastructure, loads application configuration from the API project, connects to SQL Server, and creates the database objects.

After this step, `JobTrackrDb` should exist with the first `Users` and `Tasks` tables. EF Core also creates the `__EFMigrationsHistory` table so it can track which migrations have already been applied.

The important discipline is not to confuse database creation with application persistence. The API can still use in-memory services even after the database exists. Creating the database is one step. Updating the services to use EF Core is a later step.

Reader Exercise

Open SQL Server Management Studio and confirm:

- `JobTrackrDb` exists
- `dbo.Users` exists
- `dbo.Tasks` exists
- `dbo.__EFMigrationsHistory` exists
- `InitialCreate` appears in the migration history table

Then answer:

- Why does EF Core need the startup project?
- Why is `__EFMigrationsHistory` important?
- Why does the API still use memory after the database is created?

Future Improvement

The next persistence steps should include:

- choosing which service to convert first
- replacing one in-memory service with EF Core queries
- saving new records to SQL Server
- reading records from SQL Server
- testing API behavior after database-backed persistence is introduced

Next Lesson

Day 26 creates the real SQL Server database.

The next lesson can begin connecting application behavior to that database.

Day 27 - Move User Storage To SQL Server

Lesson Goal

Move user storage from memory to SQL Server using EF Core.

Day 26 created the real database.

Day 27 makes the User API use that database for user records.

Why This Is Day 27

Until now, users were stored in an in-memory list inside `UserService`.

That allowed the API to work during early development, but data disappeared when the API restarted.

Now that SQL Server exists, the user workflow can move to permanent storage.

Before Day 27:

```
UsersController -> IUserService -> in-memory list
```

After Day 27:

```
UsersController -> IUserService -> AppDbContext -> SQL Server
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Users/UserService.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
```

The important change is that the active `UserService` implementation now belongs in Infrastructure because it uses `AppDbContext`.

Architecture Decision

`IUserService`, request DTOs, and response DTOs stay in Application.

The database-backed implementation moves to Infrastructure.

This keeps the dependency direction clear:

```
Application -> service contracts and DTOs
Infrastructure -> database-backed implementation
API -> dependency injection registration
```

Application should not reference Infrastructure.

Infrastructure can reference Application contracts and Domain entities.

Database-Backed UserService

The new `UserService` receives `AppDbContext` through dependency injection:

```
private readonly AppDbContext _dbContext;

public UserService(AppDbContext dbContext)
{
    _dbContext = dbContext;
}
```

This allows the service to read and write users through EF Core.

User Operations

After Day 27, the User API should use SQL Server for:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected behavior:

- creating a user saves it in SQL Server
- listing users reads from SQL Server
- getting one user reads from SQL Server
- updating a user changes the database record
- deleting a user removes the database record

The API routes and response shapes should stay the same.

Program.cs Change

The API dependency injection setup must register the Infrastructure implementation of `IUserService`.

That means `Program.cs` should use the database-backed service, not the old in-memory service.

Task storage should remain unchanged for now.

What Not To Do Today

Do not move task storage to SQL Server.

Do not create repositories yet.

Do not redesign the full architecture.

Do not change controller routes.

Day 27 only moves user storage to the database.

Testing Checklist

Use Swagger or HTTP requests to test:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Then confirm in SQL Server that the `Users` table reflects the changes.

Useful SQL check:

```
SELECT *
FROM Users;
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
30f7f97 Day 27: Move user storage to database
```

Files changed in that commit:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Users/UserService.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
```

Lesson Explanation

Day 27 is the first lesson where the API starts using SQL Server for real application behavior.

Earlier lessons created the infrastructure for persistence: `AppDbContext`, the connection string, the first migration, and the database. But the API still used in-memory services.

In this lesson, the user workflow moves to EF Core. The `IUserService` contract remains in Application, but the implementation that talks to the database belongs in Infrastructure.

This is an important Clean Architecture step. The controller does not need to know whether users are stored in memory or SQL Server. It only depends on `IUserService`. The implementation can change behind that interface.

The result is a more realistic backend: user data can survive API restarts because it is stored in SQL Server.

Reader Exercise

Create a user through Swagger.

Then:

- stop the API
- start the API again
- call `GET /api/users`
- confirm the user still exists

That proves the user is no longer stored only in memory.

Future Improvement

The next backend steps can include:

- moving task storage to SQL Server
- connecting tasks to database users
- adding repository classes if the service grows too large
- improving validation
- adding tests around database-backed behavior

Next Lesson

Day 27 moves users to SQL Server.

The next logical step is to move task storage to SQL Server while preserving the existing task API behavior.

Day 28 - Move Task Storage To SQL Server

Lesson Goal

Move task storage from memory to SQL Server using EF Core.

Day 27 moved users to SQL Server.

Day 28 moves tasks to SQL Server so both main resources are now persistent.

Why This Is Day 28

Until now, tasks were stored in an in-memory list inside `TaskService`.

That made the task API usable, but task data disappeared when the API restarted.

After users moved to SQL Server, tasks need the same persistence treatment.

Before Day 28:

```
TasksController -> ITaskService -> in-memory list
```

After Day 28:

```
TasksController -> ITaskService -> AppDbContext -> SQL Server
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The active `TaskService` implementation now belongs in Infrastructure because it uses `AppDbContext`.

Architecture Decision

`ITaskService`, request DTOs, and response DTOs stay in Application.

The EF Core implementation belongs in Infrastructure.

This keeps the dependency direction clear:

```
Application -> service contracts and DTOs
Infrastructure -> database-backed implementation
API -> dependency injection registration
```

Application should not depend on Infrastructure.

Infrastructure can depend on Application contracts and Domain entities.

Database-Backed TaskService

The new `TaskService` receives `AppDbContext` through dependency injection:

```
private readonly AppDbContext _dbContext;

public TaskService(AppDbContext dbContext)
{
    _dbContext = dbContext;
}
```

The service uses EF Core to read and write `JobTask` records.

Task Operations

After Day 28, the Task API should use SQL Server for:

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Existing behavior should remain the same from the API user's perspective.

The storage layer changes, not the public routes.

Search And Filtering

Task search and completion filtering should continue to work after moving to SQL Server.

Examples:

```
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

The database-backed service builds an EF Core query and applies filters before returning results.

User Validation

When creating a task, the service should validate that `UserId` belongs to an existing user:

```
Create task -> check Users table -> save task
```

This keeps tasks connected to real users.

Program.cs Change

The API dependency injection setup must register the Infrastructure implementation of `ITaskService`.

That means `Program.cs` should use the database-backed task service instead of the old in-memory implementation.

What Not To Do Today

Do not add repositories.

Do not add authentication.

Do not redesign the full architecture.

Do not change controller routes.

Day 28 only moves task storage to SQL Server.

Testing Checklist

Use Swagger or HTTP requests to test:

```
POST /api/users
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

```
DELETE /api/tasks/{id}
GET /api/tasks?search=example
GET /api/tasks?isCompleted=true
```

Then confirm in SQL Server that the `Tasks` table reflects the changes.

Useful SQL check:

```
SELECT *
FROM Tasks;
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
7cdd1ba Day 28: Move task storage to database
```

Files changed in that commit:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 28 completes the first major persistence migration for the core JobTrackr resources.

Day 27 moved users from memory to SQL Server. Day 28 applies the same idea to tasks. The controller still depends on `ITaskService`, but the implementation behind that interface now uses `AppDbContext`.

This is an important architecture lesson. The API route design does not need to change just because the storage mechanism changes. The external behavior can remain stable while the internal implementation becomes more realistic.

The task service now saves, reads, updates, completes, reopens, searches, filters, and deletes tasks through SQL Server. It also checks that a user exists before creating a task for that user.

At this point, both users and tasks are backed by SQL Server. The project is no longer just an in-memory API demo. It has crossed into real backend persistence.

Reader Exercise

Create a user.

Create a task for that user.

Then:

- stop the API
- start the API again
- call `GET /api/tasks`
- confirm the task still exists

- complete the task
- confirm the database value changes

That proves task storage has moved from memory to SQL Server.

Future Improvement

The next backend steps can include:

- improving validation
- adding database constraints
- adding repository classes if services become too large
- adding automated tests around EF Core behavior
- introducing authentication and ownership rules later

Next Lesson

Day 28 moves tasks to SQL Server.

The next lessons can review the database-backed API, clean up persistence behavior, and prepare the project for more realistic backend features.

Day 29 - Test Database Persistence

Lesson Goal

Confirm that user and task data is persisted in SQL Server.

Day 27 moved users to SQL Server.

Day 28 moved tasks to SQL Server.

Day 29 proves that the data survives after the API restarts.

Why This Is Day 29

Earlier in the project, data was stored in memory.

Memory storage disappears when the API stops.

Now users and tasks are stored in SQL Server, so the correct test is:

```
Create data -> restart API -> data should still exist
```

If the same user and task still exist after restarting the API, persistence is working.

Files To Check

No new file is required for Day 29.

Check these files only if something fails:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/appsettings.json
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger using the URL shown in the terminal.

Create A User

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "Day 29 User",
```

```
  "email": "day29@example.com"
}
```

Expected result:

```
200 OK
```

Save the returned user `id`.

Confirm The User From The API

Use:

```
GET /api/users
GET /api/users/{id}
```

The created user should appear in both checks.

Create A Task For The User

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "Day 29 persistence test",
  "description": "Confirm task remains after API restart",
  "userId": 1
}
```

Use the real user id returned when the user was created.

Expected result:

```
201 Created
```

Save the returned task `id`.

Confirm The Task From The API

Use:

```
GET /api/tasks
GET /api/tasks/{id}
```

The created task should appear in both checks.

Confirm Rows In SQL Server

Open SQL Server Management Studio and select:

```
JobTrackrDb
```

Check users:

```
SELECT *
FROM Users;
```

Check tasks:

```
SELECT *
FROM Tasks;
```

The Day 29 user and task should both appear.

Restart Test

Stop the API.

Start it again:

```
dotnet run --project src/JobTrackr.Api
```

Then call:

```
GET /api/users
GET /api/tasks
```

Expected result:

The same user and task should still exist.

That confirms SQL Server persistence is working.

What Not To Do Today

Do not add new features.

Do not change service logic unless a real bug appears.

Do not add repositories.

Do not add authentication.

Day 29 is a testing and verification day.

Lesson Explanation

Day 29 is where the database-backed API is proven.

Moving services to SQL Server is only useful if data actually survives beyond the running API process. Earlier in the project, users and tasks lived in memory. That meant every restart cleared the data.

The test for persistence is simple and important: create a user, create a task, confirm both records exist in SQL Server, restart the API, and confirm the same records are still available.

This lesson does not need new features. Its value is confidence. It verifies that the work from Day 27 and Day 28 changed the application from a temporary in-memory API into a backend that can store data permanently.

Reader Exercise

Create a user and task through Swagger.

Then:

- check the rows in SQL Server
- stop the API
- restart the API
- call `GET /api/users`
- call `GET /api/tasks`

If both records still appear, write down why this proves persistence is working.

Future Improvement

The next backend steps can include:

- cleaning up database-backed service code
- adding stronger validation
- adding automated integration tests
- adding repository classes only if the service code becomes too large
- preparing authentication and ownership rules later

Next Lesson

Day 29 verifies persistence.

The next lesson can improve the database-backed API now that the main user and task data flows are stored in SQL Server.

Day 30 - First Backend Month Review

Lesson Goal

Review the first 30 backend days and confirm the JobTrackr foundation is stable.

Day 30 does not add a new feature.

The goal is review, verification, documentation, and cleanup.

Why This Is Day 30

The first month built the backend foundation.

At this point, JobTrackr has:

```
ASP.NET Core Web API
Application layer DTOs and interfaces
Domain entities
Infrastructure EF Core services
SQL Server persistence
User CRUD
Task CRUD
Task filtering and search
Task complete and reopen endpoints
```

Before starting the next 30-day phase, the foundation should be checked.

Files To Review

```
README.md
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

The first monthly review should not continue if the project does not build.

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger and confirm the API is available.

User Endpoint Review

Test:

```
GET /api/users
POST /api/users
```

```
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected behavior:

- create user works
- get all users works
- get user by id works
- update user works
- delete user works
- missing user returns `404 Not Found`
- invalid user input returns `400 Bad Request`

Task Endpoint Review

Test:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=value
GET /api/tasks?isCompleted=false&search=value
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
DELETE /api/tasks/{id}
```

Expected behavior:

- create task works when a valid `userId` exists
- create task fails when the user does not exist
- get all tasks works
- get task by id works
- update task works
- delete task works
- complete task works
- reopen task works
- search works
- completion filter works
- missing task returns `404 Not Found`
- empty title returns `400 Bad Request`

SQL Server Persistence Review

Open SQL Server Management Studio and select:

```
JobTrackrDb
```

Check users:

```
SELECT *
FROM Users;
```

Check tasks:

```
SELECT *
FROM Tasks;
```

Expected behavior:

- users created from the API appear in `Users`
- tasks created from the API appear in `Tasks`
- data still exists after the API restarts

README Review

The README should clearly mention:

```
ASP.NET Core Web API
.NET 8
SQL Server
Entity Framework Core
Layered structure
User CRUD
Task CRUD
Task search and filtering
Task complete and reopen
Database persistence
```

Keep the README simple and current.

The goal is clarity, not a large document.

What Not To Do Today

Do not add a new feature.

Do not redesign the architecture.

Do not add authentication.

Do not add repositories.

Day 30 is a review checkpoint.

Lesson Explanation

Day 30 is the first monthly checkpoint for the JobTrackr backend.

The first month moved the project from an empty backend idea into a working ASP.NET Core Web API with users, tasks, Clean Architecture style layering, EF Core, SQL Server, and persistent storage.

This kind of review day matters because long projects can drift. A stable foundation should be checked before adding the next layer of complexity.

The review confirms that the project builds, the user API works, the task API works, SQL Server persistence works, and the documentation still matches the current state of the code.

No new feature is required. The value of Day 30 is confidence.

Reader Exercise

Run the API and test the main user and task flows.

Then answer:

- Which endpoints are now complete?
- Which endpoints use SQL Server persistence?
- What behavior should be tested after restarting the API?
- What should the next 30-day phase focus on?

Future Improvement

The next backend phase can focus on:

- stronger validation
- cleaner error handling
- integration tests
- authentication
- authorization
- ownership rules
- better project documentation

Next Lesson

Day 30 closes the first backend month.

The next lesson can begin the second phase with a stable, database-backed API foundation.

MONTH 2

ARCHITECTURE AND AUTHENTICATION

Strengthen boundaries, validation, error handling, and JWT authentication.

Day 31 - Review Database CRUD

Lesson Goal

Start the second backend phase by reviewing the database-backed CRUD system.

Day 31 does not add a new feature.

The goal is to confirm that users, tasks, search, filtering, complete/reopen behavior, and SQL Server persistence are stable before changing the database model again.

Why This Is Day 31

The first 30 days built the foundation.

Days 27 and 28 moved users and tasks from memory to SQL Server.

Day 29 tested basic persistence.

Day 30 reviewed the first month.

Before adding relationship features or changing the model again, the current database-backed system should be verified.

Current state:

```
Controllers -> Application interfaces -> Infrastructure services -> AppDbContext -> SQL Server
```

Files To Review

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Users
src/JobTrackr.Application/Tasks
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger and confirm the API starts correctly.

User CRUD Review

Test:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Create request:

```
{
  "fullName": "Day 31 User",
  "email": "day31@example.com"
}
```

Update request:

```
{
  "fullName": "Day 31 Updated User",
  "email": "day31.updated@example.com"
}
```

Expected behavior:

- create returns `200 OK`
- get all returns users
- get by id returns the user
- update changes full name and email
- delete returns `204 No Content`
- missing user returns `404 Not Found`

Task CRUD Review

Create or use an existing user first.

Then test:

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

Create request:

```
{
  "title": "Day 31 task",
  "description": "Review database-backed task CRUD",
  "userId": 1
}
```

Update request:

```
{
  "title": "Day 31 updated task",
  "description": "Updated task description"
}
```

Expected behavior:

- create returns `201 Created`
- create fails if `userId` does not exist
- get all returns tasks
- get by id returns the task
- update changes title and description

- delete returns `204 No Content`
- missing task returns `404 Not Found`

Complete And Reopen Review

Create a task or use an existing task.

Complete:

```
PATCH /api/tasks/{id}/complete
```

Expected:

```
isCompleted = true
```

Reopen:

```
PATCH /api/tasks/{id}/reopen
```

Expected:

```
isCompleted = false
```

Search And Filter Review

Test:

```
GET /api/tasks
GET /api/tasks?search=Day
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?isCompleted=false&search=Day
```

Expected behavior:

- search returns matching task titles
- completed filter returns completed tasks
- not completed filter returns open tasks
- combined search and filter works

Restart Persistence Test

Create one user and one task.

Stop the API:

```
Ctrl + C
```

Restart:

```
dotnet run --project src/JobTrackr.Api
```

Then call:

```
GET /api/users
GET /api/tasks
```

Expected result:

The same user and task should still exist.

Database Review

Open SQL Server Management Studio and check:

```
SELECT *
FROM Users;

SELECT *
FROM Tasks;

SELECT *
FROM __EFMigrationsHistory;
```

Expected result:

- users created through the API appear in `Users`
- tasks created through the API appear in `Tasks`
- migrations are visible in `__EFMigrationsHistory`

What Not To Do Today

Do not add a new feature.

Do not add relationships or navigation properties.

Do not create a migration unless a real bug requires it.

Fix only obvious issues.

Day 31 is review and stability only.

Lesson Explanation

Day 31 starts the second backend phase by slowing down and checking the current system.

The project now has database-backed user and task services. Before adding new database relationships or larger features, the existing CRUD behavior needs to be trusted.

This review confirms that the API still builds, the user endpoints work, the task endpoints work, search and filtering still work, complete and reopen still work, data survives an API restart, and EF Core migrations are applied.

The main lesson is that a long backend project should not only move forward by adding features. It also needs checkpoints where the existing behavior is verified before new complexity is introduced.

Reader Exercise

Run through the full Day 31 checklist:

- create a user
- update the user
- create a task for that user
- complete and reopen the task
- search for the task
- filter tasks by completion status
- restart the API
- confirm the data still exists

Then write down any behavior that does not match the expected result.

Future Improvement

The next backend phase can focus on:

- relationship cleanup
- better validation
- more useful error responses
- integration testing
- authentication and ownership rules later

Next Lesson

Day 31 confirms the database-backed CRUD foundation.

The next lesson can safely start improving the model or API behavior because the current system has been reviewed.

Day 32 - Add Task User Relationship

Lesson Goal

Make the task-user relationship explicit in EF Core.

The API already stores `UserId` on tasks.

Day 32 teaches EF Core that one user can have many tasks, and each task belongs to one user.

Why This Is Day 32

The database already has a `UserId` column in the `Tasks` table.

That connects tasks to users at a basic data level.

But the code should also express the relationship clearly:

```
User -> many Tasks
Task -> one User
```

This prepares the project for future user-specific task queries.

Files Changed

```
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.
Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

User Entity Change

`User` now has a collection of tasks:

```
public List<JobTask> Tasks { get; set; } = [];
```

This means one user can have many tasks.

JobTask Entity Change

`JobTask` keeps `UserId` as the foreign key:

```
public int UserId { get; set; }
```

It also adds the navigation property:

```
public User? User { get; set; }
```

This tells EF Core that a task belongs to a user.

DbContext Relationship Configuration

`AppDbContext` now configures the relationship:

```
modelBuilder.Entity<JobTask>()
    .HasOne(task => task.User)
    .WithMany(user => user.Tasks)
    .HasForeignKey(task => task.UserId)
```

```
    .OnDelete(DeleteBehavior.Cascade);
```

Meaning:

```
HasOne(task => task.User)
```

Each task has one user.

```
WithMany(user => user.Tasks)
```

Each user can have many tasks.

```
HasForeignKey(task => task.UserId)
```

The `UserId` column connects tasks to users.

```
OnDelete(DeleteBehavior.Cascade)
```

If a user is deleted, that user's tasks are deleted too.

Migration

The related migration is:

```
AddTaskUserRelationship
```

EF Core may add:

```
index on Tasks.UserId
foreign key from Tasks.UserId to Users.Id
```

This makes the relationship explicit in SQL Server, not only in C# code.

Data Check Before Migration

Before applying the foreign key, existing tasks should have valid users.

SQL check:

```
SELECT t.*
FROM Tasks t
LEFT JOIN Users u ON t.UserId = u.Id
WHERE u.Id IS NULL;
```

Expected result:

```
No rows
```

If rows appear, those tasks have invalid `UserId` values and must be fixed before applying the migration.

What Not To Do Today

Do not add a new API endpoint.

Do not add `GET /api/users/{id}/tasks` yet.

Do not change DTOs unless the build requires it.

Do not return navigation properties from API responses.

Day 32 is database model relationship work only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
813823e Day 32: Add task user relationship
```

Files changed in that commit:

```
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.
Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

Lesson Explanation

Day 32 makes the relationship between users and tasks explicit.

The project already stored `UserId` on tasks, but EF Core relationships are clearer when the entity model includes navigation properties and the `DbContext` configures the relationship.

The `User` entity now has a `Tasks` collection. The `JobTask` entity now has a `User` navigation property. `AppDbContext` connects both sides with `HasOne`, `WithMany`, and `HasForeignKey`.

This makes the model more accurate: one user can own many tasks, and each task belongs to one user.

The migration then updates SQL Server so the relationship exists at the database level through an index and foreign key.

Reader Exercise

Open `User.cs`, `JobTask.cs`, and `AppDbContext.cs`.

Then answer:

- Which property shows that a user can have many tasks?
- Which property shows that a task belongs to one user?
- Which column is the foreign key?
- What does cascade delete mean in this relationship?

Future Improvement

The next backend step can use this relationship to add user-specific task queries, such as fetching all tasks for one user.

That should be added through a clear API endpoint rather than exposing navigation properties directly in response models.

Next Lesson

Day 32 defines the relationship.

The next lesson can use that relationship to build a user tasks endpoint or improve task query behavior.

Day 33 - Get Tasks By User Endpoint

Lesson Goal

Add an endpoint to fetch all tasks for a specific user.

Day 32 made the task-user relationship explicit in EF Core.

Day 33 uses that relationship in the API.

Why This Is Day 33

The database now understands this relationship:

```
User -> many Tasks
Task -> one User
```

The next useful API behavior is:

```
Show me all tasks for one user.
```

The chosen route is:

```
GET /api/users/{userId}/tasks
```

This route is clear because tasks are being requested in the context of a specific user.

Files Changed

```
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

No new migration is required for Day 33 because the database relationship was already handled on Day 32.

Service Interface Change

`ITaskService` now includes a method for user-specific task lookup:

```
Task<List<TaskResponse>?> GetByUserId(int userId);
```

The nullable list is intentional:

```
null = user does not exist
empty list = user exists but has no tasks
```

This lets the API return the correct HTTP behavior.

Infrastructure Service Change

The database-backed `TaskService` implements the new method.

It first checks whether the user exists:

```
var userExists = await _dbContext.Users.AnyAsync(user => user.Id == userId);
```

If the user does not exist, the service returns `null`.

If the user exists, the service returns tasks filtered by `UserId`.

Controller Change

`UsersController` now depends on both:

```
IUserService
ITaskService
```

The new endpoint is added inside `UsersController`:

```
GET /api/users/{userId}/tasks
```

Expected behavior:

```
User does not exist -> 404 Not Found
User exists with no tasks -> 200 OK with []
User exists with tasks -> 200 OK with task list
```

Why This Endpoint Belongs In UsersController

The endpoint asks for tasks in the context of a user.

That makes this route easy to understand:

```
/api/users/{userId}/tasks
```

It reads as:

```
Get tasks for this user.
```

The existing general task endpoints remain unchanged.

What Not To Do Today

Do not add filtering by `userId` to `GET /api/tasks`.

Do not add due dates.

Do not add priority.

Do not create a migration.

Day 33 only adds the get-tasks-by-user endpoint.

Testing Checklist

Run the API and open Swagger.

Confirm this endpoint appears:

```
GET /api/users/{userId}/tasks
```

Test user does not exist:

```
GET /api/users/999999/tasks
```

Expected:

```
404 Not Found
```

Test user exists with no tasks:

```
GET /api/users/{userId}/tasks
```

Expected:

```
200 OK
[]
```

Test user exists with tasks:

```
POST /api/tasks
GET /api/users/{userId}/tasks
```

Expected:

```
200 OK
task list for that user
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
43ac224 Day 33: Add get tasks by user endpoint
```

Files changed in that commit:

```
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 33 uses the relationship created on Day 32.

The database already knows that users and tasks are connected. Now the API exposes a practical endpoint for that relationship: getting all tasks for a specific user.

The service method returns `null` when the user does not exist and an empty list when the user exists but has no tasks. This distinction matters because both cases should not produce the same HTTP response.

The controller returns `404 Not Found` for a missing user and `200 OK` for an existing user. If the user has no tasks, the response is still successful, but the list is empty.

This is a clean backend step: no schema change, no new feature category, and no route redesign. The API simply starts using the relationship that already exists.

Reader Exercise

Create two users.

Create tasks for only one of them.

Then call:

```
GET /api/users/{firstUserId}/tasks
GET /api/users/{secondUserId}/tasks
GET /api/users/999999/tasks
```

Confirm:

- the first user returns tasks
- the second user returns an empty list
- the missing user returns `404 Not Found`

Future Improvement

Later lessons can build on this by adding:

- pagination for user tasks
- filtering user tasks by completion status
- searching user tasks by title
- authentication and ownership checks

Next Lesson

Day 33 adds the user-specific task endpoint.

The next lesson can improve query behavior around users and tasks or begin adding more realistic API rules.

Day 34 - Add UserId Filter To Task List

Lesson Goal

Improve `GET /api/tasks` by adding `userId` as a query parameter.

Day 33 added:

```
GET /api/users/{userId}/tasks
```

Day 34 adds:

```
GET /api/tasks?userId=1
```

Why This Is Day 34

The API now supports getting tasks from a user context.

That route is useful:

```
GET /api/users/{userId}/tasks
```

But the main task list endpoint should also support filtering.

This lets clients combine query options:

```
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

That is more flexible for task listing screens.

Files Changed

```
README.md
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

No migration is required because this is a query behavior change, not a database schema change.

Service Interface Change

`ITaskService.GetAll` changed from:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search);
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search, int? userId);
```

The nullable `userId` means:

```
null = do not filter by user
value = return tasks for that user only
```

Infrastructure Service Change

The database-backed `TaskService` now applies the user filter when `userId` is provided:

```
if (userId is not null)
{
    query = query.Where(task => task.UserId == userId);
}
```

The other filters still run:

```
isCompleted
search
```

The result is one EF Core query that can combine multiple conditions.

Controller Change

`TasksController.GetAll` now accepts:

```
isCompleted
search
userId
```

The endpoint remains:

```
GET /api/tasks
```

But it now supports these examples:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=true
GET /api/tasks?search=resume
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

Why Keep Both Routes

Both routes are useful:

```
GET /api/users/{userId}/tasks
```

Use this when the client is already working from a user page.

```
GET /api/tasks?userId=1
```

Use this when the client is working from a task list page and wants filtering.

The two routes can coexist because they serve different API usage styles.

Testing Checklist

Run the API and open Swagger.

Check:

```
GET /api/tasks
```

Swagger should show:

```
isCompleted
search
userId
```

Test all tasks:

```
GET /api/tasks
```

Test tasks by user:

```
GET /api/tasks?userId=1
```

Test completed tasks:

```
GET /api/tasks?isCompleted=true
```

Test open tasks:

```
GET /api/tasks?isCompleted=false
```

Test search:

```
GET /api/tasks?search=resume
```

Test combined filters:

```
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

What Not To Do Today

Do not add pagination.

Do not add sorting.

Do not add due dates.

Do not add priority.

Do not create a migration.

Day 34 only improves task query filtering.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
d898eb6 Day 34: Add task user filter
```

Files changed in that commit:

```
README.md
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 34 improves task querying without changing the database schema.

Day 33 added a user-specific route: `GET /api/users/{userId}/tasks`. That route is useful when the API client is working from a user context.

Day 34 makes the general task list more flexible by adding a `userId` query parameter. Now `GET /api/tasks` can return all tasks, tasks for one user, completed tasks, searched tasks, or a combination of those filters.

This is a practical API design step. Not every improvement requires a new endpoint or migration. Sometimes the better change is to make an existing query endpoint more useful while keeping its behavior predictable.

Reader Exercise

Create two users.

Create tasks for both users.

Then test:

```
GET /api/tasks
GET /api/tasks?userId=1
```

```
GET /api/tasks?userId=2
GET /api/tasks?userId=1&isCompleted=false
```

Confirm each response only returns the expected tasks.

Future Improvement

Later task query improvements can include:

- pagination
- sorting
- due dates
- priority
- user-specific search
- user-specific completion filtering

Next Lesson

Day 34 improves task filtering.

The next lesson can continue strengthening task query behavior or move into the next backend concern now that user-specific task access is available.

Day 35 - Add Optional Due Date To Tasks

Lesson Goal

Add an optional due date field to tasks.

Day 34 improved task query filtering.

Day 35 adds a simple task deadline field:

```
DueDateUtc
```

The field is optional, so tasks can still exist without a due date.

Why This Is Day 35

Tasks often need deadlines.

The first version should stay simple:

```
null = no due date
date value = task has a due date
```

The property name uses `Utc` because backend systems should be clear about time storage.

Files Changed

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

Domain Entity Change

`JobTask` now has:

```
public DateTime? DueDateUtc { get; set; }
```

The nullable `DateTime?` means the field is not required.

Request Model Changes

`CreateTaskRequest` now accepts:

```
public DateTime? DueDateUtc { get; set; }
```

`UpdateTaskRequest` also accepts:

```
public DateTime? DueDateUtc { get; set; }
```

This allows clients to set or remove the due date during create and update operations.

Response Model Change

`TaskResponse` now returns:

```
public DateTime? DueDateUtc { get; set; }
```

This lets API clients see whether a task has a deadline.

TaskService Changes

When creating a task, the service saves the due date:

```
DueDateUtc = request.DueDateUtc,
```

When updating a task, the service updates the due date:

```
task.DueDateUtc = request.DueDateUtc;
```

When returning task data, the service maps the due date into `TaskResponse`.

This must be done in:

```
CreateTask
UpdateTask
MapToResponse
GetAll projection
GetByUserId projection
```

Any response path that returns a task should include `DueDateUtc`.

Migration

The related migration is:

```
AddTaskDueDate
```

Expected schema change:

```
Tasks table -> nullable DueDateUtc column
```

This is a database schema change, so a migration is required.

Testing Checklist

Create a task without a due date:

```
{
  "title": "Task without due date",
  "description": "No deadline yet",
  "userId": 1
}
```

Expected:

```
DueDateUtc = null
```

Create a task with a due date:

```
{
  "title": "Task with due date",
  "description": "Deadline test",
  "dueDateUtc": "2026-06-30T00:00:00Z",
  "userId": 1
}
```

Expected:

```
DueDateUtc has the supplied value
```

Update a task and change the due date.

Then update it again with `DueDateUtc` as `null` to confirm the due date can be removed.

What Not To Do Today

Do not add reminders.

Do not add notifications.

Do not add overdue filtering.

Do not add priority.

Day 35 only adds an optional due date field.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet `NU1900` timeout warnings can happen if package vulnerability metadata cannot be reached.

Those are not code errors.

Git Check

The related commit is:

```
90fe90c Day 35: Add task due date
```

Files changed in that commit:

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 35 adds a realistic task field without making the feature too large.

A task often needs a deadline, but not every task has one. That is why `DueDateUtc` is nullable. The API can create tasks with no due date, create tasks with a due date, update the due date, or remove it later.

The change crosses multiple layers. The Domain entity stores the value. The request models accept it. The response model returns it. The Infrastructure service saves and maps it. The EF Core migration adds the nullable column to SQL Server.

This is a useful example of how one simple API field moves through a layered backend.

Reader Exercise

Create two tasks:

- one without `DueDateUtc`
- one with `DueDateUtc`

Then call:

```
GET /api/tasks
GET /api/tasks/{id}
GET /api/users/{userId}/tasks
```

Confirm that each response returns the expected due date value.

Future Improvement

Later lessons can build on due dates with:

- overdue task filtering
- sorting by due date
- reminders
- notifications
- due date validation rules

Next Lesson

Day 35 adds the due date field.

The next lesson can use that field for filtering, sorting, or more realistic task planning behavior.

Day 36 - Add Simple Task Priority

Lesson Goal

Add simple priority support to tasks.

Day 35 added optional due dates.

Day 36 adds a priority field so tasks can be marked as:

```
Low
Medium
High
```

Why This Is Day 36

Tasks are easier to manage when importance is visible.

For the first version, priority is stored as a string:

```
public string Priority { get; set; } = "Medium";
```

This is beginner-friendly and easy to inspect in Swagger and SQL Server.

Later, the project can improve this with enum mapping or stronger validation.

Files Changed

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

Domain Entity Change

`JobTask` now has:

```
public string Priority { get; set; } = "Medium";
```

Default priority:

```
Medium
```

Request Model Changes

`CreateTaskRequest` now accepts:

```
public string Priority { get; set; } = "Medium";
```

`UpdateTaskRequest` also accepts:

```
public string Priority { get; set; } = "Medium";
```

This allows clients to set task priority during create and update operations.

Response Model Change

`TaskResponse` now returns:

```
public string Priority { get; set; } = string.Empty;
```

This lets API clients display the priority value.

TaskService Changes

When creating a task, the service validates priority:

```
if (string.IsNullOrWhiteSpace(request.Priority))
{
    throw new ArgumentException("Task priority is required.");
}
```

Then it saves:

```
Priority = request.Priority,
```

When updating a task, the service validates and updates:

```
task.Priority = request.Priority;
```

When returning task data, the service maps priority into `TaskResponse`.

This should be included in:

```
GetAll projection
GetByUserId projection
MapToResponse
```

Migration

The related migration is:

```
AddTaskPriority
```

Expected schema change:

```
Tasks table -> Priority column
```

This is a database schema change, so a migration is required.

Testing Checklist

Create a task with default priority:

```
{
  "title": "Default priority task",
  "description": "Priority should be Medium",
  "userId": 1
}
```

Create a high priority task:

```
{
  "title": "High priority task",
  "description": "Important work",
  "dueDateUtc": "2026-06-30T00:00:00Z",
  "priority": "High",
  "userId": 1
}
```

Update a task to low priority:

```
{
  "title": "Updated low priority task",
  "description": "Lower importance",
  "dueDateUtc": null,
```

```
  "priority": "Low"
}
```

Confirm priority appears in:

```
GET /api/tasks
GET /api/tasks/{id}
GET /api/users/{userId}/tasks
```

What Not To Do Today

Do not add priority filtering.

Do not add sorting.

Do not add enum mapping.

Do not add complex validation.

Day 36 only adds a simple priority field.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet `NU1900` timeout warnings can happen if package vulnerability metadata cannot be reached.

Those are not code errors.

Git Check

The related commit is:

```
b1b328d Day 36: Add task priority
```

Files changed in that commit:

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 36 adds task priority in the simplest useful form.

The project already supports task ownership, filtering, due dates, and SQL Server persistence. Priority is another common task field, but this lesson keeps the implementation small by using a string instead of an enum.

The change still moves through every important backend layer. The Domain entity stores priority. The request models accept it. The response model returns it. The Infrastructure service validates, saves, updates, and maps it. The EF Core migration adds the new database column.

This shows how a single field becomes part of a real API feature across a layered backend.

Reader Exercise

Create three tasks:

- one with `Low`
- one with `Medium`
- one with `High`

Then call:

```
GET /api/tasks
GET /api/users/{userId}/tasks
```

Confirm that each response includes the expected priority value.

Future Improvement

Later lessons can improve priority with:

- enum mapping
- stricter validation
- priority filtering
- priority sorting
- better API error messages for invalid values

Next Lesson

Day 36 adds priority.

The next lesson can use due dates and priority for richer task filtering or start improving validation rules.

Day 37 - Review Database Task Changes

Lesson Goal

Review and stabilize the recent database-backed task changes.

Day 37 is not a new feature day.

It is a checkpoint for relationship, due date, priority, filtering, migrations, SQL Server tables, and README accuracy.

Why This Is Day 37

Days 32 to 36 changed the task system several times:

```
Task-user relationship
Get tasks by user endpoint
UserId task filtering
Optional due date
Simple task priority
```

Before adding validation and error handling work, the current database-backed task system should be verified.

Recent migrations include:

```
AddTaskUserRelationship
AddTaskDueDate
AddTaskPriority
```

Files To Review

```
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Api/Controllers/UsersController.cs
README.md
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet `NU1900` warnings can appear if vulnerability metadata times out. Those are network metadata warnings, not code errors.

Migration Review

Run:

```
dotnet ef migrations list --project src/JobTrackr.Infrastructure --startup-project src/
JobTrackr.Api
```

Expected migrations:

```
InitialCreate
AddTaskUserRelationship
AddTaskDueDate
AddTaskPriority
```

All should be applied.

SQL Server Review

Open SQL Server Management Studio and use:

```
JobTrackrDb
```

Check users:

```
SELECT TOP 10 *
FROM Users;
```

Check tasks:

```
SELECT TOP 10 Id, Title, UserId, DueDateUtc, Priority, IsCompleted
FROM Tasks;
```

Expected result:

- `Users` table exists
- `Tasks` table exists
- `Tasks.UserId` exists
- `Tasks.DueDateUtc` exists
- `Tasks.Priority` exists
- task rows show priority values
- due date can be `NULL`

Relationship Review

Check the task-user join:

```
SELECT t.Id, t.Title, t.UserId, u.FullName
FROM Tasks t
INNER JOIN Users u ON t.UserId = u.Id;
```

Expected result:

Tasks should join correctly with users.

If a task appears without a valid user, the relationship or existing data needs review.

API Review

Run the API:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger.

Test creating a task with due date and priority:

```
{
  "title": "Day 37 review task",
  "description": "Testing relationship, due date, and priority",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "High",
  "userId": 1
}
```

Expected:

```
201 Created
```

Response should include:

```
"dueDateUtc": "2026-07-01T00:00:00Z",
"priority": "High"
```

Test creating a task without due date:

```
{
  "title": "Day 37 no due date task",
  "description": "Testing nullable due date",
  "priority": "Medium",
  "userId": 1
}
```

Expected response should include:

```
"dueDateUtc": null,
"priority": "Medium"
```

Update Review

Use:

```
PUT /api/tasks/{id}
```

Request:

```
{
  "title": "Day 37 updated task",
  "description": "Updated due date and priority",
  "dueDateUtc": null,
  "priority": "Low"
}
```

Expected:

```
200 OK
```

The response should show `DueDateUtc` as `null` and `Priority` as `Low`.

Query Review

Test:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=false
GET /api/tasks?search=Day
GET /api/tasks?userId=1&isCompleted=false&search=Day
GET /api/users/1/tasks
```

Expected result:

All filters and user-specific task queries should still work after the due date and priority changes.

What Not To Do Today

Do not add a new feature.

Do not add another migration unless something is broken.

Do not add priority filtering.

Do not add due date filtering.

Do not add authentication.

Day 37 is review and cleanup only.

Lesson Explanation

Day 37 is a stability checkpoint after several database changes.

The project recently added a task-user relationship, user-specific task endpoints, task filtering by user, optional due dates, and task priority. Each of those changes is useful, but together they increase the chance of small bugs or missing mappings.

A review day protects the project from moving forward on top of unstable behavior. The build should pass, migrations should be applied, SQL Server should contain the expected columns, API responses should include due date and priority, and task filters should still work.

The main lesson is that backend progress is not only about adding features. A serious project also needs verification days where the current system is checked before more complexity is added.

Reader Exercise

Run the Day 37 checklist:

- build the project
- check migrations
- inspect `Users` and `Tasks` tables
- create a task with due date and priority
- create a task without due date
- update due date and priority
- test task filters
- test `GET /api/users/{userId}/tasks`

Write down any behavior that does not match the expected result.

Future Improvement

The next backend phase can focus on:

- stronger validation
- better error responses
- due date filtering
- priority filtering
- automated tests
- authentication later

Next Lesson

Day 37 verifies the database-backed task system.

The next lesson can start validation and error handling work with a stable foundation.

Day 38 - Add Data Annotation Validation

Lesson Goal

Start the validation and error handling phase by adding data annotation validation to request DTOs.

Day 38 moves basic request shape validation closer to the DTOs.

Examples:

```
Title is required
Email must be valid
FullName has a max length
UserId must be greater than 0
```

Why This Is Day 38

Earlier validation lived mostly inside services.

That is acceptable for business rules, but request shape validation should be closer to the request models.

ASP.NET Core can automatically validate request DTOs when controllers use:

```
[ApiController]
```

The project already uses `[ApiController]`, so invalid request models can return `400 Bad Request` automatically.

Files Changed

```
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
```

No migration is required because this is API request validation only.

DataAnnotations Import

Each request DTO uses:

```
using System.ComponentModel.DataAnnotations;
```

This enables attributes such as:

```
Required
EmailAddress
MaxLength
Range
```

User Request Validation

`CreateUserRequest` and `UpdateUserRequest` should validate:

```
FullName is required
FullName max length is 100
Email is required
Email must be a valid email address
Email max length is 200
```

Expected attributes:

```
[Required]
[MaxLength(100)]
public string FullName { get; set; } = string.Empty;
```

```
[Required]
[EmailAddress]
[MaxLength(200)]
public string Email { get; set; } = string.Empty;
```

Task Request Validation

`CreateTaskRequest` should validate:

```
Title is required
Title max length is 150
Description max length is 1000
Priority is required
Priority max length is 20
UserId must be greater than 0
```

Expected attributes:

```
[Required]
[MaxLength(150)]
public string Title { get; set; } = string.Empty;

[MaxLength(1000)]
public string Description { get; set; } = string.Empty;

[Required]
[MaxLength(20)]
public string Priority { get; set; } = "Medium";

[Range(1, int.MaxValue)]
public int UserId { get; set; }
```

`UpdateTaskRequest` should validate title, description, and priority in the same way.

It does not need `UserId` because updating a task does not move it to a different user in this lesson.

Service Validation Still Matters

Service validation should stay for business rules.

Examples:

```
User not found
Task not found
UserId must refer to a real user
```

Data annotations validate request shape.

Services validate business and data rules.

Both layers can exist together.

Bad User Request Test

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "",
  "email": "not-an-email"
}
```

Expected:

```
400 Bad Request
```

Validation errors should mention:

```
FullName
Email
```

Bad Task Request Test

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "",
  "description": "Invalid request",
  "priority": "",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

Validation errors should mention:

```
Title
Priority
UserId
```

Valid Task Request Test

Use a valid request to confirm normal behavior still works:

```
{
  "title": "Day 38 valid task",
  "description": "Testing valid request after data annotations",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "Medium",
  "userId": 1
}
```

Expected:

```
201 Created
```

What Not To Do Today

Do not build custom validation middleware.

Do not change the validation response shape.

Do not remove business validation from services unless it is clearly duplicated and safe.

Do not add FluentValidation.

Day 38 uses built-in data annotations only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet `NU1900` warnings can appear if vulnerability metadata times out. Those are network metadata warnings, not code errors.

Git Check

The related commit is:

```
49901e0 Day 38: Add data annotation validation
```

Files changed in that commit:

```
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
```

Lesson Explanation

Day 38 starts the validation phase by adding built-in data annotations to request DTOs.

The project already has working user and task APIs. Now the next step is to make invalid requests fail earlier and more consistently.

Data annotations are a simple ASP.NET Core feature. They let request models describe basic rules such as required fields, email format, max length, and numeric ranges. Because the controllers use `[ApiController]`, ASP.NET Core can automatically return `400 Bad Request` when those rules fail.

This keeps basic request shape validation close to the DTOs. Service methods can still keep business validation such as checking whether a user exists.

Reader Exercise

Send one invalid user request and one invalid task request.

Confirm both return:

```
400 Bad Request
```

Then send valid requests and confirm normal API behavior still works.

Future Improvement

Later validation work can include:

- custom error response shape
- stronger priority validation
- due date validation
- reusable validation helpers
- FluentValidation if the project needs more complex rules

Next Lesson

Day 38 adds basic request validation.

The next lesson can continue improving error handling and make API responses more consistent.

Day 39 - Review ASP.NET Core Validation Responses

Lesson Goal

Review the default ASP.NET Core validation response shape.

Day 38 added data annotation validation to request DTOs.

Day 39 checks what ASP.NET Core returns by default when validation fails.

Why This Is Day 39

The controllers use:

```
[ApiController]
```

Because of that, ASP.NET Core can automatically validate request DTOs and return:

```
400 Bad Request
```

Before building custom validation middleware or custom response classes, the default framework behavior should be understood.

For this project stage, the default response is likely good enough if:

```
it returns 400
it shows which fields failed
it gives readable error messages
it does not expose sensitive data
```

Files To Check

No file change is required for Day 39 unless something is broken.

Files to review if validation behaves incorrectly:

```
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet `NU1900` warnings can appear if package vulnerability metadata times out. Those are network metadata warnings, not code errors.

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger and test invalid requests.

Invalid Create User Test

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "",
  "email": "not-an-email"
}
```

Expected:

```
400 Bad Request
```

The response body should include validation errors for:

```
FullName
Email
```

Invalid Update User Test

Use:

```
PUT /api/users/{id}
```

Request:

```
{
  "fullName": "",
  "email": "wrong-email"
}
```

Expected:

```
400 Bad Request
```

The response should show validation errors for the invalid fields.

Invalid Create Task Test

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "",
  "description": "Invalid task request",
  "priority": "",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

The response should include validation errors for:

```
Title
Priority
UserId
```

Invalid Update Task Test

Use:

```
PUT /api/tasks/{id}
```

Request:

```
{
  "title": "",
  "description": "Invalid update",
  "priority": ""
}
```

Expected:

```
400 Bad Request
```

The response should include validation errors for:

```
Title
Priority
```

Valid Request Test

Use a valid task request to confirm normal behavior still works:

```
{
  "title": "Day 39 valid task",
  "description": "Validation response review",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "Medium",
  "userId": 1
}
```

Expected:

```
201 Created
```

Decision

Keep the default ASP.NET Core validation response for now.

Reason:

```
It is clear enough.
It is built into ASP.NET Core.
It avoids unnecessary custom middleware.
It lets the project move forward without overengineering.
```

What Not To Do Today

Do not build custom middleware.

Do not create a custom validation response class.

Do not add FluentValidation.

Do not change global exception handling.

Day 39 is review and decision only unless something is clearly broken.

Lesson Explanation

Day 39 reviews what ASP.NET Core already provides for validation errors.

After Day 38, request DTOs contain data annotations such as `Required`, `EmailAddress`, `MaxLength`, and `Range`. Since the controllers use `[ApiController]`, ASP.NET Core automatically checks those rules before controller action logic runs.

When validation fails, the framework returns `400 Bad Request` with a response body that identifies the failed fields and their messages.

The important lesson is not to customize too early. The default response is readable enough for this stage of the project. Custom validation responses can be added later when the API has a stronger reason for them.

Reader Exercise

Send invalid requests for:

- create user
- update user
- create task
- update task

Confirm each returns:

```
400 Bad Request
```

Then inspect the response body and identify which fields failed validation.

Future Improvement

Later validation and error handling work can include:

- custom error response shape
- consistent business error responses
- due date validation
- priority validation
- global exception handling
- FluentValidation if rules become complex

Next Lesson

Day 39 confirms the default validation response shape.

The next lesson can improve error handling or continue toward more consistent API responses.

Day 40 - Centralize Common API Messages

Lesson Goal

Centralize obvious repeated API message strings into one shared file.

Day 40 is a small cleanup day.

It does not add global exception handling, custom middleware, or a new error response model.

Why This Is Day 40

The API now has repeated messages in controllers and services.

Examples:

```
User not found.
Task not found.
Task title is required.
UserId is required.
```

Repeated strings are easy to mistype and harder to keep consistent.

The goal is to move common message text into one place without changing API behavior.

Files Changed

```
src/JobTrackr.Application/Common/ErrorMessages.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
```

ErrorMessages Class

The new file is:

```
src/JobTrackr.Application/Common/ErrorMessages.cs
```

It stores common message constants:

```
namespace JobTrackr.Application.Common;

public static class ErrorMessages
{
    public const string UserNotFound = "User not found.";
    public const string TaskNotFound = "Task not found.";
    public const string TaskTitleRequired = "Task title is required.";
    public const string TaskPriorityRequired = "Task priority is required.";
    public const string UserIdRequired = "UserId is required.";
    public const string UserFullNameRequired = "User full name is required.";
    public const string UserEmailRequired = "User email is required.";
}
```

Controller Changes

Controllers now use shared messages instead of inline string literals.

Example:

```
return NotFound(ErrorMessages.TaskNotFound);
```

instead of:

```
return NotFound("Task not found.");
```

The controller behavior should stay the same.

Only the source of the message text changes.

Service Changes

Services now use shared validation messages.

Examples:

```
throw new ArgumentException(ErrorMessages.UserIdRequired);
throw new ArgumentException(ErrorMessages.UserNotFound);
throw new ArgumentException(ErrorMessages.TaskTitleRequired);
throw new ArgumentException(ErrorMessages.TaskPriorityRequired);
```

User service validation can also use:

```
throw new ArgumentException(ErrorMessages.UserFullNameRequired);
throw new ArgumentException(ErrorMessages.UserEmailRequired);
```

What Stays The Same

Day 40 should not change API behavior.

Expected results should remain:

```
missing task -> 404 Not Found -> Task not found.
missing user -> 404 Not Found -> User not found.
invalid task title -> 400 Bad Request -> Task title is required.
invalid user id -> 400 Bad Request -> UserId is required.
```

This is a cleanup, not a feature redesign.

What Not To Do Today

Do not create custom middleware.

Do not add global exception handling.

Do not create a custom error response model.

Do not build a large message abstraction.

Do not redesign the project.

Day 40 only centralizes obvious repeated messages.

Test Examples

Test task not found:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Test user not found:

```
GET /api/users/999999
```

Expected:

```
404 Not Found
User not found.
```

Test create task with missing user:

```
{
  "title": "Prepare resume",
  "description": "Update backend project details",
  "userId": 999999,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
400 Bad Request
User not found.
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
6567f1d Day 40: Centralize common messages
```

Lesson Explanation

Day 40 is a small but useful cleanup lesson.

As APIs grow, repeated strings start appearing in controllers and services. At first this is harmless, but over time it becomes harder to keep messages consistent.

Instead of writing `"User not found."` or `"Task not found."` in many places, the project now has one `ErrorMessages` class in the Application layer. Controllers and services can reference the same constants.

This keeps message text consistent without changing API behavior. It also avoids overengineering. The project does not need custom middleware or global exception handling yet. It only needs a simple shared place for obvious repeated messages.

Reader Exercise

Find every place where the API returns:

```
User not found.
Task not found.
```

Confirm those messages now come from `ErrorMessages`.

Then test the endpoints and confirm the HTTP behavior is unchanged.

Future Improvement

Later error handling work can include:

- consistent error response models
- global exception handling
- problem details customization
- business error codes

- localization if the project ever needs it

Next Lesson

Day 40 centralizes common messages.

The next lesson can continue improving error handling without losing the simple behavior already working in the API.

Day 41 - Add Basic Global Exception Handling

Lesson Goal

Add basic global exception handling middleware to the API.

Day 41 prevents unexpected exceptions from leaking technical details to API users.

Validation errors and normal not-found responses should keep their existing behavior.

Why This Is Day 41

The API already has request validation and common error messages.

The next error handling step is to catch unexpected server errors in one place.

Instead of exposing raw exception details, the API should return:

```
500 Internal Server Error
An unexpected error occurred.
```

This is for unexpected server errors only.

Normal validation errors should still return:

```
400 Bad Request
```

Normal missing records should still return:

```
404 Not Found
```

Files Changed

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
src/JobTrackr.Api/Program.cs
```

Middleware File

The new middleware file is:

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
```

It wraps the request pipeline in a `try/catch`.

If an unexpected exception happens, it sets:

```
StatusCode = 500
ContentType = text/plain
```

Then it returns:

```
An unexpected error occurred.
```

Middleware Code Shape

The middleware follows this structure:

```
public async Task InvokeAsync(HttpContext context)
{
    try
    {
        await _next(context);
```

```
    }
    catch (Exception)
    {
        context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;
        context.Response.ContentType = "text/plain";

        await context.Response.WriteAsync("An unexpected error occurred.");
    }
}
```

Program.cs Registration

`Program.cs` imports the middleware namespace:

```
using JobTrackr.Api.Middleware;
```

Then it registers the middleware before controller mapping:

```
app.UseMiddleware<ExceptionHandlingMiddleware>();

app.MapControllers();
```

This makes every API request pass through the exception handler.

What Should Stay The Same

Task not found should still return:

```
404 Not Found
Task not found.
```

User not found should still return:

```
404 Not Found
User not found.
```

Invalid request DTOs should still return:

```
400 Bad Request
```

ASP.NET Core validation should continue to handle those validation failures.

What Not To Do Today

Do not create a custom error response model.

Do not add a logging service.

Do not change validation responses.

Do not create custom exception classes.

Do not build a large middleware system.

Day 41 only adds basic global exception handling.

Test Examples

Test normal not found:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Test validation:

```
POST /api/tasks
```

Invalid body:

```
{
  "title": "",
  "description": "Test task",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
400 Bad Request
```

Test unexpected exception only with temporary code.

If a controller action temporarily throws an exception, expected response:

```
500 Internal Server Error
An unexpected error occurred.
```

Remove the temporary throw after testing.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
e2fc4b4 Day 41: Add basic exception handling middleware
```

Files changed in that commit:

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
src/JobTrackr.Api/Program.cs
```

Lesson Explanation

Day 41 adds the first global exception handling middleware.

Validation and not-found cases are already handled by the API. But unexpected exceptions need a safe fallback so technical details are not leaked to users.

The middleware wraps the request pipeline in a `try/catch`. If the request completes normally, nothing changes. If an unexpected exception is thrown, the middleware returns a simple `500 Internal Server Error` response.

This is intentionally small. The project does not need a full error response system yet. It only needs a safe default for unexpected server failures.

Reader Exercise

Test three cases:

- task not found

- invalid task request

- temporary unexpected exception

Confirm:

```
not found -> 404
validation -> 400
unexpected exception -> 500
```

Then remove any temporary exception test code.

Future Improvement

Later error handling work can include:

- structured error response models
- request logging
- exception logging
- custom exception types
- problem details responses
- environment-specific error details

Next Lesson

Day 41 adds a safe fallback for unexpected errors.

The next lesson can continue improving API error handling while keeping validation and normal not-found behavior stable.

Day 42 - Review HTTP Status Codes

Lesson Goal

Review the API response status codes and fix user creation so it returns the correct status code.

Day 42 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 42

The API now has users, tasks, validation, shared messages, and basic global exception handling.

Before adding more behavior, the existing endpoints should use consistent HTTP status codes.

The expected rules are:

```
successful GET -> 200 OK
successful POST create -> 201 Created
successful PUT update -> 200 OK
successful PATCH -> 200 OK
successful DELETE -> 204 No Content
missing resource -> 404 Not Found
invalid input -> 400 Bad Request
unexpected server error -> 500 Internal Server Error
```

Endpoints Reviewed

Task endpoints:

```
GET /api/tasks
GET /api/tasks/{id}
POST /api/tasks
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

User endpoints:

```
GET /api/users
GET /api/users/{id}
GET /api/users/{userId}/tasks
POST /api/users
PUT /api/users/{id}
DELETE /api/users/{id}
```

Main Fix

If user creation returns `200 OK`, it should be changed to `201 Created`.

Expected create response:

```
return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
```

This makes user creation consistent with task creation.

GET Rules

Successful list request:

```
return Ok(items);
```

Expected:

```
200 OK
```

Successful single item request:

```
return Ok(response);
```

Expected:

```
200 OK
```

Missing item:

```
return NotFound(ErrorMessages.TaskNotFound);
```

Expected:

```
404 Not Found
```

POST Rules

Successful create should return:

```
201 Created
```

This applies to:

```
POST /api/users
POST /api/tasks
```

The response should include the created resource and a location reference when possible.

PUT Rules

Successful update:

```
return Ok(response);
```

Expected:

```
200 OK
```

Missing item:

```
return NotFound(ErrorMessages.UserNotFound);
```

Expected:

```
404 Not Found
```

Invalid input:

```
return BadRequest(ex.Message);
```

Expected:

```
400 Bad Request
```

PATCH Rules

Successful partial update:

```
return Ok(response);
```

Expected:

```
200 OK
```

Missing task:

```
return NotFound(ErrorMessages.TaskNotFound);
```

Expected:

```
404 Not Found
```

DELETE Rules

Successful delete:

```
return NoContent();
```

Expected:

```
204 No Content
```

Missing item:

```
return NotFound(ErrorMessages.UserNotFound);
```

Expected:

```
404 Not Found
```

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not add new middleware.

Do not create custom response wrappers.

Do not make database changes.

Day 42 only checks and fixes HTTP status code consistency.

Testing Checklist

Create user:

```
POST /api/users
```

Expected:

```
201 Created
```

Create task:

```
POST /api/tasks
```

Expected:

```
201 Created
```

Get missing user:

```
GET /api/users/999999
```

Expected:

```
404 Not Found
User not found.
```

Get missing task:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Delete existing user:

```
DELETE /api/users/{id}
```

Expected:

```
204 No Content
```

Invalid request:

```
POST /api/tasks
```

Expected:

```
400 Bad Request
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
5056756 Day 42: Fix user create status code
```

Lesson Explanation

Day 42 reviews one of the most practical parts of API design: HTTP status codes.

Status codes are part of the API contract. A client should be able to understand what happened from the status code before reading the response body.

For create operations, `201 Created` is more accurate than `200 OK` because the request created a new resource. For deletes, `204 No Content` is appropriate because the operation succeeded and there is no response body. Missing resources should return `404 Not Found`, validation problems should return `400 Bad Request`, and unexpected failures should be handled by the global exception middleware as `500 Internal Server Error`.

The main fix in this lesson is user creation. If it returns `200 OK`, it should be updated to `201 Created` so it matches task creation and follows normal HTTP API behavior.

Reader Exercise

Open the users and tasks controllers.

For each action, write down:

- what happens when the request succeeds
- what happens when the resource is missing
- what happens when the request is invalid

Then confirm the status code matches the expected rule.

Future Improvement

Later API response improvements can include:

- consistent problem details responses
- response documentation
- integration tests for status codes
- better error response models
- automated API contract checks

Next Lesson

Day 42 makes response status codes more consistent.

The next lesson can continue improving API reliability through tests or cleaner error response behavior.

Day 43 - Review Response DTO Consistency

Lesson Goal

Review API response DTO consistency across the users and tasks endpoints.

Day 43 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 43

The API should return response DTOs, not database entities directly.

Expected response models:

```
TaskResponse
UserResponse
```

Types that should not be returned directly from the public API:

```
JobTask
User
EF Core entity objects
```

DTOs protect the API contract. The internal database model can change later without forcing API clients to change.

Files Reviewed

Main areas to review:

```
TasksController
UsersController
TaskService
UserService
ITaskService
IUserService
TaskResponse
UserResponse
```

The goal is to confirm that controllers return service responses and services return DTOs.

Controller Pattern

Controllers should receive response models from services:

```
var response = await _taskService.GetById(id);

if (response is null)
{
    return NotFound(ErrorMessages.TaskNotFound);
}

return Ok(response);
```

The controller should not query EF Core directly and return database entities.

Service Pattern

Services can use EF Core entities internally.

But public service methods should return DTOs:

```
TaskResponse
TaskResponse?
List<TaskResponse>
List<TaskResponse>?
UserResponse
UserResponse?
List<UserResponse>
```

Example mapping:

```
return new TaskResponse
{
    Id = task.Id,
    Title = task.Title,
    Description = task.Description,
    CreatedAtUtc = task.CreatedAtUtc,
    DueDateUtc = task.DueDateUtc,
    Priority = task.Priority,
    IsCompleted = task.IsCompleted,
    UserId = task.UserId
};
```

User Response Mapping

User responses should expose only the fields intended for API clients:

```
return new UserResponse
{
    Id = user.Id,
    FullName = user.FullName,
    Email = user.Email,
    CreatedAtUtc = user.CreatedAtUtc
};
```

This avoids returning the full `User` entity directly.

Why DTOs Matter

DTOs keep the public API separate from the database model.

This gives the project flexibility:

```
database entity can change
API response can stay stable
internal fields can stay hidden
EF Core navigation properties do not leak accidentally
```

This is especially important now that the project has relationships between users and tasks.

What Not To Do Today

Do not add new endpoints.

Do not add database columns.

Do not create migrations.

Do not add authentication.

Do not add a response wrapper model.

Do not add AutoMapper.

Do not create a large abstraction.

Day 43 only checks response DTO consistency.

Testing Checklist

Get all tasks:

```
GET /api/tasks
```

Expected:

```
200 OK
```

Response should show task response fields only.

Get all users:

```
GET /api/users
```

Expected:

```
200 OK
```

Response should show user response fields only.

Get tasks for a user:

```
GET /api/users/{userId}/tasks
```

Expected when the user exists:

```
200 OK
```

Response should return task DTOs, not full user entities.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 43 reviews the boundary between the internal model and the public API contract.

The project uses EF Core entities such as `User` and `JobTask` to represent database records. Those entities are useful inside the application, but they should not be exposed directly from API endpoints.

Instead, the API should return response DTOs such as `UserResponse` and `TaskResponse`. These models define exactly what the API client is allowed to see.

This keeps the API stable. If the database entity gains a new property, relationship, or internal field later, it does not automatically leak into the public response.

The lesson is simple but important: controllers should return response models, services should map entities to DTOs, and EF Core entities should remain internal implementation details.

Reader Exercise

Open the users and tasks API flow.

For each endpoint, check:

- what the controller returns

- what the service method returns
- whether any EF Core entity is returned directly
- whether the response DTO contains only public fields

Then test the API responses in Swagger and confirm the output shape is clean.

Future Improvement

Later response design improvements can include:

- response documentation
- integration tests for response shape
- consistent error response DTOs
- pagination response DTOs
- separate summary and detail response models

Next Lesson

Day 43 confirms response DTO consistency.

The next lesson can continue strengthening the API contract through tests, documentation, or cleaner response behavior.

Day 44 - Weekly Validation Review

Lesson Goal

Review and stabilize validation and response behavior across the API.

Day 44 is a review and testing day.

No new feature is added.

Why This Is Day 44

The project recently added validation, common API messages, global exception handling, HTTP status code cleanup, and response DTO consistency checks.

Before moving into more backend features, the current API behavior should be tested as a complete flow.

The review focuses on:

```
invalid create requests
invalid update requests
missing user records
missing task records
unexpected server errors
response status codes
response messages
```

Areas Reviewed

Request DTOs:

```
CreateTaskRequest
UpdateTaskRequest
CreateUserRequest
UpdateUserRequest
```

Controllers:

```
TasksController
UsersController
```

Shared messages:

```
ErrorMessages
```

Documentation:

```
README
```

Expected Validation Behavior

Invalid request body should return:

```
400 Bad Request
```

Examples:

```
empty task title
missing task title
invalid user id
empty user full name
invalid user email
missing priority
```

This confirms data annotation validation is still working.

Expected Missing Record Behavior

Missing task:

```
404 Not Found
Task not found.
```

Missing user:

```
404 Not Found
User not found.
```

This confirms shared messages and controller responses are consistent.

Expected Success Behavior

Create user:

```
201 Created
```

Create task:

```
201 Created
```

Update user:

```
200 OK
```

Update task:

```
200 OK
```

Delete user:

```
204 No Content
```

Delete task:

```
204 No Content
```

Expected Unexpected Error Behavior

Unexpected server error:

```
500 Internal Server Error
An unexpected error occurred.
```

This confirms the basic global exception handling middleware is still active.

Test: Invalid Create User

Endpoint:

```
POST /api/users
```

Example body:

```
{
  "fullName": "",
  "email": "not-valid-email"
}
```

Expected:

```
400 Bad Request
```

Test: Missing User

Endpoint:

```
GET /api/users/999999
```

Expected:

```
404 Not Found
User not found.
```

Test: Invalid Create Task

Endpoint:

```
POST /api/tasks
```

Example body:

```
{
  "title": "",
  "description": "Test validation",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
400 Bad Request
```

Test: Missing Task

Endpoint:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Test: Create User

Endpoint:

```
POST /api/users
```

Example body:

```
{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Expected:

```
201 Created
```

Test: Create Task

Endpoint:

```
POST /api/tasks
```

Example body:

```
{
  "title": "Review validation",
  "description": "Weekly validation review",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
201 Created
```

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add database changes.

Do not create migrations.

Do not add authentication.

Do not add a custom response wrapper.

Do not redesign architecture.

Day 44 only tests and stabilizes current validation behavior.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 44 is a weekly validation review.

The API now has several layers of response behavior: data annotations for invalid requests, shared messages for repeated errors, consistent status codes, global exception handling for unexpected failures, and DTO-based responses.

A review day confirms those pieces work together.

The value of this lesson is stability. Instead of adding a new feature, the API is tested against expected behavior: invalid requests return `400`, missing records return `404`, successful creates return `201`, successful updates return `200`, successful deletes return `204`, and unexpected errors return a safe `500`.

This keeps the backend reliable before more features are added.

Reader Exercise

Run the validation review checklist:

- send an invalid create user request
- request a missing user
- send an invalid create task request
- request a missing task
- create a valid user
- create a valid task
- update an existing record

- delete an existing record

Confirm every response uses the expected status code and message.

Future Improvement

Later improvements can include:

- automated integration tests
- documented response examples
- custom problem details responses
- stricter priority validation
- due date validation
- consistent business error response models

Next Lesson

Day 44 confirms validation and response behavior.

The next lesson can move forward with stronger confidence that the API contract is stable.

Day 45 - Review Application Layer Boundaries

Lesson Goal

Review the application layer boundaries and confirm the project structure still makes sense.

Day 45 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 45

The project now has controllers, service interfaces, request DTOs, response DTOs, domain entities, EF Core services, validation, common messages, and basic exception handling.

Before adding more backend complexity, the current boundaries should be reviewed.

The main question is:

```
Is each part of the project doing the right job?
```

Current Project Layers

The project is organized around these layers:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Expected responsibilities:

```
Api             -> controllers, HTTP requests, HTTP responses
Application     -> service interfaces, request DTOs, response DTOs
Domain          -> core entities
Infrastructure  -> EF Core database work and service implementations
```

Controller Review

Controllers should stay thin.

They should:

```
receive HTTP requests
call service interfaces
return HTTP responses
handle simple status code decisions
```

Controllers should not:

```
use AppDbContext directly
create or update entities directly
contain main business workflow logic
return EF Core entities directly
```

Acceptable controller pattern:

```
var response = await _taskService.GetById(id);

if (response is null)
{
    return NotFound(ErrorMessages.TaskNotFound);
}
```

```
return Ok(response);
```

This is acceptable because the controller delegates the use case to the service and only decides the HTTP response.

Service Interface Review

`ITaskService` should describe task use cases clearly:

```
get tasks
get task by id
create task
update task
delete task
complete task
reopen task
get tasks by user id
```

`IUserService` should describe user use cases clearly:

```
get users
get user by id
create user
update user
delete user
```

The API should depend on these interfaces instead of directly depending on database code.

DTO Review

Request DTOs should be used for API input:

```
CreateUserRequest
UpdateUserRequest
CreateTaskRequest
UpdateTaskRequest
```

Response DTOs should be used for API output:

```
UserResponse
TaskResponse
```

Controllers should not return domain entities like:

```
User
JobTask
```

DTOs keep the public API contract separate from the internal database model.

Infrastructure Review

The current service implementations live in Infrastructure because they use EF Core.

That is acceptable at this stage.

Example:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

This is acceptable inside `TaskService` because the service implementation is part of Infrastructure.

It would not be acceptable inside a controller.

What Is Acceptable Right Now

It is acceptable that:

```
controllers call services
services contain the main workflow logic
```

```
Infrastructure services use AppDbContext
Application defines interfaces and DTOs
Domain contains entities
controllers return DTO responses
```

The project does not need a larger architecture yet.

What Is Not Acceptable

Do not put this inside a controller:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Reason:

```
Controllers should not talk directly to the database.
```

Do not return this from a controller:

```
return Ok(taskEntity);
```

Reason:

```
Controllers should return DTOs through service responses.
```

What Not To Add Today

Do not add repositories.

Do not add MediatR.

Do not add CQRS.

Do not add AutoMapper.

Do not redesign the architecture.

Do not add database changes.

Do not add new endpoints.

Day 45 is only about reviewing boundaries and fixing obvious misplaced logic.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 45 reviews the boundaries of the backend architecture.

Clean Architecture is not about adding as many patterns as possible. It is about keeping responsibilities clear. Controllers should handle HTTP behavior. Application should describe use cases and DTOs. Domain should hold core entities. Infrastructure should handle database-specific work.

At this stage, the project does not need repositories, CQRS, MediatR, or AutoMapper. Adding those too early would increase complexity without solving a real problem.

The useful review is simpler: controllers should stay thin, services should contain workflow logic, database access should stay out of controllers, and API responses should use DTOs instead of EF Core entities.

This confirms that the project can continue growing without turning controllers into database scripts or exposing internal models directly to API clients.

Reader Exercise

Open the users and tasks flow.

For each endpoint, check:

- does the controller call a service?
- does the controller avoid direct database access?
- does the service return a response DTO?
- does Infrastructure handle EF Core work?
- are domain entities kept out of public API responses?

If the answer is yes, the current boundaries are healthy.

Future Improvement

Later architecture improvements can include:

- repositories if service code becomes too large
- separate application use-case classes
- query objects for complex filtering
- AutoMapper if manual mapping becomes repetitive
- integration tests for boundary behavior

Those should be added when the project needs them, not before.

Next Lesson

Day 45 confirms the application layer boundaries.

The next lesson can continue strengthening the architecture while keeping the current simple structure intact.

Day 46 - Review Infrastructure Layer

Lesson Goal

Review the Infrastructure layer and confirm database logic is in the right place.

Day 46 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 46

Day 45 reviewed the Application layer boundaries.

Day 46 reviews the Infrastructure layer because the project now uses EF Core, SQL Server, migrations, and database-backed services.

The main question is:

```
Is database-specific code staying in Infrastructure?
```

Infrastructure Responsibilities

Infrastructure is responsible for database and external system details.

In this project, that currently means:

```
AppDbContext
EF Core setup
SQL Server persistence
migrations
database-backed TaskService
database-backed UserService
```

Infrastructure is allowed to use EF Core.

Controllers and Domain should not directly depend on EF Core.

Files Reviewed

Main areas to review:

```
AppDbContext
Program.cs
appsettings.json
TaskService
UserService
EF Core migrations
project references
```

AppDbContext Review

`AppDbContext` should live in Infrastructure.

It should contain:

```
DbSet<User>
DbSet<JobTask>
task-user relationship configuration
```

Expected idea:

```
public DbSet<User> Users { get; set; }
public DbSet<JobTask> Tasks { get; set; }
```

The relationship between users and tasks should also be configured in the context.

Program.cs Review

The API project starts the application and wires dependencies together.

It is acceptable for `Program.cs` to register `AppDbContext`:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

It should also register services with scoped lifetime:

```
builder.Services.AddScoped<ITaskService, TaskService>();
builder.Services.AddScoped<IUserService, UserService>();
```

This keeps EF Core services aligned with request-based database context usage.

Connection String Review

The SQL Server connection string belongs in application configuration.

Do not paste passwords or secrets into notes.

For this project stage, the important check is that the API can read the connection string and create `AppDbContext`.

TaskService Review

Task database operations should live in the Infrastructure task service.

Examples:

```
add task
find task by id
remove task
save changes
query task filters
```

This is acceptable inside Infrastructure because Infrastructure owns database access.

It is not acceptable inside controllers.

UserService Review

User database operations should live in the Infrastructure user service.

Examples:

```
add user
find user by id
remove user
save changes
query users
```

Controllers should call `IUserService`.

They should not use `AppDbContext` directly.

Migrations Review

EF Core migrations should stay in Infrastructure.

Migration files represent database structure changes, so they belong beside persistence code.

Do not edit migration files manually during this review.

Project Reference Review

Expected dependency direction:

```
Api -> Application
Api -> Infrastructure
Infrastructure -> Application
Infrastructure -> Domain
Application -> Domain if needed
```

Controllers should not need direct EF Core package usage.

Domain should not reference EF Core.

What Is Acceptable Right Now

This is acceptable inside an Infrastructure service:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Reason:

```
Infrastructure owns database access.
```

This is acceptable inside `Program.cs`:

```
builder.Services.AddDbContext<AppDbContext>(...);
```

Reason:

```
The API project starts the app and wires dependencies together.
```

What Is Not Acceptable

Do not put this inside controllers:

```
private readonly AppDbContext _dbContext;
```

Reason:

```
Controllers should not directly access the database.
```

Do not move EF Core code into Domain:

```
using Microsoft.EntityFrameworkCore;
```

Reason:

```
Domain should stay independent from EF Core.
```

What Not To Add Today

Do not add repositories.

Do not add Unit of Work.

Do not add CQRS.

Do not add MediatR.

Do not create new migrations.

Do not add database tables.

Do not add endpoints.

Do not redesign the architecture.

Day 46 only reviews Infrastructure boundaries and fixes obvious misplaced database logic.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 46 reviews the database side of the project architecture.

The API now uses SQL Server through EF Core. That means the project needs a clear place for database-specific logic. In this structure, that place is Infrastructure.

`AppDbContext`, migrations, and EF Core-backed service implementations belong in Infrastructure. Controllers should not query the database directly. Domain entities should not know about EF Core. Application should describe use cases and DTOs without owning database details.

This review keeps the project from drifting into a common backend problem: controllers becoming database scripts. By keeping database access in Infrastructure, the API layer stays focused on HTTP behavior and the rest of the project remains easier to reason about.

Reader Exercise

Review the project references and answer:

- Does the API reference Application and Infrastructure?
- Does Infrastructure reference Application and Domain?
- Does Domain avoid EF Core references?
- Do controllers avoid `AppDbContext`?
- Are migrations stored in Infrastructure?

If the answer is yes, the Infrastructure boundary is healthy.

Future Improvement

Later infrastructure improvements can include:

- repository classes if services become too large
- better EF Core configuration files
- seed data
- integration tests
- production-safe connection string handling
- logging database errors

Those should be added when the project needs them.

Next Lesson

Day 46 confirms Infrastructure boundaries.

The next lesson can continue strengthening the project structure without adding unnecessary architecture too early.

Day 47 - Review Service Interfaces

Lesson Goal

Review service interfaces and confirm method names are clean, useful, and consistent.

Day 47 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 47

Day 46 reviewed the Infrastructure layer.

Day 47 reviews the service interfaces in the Application layer.

The main question is:

```
Do the interfaces clearly describe what the application can do?
```

Service interfaces should:

```
describe clear use cases
use consistent method names
return DTOs instead of database entities
avoid EF Core details
be easy to explain in an interview
```

Interfaces Reviewed

Task service interface:

```
ITaskService
```

User service interface:

```
IUserService
```

Related implementations and callers:

```
TaskService
UserService
TasksController
UsersController
```

Task Service Review

The task interface should describe task use cases clearly.

Examples:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search, int? userId);
Task<TaskResponse?> GetById(int id);
Task<TaskResponse> CreateTask(CreateTaskRequest request);
Task<TaskResponse?> UpdateTask(int id, UpdateTaskRequest request);
Task<bool> DeleteTask(int id);
Task<TaskResponse?> CompleteTask(int id);
Task<TaskResponse?> ReopenTask(int id);
Task<List<TaskResponse>?> GetByUserId(int userId);
```

These method names are acceptable because they describe real application actions.

User Service Review

The user interface should describe user use cases clearly.

Examples:

```
Task<UserResponse> CreateUser(CreateUserRequest request);
Task<List<UserResponse>> GetAll();
Task<UserResponse?> GetById(int id);
Task<UserResponse?> UpdateUser(int id, UpdateUserRequest request);
Task<bool> DeleteUser(int id);
```

These names are clear enough for the current project stage.

Naming Rules

Use simple names.

Good examples:

```
GetAll
GetById
CreateTask
UpdateTask
DeleteTask
CompleteTask
ReopenTask
GetByUserId
```

Avoid vague names:

```
Process
Handle
DoTask
Manage
Execute
```

Avoid names that expose database details:

```
InsertIntoDatabase
SaveToSqlServer
QueryDbSet
```

What Is Acceptable Right Now

It is acceptable if task methods include `Task` in the name:

```
CreateTask
UpdateTask
DeleteTask
CompleteTask
ReopenTask
```

Reason:

```
The service is task-focused and the names are beginner-friendly.
```

It is also acceptable if read methods are generic:

```
GetAll
GetById
```

Reason:

```
Inside ITaskService or IUserService, the context is already clear.
```

Possible Cleanup

More explicit names can also work:

```
GetAllTasks
GetTaskById
GetTasksByUserId
GetAllUsers
GetUserById
```

But renaming only for style is not necessary if the current names are already clear.

Consistency matters more than personal preference.

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not add repositories.

Do not make database changes.

Do not create migrations.

Do not do async suffix cleanup today.

Day 47 is only about service interface naming and clarity.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Interview Explanation

A simple explanation:

```
The Application project contains service interfaces because it defines what the application can
do.
The Infrastructure project implements those interfaces because it contains the database-backed
code.
Controllers depend on interfaces, not directly on database classes.
```

Lesson Explanation

Day 47 reviews service interface clarity.

Interfaces are important because they describe the use cases of the application. Controllers call these interfaces, and Infrastructure provides the database-backed implementations.

The service interface should not expose EF Core details. It should not mention SQL Server, DbSet, or database-specific operations. It should describe application behavior such as creating a task, updating a user, completing a task, or getting tasks for a user.

This keeps the API layer separated from the database implementation and makes the code easier to explain.

The lesson is also about restraint. If names are already clear, there is no need to rename everything for style. A good interface is understandable, consistent, and focused on application use cases.

Reader Exercise

Open the task and user service interfaces.

For each method, answer:

- does the name describe a real use case?
- does the return type use a response DTO?
- does the method avoid EF Core details?
- would the method name make sense in an interview explanation?

If the answer is yes, the interface is healthy.

Future Improvement

Later cleanup can include:

- async naming review
- splitting large services if they grow too much
- use-case classes if workflow logic becomes complex
- tests for interface behavior
- repository abstraction only if the project needs it

Next Lesson

Day 47 reviews service interfaces.

The next lesson can focus on async naming cleanup or another small architecture review step.

Day 48 - Add Async Suffix To Service Methods

Lesson Goal

Improve C# async method naming consistency across the service layer.

Day 48 is a naming cleanup day.

No new feature is added.

Why This Is Day 48

Day 47 reviewed the service interfaces.

Day 48 applies a common C# convention:

```
methods that return Task or Task<T> should use the Async suffix
```

Example:

```
GetAll()
```

becomes:

```
GetAllAsync()
```

This makes asynchronous code easier for other .NET developers to recognize.

Files Updated

Main areas updated:

```
ITaskService
IUserService
TaskService
UserService
TasksController
UsersController
```

The service interfaces define the method names.

The Infrastructure implementations must match those names.

The controllers must call the renamed methods.

Task Service Rename

Task service methods are renamed with the `Async` suffix:

```
GetAll -> GetAllAsync
GetById -> GetByIdAsync
CreateTask -> CreateTaskAsync
UpdateTask -> UpdateTaskAsync
DeleteTask -> DeleteTaskAsync
CompleteTask -> CompleteTaskAsync
ReopenTask -> ReopenTaskAsync
GetByUserId -> GetByUserIdAsync
```

Expected shape:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int? userId);
Task<TaskResponse?> GetByIdAsync(int id);
Task<TaskResponse> CreateTaskAsync(CreateTaskRequest request);
```

```
Task<TaskResponse?> UpdateTaskAsync(int id, UpdateTaskRequest request);
Task<bool> DeleteTaskAsync(int id);
Task<TaskResponse?> CompleteTaskAsync(int id);
Task<TaskResponse?> ReopenTaskAsync(int id);
Task<List<TaskResponse>?> GetByUserIdAsync(int userId);
```

User Service Rename

User service methods are also renamed:

```
CreateUser -> CreateUserAsync
GetAll -> GetAllAsync
GetById -> GetByIdAsync
UpdateUser -> UpdateUserAsync
DeleteUser -> DeleteUserAsync
```

Expected shape:

```
Task<UserResponse> CreateUserAsync(CreateUserRequest request);
Task<List<UserResponse>> GetAllAsync();
Task<UserResponse?> GetByIdAsync(int id);
Task<UserResponse?> UpdateUserAsync(int id, UpdateUserRequest request);
Task<bool> DeleteUserAsync(int id);
```

Controller Updates

Controllers should call the renamed service methods.

Example:

```
var response = await _taskService.GetByIdAsync(id);
```

instead of:

```
var response = await _taskService.GetById(id);
```

The same applies for user service calls.

Controller Action Names

Do not rename controller action methods today unless required.

This is acceptable:

```
public async Task<IActionResult> GetById(int id)
{
    var response = await _taskService.GetByIdAsync(id);
}
```

Reason:

```
Controller action names are also used by routing and CreatedAtAction.
```

Day 48 only renames service methods.

Why This Matters

The `Async` suffix tells C# developers that a method is asynchronous.

It improves readability in:

```
interfaces
service implementations
controller calls
code reviews
interview explanations
```

It also follows common .NET naming conventions.

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not make database changes.

Do not create migrations.

Do not add repositories.

Do not redesign architecture.

Day 48 only cleans up async method names.

Search Check

After renaming, search for old service method calls:

```
GetAll(
GetById(
CreateTask(
UpdateTask(
DeleteTask(
CompleteTask(
ReopenTask(
GetByUserId(
CreateUser(
UpdateUser(
DeleteUser(
```

It is acceptable if controller action method names still use some of these names.

The important part is that service interface and implementation methods use `Async`.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
5d25572 Day 48: Add async suffix to service methods
```

Lesson Explanation

Day 48 is a naming cleanup lesson.

The project already uses asynchronous service methods because database access through EF Core is asynchronous. The method signatures return `Task` or `Task<T>`, but the method names did not clearly show that.

Adding the `Async` suffix makes the code more idiomatic for C# developers. It also makes interfaces easier to scan. When a developer sees `GetByIdAsync`, they immediately know the method should be awaited.

This is a small change, but it improves consistency across the Application interfaces, Infrastructure implementations, and API controllers.

The key discipline is to keep the cleanup focused. No endpoints, DTOs, database changes, or architecture changes are needed.

Reader Exercise

Open the service interfaces and check each method.

For every method that returns:

```
Task
Task<T>
```

Confirm the method name ends with:

```
Async
```

Then check controller calls and confirm they call the renamed service methods.

Future Improvement

Later cleanup can include:

- integration tests after renames
- consistent naming for query helpers
- reviewing private helper method names
- separating large services if needed

Next Lesson

Day 48 improves async naming consistency.

The next lesson can continue with small architecture cleanup or move toward testing the service layer.

Day 49 - Remove Dead Code And Starter Files

Lesson Goal

Review the repository and remove safe starter code or dead code.

Day 49 is a cleanup day.

No new feature is added.

Why This Is Day 49

Day 48 cleaned up async service method names.

Day 49 focuses on keeping the repository clean before adding more backend features.

As a project grows, leftover starter files and unused code make the codebase harder to understand.

The goal is to remove only code that is clearly safe to remove.

What To Search For

Review the project for:

```
unused starter files
unused classes
unused methods
unused using statements
old WeatherForecast code
old Class1.cs files
unused template comments
```

Cleanup should be careful.

Do not delete code just because it looks unfamiliar.

Projects Reviewed

API project:

```
check for WeatherForecast files
check for unused controllers
check for template comments
check for unused using statements
```

Application project:

```
check for Class1.cs
check for unused DTO files
check for unused interface files
check for unused using statements
```

Domain project:

```
check for Class1.cs
check for unused entity files
check for unused using statements
```

Infrastructure project:

```
check for Class1.cs
check for unused service files
check for unused persistence files
```

```
check for unused using statements
```

Search Commands

Search for starter files:

```
Get-ChildItem -Recurse -Filter Class1.cs
Get-ChildItem -Recurse -Filter *Weather*
```

Search for old template text:

```
rg "WeatherForecast|Class1|TODO|template|sample|example" src
```

Search for public methods that may need manual review:

```
rg "public .*\\(" src
```

Use judgment before deleting anything.

Safe Cleanup Examples

Safe examples:

```
Class1.cs
WeatherForecast.cs
WeatherForecastController.cs
unused template comments
unused using statements
```

Only remove them if the project builds after removal.

What Not To Remove

Do not remove:

```
DTOs currently used by controllers or services
service interfaces
service implementations
entity classes
migrations
AppDbContext
Program.cs registrations
appsettings.json
```

Do not remove migration files during this cleanup.

Migrations are part of the database history.

What Not To Add Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not make database changes.

Do not create migrations.

Do not add repositories.

Do not redesign architecture.

Day 49 is only about safe cleanup.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

The build is the final safety check after removing dead code.

Lesson Explanation

Day 49 is about keeping the repository clean.

A backend project can become confusing if starter files, unused classes, and old template code remain in the solution. These files distract from the real application and make it harder for another developer to understand what matters.

Cleanup should be conservative. Remove obvious starter code such as old `WeatherForecast` files or empty `Class1.cs` files, but do not remove DTOs, services, entities, migrations, `AppDbContext`, or application startup code.

The important rule is simple: delete only what is clearly safe, then build the project.

Reader Exercise

Search the project for:

```
WeatherForecast
Class1
TODO
sample
template
```

For each result, decide:

- is this still used?
- is it starter/template code?
- will the project build without it?

Remove only the safe items and run the build.

Future Improvement

Later cleanup can include:

- organizing folders
- adding code analyzers
- enabling stricter compiler warnings
- reviewing duplicated mapping code
- adding tests to protect cleanup work

Next Lesson

Day 49 removes safe dead code.

The next lesson can continue improving code quality or begin the next backend feature on a cleaner project.

Day 50 - Architecture Review Day

Lesson Goal

Review the current JobTrackr architecture and confirm the four-project structure still makes sense.

Day 50 is a review day.

No new feature is added.

Why This Is Day 50

Day 49 cleaned up dead code and starter files.

Day 50 reviews the full project architecture before moving into the next stage of backend development.

The main question is:

```
Are the project responsibilities still clear?
```

Projects Reviewed

The solution is split into four projects:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Each project should have a clear responsibility.

JobTrackr.Api

The API project should contain:

```
controllers
Program.cs
middleware
app settings
dependency injection setup
Swagger setup
```

It should not contain:

```
domain entities
EF Core migrations
database query logic inside controllers
```

The API project handles HTTP behavior and application startup.

JobTrackr.Application

The Application project should contain:

```
request DTOs
response DTOs
service interfaces
shared application constants like ErrorMessages
```

It should not contain:

```
AppDbContext
EF Core migrations
SQL Server setup
controller classes
```

The Application project describes what the backend can do without owning database details.

JobTrackr.Domain

The Domain project should contain:

```
domain entities
core business objects
```

Current entities:

```
User
JobTask
```

It should not contain:

```
controllers
DTOs
EF Core DbContext
SQL Server setup
```

Domain should stay independent from infrastructure details.

JobTrackr.Infrastructure

The Infrastructure project should contain:

```
AppDbContext
EF Core migrations
database-backed service implementations
SQL Server persistence code
```

It should not contain:

```
controllers
Swagger setup
API route logic
```

Infrastructure owns database-specific implementation details.

Project Reference Direction

Expected direction:

```
Api -> Application
Api -> Infrastructure
Application -> Domain
Infrastructure -> Application
Infrastructure -> Domain
Domain -> no project references
```

This direction is acceptable for the current stage of the project.

Search Checks

Controllers should not use `AppDbContext` directly.

Expected:

```
no AppDbContext usage inside controllers
```

Domain should not use EF Core.

Expected:

```
no Microsoft.EntityFrameworkCore usage inside Domain
```

Starter code should be gone.

Expected:

```
no WeatherForecast or Class1 starter files
```

README Review

The README should accurately describe:

```
.NET 8 Web API
SQL Server persistence
Entity Framework Core
users API
tasks API
task-user relationship
current project architecture
```

Update README only if it is outdated.

What Not To Add Today

Do not add repositories.

Do not add Unit of Work.

Do not add CQRS.

Do not add MediatR.

Do not add new endpoints.

Do not add new DTOs.

Do not make database changes.

Do not create migrations.

Do not add authentication.

Day 50 is architecture review only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Interview Explanation

A simple explanation:

```
JobTrackr is split into four projects.
Api handles HTTP requests and dependency injection.
Application contains DTOs and service interfaces.
Domain contains core entities.
Infrastructure contains EF Core, SQL Server, migrations, and database-backed service
implementations.
This keeps controllers thin and keeps database logic out of API controllers.
```

Lesson Explanation

Day 50 is an architecture review checkpoint.

The project has grown from a small API into a database-backed backend with users, tasks, validation, middleware, DTOs, EF Core, SQL Server, and multiple project layers.

At this stage, it is important to stop and confirm that the project structure still makes sense. The API project should handle HTTP behavior. Application should hold DTOs and service interfaces. Domain should hold core entities. Infrastructure should hold EF Core, migrations, and database-backed service implementations.

This keeps responsibilities clear. Controllers stay thin, database logic stays out of controllers, Domain stays independent from EF Core, and the public API does not directly expose database entities.

The lesson is not to add more architecture. The lesson is to confirm the current architecture is still understandable and useful.

Reader Exercise

Review each project and answer:

- what is this project responsible for?
- what should not be inside this project?
- does the project reference direction make sense?
- are controllers free from direct database access?
- is Domain free from EF Core?
- is README architecture information accurate?

If the answer is yes, the architecture is healthy for the current stage.

Future Improvement

Later architecture improvements can include:

- repositories if service classes grow too large
- use-case classes if workflows become complex
- integration tests
- clearer module organization
- authentication and authorization structure
- deployment configuration

Those should be added when needed, not just for appearance.

Next Lesson

Day 50 confirms the project architecture.

The next lesson can move into the next backend stage with a cleaner, reviewed foundation.

Day 51 - Plan Authentication Flow

Lesson Goal

Plan the authentication flow before writing code.

Day 51 starts the authentication phase, but it does not implement authentication yet.

Why This Is Day 51

Day 50 reviewed the current project architecture.

Before adding authentication code, the project needs a clear plan for:

```
registration
login
password hashing
JWT tokens
future auth endpoints
what should not be stored in the database
```

Authentication is sensitive enough that planning first is better than rushing into implementation.

First Authentication Scope

The first version should be simple.

Users should be able to:

```
register
login
receive a JWT token
use the token later for protected APIs
```

Do not add these yet:

```
roles
refresh tokens
email verification
password reset
```

Those features can come later.

Planned Auth Endpoints

The first authentication endpoints will be:

```
POST /api/auth/register
POST /api/auth/login
```

Do not create them during the planning day.

Day 51 only defines the direction.

Register Request

Planned registration request:

```
public class RegisterRequest
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;
```

```
}
```

Purpose:

```
FullName identifies the user
Email will be used for login
Password will be hashed before storage
```

Login Request

Planned login request:

```
public class LoginRequest
{
    public string Email { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;
}
```

Purpose:

```
email identifies the user
password proves the user owns the account
```

Auth Response

Planned authentication response:

```
public class AuthResponse
{
    public int UserId { get; set; }

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Token { get; set; } = string.Empty;
}
```

Purpose:

```
client receives basic user details
client receives JWT token for protected requests
```

Password Storage Rule

Never store plain text passwords.

Bad:

```
Password = "mypassword123"
```

Good:

```
PasswordHash = hashed password value
```

The database should store:

```
public string PasswordHash { get; set; } = string.Empty;
```

Do not return `PasswordHash` from API responses.

Password Hashing Plan

Use ASP.NET Core password hashing:

```
PasswordHasher<User>
```

Reasons:

```
built into ASP.NET Core
```

```
safer than custom hashing
beginner-friendly
production-style enough for this stage
```

Do not create a custom hashing algorithm.

JWT Plan

Login should return a JWT token.

The JWT can contain simple claims:

```
user id
email
full name if needed
```

The client will later send the token like this:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

JWT setup will be implemented later.

Do not implement JWT during Day 51.

Future User Entity Change

Later, the `User` entity may need:

```
public string PasswordHash { get; set; } = string.Empty;
```

That change will require:

```
entity update
EF Core migration
database update
```

Do not add it until the implementation day.

Planned File Structure

Possible files for the implementation phase:

```
Application/Auth/RegisterRequest.cs
Application/Auth/LoginRequest.cs
Application/Auth/AuthResponse.cs
Application/Auth/IAuthService.cs
Infrastructure/Auth/AuthService.cs
Api/Controllers/AuthController.cs
```

These files should be created when authentication implementation starts, not during planning.

What Not To Add Today

Do not add an auth controller.

Do not add an auth service.

Do not add auth DTOs.

Do not add JWT packages.

Do not add JWT settings.

Do not add a password hash column.

Do not create a migration.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Day 51 is planning only.

Interview Explanation

A simple explanation:

```
Before implementing authentication, I planned the register and login flow.
The API will store password hashes, not plain text passwords.
Login will return a JWT token, and later protected endpoints will require that token.
```

Lesson Explanation

Day 51 begins the authentication phase by planning before coding.

Authentication affects database design, API contracts, security, and future endpoint protection. Because of that, it should not be added casually.

The first version stays small: users register, users log in, passwords are hashed, and login returns a JWT token. More advanced features like roles, refresh tokens, email verification, and password reset are deliberately left out.

The most important rule is password safety. The API should never store or return plain text passwords. It should store a password hash and keep that field out of response DTOs.

This planning day gives the next implementation steps a clear direction.

Reader Exercise

Before writing authentication code, answer:

- what fields are needed for registration?
- what fields are needed for login?
- where should password hashes be stored?
- why should plain text passwords never be stored?
- what should the login response return?
- which authentication features should be delayed?

If those answers are clear, implementation will be easier.

Future Improvement

Future authentication work can include:

- register endpoint
- login endpoint
- password hashing
- JWT generation
- JWT validation
- protected endpoints
- user ownership rules

- roles later
- refresh tokens later

Next Lesson

Day 51 plans authentication.

The next lesson can start implementing authentication step by step, beginning with DTOs or the password hash field.

Day 52 - Create Auth DTOs

Lesson Goal

Create request and response DTOs for authentication.

Day 52 only creates DTO classes.

Authentication logic is not implemented yet.

Why This Is Day 52

Day 51 planned the authentication flow.

Day 52 starts implementation in a controlled way by creating the models that will define the auth API contract.

The first authentication flow will need:

```
registration input
login input
authentication output
```

These should be represented by DTOs before creating the service or controller.

Files Created

Authentication DTOs are added under the Application layer:

```
Application/Auth/RegisterRequest.cs
Application/Auth/LoginRequest.cs
Application/Auth/AuthResponse.cs
```

These files belong in Application because they define API request and response shapes.

They do not belong in Domain because they are not core entities.

They do not belong in Infrastructure because they are not database implementation details.

RegisterRequest

`RegisterRequest` represents the request body for user registration.

It contains:

```
FullName
Email
Password
```

It also includes validation attributes:

```
FullName is required
FullName max length is 100
Email is required
Email must be valid
Email max length is 150
Password is required
Password minimum length is 6
Password max length is 100
```

This keeps basic request validation close to the DTO.

LoginRequest

`LoginRequest` represents the request body for user login.

It contains:

```
Email
Password
```

It also includes validation attributes:

```
Email is required
Email must be valid
Password is required
```

Login should stay simple at this stage.

The user identifies themselves with email and proves ownership with password.

AuthResponse

`AuthResponse` represents the response after successful authentication.

It contains:

```
UserId
FullName
Email
Token
```

The `Token` field is prepared for JWT usage later.

Day 52 does not generate the token yet.

Password Safety

The request DTO accepts a password because the user must send it during registration and login.

But the API should never return the password.

The response DTO does not include:

```
Password
PasswordHash
```

Later implementation should store a password hash, not the plain text password.

What Not To Add Today

Do not add an auth controller.

Do not add an auth service.

Do not add password hashing logic.

Do not add JWT generation.

Do not add JWT packages.

Do not change the User entity.

Do not create a database migration.

Do not protect endpoints.

Day 52 only creates authentication DTOs.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
85052fb Day 52: Create auth DTOs
```

Interview Explanation

A simple explanation:

```
I created authentication DTOs before implementing authentication logic.
RegisterRequest and LoginRequest define the API input.
AuthResponse defines the API output after successful authentication.
This keeps request and response models separate from database entities.
```

Lesson Explanation

Day 52 begins authentication implementation by creating DTOs only.

This is a clean first step because authentication has multiple moving parts: registration, login, password hashing, JWT generation, database changes, and endpoint protection. Instead of doing everything at once, the project first defines the request and response models.

`RegisterRequest` describes what a client must send to create an account. `LoginRequest` describes what a client must send to sign in. `AuthResponse` describes what the API will eventually return after successful authentication.

This keeps the API contract clear before adding authentication behavior.

Reader Exercise

Review the three DTOs and answer:

- what fields are needed to register?
- what fields are needed to login?
- why does AuthResponse include a token?
- why should AuthResponse not include Password or PasswordHash?
- why do these DTOs belong in Application?

Future Improvement

The next authentication steps can include:

- adding `PasswordHash` to the User entity
- creating an EF Core migration
- adding `IAuthService`
- implementing registration
- implementing login
- adding JWT generation

- adding an auth controller

Next Lesson

Day 52 creates the auth DTOs.

The next lesson can start adding the database support needed for authentication.

Day 53 - Add Password Hashing Service

Lesson Goal

Add a password hashing service so passwords are never stored as plain text.

Day 53 only creates the hashing service.

Register and login are not implemented yet.

Why This Is Day 53

Day 52 created the authentication DTOs.

The next authentication building block is password hashing.

Before the API can register or log in users, it needs a safe way to:

```
hash a new password
verify a login password against a stored hash
```

The project should never store plain text passwords.

Package Added

The Application project uses ASP.NET Core Identity password hashing:

```
Microsoft.Extensions.Identity.Core
```

For this stage, both the password hashing interface and implementation live in the Application project.

This keeps the implementation simple while the authentication flow is still being built step by step.

Files Created

Two authentication service files are added:

```
Application/Auth/IPasswordHasherService.cs
Application/Auth/PasswordHasherService.cs
```

IPasswordHasherService

The interface defines what the password hashing service can do:

```
public interface IPasswordHasherService
{
    string HashPassword(string password);

    bool VerifyPassword(string password, string passwordHash);
}
```

Other code can depend on this interface instead of directly depending on `PasswordHasher`.

PasswordHasherService

The implementation uses ASP.NET Core Identity:

```
PasswordHasher<object>
```

It has two responsibilities:

```
create a password hash
verify a plain text password against an existing hash
```

This avoids writing a custom password hashing algorithm.

Verify Password Order

The verification call must use the correct argument order:

```
VerifyHashedPassword(new object(), passwordHash, password)
```

Meaning:

```
stored hash first
plain password second
```

This order matters.

Dependency Injection

The API registers the password hashing service:

```
builder.Services.AddScoped<IPasswordHasherService, PasswordHasherService>();
```

Later, the auth service can use this dependency during registration and login.

Security Rule

Never store:

```
Password = "mypassword123"
```

Store only:

```
PasswordHash = hashed password value
```

Also do not return password hashes from API responses.

What Not To Add Today

Do not add register endpoint.

Do not add login endpoint.

Do not add auth controller.

Do not add auth service.

Do not add JWT generation.

Do not add `PasswordHash` to the User entity.

Do not create a database migration.

Do not protect endpoints.

Day 53 only sets up password hashing.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
3f6b1fb Day 53: Add password hashing service
```

Interview Explanation

A simple explanation:

```
I added a password hashing service so the application never stores plain text passwords.
The service uses ASP.NET Core Identity's PasswordHasher.
It can hash a new password and verify a login password against a stored password hash.
```

Lesson Explanation

Day 53 adds the password hashing service, which is a core security building block for authentication.

Authentication should never store a user's plain password. During registration, the API should hash the password and store only the hash. During login, the API should verify the submitted password against the stored hash.

Instead of writing a custom hashing algorithm, the project uses ASP.NET Core Identity's built-in `PasswordHasher`. This is safer and more appropriate than inventing password hashing logic manually.

This lesson keeps the scope controlled. It does not add register or login endpoints yet. It only prepares the hashing service that those features will need.

Reader Exercise

Review the password hashing service and answer:

- why should plain text passwords never be stored?
- what method creates a password hash?
- what method verifies a password?
- why should custom hashing algorithms be avoided?
- why should password hashes not be returned from API responses?

Future Improvement

The next authentication steps can include:

- adding `PasswordHash` to the User entity
- creating an EF Core migration
- creating `IAuthService`
- implementing registration
- implementing login
- generating JWT tokens
- protecting endpoints

Next Lesson

Day 53 adds password hashing.

The next lesson can add database support for authentication by storing password hashes safely.

Day 54 - Add Register Use Case

Lesson Goal

Add the first authentication use case: user registration.

Day 54 implements register only.

Login and JWT generation are not implemented yet.

Why This Is Day 54

Day 52 created authentication DTOs.

Day 53 added the password hashing service.

Day 54 uses those pieces to register a new user safely.

Registration should:

```
accept full name, email, and password
check if the email already exists
hash the password
save the user with PasswordHash
return a safe response without password data
```

User Entity Change

The `User` entity now needs a password hash field:

```
public string PasswordHash { get; set; } = string.Empty;
```

This field stores the hashed password value.

It must never store the plain text password.

Auth Service Interface

The authentication service interface starts with one use case:

```
public interface IAuthService
{
    Task<AuthResponse> RegisterAsync(RegisterRequest request);
}
```

More authentication methods can be added later.

For Day 54, only registration is needed.

AuthService

`AuthService` handles registration logic.

It uses:

```
AppDbContext
IPasswordHasherService
```

The registration flow is:

```
check duplicate email
hash password
create user
save user
```

```
return AuthResponse
```

Duplicate emails should be rejected.

Example error:

```
Email is already registered.
```

Password Hashing

The plain password from `RegisterRequest` is passed to the password hashing service:

```
PasswordHash = _passwordHasherService.HashPassword(request.Password)
```

Only the hash is stored.

The plain password is not stored.

Auth Response

After successful registration, the API returns a safe response:

```
return new AuthResponse
{
    UserId = user.Id,
    FullName = user.FullName,
    Email = user.Email,
    Token = string.Empty
};
```

For now, `Token` is empty because JWT generation is not implemented yet.

This is acceptable for Day 54.

AuthController

The new controller exposes:

```
POST /api/auth/register
```

The controller calls:

```
RegisterAsync
```

If registration succeeds, it returns the auth response.

If the email already exists, it returns:

```
400 Bad Request
```

Dependency Injection

The API registers the auth service:

```
builder.Services.AddScoped<IAuthService, AuthService>();
```

This allows `AuthController` to depend on `IAuthService`.

Database Migration

Because the `User` entity gets a new `PasswordHash` property, EF Core needs a migration:

```
AddUserPasswordHash
```

The database must be updated after the migration.

API Test: Register User

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Test User",
  "email": "testuser@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Example response:

```
{
  "userId": 1,
  "fullName": "Test User",
  "email": "testuser@example.com",
  "token": ""
}
```

API Test: Duplicate Email

Register with the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

Security Rule

The API response must not include:

```
plain password
password hash
```

Only safe user details should be returned.

What Not To Add Today

Do not add login endpoint.

Do not add JWT generation.

Do not add protected endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Day 54 is only register.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
5a2f615 Day 54: Add register use case
```

Interview Explanation

A simple explanation:

```
I added a register use case.
The API checks if the email already exists, hashes the password, stores only PasswordHash, and
returns a safe response without password data.
Login and JWT are separate next steps.
```

Lesson Explanation

Day 54 creates the first working authentication use case.

The project already has auth DTOs and a password hashing service. Registration connects those pieces together. The API accepts full name, email, and password. It checks whether the email is already registered, hashes the password, saves the user, and returns a response that does not expose password data.

This is an important security step. The password is used only to create a hash. The database stores `PasswordHash`, not the original password. The API response returns safe user information and keeps the token empty until JWT generation is implemented later.

The scope stays controlled: register is implemented, but login and JWT are left for later.

Reader Exercise

Test registration with:

- a new email
- the same email again
- an invalid email
- a short password

Confirm:

```
new email creates a user
duplicate email returns 400
response does not include password
response does not include password hash
```

Future Improvement

The next authentication steps can include:

- implementing login
- verifying passwords
- generating JWT tokens
- protecting endpoints
- adding ownership rules
- improving auth error responses

Next Lesson

Day 54 adds registration.

The next lesson can implement login and start using the password verification service.

Day 55 - Add Login Use Case

Lesson Goal

Add the login use case for authentication.

Day 55 validates user credentials.

JWT generation is not implemented yet.

Why This Is Day 55

Day 54 added registration.

Day 55 adds the matching login flow.

Login should:

```
accept email and password
find the user by email
verify the password against PasswordHash
reject invalid credentials
return a safe response without password data
```

This completes the first basic authentication loop: register, then login.

Auth Service Interface Change

`IAuthService` now includes login:

```
Task<AuthResponse> LoginAsync(LoginRequest request);
```

The interface now describes two authentication use cases:

```
RegisterAsync
LoginAsync
```

Controllers call this interface instead of directly accessing the database or password hashing logic.

Login Logic

`AuthService` contains the login workflow.

The login flow is:

```
find user by email
return generic error if user does not exist
verify submitted password against PasswordHash
return generic error if password is invalid
return AuthResponse if credentials are valid
```

Password Verification

The password is verified through the password hashing service:

```
_passwordHasherService.VerifyPassword(request.Password, user.PasswordHash)
```

The API does not compare plain text passwords.

It verifies the submitted password against the stored hash.

Generic Error Message

Wrong email and wrong password should return the same message:

```
Invalid email or password.
```

Reason:

```
The API should not reveal whether an email exists.
```

This avoids giving attackers account discovery information.

Auth Response

Successful login returns:

```
UserId
FullName
Email
Token
```

For now:

```
Token = empty string
```

That is acceptable because JWT generation is not implemented yet.

AuthController Change

The controller exposes:

```
POST /api/auth/login
```

If login succeeds:

```
200 OK
```

If credentials are invalid:

```
400 Bad Request
Invalid email or password.
```

API Test: Correct Credentials

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "testuser@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Example response:

```
{
  "userId": 1,
  "fullName": "Test User",
  "email": "testuser@example.com",
  "token": ""
}
```

API Test: Wrong Email

Example body:

```
{
  "email": "wrong@example.com",
  "password": "Password123"
}
```

Expected:

```
400 Bad Request
Invalid email or password.
```

API Test: Wrong Password

Example body:

```
{
  "email": "testuser@example.com",
  "password": "WrongPassword"
}
```

Expected:

```
400 Bad Request
Invalid email or password.
```

Security Rules

Do not return:

```
plain password
password hash
different messages for wrong email vs wrong password
```

Use one generic message:

```
Invalid email or password.
```

What Not To Add Today

Do not add JWT generation.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Do not create a new database migration.

Day 55 is only login credential validation.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
77789ad Day 55: Add login use case
```

Interview Explanation

A simple explanation:

```
I added the login use case.
The API finds a user by email, verifies the submitted password against the stored password hash,
 and returns a safe response.
It uses the same error message for wrong email and wrong password so it does not reveal account
existence.
```

Lesson Explanation

Day 55 adds the login use case.

Registration already stores users with hashed passwords. Login now uses those stored hashes to validate credentials. The API finds the user by email, verifies the submitted password, and returns a safe auth response when the credentials are correct.

The important security detail is the error message. The API should not say "email not found" or "wrong password" separately. Both cases return `Invalid email or password.` so the API does not reveal whether an account exists.

JWT generation is still intentionally delayed. The login response has a token field, but the real token will be added in a later step.

Reader Exercise

Test login with:

- correct email and password
- wrong email
- wrong password

Confirm:

```
correct credentials return 200
wrong email returns the generic error
wrong password returns the same generic error
response does not include password
response does not include password hash
```

Future Improvement

The next authentication steps can include:

- JWT generation
- JWT settings
- token claims
- protected endpoints
- authentication middleware setup
- ownership rules for user-owned data

Next Lesson

Day 55 adds login.

The next lesson can start JWT generation so login returns a real token.

Day 56 - Review Auth Controller Endpoints

Lesson Goal

Review and stabilize the authentication controller endpoints.

Day 56 confirms that register and login are exposed through `AuthController`.

JWT generation is not added yet.

Why This Is Day 56

Day 54 added registration.

Day 55 added login.

Day 56 checks that the API surface is clear and stable:

```
POST /api/auth/register
POST /api/auth/login
```

Both endpoints should call `IAuthService`, return safe responses, and avoid exposing password data.

AuthController Review

`AuthController` should expose:

```
POST /api/auth/register
POST /api/auth/login
```

The controller should depend on:

```
IAuthService
```

It should not directly use database code or password hashing logic.

Those details belong inside the auth service.

Register Endpoint

Register endpoint:

```
POST /api/auth/register
```

Example request:

```
{
  "fullName": "Day 56 User",
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

The response should not include:

```
password
passwordHash
```

Login Endpoint

Login endpoint:

```
POST /api/auth/login
```

Example request:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

For now, this is acceptable:

```
{
  "token": ""
}
```

JWT generation will be added later.

Duplicate Register Test

Register the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

This confirms duplicate email handling still works.

Invalid Login Test

Use the wrong password.

Expected:

```
400 Bad Request
Invalid email or password.
```

Login should use the same generic message for wrong email and wrong password.

Security Rules

Do not return:

```
plain password
password hash
different messages for wrong email and wrong password
```

Login should use:

```
Invalid email or password.
```

This avoids revealing whether an email exists.

Dependency Injection Review

The API should register:

```
builder.Services.AddScoped<IAuthService, AuthService>();
```

This allows `AuthController` to call the auth service through the interface.

The controller stays thin.

The auth service owns the registration and login workflow.

Swagger Review

Swagger should show:

```
POST /api/auth/register
POST /api/auth/login
```

Both endpoints should accept the expected request bodies and return safe auth responses.

What Not To Add Today

Do not add JWT generation.

Do not add authorization attributes.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Do not create a new database migration.

Day 56 is only auth controller endpoint review and stabilization.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
6f3405d Day 56: Update README auth endpoints
```

Lesson Explanation

Day 56 reviews the authentication controller after registration and login have been added.

At this stage, the API should expose two authentication endpoints: register and login. The controller should be simple. It receives the request, calls `IAuthService`, and returns the appropriate HTTP response.

The important part is separation of responsibility. The controller should not hash passwords, query the database directly, or generate auth logic itself. Those responsibilities belong in the auth service and supporting services.

The review also checks security behavior. Register and login responses must not expose passwords or password hashes. Invalid login should use a generic message so the API does not reveal whether an email exists.

JWT is intentionally still missing. The token field can remain empty until the token generation lesson.

Reader Exercise

Open Swagger and test:

- register with a new email
- register with the same email again
- login with correct credentials
- login with the wrong password

Confirm:

```
register endpoint appears
login endpoint appears
password is never returned
passwordHash is never returned
invalid login uses a generic message
```

Future Improvement

Next authentication steps can include:

- JWT generation
- token settings
- token claims
- authentication middleware
- protected endpoints
- ownership rules
- roles later

Next Lesson

Day 56 stabilizes the auth controller endpoints.

The next lesson can start generating real JWT tokens for successful login.

Day 57 - Add JWT Configuration

Lesson Goal

Prepare the API for JWT token generation.

Day 57 only adds JWT configuration.

It does not generate tokens yet.

Why This Is Day 57

Day 56 reviewed the authentication controller endpoints.

The next authentication step is preparing the API to understand JWT bearer tokens.

Before login can return a real token, the API needs:

```
JWT package
JWT settings
JWT authentication registration
authentication middleware
```

Day 57 adds that setup.

Package Added

The API project uses:

```
Microsoft.AspNetCore.Authentication.JwtBearer
```

This package belongs in the API project because authentication middleware is configured in `Program.cs`.

JWT Settings

The API configuration now includes JWT settings:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr",
    "Key": "THIS_IS_A_DEVELOPMENT_SECRET_KEY_CHANGE_LATER"
  }
}
```

These values are used when validating JWT tokens.

For learning, a development key is acceptable.

For production, secrets should not be committed to Git.

Program.cs Configuration

`Program.cs` reads:

```
Jwt:Issuer
Jwt:Audience
Jwt:Key
```

Then it configures JWT bearer authentication.

Important validation settings:

```
ValidateIssuer
```

```
ValidateAudience
ValidateLifetime
ValidateIssuerSigningKey
```

This tells ASP.NET Core how to validate tokens later.

Middleware Order

The authentication middleware should be registered before controller mapping:

```
app.UseHttpsRedirection();
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

This prepares the request pipeline for protected endpoints later.

What Each Change Does

JWT package:

```
adds support for validating JWT bearer tokens
```

JWT settings:

```
store issuer, audience, and signing key configuration
```

Program.cs:

```
registers JWT authentication services and middleware
```

API Behavior Today

Register should still work.

Login should still work.

Login still returns:

```
{
  "token": ""
}
```

That is expected.

Day 58 can generate the real token.

What Not To Add Today

Do not add JWT token generation.

Do not return a real token from login yet.

Do not add `[Authorize]` attributes.

Do not protect endpoints.

Do not add refresh tokens.

Do not add roles.

Do not make database changes.

Do not create migrations.

Day 57 is only JWT configuration.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
3b17ce4 Day 57: Add JWT configuration
```

Interview Explanation

A simple explanation:

```
I added JWT authentication configuration before generating tokens.
The API now knows how to validate JWT tokens using issuer, audience, and signing key settings.
Token generation will be added separately so the setup stays easy to test.
```

Lesson Explanation

Day 57 prepares the API for token-based authentication.

The project already has register and login endpoints, but login still returns an empty token. Before generating real tokens, the API must know how JWT validation will work.

This lesson adds the JWT bearer package, configuration values for issuer, audience, and signing key, and the authentication middleware setup in `Program.cs`.

The scope stays controlled. The API can now be prepared for JWT validation, but it does not generate tokens and does not protect endpoints yet.

This keeps authentication implementation step by step: configure first, generate tokens later, protect endpoints after that.

Reader Exercise

Review the JWT configuration and answer:

- where are issuer and audience configured?
- where is the signing key configured?
- why does the API need JWT bearer authentication?
- why should production secrets not be committed?
- why is login still returning an empty token today?

Future Improvement

The next authentication steps can include:

- creating a JWT token service
- generating tokens during login
- adding claims

- protecting endpoints with `[Authorize]`
- reading the current user id from claims
- adding ownership rules

Next Lesson

Day 57 adds JWT configuration.

The next lesson can generate a real JWT token during login.

Day 58 - Generate JWT Token On Login

Lesson Goal

Return a real JWT token after successful login.

Day 58 generates the token.

Protected endpoints are not added yet.

Why This Is Day 58

Day 57 added JWT configuration.

The API now has issuer, audience, signing key, and JWT bearer authentication setup.

Day 58 uses that setup to create a signed token when login succeeds.

The flow becomes:

```
login request
validate email and password
generate JWT token
return token in AuthResponse
```

Files Created

JWT token generation uses two new files:

```
Application/Auth/IJwtTokenService.cs
Infrastructure/Auth/JwtTokenService.cs
```

The interface belongs in Application because it defines what the application needs.

The implementation belongs in Infrastructure because it uses configuration and token-generation details.

IJwtTokenService

The token service interface exposes:

```
string GenerateToken(User user);
```

It accepts the authenticated user and returns a JWT token string.

JwtTokenService

The implementation reads JWT settings from configuration:

```
Jwt:Issuer
Jwt:Audience
Jwt:Key
```

Then it creates claims, signs the token, and returns it as a string.

Token Claims

The generated token includes simple user claims:

```
user id
email
full name
```

These claims can be used later when protected endpoints need to know who the current user is.

Token Signing

The token is signed with:

```
HmacSha256
```

The signing key comes from JWT configuration.

If the key is missing, the service should fail clearly instead of silently generating an invalid token.

AuthService Update

`AuthService` now depends on:

```
AppDbContext
IPasswordHasherService
IJwtTokenService
```

Login still validates credentials first.

Only after credentials are valid does it generate a token:

```
var token = _jwtTokenService.GenerateToken(user);
```

The login response now returns:

```
Token = token
```

Register Behavior

Register can still return:

```
Token = empty string
```

That is acceptable for Day 58.

Today only login returns a JWT token.

Dependency Injection

The API registers the token service:

```
builder.Services.AddScoped<IJwtTokenService, JwtTokenService>();
```

It should be registered near the other authentication services:

```
IPasswordHasherService
IJwtTokenService
IAuthService
```

API Test: Login

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Response should contain a real token:

```
{
  "userId": 1,
  "fullName": "Day 56 User",
  "email": "day56@example.com",
  "token": "eyJhbGciOi..."
}
```

The token value should no longer be empty.

What Not To Add Today

Do not add `[Authorize]`.

Do not protect endpoints.

Do not add user-specific task filtering by token.

Do not add refresh tokens.

Do not add roles.

Do not add password reset.

Do not create a database migration.

Day 58 is only JWT token generation on login.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
ce3a362 Day 58: Generate JWT token on login
```

Interview Explanation

A simple explanation:

```
I added JWT token generation during login.
After credentials are valid, the API creates a signed token containing the user id, email, and
full name.
The token is returned in AuthResponse and will be used later to access protected endpoints.
```

Lesson Explanation

Day 58 makes login return a real JWT token.

The project already had registration, login validation, and JWT configuration. This lesson adds the token generation service and connects it to the login flow.

The token service reads issuer, audience, and signing key values from configuration. It creates claims for the authenticated user, signs the token, and returns the token string.

Login now does more than validate credentials. After the email and password are confirmed, the API generates a signed JWT and returns it in `AuthResponse`.

Protected endpoints are intentionally delayed. This keeps the authentication work clear: configure JWT, generate JWT, then protect endpoints later.

Reader Exercise

Login with valid credentials.

Confirm:

```
response is 200 OK
token is not empty
token starts with eyJ
invalid credentials still fail
register still works
```

Then explain why token generation should only happen after password verification succeeds.

Future Improvement

Next authentication steps can include:

- adding `[Authorize]`
- protecting task and user endpoints
- reading the current user id from token claims
- adding ownership rules
- adding token expiration handling
- improving JWT secret management

Next Lesson

Day 58 generates JWT tokens on login.

The next lesson can start protecting endpoints with authentication.

Day 59 - Protect Task Endpoints

Lesson Goal

Require authentication for task operations.

Day 59 protects task endpoints with JWT authentication.

User endpoints are not protected yet.

Why This Is Day 59

Day 58 made login return a real JWT token.

Day 59 starts using that token to protect API endpoints.

The first protected area is task operations:

```
GET /api/tasks
GET /api/tasks/{id}
POST /api/tasks
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Auth endpoints must stay public:

```
POST /api/auth/register
POST /api/auth/login
```

Users need those endpoints to get a token.

Main Change

`TasksController` now uses:

```
[Authorize]
```

This requires a valid JWT bearer token before task endpoints can be used.

The controller should also import:

```
using Microsoft.AspNetCore.Authorization;
```

Controller Header

Expected controller setup:

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class TasksController : ControllerBase
{
}
```

This protects every action inside `TasksController`.

What Stays Public

Do not add `[Authorize]` to `AuthController`.

These endpoints must remain public:

```
POST /api/auth/register
POST /api/auth/login
```

If login were protected, users would not be able to get a token.

Test Without Token

Request:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

This confirms the task endpoint is protected.

Test With Token

First login:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Copy the token from the response.

Then call:

```
GET /api/tasks
```

With header:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Expected:

```
200 OK
```

This confirms valid JWT authentication works.

Curl Test Without Token

```
curl -k https://localhost:7024/api/tasks
```

Expected:

```
401 Unauthorized
```

Curl Test With Token

```
curl -k https://localhost:7024/api/tasks -H "Authorization: Bearer YOUR_TOKEN_HERE"
```

Expected:

```
200 OK
```

Swagger Note

If Swagger does not show an authorize button yet, use another client for testing:

```
Postman
Thunder Client
curl
```

Swagger JWT setup can be improved later.

Do not spend Day 59 redesigning Swagger.

What Not To Add Today

Do not add user-specific task ownership from token.

Do not protect user endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add Swagger JWT button setup unless required.

Do not make database changes.

Do not create migrations.

Day 59 is only protecting task endpoints.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
9298b61 Day 59: Protect task endpoints
```

Interview Explanation

A simple explanation:

```
I protected task endpoints with JWT authentication by adding [Authorize] to TasksController.
Unauthenticated requests now return 401 Unauthorized.
Authenticated requests with a valid Bearer token can access task endpoints.
Auth endpoints remain public so users can register and login.
```

Lesson Explanation

Day 59 is the first step where JWT authentication actually controls API access.

The project already has registration, login, JWT configuration, and token generation. Now the token is used to protect task endpoints.

Adding `[Authorize]` to `TasksController` means every task action requires authentication. Without a token, the API returns `401 Unauthorized`. With a valid bearer token, the request can continue.

The auth endpoints remain public because users need to register and log in before they can receive a token.

This lesson does not yet enforce task ownership. It only checks whether the caller is authenticated.

Reader Exercise

Test the protected task API:

- call `GET /api/tasks` without a token
- confirm `401 Unauthorized`
- login and copy the token
- call `GET /api/tasks` with the bearer token
- confirm `200 OK`

Then explain why `AuthController` should not be protected.

Future Improvement

Next authentication steps can include:

- protecting user endpoints
- adding Swagger JWT support
- reading current user id from claims
- filtering tasks by the authenticated user
- enforcing task ownership
- adding role-based authorization later

Next Lesson

Day 59 protects task endpoints.

The next lesson can continue securing the API by improving token usage or adding ownership rules.

Day 60 - Month 2 Review

Lesson Goal

Review the second month of backend work.

Day 60 is a checkpoint day.

No new feature is added unless something is clearly broken.

Why This Is Day 60

The second month moved JobTrackr from a database-backed API into an API with authentication foundations.

By Day 60, the project should have:

```
SQL Server persistence
EF Core migrations
user/task relationship
task filters
due date and priority fields
stronger validation
consistent API responses
cleaner application layer
authentication DTOs
register/login flow
basic JWT authentication
protected task endpoints
```

This review confirms that the current backend foundation is still working before the next phase begins.

Areas Reviewed

The review covers:

```
user endpoints
task endpoints
auth endpoints
protected task endpoints
database persistence
README accuracy
```

README Review

The README should mention:

```
SQL Server
EF Core
users API
tasks API
auth register endpoint
auth login endpoint
JWT token generation
protected task endpoints
```

Update README only if it is outdated.

Auth Review

Auth endpoints should exist:

```
POST /api/auth/register
POST /api/auth/login
```

Registration should:

```
hash passwords
reject duplicate emails
return safe user data
avoid returning password or PasswordHash
```

Login should:

```
verify password
return a JWT token
reject invalid credentials
use a generic invalid credential message
```

JWT Review

The token service should include claims for:

```
user id
email
full name
```

The API pipeline should use:

```
app.UseAuthentication();
app.UseAuthorization();
```

This confirms JWT authentication is wired into the request pipeline.

Protected Task Review

`TasksController` should require authentication.

Without token:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

With a valid bearer token:

```
GET /api/tasks
```

Expected:

```
200 OK
```

Auth Test: Register

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Month Two User",
  "email": "monthtwo@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Response should not include:

```
password
passwordHash
```

Auth Test: Duplicate Register

Use the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

Auth Test: Login

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "monthtwo@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Response should include a token:

```
{
  "token": "eyJ..."
}
```

User Endpoint Review

Check:

```
GET /api/users
GET /api/users/{id}
POST /api/users
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected:

```
successful reads -> 200 OK
create -> 201 Created
delete -> 204 No Content
missing user -> 404 Not Found
```

Task Endpoint Review

With a valid token, check:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
POST /api/tasks
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
DELETE /api/tasks/{id}
```

Expected:

```
successful reads -> 200 OK
create -> 201 Created
update -> 200 OK
delete -> 204 No Content
missing task -> 404 Not Found
```

What Not To Add Today

Do not add new auth features.

Do not add roles.

Do not add refresh tokens.

Do not add ownership rules.

Do not add deployment.

Do not add tests.

Do not create migrations.

Day 60 is review only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 60 is the second monthly checkpoint for JobTrackr.

The first month built the core API foundation. The second month strengthened the backend with SQL Server persistence, relationships, task fields, validation, cleaner responses, architecture cleanup, and authentication basics.

By this point, the API can register users, log them in, return JWT tokens, and protect task endpoints. That is a major backend milestone.

This review day exists to confirm the pieces work together: users, tasks, auth, JWT login, protected task routes, database persistence, and documentation.

The lesson is that long projects need checkpoints. Before starting the next backend phase, verify that the current foundation is stable.

Reader Exercise

Run the Month 2 review checklist:

- register a user
- try duplicate registration
- login and copy the token
- call task endpoints without token
- call task endpoints with token
- test user endpoints
- test task filters
- build the project

- review the README

Write down anything that does not match the expected behavior.

Next Backend Phase

After Day 60, the project is ready for:

```
authorization rules
user-owned tasks
better Swagger JWT support
unit testing
integration testing
deployment basics
```

Next Lesson

Day 60 closes the second month.

The next lesson can start the next backend phase from a reviewed, authenticated API foundation.

MONTH 3

OWNERSHIP, TESTING, AND RELIABILITY

Protect user data, automate verification, add CI, and prepare the project for continued growth.

Day 61 - Review Authentication

Lesson Goal

Review the authentication work completed so far.

Day 61 is a verification day. The goal is not to add a new feature. The goal is to prove that the authentication flow still works from registration to protected API access.

Why This Review Matters

Authentication touches several parts of the backend:

```
API controllers
application services
password hashing
JWT token generation
middleware configuration
protected endpoints
HTTP status codes
```

If one part is wired incorrectly, the API can look finished but still fail in real use. This review checks the full flow instead of only checking individual files.

Current Authentication Features

By this point, JobTrackr has:

```
auth request and response DTOs
password hashing
register use case
login use case
JWT settings
JWT token generation
protected task endpoints
anonymous request rejection
```

This gives the project a basic but realistic authentication foundation.

Files Reviewed

The review focuses on these areas:

```
AuthController
AuthService
JwtTokenService
TasksController
Program
```

`AuthController` should expose:

```
POST /api/auth/register
POST /api/auth/login
```

`AuthService` should:

```
hash passwords during registration
reject duplicate email addresses
validate password during login
return a token after successful login
avoid returning password or password hash data
```

`JwtTokenService` should create a token with useful user claims:

```
user id
email
```

```
full name
```

`TasksController` should require authentication for task endpoints:

```
[Authorize]
```

`Program` should register the authentication middleware in the correct request pipeline:

```
app.UseAuthentication();
app.UseAuthorization();
```

Test 1 - Register A User

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Day 61 User",
  "email": "day61@example.com",
  "password": "Password123"
}
```

Expected result:

```
200 OK
```

The response should not include:

```
password
passwordHash
```

That confirms sensitive password data is not being exposed through the API response.

Test 2 - Reject Duplicate Registration

Register the same email address again.

Expected result:

```
400 Bad Request
Email is already registered.
```

This confirms the API does not allow duplicate users with the same email.

Test 3 - Login With Valid Credentials

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day61@example.com",
  "password": "Password123"
}
```

Expected result:

```
200 OK
```

The response should include a JWT token:

```
{
  "token": "eyJ..."
}
```

This proves the login flow can authenticate a registered user and issue a bearer token.

Test 4 - Reject Invalid Login

Use the correct email with the wrong password.

Expected result:

```
400 Bad Request
Invalid email or password.
```

The message should stay generic. It should not reveal whether the email exists or only the password is wrong.

Test 5 - Block Anonymous Task Access

Request:

```
GET /api/tasks
```

Do not send an authorization header.

Expected result:

```
401 Unauthorized
```

This confirms task endpoints are no longer open to anonymous users.

Test 6 - Allow Task Access With Token

Use the token returned from login.

Header:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Request:

```
GET /api/tasks
```

Expected result:

```
200 OK
```

This confirms the protected endpoint accepts a valid JWT token.

Build Check

The project should still build successfully:

```
dotnet build
```

Expected result:

```
Build succeeded.
```

The build check matters because authentication changes often involve package references, configuration, middleware, and service registrations.

What This Day Teaches

This lesson teaches that authentication is not one file or one endpoint.

A working backend authentication flow needs:

```
safe registration
password hashing
duplicate user protection
credential validation
token generation
middleware setup
```

```
endpoint authorization
manual verification
```

Review days make the project stronger because they catch integration issues before the next feature is added.

Reader Exercise

Repeat the full authentication test flow:

```
register user
try duplicate registration
login user
try invalid login
call tasks without token
call tasks with token
run the build
```

Then write down which layer handled each part of the flow.

Next Step

After confirming authentication works, the next step can move toward user-specific behavior and stronger authorization rules.

Day 62 - Add User Id Claim Usage

Lesson Goal

Read the authenticated user id from JWT claims.

Day 62 adds a small helper inside the tasks API controller. The goal is not to change task ownership yet. The goal is to prepare the controller so future task operations can know which user is making the request.

Why This Step Matters

Authentication is useful only when the API can connect the current request to a real user.

After login, the API returns a JWT token. That token contains claims about the authenticated user. One of the most important claims is the user id.

This day connects two earlier pieces:

```
Day 58 generated a JWT token with the user id claim.
Day 62 reads that user id claim from the authenticated request.
```

This prepares the project for user-owned task behavior.

What Was Added

The tasks controller now has helper logic that reads the current user id from the authenticated user's JWT claims.

The helper uses:

```
User.FindFirstValue(ClaimTypes.NameIdentifier)
```

That reads the user id stored in the JWT token.

The value is parsed into an integer:

```
if (int.TryParse(userIdValue, out var userId))
{
    return userId;
}
```

If the claim is missing or invalid, the helper returns:

```
null
```

This keeps the method defensive and avoids throwing an exception for a malformed or missing claim.

Code Shape

The helper method is simple:

```
private int? GetCurrentUserId()
{
    var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (int.TryParse(userIdValue, out var userId))
    {
        return userId;
    }

    return null;
}
```

The controller also needs:

```
using System.Security.Claims;
```

This import provides access to `ClaimTypes` and the claim helper extension method.

What Did Not Change

Day 62 intentionally does not change task behavior.

No changes were made to:

```
task ownership
create task request shape
task service behavior
database schema
migrations
roles
refresh tokens
new endpoints
```

This keeps the lesson focused. The project now knows how to read the current user id, but it does not yet use that id to filter or assign tasks.

Why Use ClaimTypes.NameIdentifier

The JWT token was created with the user id stored as:

```
ClaimTypes.NameIdentifier
```

Reading the same claim type in the controller keeps both sides consistent.

The token generation side writes the claim. The controller side reads the claim.

That gives the API a clean path from login to authenticated request context.

Verification

The project should still build successfully:

```
dotnet build
```

Expected result:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

This confirms the helper method, namespace import, and controller changes compile correctly.

What This Day Teaches

This lesson teaches an important backend pattern:

```
Do not pass user id from the client when the API can read it from the authenticated token.
```

Once a user is authenticated, the API should trust the token context more than request body fields for identity decisions.

That prevents users from sending another user's id in a request and accidentally or deliberately working with data that is not theirs.

Reader Exercise

Find where the JWT token is created and confirm the user id is added as a `NameIdentifier` claim.

Then find the tasks controller helper and confirm it reads the same claim type.

The important connection is:

```
token creation claim type = token reading claim type
```

Next Step

The next step is to use the current user id when creating or reading tasks, so task data can belong to the authenticated user instead of being controlled by request body values.

Day 63 - Make Tasks User-Owned

Lesson Goal

Make newly created tasks belong to the logged-in user.

Day 63 changes task creation only. The API now reads the user id from the authenticated JWT token instead of trusting a `UserId` value sent in the request body.

Why This Matters

Before authentication, `CreateTaskRequest` accepted `UserId` because the API did not yet know who the current user was.

After adding JWT authentication, the backend can identify the logged-in user from the token. That means the client should not decide which user owns a new task.

This is an important security improvement:

```
The API should use authenticated identity for ownership decisions.
The request body should not be trusted for user identity.
```

Without this change, a user could send another user's id while creating a task.

What Changed

Task creation now follows this flow:

```
user logs in
API returns JWT token
client sends token when creating a task
TasksController reads current user id from token claims
TaskService creates the task with that authenticated user id
```

The task still has a `UserId`, but that value now comes from the authenticated request context.

Request Model Change

`CreateTaskRequest` no longer needs `UserId`.

Old request idea:

```
{
  "title": "Task title",
  "description": "Task description",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

New request idea:

```
{
  "title": "Task title",
  "description": "Task description",
  "dueDateUtc": null,
  "priority": "Medium"
}
```

This makes the API cleaner and safer. The client sends task information, while the server decides ownership.

Service Contract Change

The task service create method now accepts the authenticated user id separately:

```
Task<TaskResponse> CreateTaskAsync(CreateTaskRequest request, int userId);
```

This makes the method explicit.

The request contains task data. The `userId` parameter contains identity data from the authenticated user.

Controller Flow

The create endpoint reads the current user id first:

```
var currentUserId = GetCurrentUserId();
```

If no valid user id exists, the API returns:

```
401 Unauthorized
```

If the user id exists, the controller passes it to the service:

```
var response = await _taskService.CreateTaskAsync(request, currentUserId.Value);
```

This keeps the controller responsible for authenticated request context and keeps the service focused on task creation rules.

Task Service Flow

The task service uses the `userId` parameter:

```
UserId = userId
```

It should not use:

```
UserId = request.UserId
```

The service can also validate that the user exists before saving the task.

Swagger Result

Swagger should no longer show `userId` in the create task request body.

That is a useful sign that the API contract has improved:

```
Clients create tasks.
The authenticated backend assigns ownership.
```

Test Flow

First, log in:

```
POST /api/auth/login
```

Then create a task with the bearer token:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Request:

```
{
  "title": "User owned task",
  "description": "Created from authenticated user",
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected result:

```
201 Created
```

The response should include a `userId` matching the logged-in user.

If the same request is sent without a token, the API should reject it:

```
401 Unauthorized
```

What Did Not Change

Day 63 does not yet restrict every task read, update, or delete operation by user.

This day only changes task creation.

That keeps the lesson focused and makes it easier to verify one behavior at a time.

What This Day Teaches

This lesson teaches a core backend rule:

```
Authenticated user identity should come from the server-side auth context, not from client-
submitted fields.
```

That rule becomes more important as the API grows. User-owned data, authorization checks, and secure multi-user behavior all depend on it.

Reader Exercise

Create a task using a logged-in user's token and confirm the returned `userId`.

Then try to send a fake `userId` in the request body. The API should no longer need that field for task creation.

Next Step

The next step is to apply authenticated user ownership to more task operations, such as reading only the current user's tasks or blocking updates to another user's task.

Day 64 - Restrict Get All Tasks

Lesson Goal

Return only tasks owned by the logged-in user.

Day 64 changes only the `GET /api/tasks` endpoint. The endpoint still supports filtering, but task ownership now comes from the authenticated JWT token instead of a public query parameter.

Why This Matters

Before this change, the task list endpoint could accept a `userId` query parameter.

Example:

```
GET /api/tasks?userId=2
```

That is risky after authentication exists. A logged-in user should not be able to request another user's tasks by changing a query string value.

The backend should decide ownership from the authenticated request:

```
JWT token -> current user id -> only that user's tasks
```

This is a key step toward secure multi-user behavior.

What Changed

The public task list endpoint now:

```
reads the current user id from JWT claims
returns only tasks owned by that user
keeps isCompleted filtering
keeps title search
removes userId as a public query parameter
```

The API contract becomes safer because clients can no longer choose which user's task list to request.

Service Contract Change

The task service method changed from an optional user filter:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int? userId);
```

To a required authenticated user id:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int userId);
```

This makes ownership explicit. Getting all tasks now means getting all tasks for the authenticated user.

Query Behavior

The query starts with the ownership filter:

```
var query = _dbContext.Tasks
    .Where(task => task.UserId == userId);
```

Then the optional filters are applied inside that owned task set:

```
if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}
```

```
if (!string.IsNullOrWhiteSpace(search))
{
    query = query.Where(task => task.Title.Contains(search));
}
```

The order matters conceptually:

```
first restrict by owner
then filter that owner's tasks
```

Controller Flow

The controller no longer accepts `userId` as a query parameter.

Old shape:

```
public async Task<IActionResult> GetAll(
    bool? isCompleted,
    string? search,
    int? userId)
```

New shape:

```
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
```

The controller reads the current user id from the authenticated request:

```
var currentUserId = GetCurrentUserId();
```

If the user id is missing, the API returns:

```
401 Unauthorized
```

If the user id is valid, it is passed into the task service:

```
var tasks = await _taskService.GetAllAsync(
    isCompleted,
    search,
    currentUserId.Value);
```

Swagger Result

Swagger should still show:

```
isCompleted
search
```

Swagger should no longer show:

```
userId
```

That is an important API design improvement. The client can filter tasks, but it cannot choose the owner.

Test Flow

Create tasks for two different users.

Then call the task list endpoint as user one:

```
GET /api/tasks
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

The response should contain only user one's tasks.

Call the same endpoint as user two:

```
GET /api/tasks
```

```
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
200 OK
```

The response should contain only user two's tasks.

Filter Test

Filtering should still work inside the logged-in user's own task list.

Example:

```
GET /api/tasks?isCompleted=false&search=resume
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

The response should include only user one's incomplete tasks that match the search term.

Anonymous Request Test

Request:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

This confirms the endpoint is protected and depends on authenticated identity.

What Did Not Change

Day 64 does not change ownership rules for:

```
get task by id
update task
delete task
complete task
reopen task
```

Those endpoints need their own focused authorization checks.

What This Day Teaches

This lesson teaches a practical authorization rule:

```
Filtering by owner should happen on the server from authenticated identity.
```

Query parameters are useful for search and status filters. They should not be used to decide which user's private data is visible.

Reader Exercise

Create tasks for two users.

Log in as each user and call:

```
GET /api/tasks
```

Confirm each user sees only their own tasks.

Then verify that `isCompleted` and `search` still work inside the authenticated user's own task list.

Next Step

The next step is to continue applying ownership rules to more task endpoints, especially reading a single task by id and blocking access to another user's task.

Day 65 - Restrict Get Task By Id

Lesson Goal

Return a task only when it belongs to the logged-in user.

Day 65 changes only the `GET /api/tasks/{id}` endpoint. The endpoint now checks both the task id and the authenticated user id before returning task data.

Why This Matters

Before this change, the service could search for a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is not enough in a multi-user API.

If the API only checks the task id, a logged-in user could change the id in the URL and potentially read another user's task.

The safer rule is:

```
task id must match
owner user id must also match
```

This prevents users from reading private task data that does not belong to them.

What Changed

The `Get By Id` flow now uses authenticated ownership:

```
controller reads current user id from JWT claims
controller passes task id and user id to the service
service queries by both task id and user id
API returns the task only when both values match
```

If the task is missing or belongs to another user, the API returns the same result:

```
404 Not Found
```

Service Contract Change

The task service method changed from:

```
Task<TaskResponse?> GetByIdAsync(int id);
```

To:

```
Task<TaskResponse?> GetByIdAsync(int id, int userId);
```

The authenticated user id is now required because the service must verify ownership before returning the task.

Query Change

The task lookup should use both task id and user id:

```
var task = await _dbContext.Tasks
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

This replaces id-only lookup logic.

The important difference is:

```
FindAsync(id) checks only the task id.
FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId) checks both task id and
owner.
```

Controller Flow

The controller reads the authenticated user id:

```
var currentUserId = GetCurrentUserId();
```

If no valid user id exists, the API returns:

```
401 Unauthorized
```

If a user id exists, the controller passes it into the service:

```
var response = await _taskService.GetByIdAsync(id, currentUserId.Value);
```

If no matching task is found, the controller returns:

```
404 Not Found
```

If the task exists and belongs to the logged-in user, the controller returns:

```
200 OK
```

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

That is why both cases use the same response:

```
missing task -> 404 Not Found
another user's task -> 404 Not Found
```

This is safer than returning a different response that confirms the task exists but is owned by someone else.

Test Flow

Create two users and get a JWT token for each user.

Create a task while authenticated as user one.

Then test the owner request:

```
GET /api/tasks/1
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

Now test another user requesting the same task:

```
GET /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
404 Not Found
```

Then test a missing task:

```
GET /api/tasks/99999
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

Finally, test without a token:

```
GET /api/tasks/1
```

Expected:

```
401 Unauthorized
```

What Did Not Change

Day 65 does not change ownership rules for:

```
update task
delete task
complete task
reopen task
```

Those endpoints still need their own authorization checks.

What This Day Teaches

This lesson teaches a core multi-user backend rule:

```
Every private resource lookup must include the authenticated owner condition.
```

For user-owned data, checking the resource id alone is not enough.

Reader Exercise

Find every endpoint that reads or changes a task by id.

Ask this question for each endpoint:

```
Does this query check both task id and authenticated user id?
```

If the answer is no, that endpoint still needs an ownership restriction.

Next Step

The next step is to continue applying ownership checks to task mutation endpoints such as update, delete, complete, and reopen.

Day 66 - Restrict Update Task

Lesson Goal

Allow a task to be updated only by the authenticated user who owns it.

Day 66 changes only the `PUT /api/tasks/{id}` endpoint. The update operation now checks both the task id and the authenticated user id before changing task data.

Why This Matters

An update endpoint is more sensitive than a read endpoint because it changes data.

Before this change, the task service could find a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is not enough for user-owned data.

If a logged-in user can guess or change another task id in the URL, they could update another user's task unless the backend checks ownership.

The correct rule is:

```
task id must match
authenticated user id must also match
```

What Changed

The update flow now works like this:

```
client sends PUT /api/tasks/{id}
controller reads the current user id from JWT claims
controller passes id, request, and user id to the service
service searches by both task id and owner user id
service updates the task only when ownership matches
```

If the task is missing or belongs to another user, the API returns:

```
404 Not Found
```

Service Contract Change

The task service update method changed from:

```
Task<TaskResponse?> UpdateTaskAsync(int id, UpdateTaskRequest request);
```

To:

```
Task<TaskResponse?> UpdateTaskAsync(int id, UpdateTaskRequest request, int userId);
```

The service needs `userId` because ownership must be checked before applying updates.

Query Change

The service should not use id-only lookup for updates.

Instead of:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Use:

```
var task = await _dbContext.Tasks
```

```
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

This makes the update operation ownership-aware.

Validation Still Applies

The endpoint should still validate update data.

Examples:

```
title is required
priority is required
```

Invalid request data should return:

```
400 Bad Request
```

Ownership checks and validation are both needed. Ownership decides whether the user can update the task. Validation decides whether the submitted task data is acceptable.

Controller Flow

The controller gets the current user id:

```
var currentUserId = GetCurrentUserId();
```

If the JWT token is missing or invalid, the API returns:

```
401 Unauthorized
```

Then the controller calls the service:

```
var response = await _taskService.UpdateTaskAsync(
    id,
    request,
    currentUserId.Value);
```

If the service returns `null`, the controller returns:

```
404 Not Found
```

If the task belongs to the logged-in user and the request is valid, the API returns:

```
200 OK
```

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

These two cases should look the same to the caller:

```
task does not exist
task exists but belongs to another user
```

Both return:

```
404 Not Found
```

This avoids leaking private resource information.

Test Flow

Create two users and obtain a JWT token for each user.

Create a task while authenticated as user one.

Then update it as the owner:

```
PUT /api/tasks/1
```

```
Authorization: Bearer USER_ONE_TOKEN
Content-Type: application/json
```

Body:

```
{
  "title": "Updated owned task",
  "description": "Updated by the owner",
  "dueDateUtc": null,
  "priority": "High"
}
```

Expected:

```
200 OK
```

Now try the same task id with user two's token:

```
PUT /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
Content-Type: application/json
```

Expected:

```
404 Not Found
```

The task should remain unchanged.

Anonymous Request Test

Request:

```
PUT /api/tasks/1
```

Expected:

```
401 Unauthorized
```

This confirms update requires authentication.

What Did Not Change

Day 66 does not change ownership rules for:

```
delete task
complete task
reopen task
```

Those mutation endpoints still need separate ownership checks.

What This Day Teaches

This lesson teaches a practical authorization rule:

```
Every write operation on user-owned data must verify ownership before changing data.
```

Reading private data needs ownership checks. Updating private data needs them even more.

Reader Exercise

Try to update a task with the owner's token and confirm the response is `200 OK`.

Then try to update the same task with another user's token and confirm the response is `404 Not Found`.

Finally, verify that invalid update data still returns `400 Bad Request`.

Next Step

The next step is to apply the same ownership pattern to delete, complete, and reopen task endpoints.

Day 67 - Restrict Delete Task

Lesson Goal

Allow a task to be deleted only by the authenticated user who owns it.

Day 67 changes only the `DELETE /api/tasks/{id}` endpoint. The delete operation now checks both the task id and authenticated user id before removing task data.

Why This Matters

Delete endpoints are high risk because they permanently remove data.

Before this change, the task service could find a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is unsafe for user-owned data.

If a logged-in user can change the id in the URL and the backend does not check ownership, that user could delete another user's task.

The correct rule is:

```
task id must match
authenticated user id must also match
```

What Changed

The delete flow now works like this:

```
client sends DELETE /api/tasks/{id}
controller reads the current user id from JWT claims
controller passes id and user id to the service
service searches by both task id and owner user id
service deletes the task only when ownership matches
```

If the task is missing or belongs to another user, the API returns:

```
404 Not Found
```

If the owner deletes the task successfully, the API returns:

```
204 No Content
```

Service Contract Change

The task service delete method changed from:

```
Task<bool> DeleteTaskAsync(int id);
```

To:

```
Task<bool> DeleteTaskAsync(int id, int userId);
```

The service needs `userId` because deletion must verify ownership before removing the row.

Query Change

The service should not use id-only lookup for deletion.

Instead of:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Use:

```
var task = await _dbContext.Tasks
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

If this query returns `null`, the task either does not exist or does not belong to the logged-in user.

Both cases return `false` from the service, and the controller converts that into:

```
404 Not Found
```

Controller Flow

The controller reads the current user id:

```
var currentUserId = GetCurrentUserId();
```

If the JWT token is missing or invalid, the API returns:

```
401 Unauthorized
```

Then the controller calls the service:

```
var isDeleted = await _taskService.DeleteTaskAsync(
    id,
    currentUserId.Value);
```

If the service returns `false`, the controller returns:

```
404 Not Found
```

If the delete succeeds, the controller returns:

```
204 No Content
```

Why 204 No Content

For a successful delete, the API does not need to return the deleted task.

`204 No Content` means:

```
the request succeeded
there is no response body
```

That is a standard response for successful delete operations.

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

These two cases should look the same:

```
task does not exist
task exists but belongs to another user
```

Both return:

```
404 Not Found
```

This avoids leaking private resource information.

Test Flow

Create two users and obtain a JWT token for each user.

Create a task while authenticated as user one.

First, try deleting it as another user:

```
DELETE /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
404 Not Found
```

The task should still exist for user one.

Then delete it as the owner:

```
DELETE /api/tasks/1
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
204 No Content
```

The response should not contain a body.

After deletion, requesting the same task should return:

```
404 Not Found
```

Anonymous Request Test

Request:

```
DELETE /api/tasks/1
```

Expected:

```
401 Unauthorized
```

This confirms deletion requires authentication.

What Did Not Change

Day 67 does not change ownership rules for:

```
complete task
reopen task
```

Those endpoints still need separate ownership checks.

What This Day Teaches

This lesson teaches a direct authorization rule:

```
Before deleting user-owned data, verify the authenticated user owns the resource.
```

For delete operations, ownership checks are not optional. They protect user data from accidental or malicious deletion.

Reader Exercise

Delete a task with the owner's token and confirm the response is `204 No Content`.

Then try to delete another user's task and confirm the response is `404 Not Found`.

Finally, confirm the task still exists for the correct owner after the failed delete attempt.

Next Step

The next step is to apply the same ownership pattern to complete and reopen task endpoints.

Day 68 - Review User Ownership

Lesson Goal

Review task ownership with two different users and confirm that the protections added during Days 63-67 work together.

Day 68 is mainly a testing and review day. The goal is not to add a new architecture or feature. The goal is to verify that the user ownership rules behave correctly across the main task endpoints.

What Is Reviewed

The review checks that authenticated users can:

```
create tasks only for themselves
list only their own tasks
retrieve only their own task by id
update only their own task
delete only their own task
```

This confirms that the API is moving from simple authentication toward practical authorization.

Important Current Limitation

The complete and reopen endpoints are protected by JWT authentication, but they do not yet check task ownership.

These endpoints are not part of the Day 68 ownership confirmation:

```
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Until those endpoints also check the authenticated user id, the project should not claim that every task endpoint is owner-restricted.

Ownership Test Setup

The review uses two users.

Example user one:

```
{
  "fullName": "Ownership User One",
  "email": "owner.one@example.com",
  "password": "Password123"
}
```

Example user two:

```
{
  "fullName": "Ownership User Two",
  "email": "owner.two@example.com",
  "password": "Password123"
}
```

After registering both users, log in as each user and keep the returned JWT tokens:

```
USER_ONE_TOKEN
USER_TWO_TOKEN
```

Swagger can authorize one token at a time, so the token must be switched when testing each user's behavior.

Create Task Ownership

Create one task while authenticated as user one:

```
{
  "title": "User one private task",
  "description": "Owned by user one",
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Then create one task while authenticated as user two:

```
{
  "title": "User two private task",
  "description": "Owned by user two",
  "dueDateUtc": null,
  "priority": "High"
}
```

The request body should not accept `userId`.

The response should include a `userId` that matches the logged-in user.

This confirms task ownership is assigned by the backend from JWT identity, not from client-submitted data.

Get All Ownership

When user one calls:

```
GET /api/tasks
Authorization: Bearer USER_ONE_TOKEN
```

The response should include only user one's tasks.

When user two calls the same endpoint with `USER_TWO_TOKEN`, the response should include only user two's tasks.

The filters should still work inside the logged-in user's task list:

```
GET /api/tasks?isCompleted=false
GET /api/tasks?search=private
```

This confirms filtering works after ownership is applied.

Get By Id Ownership

When user one requests their own task:

```
GET /api/tasks/USER_ONE_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

When user one requests user two's task:

```
GET /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

This confirms task id lookup is restricted by owner.

Update Ownership

When user one tries to update user two's task:

```
PUT /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

The task should remain unchanged.

When user two updates the same task with their own token:

```
200 OK
```

This confirms update operations are owner-restricted.

Delete Ownership

When user one tries to delete user two's task:

```
DELETE /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

The task should still exist for user two.

When user two deletes their own task:

```
204 No Content
```

This confirms delete operations are owner-restricted.

Ownership Checklist

Use this checklist during the review:

```
Create Task uses the authenticated user id
Get All returns only the authenticated user's tasks
Completion and search filters stay inside the user's task list
Get By Id returns 404 for another user's task
Update returns 404 for another user's task
An unsuccessful update does not change the task
Delete returns 404 for another user's task
An unsuccessful delete does not remove the task
Owner delete returns 204 No Content
Requests without a token return 401 Unauthorized
```

README Accuracy

The project README should describe the current API behavior accurately.

The README should mention:

```
JWT token generation on login
protected task endpoints with JWT authentication
created tasks belong to the authenticated user
task listing, retrieval, update, and deletion are restricted to the task owner
```

The README should not show old examples that use `userId` as a public task list query parameter.

For create task behavior, the README should explain:

```
POST /api/tasks uses the authenticated user id from the JWT token and returns 201 Created.
```

For login behavior, the README should explain:

```
Successful login returns a JWT token for authenticating protected task endpoints.
```

What This Day Teaches

This review teaches that authorization is not one endpoint.

For user-owned data, each endpoint needs to answer a direct question:

```
Is the authenticated user allowed to access or change this resource?
```

Days 63-67 added this pattern step by step for task creation, listing, retrieval, update, and deletion.

Reader Exercise

Run the full two-user ownership test.

Use separate tokens for user one and user two. Create one task per user, then test list, get by id, update, and delete behavior from both identities.

Write down every endpoint that returns `404 Not Found` for another user's task.

Next Step

The next step is to apply ownership protection to the complete and reopen endpoints so task status changes also respect authenticated ownership.

Day 69 - Add Basic Unit Test Project

Lesson Goal

Create the first automated test project and add one simple service test.

Day 69 starts the testing phase of JobTrackr. The goal is to add an xUnit test project, connect it to the solution, and write the first small unit test for password hashing.

Why This Matters

Manual testing through Swagger is useful while building early features, but it does not scale well.

As the backend grows, the project needs automated tests that can be run repeatedly:

```
before commits
before refactoring
before adding new features
before publishing code
```

Automated tests help catch regressions earlier.

What Is A Unit Test

A unit test checks a small piece of code automatically.

For this first test, the project does not test database behavior. It tests `PasswordHasherService` because password hashing is isolated and does not require EF Core or SQL Server.

The test verifies two things:

```
the password hash is not the original password
the correct password verifies successfully against the hash
```

Project Structure

The test project is placed under `tests`, outside the production `src` folder:

```
JobTrackr
|-- src
|   |-- JobTrackr.Api
|   |-- JobTrackr.Application
|   |-- JobTrackr.Domain
|   `-- JobTrackr.Infrastructure
`-- tests
    `-- JobTrackr.Tests
        `-- Auth
            `-- PasswordHasherServiceTests.cs
```

This keeps test code separate from the running API code.

Test Project Setup

The xUnit project is created with:

```
dotnet new xunit -n JobTrackr.Tests -o tests\JobTrackr.Tests --framework net8.0
```

Then the test project is added to the solution:

```
dotnet sln JobTrackr.slnx add tests\JobTrackr.Tests\JobTrackr.Tests.csproj
```

The test project references the Application layer:

```
dotnet add tests\JobTrackr.Tests\JobTrackr.Tests.csproj reference src\JobTrackr.Application\
```

```
JobTrackr.Application.csproj
```

That allows the test project to use application services such as `PasswordHasherService`.

First Test File

The generated example test is removed, and a real auth test is added:

```
tests\JobTrackr.Tests\Auth\PasswordHasherServiceTests.cs
```

The test class focuses on password hashing behavior.

First Test

The first test is:

```
using JobTrackr.Application.Auth;

namespace JobTrackr.Tests.Auth
{
    public class PasswordHasherServiceTests
    {
        [Fact]
        public void HashAndVerifyPassword_WithCorrectPassword_ReturnsTrue()
        {
            var passwordHasher = new PasswordHasherService();
            const string password = "Password123";

            var passwordHash = passwordHasher.HashPassword(password);
            var isValid = passwordHasher.VerifyPassword(password, passwordHash);

            Assert.NotEqual(password, passwordHash);
            Assert.True(isValid);
        }
    }
}
```

This test creates the password hasher, hashes a password, verifies the password, and checks the result.

What The Assertions Mean

This assertion confirms the stored value is not the plain password:

```
Assert.NotEqual(password, passwordHash);
```

This assertion confirms the correct password can be verified:

```
Assert.True(isValid);
```

Together, these checks confirm a basic but important authentication behavior.

Test Naming

The test name follows this pattern:

```
MethodOrBehavior_Scenario_ExpectedResult
```

The test name is:

```
HashAndVerifyPassword_WithCorrectPassword_ReturnsTrue
```

That makes the purpose of the test readable without opening the implementation.

Commands To Verify

Run all tests:

```
dotnet test
```

Expected result:

```
Passed: 1
Failed: 0
```

Then build the solution:

```
dotnet build
```

Expected result:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

What Did Not Change

Day 69 intentionally avoids extra testing complexity.

No changes are made to:

```
production authentication code
EF Core test setup
TaskService tests
mocking libraries
database behavior
```

The first test stays simple so the testing foundation is easy to understand.

What This Day Teaches

This lesson teaches that testing should start with a small, isolated behavior.

Password hashing is a good first unit test because it has clear input and output:

```
input: plain password
output: hash and verification result
```

It does not need controllers, HTTP requests, Swagger, a database, or authentication middleware.

Reader Exercise

Run the test project and confirm that one test passes.

Then add a second test that verifies the wrong password does not validate against the generated hash.

Expected idea:

```
HashAndVerifyPassword_WithWrongPassword_ReturnsFalse
```

Next Step

The next step is to begin testing task service behavior, where the project will need to think carefully about database setup and test isolation.

Day 70 - Test Task Creation

Lesson Goal

Add automated tests for creating a task through `TaskService`.

Day 70 continues the testing phase started on Day 69. The goal is to test task creation with a temporary EF Core in-memory database, without connecting to SQL Server and without changing production code.

What Is Tested

The tests focus on two behaviors:

```
valid task creation
empty task title validation
```

This keeps the lesson focused on the create task flow.

Why Use An In-Memory Database

`TaskService` depends on `AppDbContext`, so the service needs a database context during testing.

The EF Core in-memory provider creates a temporary database inside the test process.

That means the tests can run without:

```
SQL Server
real application data
manual database setup
Swagger
HTTP requests
```

These are service-level tests using the real `TaskService` and a temporary `AppDbContext`.

Test Project Updates

The test project now references the Infrastructure project:

```
dotnet add tests\JobTrackr.Tests\JobTrackr.Tests.csproj reference src\JobTrackr.Infrastructure\
JobTrackr.Infrastructure.csproj
```

This gives the test project access to:

```
TaskService
AppDbContext
Infrastructure dependencies
```

The test project also uses EF Core InMemory:

```
dotnet add tests\JobTrackr.Tests\JobTrackr.Tests.csproj package Microsoft.EntityFrameworkCore.
InMemory --version 8.0.27
```

The version should match the EF Core version used by the project.

Test File

The task creation tests are placed in:

```
tests\JobTrackr.Tests\Tasks\TaskServiceCreateTests.cs
```

The file contains tests for valid creation and title validation.

Fresh Database Per Test

Each test creates a fresh in-memory database:

```
private static AppDbContext CreateDbContext()
{
    var options = new DbContextOptionsBuilder<AppDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    return new AppDbContext(options);
}
```

Using a unique database name matters:

```
data from one test cannot affect another test
test order does not matter
each test starts clean
```

Seeding A User

Task creation requires a valid user id.

The test adds a user first:

```
private static async Task<User> AddUserAsync(AppDbContext dbContext)
{
    var user = new User
    {
        FullName = "Task Test User",
        Email = "task.test@example.com",
        PasswordHash = "test-password-hash"
    };

    dbContext.Users.Add(user);
    await dbContext.SaveChangesAsync();

    return user;
}
```

This gives the test a real user row for the task ownership check.

Valid Task Creation Test

The valid test creates:

```
database context
test user
real TaskService
CreateTaskRequest
```

Then it calls:

```
var response = await taskService.CreateTaskAsync(request, user.Id);
```

The important assertions are:

```
Assert.Equal(request.Title, response.Title);
Assert.Equal(request.Description, response.Description);
Assert.Equal(request.Priority, response.Priority);
Assert.Equal(user.Id, response.UserId);
Assert.False(response.IsCompleted);
Assert.Equal(1, await dbContext.Tasks.CountAsync());
```

These checks confirm that the task was created, assigned to the correct user, starts incomplete, and was saved to the database context.

Empty Title Test

The second test sends an invalid request with an empty title.

It expects:

```
Assert.ThrowsAsync<ArgumentException>
```

Then it confirms the exact validation message:

```
Assert.Equal(ErrorMessages.TaskTitleRequired, exception.Message);
```

Finally, it confirms the invalid task was not saved:

```
Assert.Empty(dbContext.Tasks);
```

This is important because validation should stop bad data before it reaches storage.

Expected Test Result

After Day 70, running:

```
dotnet test
```

Should show three passing tests:

```
password hashing test
valid task creation test
empty title validation test
```

Expected summary:

```
Passed: 3
Failed: 0
```

What Did Not Change

Day 70 intentionally avoids extra complexity.

No changes are made to:

```
production service code
SQL Server test setup
mocking libraries
controller tests
TaskService update/delete tests
```

The tests stay focused on create task behavior.

What This Day Teaches

This lesson teaches how to test a service that depends on EF Core.

The key idea is:

```
use the real service
use a temporary database
seed only the data required for the test
assert both the response and saved data
```

That gives useful confidence without requiring a full API or SQL Server setup.

Reader Exercise

Add one more create task test:

```
CreateTaskAsync_WithEmptyPriority_ThrowsArgumentException
```

It should verify that an empty priority throws the expected error and does not save a task.

Next Step

The next step is to expand task service tests beyond creation, such as testing task listing, filtering, or ownership behavior.

Day 71 - Test Task Get By Id

Lesson Goal

Add automated tests for retrieving one task through `TaskService.GetByIdAsync`.

Day 71 continues the service testing work from Day 70. The goal is to test the Get By Id behavior with an EF Core in-memory database while keeping the tests focused and simple.

What Is Tested

The tests cover two cases:

```
an existing task owned by the user is returned
a missing task returns null
```

These are core behaviors for the `GET /api/tasks/{id}` API flow.

No New Packages

Day 70 already added the required test setup:

```
Infrastructure project reference
Microsoft.EntityFrameworkCore.InMemory 8.0.27
```

Day 71 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Tasks\TaskServiceGetByIdTests.cs
```

It tests `TaskService.GetByIdAsync` using a temporary in-memory database.

Existing Owned Task Test

The first test creates a user and a task:

```
var user = new User
{
    FullName = "Get Task Test User",
    Email = "get.task@example.com",
    PasswordHash = "test-password-hash"
};
```

Then it creates a task owned by that user:

```
var task = new JobTask
{
    Title = "Review Get By Id",
    Description = "Test existing task behavior",
    Priority = "High",
    User = user
};
```

Using `User = user` creates the relationship between the task and the user. EF Core assigns the ids when the data is saved.

Calling The Service

The test creates the real service:

```
var taskService = new TaskService(dbContext);
```

Then it calls:

```
var response = await taskService.GetByIdAsync(task.Id, user.Id);
```

The method receives both:

```
task id
authenticated user id
```

That matches the ownership-aware behavior added earlier.

Assertions

The test first confirms the response exists:

```
Assert.NotNull(response);
```

Then it confirms the returned data is mapped correctly:

```
Assert.Equal(task.Id, response.Id);
Assert.Equal(task.Title, response.Title);
Assert.Equal(task.Description, response.Description);
Assert.Equal(task.Priority, response.Priority);
Assert.Equal(user.Id, response.UserId);
```

This proves the service can retrieve an owned task and return the expected response model.

Missing Task Test

The second test uses a fresh empty database and calls:

```
var response = await taskService.GetByIdAsync(999, 1);
```

Because no matching task exists, the service should return:

```
null
```

The test confirms that with:

```
Assert.Null(response);
```

In the real API, the controller converts this `null` result into:

```
404 Not Found
```

Isolated Database Per Test

Each test uses a unique in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That prevents one test from leaking data into another test.

This matters because the missing-task test must not accidentally see the task created by the existing-task test.

Expected Test Result

After Day 71, running:

```
dotnet test
```

Should show five passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
```

Expected summary:

```
Passed: 5
Failed: 0
```

What Did Not Change

Day 71 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
update/delete tests
```

The lesson stays focused on retrieving a task by id.

What This Day Teaches

This lesson teaches how to test a service method that returns either data or `null`.

That behavior matters because service methods often separate business logic from HTTP status codes:

```
service returns null
controller returns 404 Not Found
```

Keeping that boundary clear makes the code easier to test.

Reader Exercise

Add a third test:

```
GetByIdAsync_WithTaskOwnedByAnotherUser_ReturnsNull
```

Create two users, create a task for user one, then call `GetByIdAsync` with user two's id.

Expected result:

```
null
```

This verifies ownership protection directly in the service test.

Next Step

The next step is to continue expanding TaskService tests for list, update, delete, complete, or reopen behavior.

Day 72 - Test Task Update

Lesson Goal

Add automated tests for updating a task through `TaskService.UpdateTaskAsync`.

Day 72 continues the TaskService testing work. The goal is to verify successful updates, missing task behavior, and validation failure behavior using an EF Core in-memory database.

What Is Tested

The tests cover three cases:

```
a valid owned task update succeeds
a missing task returns null
an empty title throws the expected validation error
```

The tests also confirm that invalid update data does not change the saved task.

No New Packages

The test project already has the required setup:

```
Infrastructure project reference
Microsoft.EntityFrameworkCore.InMemory 8.0.27
```

Day 72 does not add new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Tasks\TaskServiceUpdateTests.cs
```

It tests `TaskService.UpdateTaskAsync` using an isolated in-memory database for each test.

Valid Update Test

The first test creates and saves an original task:

```
var task = await AddTaskAsync(dbContext);
```

Then it creates an update request:

```
var request = new UpdateTaskRequest
{
    Title = "Updated task title",
    Description = "Updated task description",
    DueDateUtc = DateTime.UtcNow.AddDays(7),
    Priority = "High"
};
```

The service is called with the task id, update request, and owner user id:

```
var response = await taskService.UpdateTaskAsync(
    task.Id,
    request,
    task.UserId);
```

This matches the ownership-aware update method added earlier.

Response Assertions

The test checks that the response contains the updated values:

```
Assert.NotNull(response);
Assert.Equal(request.Title, response.Title);
Assert.Equal(request.Description, response.Description);
Assert.Equal(request.DueDateUtc, response.DueDateUtc);
Assert.Equal(request.Priority, response.Priority);
Assert.Equal(task.UserId, response.UserId);
```

These assertions confirm that the service returned the expected updated task response.

Database Assertions

The test also reads the saved task from the in-memory database:

```
var savedTask = await dbContext.Tasks.SingleAsync();
```

Then it confirms the values were actually persisted:

```
Assert.Equal(request.Title, savedTask.Title);
Assert.Equal(request.Description, savedTask.Description);
Assert.Equal(request.DueDateUtc, savedTask.DueDateUtc);
Assert.Equal(request.Priority, savedTask.Priority);
```

This is important because the response alone is not enough. A useful service test should confirm the database state changed as expected.

Missing Task Test

The missing task test calls:

```
var response = await taskService.UpdateTaskAsync(999, request, 1);
```

Because no matching task exists, the service returns:

```
null
```

The test confirms:

```
Assert.Null(response);
```

In the real API, the controller converts this `null` result into:

```
404 Not Found
```

Empty Title Test

The empty title test sends:

```
Title = ""
```

The service should reject this invalid update by throwing:

```
ArgumentException
```

The test confirms the exact validation message:

```
Assert.Equal(ErrorMessages.TaskTitleRequired, exception.Message);
```

Then it reloads the saved task and confirms the original title is still present:

```
Assert.Equal(originalTitle, savedTask.Title);
```

This proves that invalid data was not saved.

Isolated Database Per Test

Each test uses:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That gives every test a clean database.

This prevents a task created in one test from affecting another test.

Expected Test Result

After Day 72, running:

```
dotnet test
```

Should show eight passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
```

Expected summary:

```
Passed: 8
Failed: 0
```

What Did Not Change

Day 72 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
delete tests
complete or reopen tests
```

The lesson stays focused on update behavior.

What This Day Teaches

This lesson teaches that update tests should verify both:

```
the response returned by the service
the actual saved database state
```

It also teaches that validation tests should confirm bad data does not partially update the stored entity.

Reader Exercise

Add one more update test:

```
UpdateTaskAsync_WithTaskOwnedByAnotherUser_ReturnsNull
```

Create a task for user one, then call `UpdateTaskAsync` with user two's id.

Expected result:

```
null
```

Also confirm the task data was not changed.

Next Step

The next step is to continue expanding service tests for delete, list, complete, or reopen task behavior.

Day 73 - Test Task Delete

Lesson Goal

Add automated tests for deleting a task through `TaskService.DeleteTaskAsync`.

Day 73 continues the TaskService testing phase. The goal is to verify successful task deletion and missing-task behavior using an EF Core in-memory database.

What Is Tested

The tests cover two cases:

```
an owned task can be deleted successfully
deleting a missing task returns false
```

The successful delete test also confirms that the task is actually removed from the database.

No New Packages

The test project already has everything required:

```
Infrastructure project reference
EF Core InMemory package
xUnit
```

Day 73 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Tasks\TaskServiceDeleteTests.cs
```

It tests `TaskService.DeleteTaskAsync` with an isolated in-memory database.

Successful Delete Test

The first test creates one user and one owned task:

```
var task = await AddTaskAsync(dbContext);
```

Then it calls the real service:

```
var isDeleted = await taskService.DeleteTaskAsync(
    task.Id,
    task.UserId);
```

The method receives both:

```
task id
authenticated owner user id
```

That matches the ownership-aware delete behavior added earlier.

Delete Assertions

The first assertion checks the service result:

```
Assert.True(isDeleted);
```

This confirms the service reported a successful deletion.

The next assertion checks the database state:

```
Assert.Equal(0, await dbContext.Tasks.CountAsync());
```

This confirms the task was actually removed.

The final assertion confirms the owner was not deleted:

```
Assert.Equal(1, await dbContext.Users.CountAsync());
```

Deleting a task should remove the task only. It should not remove the user who owned it.

Missing Task Test

The second test uses an empty database and calls:

```
var isDeleted = await taskService.DeleteTaskAsync(999, 1);
```

Because no matching task exists, the service returns:

```
false
```

The test confirms:

```
Assert.False(isDeleted);
```

In the real API, the controller converts this result into:

```
404 Not Found
```

Why Check The Database

Checking only the boolean result is not enough.

This assertion confirms actual persistence behavior:

```
Assert.Equal(0, await dbContext.Tasks.CountAsync());
```

Good service tests verify both:

```
the returned value
the saved database state
```

Isolated Database Per Test

Each test uses a fresh in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

This prevents data from one test from affecting another test.

The missing-task test must remain empty, regardless of what the successful delete test created.

Expected Test Result

After Day 73, running:

```
dotnet test
```

Should show ten passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
2 delete task tests
```

Expected summary:

```
Passed: 10
```

```
Failed: 0
```

What Did Not Change

Day 73 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
complete or reopen tests
```

The lesson stays focused on delete behavior.

What This Day Teaches

This lesson teaches that delete tests should verify more than the returned status.

The service should:

```
return true when an owned task is deleted
remove the task from storage
leave the owning user intact
return false when no matching task exists
```

This gives confidence that the delete behavior is correct before testing more complex task flows.

Reader Exercise

Add one more delete test:

```
DeleteTaskAsync_WithTaskOwnedByAnotherUser_ReturnsFalse
```

Create a task for user one, then call `DeleteTaskAsync` with user two's id.

Expected result:

```
false
```

Also confirm the task still exists in the database.

Next Step

The next step is to continue expanding service tests for task listing, complete, reopen, or ownership behavior.

Day 74 - Test User Service

Lesson Goal

Add automated tests for basic `UserService` behavior.

Day 74 expands the automated test suite beyond tasks. The goal is to test user creation and user retrieval by id through the real `UserService` using an EF Core in-memory database.

What Is Tested

The tests cover two cases:

```
a valid user is created and saved
an existing user can be retrieved by id
```

The tests also confirm that user entities are mapped correctly to `UserResponse`.

No Duplicate Email Test Today

`UserService.CreateUserAsync` validates that the email is not empty, but it does not currently check whether another user already has the same email.

That means `UserService` currently allows duplicate emails.

Day 74 does not add a duplicate-email rejection test because that behavior is not implemented in `UserService`.

Duplicate-email behavior exists separately in authentication registration and should be tested later in the auth service test area.

No New Packages

The test project already has everything required:

```
Infrastructure project reference
EF Core InMemory
xUnit
```

Day 74 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Users\UserServiceTests.cs
```

It tests `UserService` with an isolated in-memory database.

Create User Test

The create test starts with a valid request:

```
var request = new CreateUserRequest
{
    FullName = "Create User Test",
    Email = "create.user@example.com"
};
```

Then it calls the real service:

```
var response = await userService.CreateUserAsync(request);
```

The response assertions confirm that a `UserResponse` is returned with the expected values:

```
Assert.True(response.Id > 0);
Assert.Equal(request.FullName, response.FullName);
Assert.Equal(request.Email, response.Email);
```

The database assertions confirm that the user was actually saved:

```
Assert.Equal(1, await dbContext.Users.CountAsync());
```

Then the saved entity is checked directly:

```
var savedUser = await dbContext.Users.SingleAsync();

Assert.Equal(request.FullName, savedUser.FullName);
Assert.Equal(request.Email, savedUser.Email);
```

This verifies both the returned DTO and the persisted entity.

Get By Id Test

The Get By Id test first creates a user through `UserService`:

```
var createdUser = await userService.CreateUserAsync(createRequest);
```

Then it retrieves the same user by id:

```
var response = await userService.GetByIdAsync(createdUser.Id);
```

The assertions confirm:

```
the response is not null
the id matches
the full name matches
the email matches
the created date matches
```

This proves the service can retrieve and map an existing user correctly.

Why Use UserService To Seed The Test

The Get By Id test uses `CreateUserAsync` to create the user first.

This keeps the setup simple and uses the same real service behavior to generate a saved user and id.

The main behavior under test is still the retrieval step:

```
GetByIdAsync returns the existing user as UserResponse
```

Isolated Database Per Test

Each test uses a fresh in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That keeps the tests independent.

The create test and get-by-id test do not share data.

Expected Test Result

After Day 74, running:

```
dotnet test
```

Should show twelve passing tests:

```
1 password hashing test
```

```
2 task creation tests
2 Get By Id tests
3 update task tests
2 delete task tests
2 user service tests
```

Expected summary:

```
Passed: 12
Failed: 0
```

What Did Not Change

Day 74 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production UserService code
new packages
duplicate email behavior
auth registration tests
controller tests
```

The lesson stays focused on basic user service behavior.

What This Day Teaches

This lesson teaches that service tests should match the behavior that exists today.

A test should not assert duplicate-email rejection in `UserService` if `UserService` does not implement duplicate-email rejection.

Good tests document real behavior clearly:

```
valid users can be created
saved users can be retrieved by id
responses are mapped correctly
```

Reader Exercise

Add a missing-user test:

```
GetByIdAsync_WithMissingUser_ReturnsNull
```

Use an empty in-memory database, call `GetByIdAsync` with an id that does not exist, and confirm the response is `null`.

Next Step

The next step is to continue expanding tests carefully, especially around authentication registration behavior and task ownership edge cases.

Day 75 - Test Auth Service

Lesson Goal

Add automated tests for registration and login through `AuthService`.

Day 75 expands the test suite into authentication behavior. The goal is to test registration, valid login, and invalid-password login without connecting to SQL Server and without changing production authentication code.

What Is Tested

The tests cover three cases:

```
successful registration
login with valid credentials
login with an invalid password
```

These are core authentication behaviors for the backend.

Why Use A Fake JWT Service

`AuthService` depends on `IJwtTokenService`.

The real JWT token service needs signing-key configuration. These tests are not trying to verify JWT signing, token expiration, or cryptographic configuration.

The tests use a small fake implementation that returns a predictable token:

```
test-token-USER_ID
```

This keeps the test focused on authentication flow:

```
register user
hash password
verify password
call token service after successful login
return auth response
```

No mocking library is needed.

No New Packages

The test project already has everything required:

```
Application reference
Infrastructure reference
EF Core InMemory
xUnit
```

Day 75 does not install new packages.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Auth\AuthServiceTests.cs
```

It tests `AuthService` with an isolated in-memory database.

Registration Test

The registration test uses the real password hasher:

```
var passwordHasher = new PasswordHasherService();
```

Then it creates `AuthService` with:

```
AppDbContext
PasswordHasherService
FakeJwtTokenService
```

The test registers a valid user:

```
var response = await authService.RegisterAsync(request);
```

The response assertions confirm:

```
user id is generated
full name matches
email matches
registration response token is empty
```

Registration currently creates the user. Login is what returns the JWT token.

Password Hash Assertions

After registration, the test reads the saved user from the database.

It confirms the plain password was not stored:

```
Assert.NotEqual(request.Password, savedUser.PasswordHash);
```

Then it verifies the stored hash belongs to the supplied password:

```
Assert.True(passwordHasher.VerifyPassword(
    request.Password,
    savedUser.PasswordHash));
```

These checks prove the registration flow stores a hash instead of storing the original password.

Valid Login Test

The valid login test first registers a user.

That gives the database a user with a properly hashed password.

Then the test logs in with the same email and password:

```
var response = await authService.LoginAsync(loginRequest);
```

The assertions confirm:

```
user id matches the registered user
full name matches
email matches
token matches the fake token format
```

The token assertion is:

```
Assert.Equal($"test-token-{registeredUser.UserId}", response.Token);
```

This proves `AuthService` called `IJwtTokenService.GenerateToken` after successful credential validation.

Invalid Password Test

The invalid login test registers a user first.

Then it tries to log in with:

```
WrongPassword
```

The test expects:

```
ArgumentException
```

And confirms the safe message:

```
Invalid email or password.
```

This message does not reveal whether the email exists or only the password was wrong.

FakeJwtTokenService

The fake token service is private inside the test class:

```
private class FakeJwtTokenService : IJwtTokenService
{
    public string GenerateToken(User user)
    {
        return $"test-token-{user.Id}";
    }
}
```

It follows the same interface as the real JWT service but returns a predictable test value.

It is used only by tests and never by the running API.

Expected Test Result

After Day 75, running:

```
dotnet test
```

Should show fifteen passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
2 delete task tests
2 user service tests
3 auth service tests
```

Expected summary:

```
Passed: 15
Failed: 0
```

What Did Not Change

Day 75 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production authentication code
JWT configuration
mocking libraries
controller tests
real token signing behavior
```

The lesson stays focused on authentication service behavior.

What This Day Teaches

This lesson teaches how to test a service with a dependency.

The real password hasher is useful because password hashing is part of the authentication behavior being tested.

The fake JWT service is useful because token signing configuration is not the behavior being tested today.

The key idea is:

```
use real dependencies when they are part of the behavior
use a small test double when a dependency would distract from the behavior
```

Reader Exercise

Add a fourth auth test:

```
RegisterAsync_WithDuplicateEmail_ThrowsArgumentException
```

Only add this if duplicate-email rejection exists in `AuthService`.

The test should register one user, try registering the same email again, and confirm the expected error message.

Next Step

The next step is to continue expanding authentication and ownership tests, especially for edge cases that protect real user data.

Day 76 - Test Build And CI Readiness

Lesson Goal

Verify that the complete JobTrackr solution is ready for Continuous Integration before adding a GitHub Actions workflow.

This is a review and verification lesson. No feature or test is added. The focus is proving that the same restore, build, and test commands planned for CI already work from the solution root.

What CI Readiness Means

Continuous Integration automatically checks the project whenever code is pushed or a pull request is opened.

Before asking GitHub to run those checks, the commands should work locally:

```
restore dependencies
build the complete solution
run the complete test suite
leave the repository clean
```

A CI workflow cannot repair an unreliable build. It only repeats the project's commands in a consistent environment.

Verify The Starting State

Start from the repository root and check Git:

```
git status
```

The expected result is:

```
nothing to commit, working tree clean
```

This matters because generated files under `bin` and `obj` should be ignored. A build or test run should not introduce tracked changes.

Confirm The Test Project Is Discoverable

The solution file must include the test project:

```
<Folder Name="/tests/">
  <Project Path="tests/JobTrackr.Tests/JobTrackr.Tests.csproj" />
</Folder>
```

This allows solution-level commands to build and test every required project without targeting the test project manually.

Restore Dependencies

Run:

```
dotnet restore JobTrackr.slnx
```

Restore resolves the NuGet dependencies for all projects in the solution.

If the dependencies are already available, the command can report that all projects are up to date. That is also a successful result.

Build In Release Configuration

Run:

```
dotnet build JobTrackr.slnx --configuration Release --no-restore
```

The separate `--no-restore` build proves that the restore stage completed successfully.

Using Release configuration is useful because it is closer to the configuration normally used by CI and deployment than a local Debug build.

The verified Day 76 result is:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The complete solution builds, including:

```
JobTrackr.Domain
JobTrackr.Application
JobTrackr.Infrastructure
JobTrackr.Api
JobTrackr.Tests
```

Run The Complete Test Suite

Run:

```
dotnet test JobTrackr.slnx --configuration Release --no-build
```

The `--no-build` option uses the Release output from the previous step. This keeps each stage clear:

```
restore once
build once
test the completed build
```

The verified result is:

```
Passed: 15
Failed: 0
Skipped: 0
```

Current Test Coverage

The fifteen tests currently cover:

Authentication

- Password hashing and verification
- User registration
- Login with valid credentials
- Login with an invalid password

Tasks

- Creating a valid task
- Rejecting an empty task title
- Getting an existing task by ID
- Handling a missing task
- Updating a valid task
- Handling an update for a missing task

- Rejecting an empty title during update
- Deleting an owned task
- Handling deletion of a missing task

Users

- Creating a valid user
- Getting an existing user by ID

Review Test Names

Clear test names make failures easier to understand in local output and in CI logs.

The test suite follows this structure:

```
MethodOrBehavior_Scenario_ExpectedResult
```

Examples include:

```
CreateTaskAsync_WithValidRequest_CreatesTask
GetByIdAsync_WithMissingTask_ReturnsNull
UpdateTaskAsync_WithEmptyTitle_ThrowsArgumentException
DeleteTaskAsync_WithExistingOwnedTask_DeletesTask
LoginAsync_WithInvalidPassword_ThrowsArgumentException
```

Each name explains the behavior, condition, and expected outcome without requiring the reader to open the test first.

Check The Test Project

The test project should identify itself correctly:

```
<TargetFramework>net8.0</TargetFramework>
<IsTestProject>true</IsTestProject>
```

Its dependencies include:

```
xUnit
Microsoft.NET.Test.Sdk
EF Core InMemory 8.0.27
JobTrackr.Application project reference
JobTrackr.Infrastructure project reference
```

Working package versions should not be changed during a verification day without a specific reason.

Verify The Final Repository State

Run Git status again after the build and tests:

```
git status
```

The repository remained clean during the verified Day 76 run.

This confirms that generated build and test files are ignored correctly.

Why There Is No Day 76 Commit

No source code, tests, or configuration required a change.

Creating an empty commit would make the project history noisier without documenting a real code change. The previous commit remains the latest verified source state:

```
3eca56c Day 75: Test auth service
```

The Day 76 result is still valuable: it establishes a known-good baseline before CI automation is introduced.

What This Day Teaches

CI readiness is not mainly about writing a workflow file. It is about having reliable commands and a repository that behaves predictably.

The key sequence is:

```
clean repository
    -> successful restore
    -> successful Release build
    -> complete passing test suite
    -> clean repository
```

Once this sequence works locally, it can be transferred into GitHub Actions with much greater confidence.

Reader Exercise

Run the verification sequence from the repository root:

```
dotnet restore JobTrackr.slnx
dotnet build JobTrackr.slnx --configuration Release --no-restore
dotnet test JobTrackr.slnx --configuration Release --no-build
git status
```

Record:

```
the number of warnings
the number of errors
the number of passing tests
whether Git remained clean
```

Do not change code if everything passes.

Next Step

The next lesson can add a GitHub Actions workflow that runs the same restore, Release build, and test commands automatically.

Day 77 - Add GitHub Actions Build

Lesson Goal

Add the first GitHub Actions workflow to JobTrackr so every push to `main`, and every pull request targeting `main`, automatically verifies that the production projects build.

Day 76 proved that the project builds locally. Day 77 moves that check onto a clean computer managed by GitHub.

This first workflow intentionally performs build automation only. Automated tests will be added separately.

What GitHub Actions Provides

GitHub Actions can run repository commands when an event occurs.

For this workflow, the sequence is:

```
push or pull request
    -> create a temporary Ubuntu runner
    -> download the repository
    -> install .NET 8
    -> restore NuGet dependencies
    -> build the API in Release configuration
```

This catches compilation failures without relying on a developer to remember to run the build locally.

Workflow Location

GitHub discovers workflow files inside:

```
.github/workflows/
```

The new file is:

```
.github/workflows/build.yml
```

No production C# file is changed in this lesson.

The Build Workflow

The completed workflow is:

```yaml
name: Build

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v7

      - name: Set up .NET
        uses: actions/setup-dotnet@v6
```

```
        with:
          dotnet-version: 8.0.x

      - name: Restore dependencies
        run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj

      - name: Build
        run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-
restore
```

YAML uses indentation to represent structure. Spaces must be used consistently because incorrect indentation can make the workflow invalid or change its meaning.

Workflow Triggers

The workflow responds to two events:

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

The push trigger checks code after it reaches `main`.

The pull request trigger checks proposed changes before they are merged into `main`.

Together, these triggers support both direct pushes and a future pull-request workflow.

Minimal Permissions

The workflow declares:

```
permissions:
  contents: read
```

The build only needs to read the repository. It does not need permission to modify source code, create releases, or write packages.

Giving automation only the permissions it requires is a useful security habit.

The GitHub Runner

The build job uses:

```
runs-on: ubuntu-latest
```

GitHub creates a temporary Linux environment for the job and removes it afterward.

Building successfully on Ubuntu demonstrates that the production projects compile outside the local Windows development environment.

Checkout And .NET Setup

The first action downloads the repository:

```
- name: Checkout repository
  uses: actions/checkout@v7
```

The next action installs the .NET 8 SDK:

```
- name: Set up .NET
  uses: actions/setup-dotnet@v6
  with:
    dotnet-version: 8.0.x
```

The `8.0.x` value allows the runner to use a current servicing release of the .NET 8 SDK.

Why The Action Versions Were Updated

The workflow was initially added in commit:

```
8f21e1a Day 77: Add GitHub Actions build
```

The action versions were then updated in:

```
7d8e8eb Day 77: Update GitHub Actions runtime
```

The update moved the workflow away from older action versions that produced a Node.js runtime deprecation warning on GitHub's runners.

This does not change JobTrackr's target framework. The application still uses .NET 8; the Node.js runtime is an internal implementation detail of the reusable GitHub actions.

Why Build The API Project

The workflow targets:

```
src/JobTrackr.Api/JobTrackr.Api.csproj
```

The repository currently uses the newer `JobTrackr.slnx` solution format. Building the API project directly avoids depending on solution-format support across different .NET 8 SDK versions.

The API project references the other production layers:

```
JobTrackr.Api
    -> JobTrackr.Infrastructure
    -> JobTrackr.Application
    -> JobTrackr.Domain
```

Therefore, building the API also compiles all four production projects.

The test project is not part of this dependency chain and is deliberately left for the next workflow improvement.

Separate Restore And Build Stages

The restore step is:

```
- name: Restore dependencies
  run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj
```

The build step is:

```
- name: Build
  run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-restore
```

Using `--no-restore` makes the stages explicit. If dependency restoration fails, the workflow stops at restore instead of hiding the problem inside the build command.

Verified Local Result

The same commands were run locally from the repository root:

```
dotnet restore src\JobTrackr.Api\JobTrackr.Api.csproj
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release --no-restore
```

The verified result was:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The build compiled:

```
JobTrackr.Domain
JobTrackr.Application
JobTrackr.Infrastructure
JobTrackr.Api
```

Git remained clean after the verification.

Why No Secrets Or Database Are Required

This workflow compiles the code. It does not launch the API or execute database operations.

Therefore, the build does not need:

```
SQL Server
a production connection string
a JWT signing key
GitHub repository secrets
```

Runtime configuration should only be added when a workflow step actually needs to run the application or connect to an external service.

Checking The Workflow On GitHub

After the workflow is pushed, open the repository's Actions tab and inspect the newest Build run.

The job should contain these steps:

```
Checkout repository
Set up .NET
Restore dependencies
Build
```

A successful run confirms the production code builds on a clean GitHub runner.

When a workflow fails, open the failed step first. Common causes include:

```
incorrect YAML indentation
an incorrect project path
an invalid action version
a misspelled action name
an actual compilation error
```

Fix the reported problem rather than adding unrelated workflow complexity.

What This Day Teaches

Continuous Integration starts with a small, dependable check.

The first workflow answers one important question:

```
Can the production application compile from a clean checkout?
```

Keeping the workflow focused makes failures easier to understand and provides a stable foundation for adding tests next.

Reader Exercise

Create a branch, make a harmless documentation change, and open a pull request targeting `main`.

Confirm that the Build workflow starts automatically and completes successfully. Then inspect each step to see how GitHub presents logs and execution time.

Do not add deployment credentials or services to this exercise.

Next Step

The next lesson can extend Continuous Integration by running the complete automated test suite after the Release build succeeds.

Day 78 - Add GitHub Actions Tests

Lesson Goal

Extend the existing GitHub Actions build workflow so every push to `main`, and every pull request targeting `main`, runs the complete JobTrackr test suite.

Day 77 automated the production build. Day 78 adds behavioral verification without creating another workflow or changing production code.

From Build Automation To Continuous Integration

The existing workflow answers:

```
Does the production application compile?
```

The test step adds another question:

```
Do the tested backend behaviors still work?
```

The complete CI sequence is now:

```
push or pull request
    -> restore API dependencies
    -> restore test dependencies
    -> build production projects
    -> build the test project
    -> run all automated tests
    -> report success or failure
```

Update The Existing Workflow

The only changed file is:

```
.github/workflows/build.yml
```

A second workflow is unnecessary because building and testing are parts of the same validation process.

Keeping them in one job also means the Test step only runs after the earlier checkout, setup, restore, and build steps succeed.

Completed Workflow

The workflow now contains:

```yaml
name: Build

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v7
```

```
      - name: Set up .NET
        uses: actions/setup-dotnet@v6
        with:
          dotnet-version: 8.0.x

      - name: Restore API dependencies
        run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj

      - name: Restore test dependencies
        run: dotnet restore tests/JobTrackr.Tests/JobTrackr.Tests.csproj

      - name: Build
        run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-
restore

      - name: Test
        run: dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --
no-restore
```

The existing triggers, permissions, Ubuntu runner, and .NET setup remain unchanged.

Clear Restore Step Names

The original step was named:

```
- name: Restore dependencies
```

It is renamed to:

```
- name: Restore API dependencies
```

The test restore step is named separately:

```
- name: Restore test dependencies
```

These labels make GitHub's workflow log easier to scan. A failed restore immediately reveals whether the production or test project caused the problem.

Why Restore The Test Project Separately

The API project does not depend on the test project.

The test project has its own packages, including:

```
xUnit
Microsoft.NET.Test.Sdk
Microsoft.EntityFrameworkCore.InMemory
```

Restoring only the API project does not guarantee that these test-specific dependencies are available.

The workflow therefore runs:

```
run: dotnet restore tests/JobTrackr.Tests/JobTrackr.Tests.csproj
```

This creates a clear dependency stage before testing.

Run The Test Project Directly

The Test step is:

```
- name: Test
  run: dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --no-
restore
```

The repository uses the newer `JobTrackr.slnx` solution format. Targeting the test project directly avoids depending on `.slnx` support across different .NET 8 SDK servicing versions.

The test project references the Application and Infrastructure projects. Their dependencies, including Domain, are compiled automatically when the test project is built.

Understanding The Test Command

The command has three important parts:

```
dotnet test
```

Builds the test project when needed and runs all discovered tests.

```
--configuration Release
```

Uses the same Release configuration as the production build.

```
--no-restore
```

Proves that the earlier test dependency restore completed successfully.

The command does not use `--no-build` because the earlier Build step targets the API and its production dependencies, not `JobTrackr.Tests`.

Verified Result

The same workflow commands were run locally from the repository root.

The production build completed with:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The test command completed with:

```
Passed: 15
Failed: 0
Skipped: 0
Total: 15
```

The passing tests cover task operations, user operations, password hashing, registration, and login behavior.

How A Failed Test Protects The Project

`dotnet test` returns a nonzero exit code when any test fails.

GitHub Actions responds by:

```
marking the Test step as failed
marking the complete workflow run as failed
showing the failed test name
displaying the assertion or exception details
```

This gives the developer feedback before a regression is accepted as healthy code.

When branch protection is added later, a successful workflow can also become a requirement before merging a pull request.

Why CI Does Not Need SQL Server

The JobTrackr service tests use:

```
Microsoft.EntityFrameworkCore.InMemory
```

Each test creates an isolated in-memory database. The workflow does not need to install or connect to SQL Server.

It also does not need:

```
a database connection string
database migrations
a JWT signing key
```

```
GitHub secrets
```

The fake JWT service used by authentication tests avoids requiring token-signing configuration during these service-level tests.

Workflow Steps On GitHub

The Build job now presents these main steps:

```
Checkout repository
Set up .NET
Restore API dependencies
Restore test dependencies
Build
Test
```

Opening the Test step reveals the test runner output and the number of passed, failed, and skipped tests.

If the step fails, start with the failed test name and its error message. Avoid changing unrelated workflow configuration when the failure comes from application behavior.

Scope Control

Day 78 deliberately does not add:

```
a second workflow file
SQL Server services
repository secrets
deployment
code coverage reporting
test result artifacts
changes to production C# code
changes to test code
```

The goal is a small and understandable improvement: automatically run the tests that already exist.

What This Day Teaches

A successful build proves that code can compile. A passing test suite provides evidence that important behavior remains correct.

The CI pipeline now checks both:

```
structural correctness through compilation
behavioral correctness through automated tests
```

This creates a stronger safety net for future backend changes.

Reader Exercise

Open a successful workflow run in GitHub Actions and inspect the Test step.

Find:

```
the test assembly
the total number of tests
the number passed
the number failed
the number skipped
```

Then locate one test locally and connect its method name to the name shown in the CI output.

Do not intentionally break the main branch to demonstrate a failed test.

Next Step

The next lesson can improve the CI workflow with another practical quality check, or begin the next tested backend feature using the automated build-and-test baseline.

Day 79 - Review README For Testing

Lesson Goal

Update the JobTrackr repository README so it accurately describes the backend as it exists today.

This is a documentation-only lesson. No C# source code, test code, or GitHub Actions workflow is changed.

Why Documentation Must Evolve

A README is often the first part of a repository that another developer, interviewer, client, or contributor reads.

Before Day 79, the README still implied that login did not generate a JWT, task requests accepted a public user ID, task queries could expose every user's data, and testing was future work.

Those statements no longer matched JobTrackr.

Technical documentation should answer:

```
What does the project do?
How is it organized?
Which endpoints exist?
How does authentication work?
Which security behavior is complete?
Which limitations remain?
How can the project be built and tested?
What does CI verify?
```

Current Project Summary

The README now introduces JobTrackr as a learning-focused .NET 8 Web API built with production-style backend practices.

It identifies the current capabilities:

```
ASP.NET Core Web API
Clean Architecture-style project boundaries
SQL Server and Entity Framework Core
user and task endpoints
password hashing
registration and JWT login
authenticated, user-owned tasks
xUnit service tests
GitHub Actions continuous integration
```

Architecture Documentation

The five projects are documented by responsibility:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
JobTrackr.Tests
```

`JobTrackr.Api` contains controllers, middleware, authentication configuration, dependency registration, and startup.

`JobTrackr.Application` contains DTOs, interfaces, shared messages, and application contracts.

`JobTrackr.Domain` contains the core `User` and `JobTask` entities.

`JobTrackr.Infrastructure` contains EF Core access, migrations, service implementations, authentication logic, and JWT generation.

`JobTrackr.Tests` contains xUnit service tests using isolated in-memory databases.

Current Endpoint Documentation

The README groups endpoints into Authentication, Tasks, and Users.

Authentication endpoints are:

```
POST /api/auth/register
POST /api/auth/login
```

Task documentation covers listing, completion filtering, title searching, creation, retrieval, updating, deletion, completion, and reopening.

The outdated public `userId` task filter is no longer advertised.

Accurate Authentication Behavior

The README distinguishes registration from login:

```
registration creates the user and currently returns an empty token
successful login returns a JWT token
```

It also documents that duplicate emails are rejected, passwords are hashed, authentication responses do not expose password hashes, and protected task endpoints require a Bearer token.

Accurate Task Ownership

Task ownership now comes from the authenticated identity:

```
task creation does not accept UserId
new tasks belong to the authenticated user
task listing returns only that user's tasks
filters operate within that user's tasks
Get By Id checks ownership
Update checks ownership
Delete checks ownership
```

For ownership-protected operations, missing tasks and tasks belonging to another user both return `404 Not Found`.

This avoids revealing whether another user's task exists.

Document Limitations Honestly

Good documentation does not present planned security work as completed behavior.

The README explicitly records:

```
Complete and Reopen require authentication but do not yet check ownership
User CRUD endpoints are not yet protected by authorization
```

Publishing these limitations prevents readers from assuming protections that do not exist and creates an accurate roadmap for future work.

Database Documentation

The current persistence summary includes SQL Server, Entity Framework Core, migrations, Users and Tasks, password hashes, due dates, priorities, the task-to-user foreign key, and cascade deletion.

This provides useful context without turning the README into a full database specification.

Automated Tests

The README now records the current fifteen xUnit service tests.

Coverage includes:

```
password hashing and verification
registration
valid and invalid login
valid and invalid task creation
existing and missing task retrieval
valid and invalid task updates
successful and missing task deletion
valid user creation
existing user retrieval
```

The service tests use `Microsoft.EntityFrameworkCore.InMemory`, so they do not connect to SQL Server.

Reproducible Test Commands

Readers can run:

```
dotnet restore tests\JobTrackr.Tests\JobTrackr.Tests.csproj
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-restore
```

The verified result is:

```
Passed: 15
Failed: 0
Skipped: 0
```

GitHub Actions Documentation

The README explains that `.github/workflows/build.yml` runs on pushes and pull requests targeting `main`.

The workflow checks out the repository, installs .NET 8, restores API and test dependencies, builds the production projects in Release configuration, and runs all automated tests.

Because the tests use an in-memory database, CI does not need SQL Server or repository secrets.

Verified Build

The documented production build was verified:

```
dotnet restore src\JobTrackr.Api\JobTrackr.Api.csproj
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release --no-restore
```

Result:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

All fifteen tests also passed.

Scope Of The Commit

Commit `772fb66` changes only `README.md`:

```
162 lines added
102 lines removed
```

No production code, test code, or workflow configuration was modified.

What This Day Teaches

Documentation quality depends on accuracy, not only presentation.

The important habit is:

```
compare documentation with current behavior
remove outdated claims
describe verified features precisely
publish known limitations
test every command given to readers
```

A trustworthy README helps other people evaluate the project and helps the developer explain it clearly during interviews.

Reader Exercise

Review one of your own repository READMEs.

For every feature claim, identify the controller, service, test, or workflow that proves it exists. Mark unsupported claims as outdated, planned, partially complete, or verified.

Then update the README and run every documented command before publishing it.

Next Step

The next lesson can address one of the authorization limitations now documented publicly, or continue improving reliability from the verified build, test, and CI baseline.

Day 80 - Add Basic Request Logging

Lesson Goal

Add simple request logging to JobTrackr using the logging system built into ASP.NET Core.

Each completed request records:

```
HTTP method
request path
response status code
elapsed time in milliseconds
```

The implementation deliberately avoids request bodies, passwords, JWT tokens, authorization headers, signing keys, and connection strings.

Why Request Logging Matters

When an API is running, developers need a quick way to understand which routes are being called, whether they succeed, and how long they take.

A useful request log can answer:

```
Which endpoint received traffic?
Did it return success, a client error, or a server error?
How long did processing take?
```

Request logging improves visibility without changing controller or service behavior.

The Middleware

The new file is:

```
src/JobTrackr.Api/Middleware/RequestLoggingMiddleware.cs
```

It uses a stopwatch to measure the full request:

```
var stopwatch = Stopwatch.StartNew();

try
{
    await _next(context);
}
finally
{
    stopwatch.Stop();

    _logger.LogInformation(
        "HTTP {Method} {Path} responded {StatusCode} in {ElapsedMilliseconds} ms",
        context.Request.Method,
        context.Request.Path,
        context.Response.StatusCode,
        stopwatch.ElapsedMilliseconds);
}
```

The middleware starts timing before passing control to the rest of the pipeline. It writes the log after downstream processing completes.

Why Use A Finally Block

The logging code belongs in a finally block because finally runs whether downstream processing succeeds or throws an exception.

Without it, failed requests could disappear from the request log precisely when diagnostic information is most useful.

The pattern is:

```
start timer
    -> execute remaining middleware and endpoint
    -> stop timer and log result in finally
```

Structured Logging

The message template uses named placeholders:

```
HTTP {Method} {Path} responded {StatusCode} in {ElapsedMilliseconds} ms
```

This is structured logging. Method, Path, StatusCode, and ElapsedMilliseconds are supplied as separate values rather than being joined manually into one string.

Logging systems can later filter or group records using those fields. Examples include finding all 500 responses or comparing elapsed time by route.

Built-In ILogger

The middleware receives:

```
ILogger<RequestLoggingMiddleware>
```

No third-party package or custom service registration is required. ASP.NET Core supplies both the logger and the next request delegate when it creates the middleware.

The logger category is associated with RequestLoggingMiddleware, which makes its records identifiable among other application logs.

Middleware Registration

Program.cs adds one line to the request pipeline:

```
app.UseHttpsRedirection();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

The position of request logging is intentional.

Why Middleware Order Matters

Request logging is registered before exception handling.

When a downstream component throws an unexpected exception:

```
RequestLoggingMiddleware starts timing
    -> ExceptionHandlingMiddleware calls downstream code
    -> downstream code throws
    -> ExceptionHandlingMiddleware creates a 500 response
    -> control returns to RequestLoggingMiddleware
    -> request logger records status 500
```

This order allows the request log to capture the final response status produced by the exception handler.

If logging were placed inside exception handling, the behavior and final recorded status could differ.

Safe Request Metadata

An example log is:

```
HTTP GET /api/tasks responded 200 in 24 ms
```

This contains enough information for basic operational visibility without exposing user content or credentials.

Only the request path is logged. The query string is not included, reducing the chance that sensitive query values enter the logs.

Information That Must Not Be Logged

The middleware does not record:

```
request or response bodies
passwords
JWT tokens
Authorization headers
JWT signing keys
database connection strings
```

Logs often remain available longer than individual requests and may be accessed by more people than production data. A secret written to a log should be treated as exposed.

The safe default is to log metadata, not payloads.

Example Outcomes

An unauthenticated request to a protected endpoint can produce:

```
HTTP GET /api/tasks responded 401 in 12 ms
```

A successful login can produce:

```
HTTP POST /api/auth/login responded 200 in 35 ms
```

The password and returned token must not appear.

An unknown route can produce:

```
HTTP GET /api/unknown responded 404 in 1 ms
```

Elapsed time naturally varies between requests and environments.

Scope Of The Change

Commit 32fef89 changes two files:

```
RequestLoggingMiddleware.cs: new middleware
Program.cs: middleware pipeline registration
```

The commit adds 40 lines and does not change controllers, application services, domain entities, persistence behavior, or the existing exception handler.

Verified Result

The API was built in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The existing test suite also passed:

```
Passed: 15
Failed: 0
```

```
Skipped: 0
```

This confirms that adding request logging did not break the tested backend behavior.

Current Limitation

The existing tests focus on application and infrastructure services. They do not directly test the new middleware.

A future middleware or integration test could verify:

```
the next delegate is called
the log is written after completion
the method, path, status, and elapsed fields are present
exceptions still result in a request log
```

That broader test is outside Day 80's intentionally small scope.

What This Day Teaches

Request logging is a cross-cutting concern, so middleware is a natural implementation point.

The main lessons are:

```
measure the whole request pipeline
log in finally so failures are visible
use structured placeholders
place middleware deliberately
record useful metadata without recording secrets
```

The result is a small observability improvement that applies consistently across the API.

Reader Exercise

Run the API and make three requests:

```
a successful public request
an unauthorized protected request
an unknown route
```

For each request, confirm that the terminal shows method, path, final status, and elapsed time.

Also confirm that request bodies, passwords, tokens, authorization headers, and query strings do not appear.

Next Step

The next lesson can add focused middleware testing, improve log context with a safe request identifier, or address one of the remaining authorization gaps.

Day 81 - Review Structured Error Responses

Lesson Goal

Review how JobTrackr responds to invalid requests, authentication failures, missing resources, and unexpected server errors.

This is a review and testing lesson. No controller, service, DTO, middleware, or workflow file is changed.

Why Review Errors Separately

Successful responses demonstrate the normal path. Error responses define how clients understand and recover from problems.

An API error contract should answer:

```
What happened?
Which HTTP status applies?
Is the problem safe to show to the client?
Can the client parse the response consistently?
```

Day 81 finds that JobTrackr uses appropriate status codes and safe messages, but does not yet use one consistent response-body format.

Current Error Summary

Situation	Status	Current body
Invalid DTO validation	400 Bad Request	Structured validation JSON
Duplicate registration email	400 Bad Request	Plain-text message
Invalid login credentials	400 Bad Request	Plain-text message
Missing or invalid JWT	401 Unauthorized	Usually empty
Task not found	404 Not Found	Task not found.
User not found	404 Not Found	User not found.
Unexpected exception	500 Internal Server Error	An unexpected error occurred.

The status codes communicate the broad categories correctly. The inconsistency lies in whether the body is JSON, plain text, or absent.

Automatic DTO Validation

The API controllers use the ApiController attribute.

Request models use data annotations such as:

```
Required
EmailAddress
MinLength
MaxLength
```

ASP.NET Core checks these annotations before the controller action executes.

An invalid registration request can produce:

```
{
  "fullName": "",
  "email": "not-an-email",
  "password": "123"
}
```

The response is 400 Bad Request and includes field-level errors for FullName, Email, and Password.

A typical validation body includes:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "FullName": ["Full name is required."],
    "Email": ["Email is not valid."],
    "Password": ["Password must be at least 6 characters."]
  }
}
```

ASP.NET Core may also include fields such as type and traceId.

Why Automatic Validation Helps

Automatic validation creates several useful guarantees:

```
invalid DTOs do not reach application logic
field names are associated with their messages
multiple validation failures can be returned together
controllers need less repeated guard code
```

This is currently the most structured error response in JobTrackr.

Invalid Task Creation

Creating a task with an empty title also fails DTO validation:

```
{
  "title": "",
  "description": "Validation test",
  "priority": "Medium"
}
```

The API returns 400 Bad Request with a Title validation error before the task action performs normal creation work.

Because task endpoints are protected, the caller must still provide a valid JWT before reaching DTO validation for that endpoint.

Missing Authentication

TasksController uses the Authorize attribute.

Calling a protected route without a valid Bearer token returns:

```
401 Unauthorized
```

Authentication rejects the request before the task action runs. The response body is usually empty.

An empty 401 body is valid HTTP behavior, but it differs from the structured validation response and the plain-text application errors.

Invalid Login

Login with a registered email and incorrect password returns:

```
400 Bad Request
Invalid email or password.
```

The message deliberately does not reveal whether the email exists or whether only the password was incorrect.

This is an important security property. Detailed credential failure messages can help attackers discover registered accounts.

The message is safe, but the body is plain text rather than structured JSON.

Not-Found Responses

Requesting a missing task returns:

```
404 Not Found
Task not found.
```

Requesting a missing user returns:

```
404 Not Found
User not found.
```

These shared messages come from the Application project's ErrorMessages class.

Ownership-protected task operations also use 404 when the requested task belongs to another user. This avoids confirming the existence of another user's resource.

Unexpected Exceptions

ExceptionHandlingMiddleware catches exceptions that escape downstream code and returns:

```
500 Internal Server Error
An unexpected error occurred.
```

The middleware sets the content type to plain text.

The client does not receive:

```
stack traces
database details
connection strings
JWT signing keys
internal exception messages
```

The generic message prevents accidental disclosure of technical or sensitive information.

Request Logging And Errors

Day 80 request logging records the final status code for these requests.

Examples include:

```
HTTP POST /api/auth/register responded 400 in ... ms
HTTP GET /api/tasks responded 401 in ... ms
HTTP GET /api/tasks/999999 responded 404 in ... ms
```

The logger records request metadata without recording passwords, JWT tokens, authorization headers, or request bodies.

This gives the server operational visibility while preserving a safer client response.

What Is Already Correct

The review confirms several good behaviors:

```
invalid request data returns 400
```

```
missing authentication returns 401
missing or non-owned resources return 404
unexpected failures return 500
login errors do not reveal which credential failed
unexpected errors do not expose internal details
request logging records status without recording secrets
```

These behaviors should be preserved during any future response-format improvement.

The Main Inconsistency

Clients currently need to handle three response-body styles:

```
structured JSON for validation
plain text for many application and server errors
an empty body for some authentication failures
```

This makes client code more complicated because it cannot parse every error through one predictable contract.

The problem is not primarily the status codes. It is the representation of error details.

Why Not Fix It During The Review

Day 81 separates diagnosis from implementation.

Changing error responses can affect:

```
controllers
exception middleware
authentication behavior
frontend or API clients
tests
public documentation
```

The current behavior should be understood before selecting a standard format.

ASP.NET Core ProblemDetails is a candidate for Day 82 because it provides a recognized JSON structure without requiring a custom error model for every case.

Verified Baseline

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Git remains clean, and no commit is needed for this review-only day.

Testing Gap

The current fifteen tests focus mainly on application and infrastructure services.

They do not yet prove the HTTP shape of:

```
automatic validation responses
401 authentication responses
controller 404 responses
exception middleware 500 responses
```

Future integration tests can lock down those contracts after the project chooses a consistent format.

What This Day Teaches

Error handling has two related but separate concerns:

```
HTTP semantics: selecting the correct status code
response contract: returning predictable, parseable details
```

JobTrackr's status-code semantics are mostly sound. Its next opportunity is a consistent response contract that retains safe messages and useful validation detail.

Reader Exercise

Create an error-response table for another API.

For each endpoint failure, record:

```
status code
content type
response body shape
whether sensitive details are exposed
whether clients can parse it consistently
```

Separate genuine status-code problems from formatting inconsistencies before proposing changes.

Next Step

Evaluate ASP.NET Core ProblemDetails as a standard JSON format for validation, application, authentication, not-found, and unexpected-error responses without adding unnecessary complexity.

Day 82 - Add ProblemDetails Error Responses

Lesson Goal

Use ASP.NET Core ProblemDetails to make JobTrackr error responses more consistent.

Day 81 established that the API already used appropriate status codes but mixed structured JSON, plain text, and empty response bodies. Day 82 standardizes known controller errors and unexpected server failures without changing business logic.

What ProblemDetails Provides

ProblemDetails is a standard JSON representation for API errors.

A missing task can now return:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "Task not found."
}
```

The main fields are:

```
title: broad error category
status: HTTP status code
detail: safe explanation for the client
```

ASP.NET Core can also include standard fields such as type and instance when appropriate.

Scope Of The Change

Day 82 updates:

```
AuthController
TasksController
UsersController
ExceptionHandlingMiddleware
```

It does not:

```
install a package
create another error layer
change service interfaces
change successful responses
replace automatic DTO validation
customize 401 authentication challenges
expose exception messages or stack traces
```

The work stays focused on response consistency.

Known Bad Requests

Controllers previously returned exception messages as plain text:

```
return BadRequest(ex.Message);
```

They now return:

```
return Problem(
    statusCode: StatusCodes.Status400BadRequest,
    title: "Bad Request",
    detail: ex.Message);
```

The existing safe service message is preserved, but it is placed in a predictable JSON field.

Authentication Errors

Registration and login business errors now use ProblemDetails.

An invalid login can return:

```
{
  "title": "Bad Request",
  "status": 400,
  "detail": "Invalid email or password."
}
```

The message remains intentionally vague. It does not reveal whether the email exists or whether only the password was incorrect.

Duplicate registration and other known argument errors follow the same 400 response shape.

Task Not-Found Errors

TaskController now uses ProblemDetails wherever the shared task-not-found message is returned.

The pattern is:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.TaskNotFound);
```

This applies to task retrieval, update, deletion, completion, and reopening paths that return the shared missing-task result.

A missing or non-owned task therefore uses:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "Task not found."
}
```

The ownership behavior remains unchanged. Another user's task is still treated as not found.

User Not-Found Errors

UsersController applies the same pattern:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.UserNotFound);
```

The client receives:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "User not found."
}
```

User validation and business-rule errors also use the standard 400 structure.

Automatic Validation Remains Intact

Controllers use the ApiController attribute, and DTOs use data annotations.

Invalid DTOs already produce ValidationProblemDetails with an errors object:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": ["Email is not valid."],
    "Password": ["Password must be at least 6 characters."]
  }
}
```

This behavior is more detailed than ordinary ProblemDetails because validation must associate messages with specific fields.

Day 82 does not replace or duplicate it.

Unexpected Server Errors

ExceptionHandlingMiddleware previously returned a plain-text 500 response.

It now creates:

```
var problemDetails = new ProblemDetails
{
    Status = StatusCodes.Status500InternalServerError,
    Title = "Internal Server Error",
    Detail = "An unexpected error occurred."
};
```

The response content type is:

```
application/problem+json
```

The client receives safe JSON:

```
{
  "title": "Internal Server Error",
  "status": 500,
  "detail": "An unexpected error occurred."
}
```

Log The Real Exception On The Server

The middleware now receives ILogger for its own category and records the exception:

```
_logger.LogError(
    ex,
    "An unexpected error occurred while processing the request.");
```

This creates an important separation:

```
server log: exception object and technical diagnostic details
client response: generic, safe ProblemDetails message
```

Developers retain the information needed to investigate failures without exposing internals to API consumers.

Information Kept Out Of Responses

ProblemDetails responses must not contain:

```
password values
password hashes
JWT tokens
JWT signing keys
authorization headers
connection strings
stack traces
database exception details
internal exception messages
```

A structured response is not automatically safe. The values assigned to its fields still require careful judgment.

Why 401 Remains Unchanged

Missing or invalid authentication continues to return the standard:

```
401 Unauthorized
```

The body may be empty.

Customizing authentication challenge responses is possible, but it touches a different part of the ASP.NET Core pipeline. Day 82 avoids expanding scope simply to make every error look identical immediately.

Consistency is valuable, but narrow, understandable changes are valuable too.

Error Formats After Day 82

Error category	Response format
DTO validation	ValidationProblemDetails JSON
Known bad request	ProblemDetails JSON
Missing resource	ProblemDetails JSON
Unexpected exception	ProblemDetails JSON
Missing or invalid JWT	Standard 401 response

The API now has a substantially more predictable error contract while preserving specialized validation details.

Request Logging Still Works

RequestLoggingMiddleware remains outside exception handling in the pipeline.

When an unexpected exception occurs:

```
exception middleware logs technical details
exception middleware writes safe ProblemDetails
request logging records the final 500 status and elapsed time
```

These two logging responsibilities complement each other:

```
request log: what route failed and how long it took
exception log: why the server failed
```

Neither log should record credentials or tokens.

Scope Of The Commit

Commit 7dc3769 changes four files:

```
AuthController.cs
TasksController.cs
UsersController.cs
ExceptionHandlingMiddleware.cs
```

The commit contains 35 additions and 21 removals.

No service, domain, persistence, DTO, or workflow file changes.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
```

```
0 Warning(s)
0 Error(s)
```

The existing automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The test suite verifies that application behavior still works, although it does not yet assert the HTTP ProblemDetails shapes directly.

Testing Gap

Future integration tests should verify:

```
400 business errors use application/problem+json
404 responses include the expected title, status, and detail
500 responses hide exception details
automatic validation still includes its errors object
401 behavior remains intentional
```

These tests would protect the public error contract from accidental regression.

What This Day Teaches

Standardizing API errors improves predictability for clients.

The important principles are:

```
preserve correct status codes
use a recognized error representation
keep field-level validation detail
log exceptions on the server
return generic server errors to clients
avoid broad architectural changes for a focused improvement
```

ProblemDetails creates consistency without requiring a custom error DTO for every controller.

Reader Exercise

Call four failing endpoints:

```
invalid login
missing task
missing user
invalid registration DTO
```

Compare the content type and JSON shape.

Confirm that ordinary errors contain title, status, and detail, while validation adds an errors object. Also confirm that no password, token, stack trace, or database detail appears.

Next Step

Add focused integration tests for the ProblemDetails contract, or review whether the remaining standard 401 response needs safe JSON customization.

Day 83 - Add A Health Check Endpoint

Lesson Goal

Add a public health endpoint using ASP.NET Core's built-in health-check support.

The endpoint is:

```
GET /health
```

When the API process is running and can respond, it returns:

```
200 OK
Healthy
```

Why Health Checks Matter

Deployment platforms, load balancers, monitoring systems, and developers need a simple way to determine whether an application process is available.

Without a dedicated route, a monitoring tool may call a business endpoint that requires authentication, database data, or a specific request payload.

A health route provides a stable operational signal with a small response.

Scope Of Day 83

This lesson adds only a basic application health check.

It does:

```
use built-in ASP.NET Core health checks
add one public route
return a minimal response
change only Program.cs
```

It does not:

```
install a NuGet package
create a health controller
check SQL Server
require a JWT
expose configuration or secrets
redesign application startup
```

Register Health-Check Services

Program.cs now contains:

```
builder.Services.AddAuthorization();
builder.Services.AddHealthChecks();
```

AddHealthChecks registers the built-in health-check services with dependency injection.

Basic support is included with ASP.NET Core, so no external package is needed.

Map The Endpoint

The request pipeline now includes:

```
app.UseHttpsRedirection();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseAuthentication();
```

```
app.UseAuthorization();
app.MapHealthChecks("/health");
app.MapControllers();
```

MapHealthChecks creates the endpoint directly. A controller and action method are unnecessary.

Why The Endpoint Is Public

Monitoring systems must often check application availability before they can obtain or refresh an application token.

The health endpoint therefore works without a JWT.

Public access is safe here because the response reveals only:

```
Healthy
```

It does not reveal:

```
database names
connection strings
server names
JWT configuration
environment variables
application data
exception details
```

Health endpoints should provide enough information for operations without becoming an information-disclosure endpoint.

What This Basic Check Proves

The current health check confirms:

```
the JobTrackr API process has started
ASP.NET Core can accept an HTTP request
the routing pipeline can return a response
```

This is sometimes called a liveness-style signal.

It answers a narrow question:

```
Is the web application running and responsive?
```

What It Does Not Prove

The endpoint does not confirm:

```
SQL Server is available
database credentials are valid
migrations are current
external services are available
every controller action works
authentication can issue tokens
```

A healthy response should not be interpreted as proof that every dependency and feature is operational.

Dependency checks can be added later when deployment requirements justify them.

Live Verification

The API was started locally on a temporary HTTP port, and the health endpoint was called without an authorization token.

The verified response was:

```
StatusCode  : 200
Content     : Healthy
```

```
ContentType : text/plain
```

This confirms that the route is mapped, public, and returns the expected built-in response.

The temporary server was stopped after verification.

Request Logging Integration

Because RequestLoggingMiddleware wraps mapped endpoints, a health request also appears in the server logs:

```
HTTP GET /health responded 200 in ... ms
```

This can help operators see when monitoring systems are checking the API.

Elapsed time varies between machines and requests.

Swagger Behavior

The health endpoint may not appear in Swagger.

Swagger primarily discovers API actions described through controllers and endpoint metadata. The health route is mapped directly through middleware-oriented endpoint configuration.

Its absence from Swagger does not mean the endpoint is unavailable.

It can be tested with:

```
a web browser
PowerShell
curl
an uptime monitor
a deployment platform health probe
```

PowerShell Test

With the API running, a local check can use:

```
Invoke-WebRequest -Uri "https://localhost:PORT/health" -SkipCertificateCheck
```

The important values are:

```
StatusCode : 200
Content    : Healthy
```

The actual port comes from the API startup output.

Why No Database Check Yet

Database health checks add a stronger readiness signal, but they also add concerns:

```
connection timeouts
database load from frequent probes
credentials and configuration
whether temporary database failure should remove the API from service
separate liveness and readiness semantics
```

Day 83 avoids pretending those operational decisions are trivial.

The basic check is useful on its own and creates a clean foundation for later readiness checks.

Scope Of The Commit

Commit 1d6f8d9 changes only Program.cs.

It adds two lines:

```
service registration through AddHealthChecks
route mapping through MapHealthChecks
```

No controller, service, domain, persistence, authentication, or workflow file changes.

Verified Build And Tests

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The existing tests do not directly test the health endpoint, so the live HTTP verification provides evidence for the new route.

Future Test Opportunity

An integration test could start the application in memory and call:

```
GET /health
```

It could assert:

```
status is 200
body is Healthy
authentication is not required
no sensitive data is returned
```

This would protect the operational contract automatically.

What This Day Teaches

A useful health check should have a clear meaning.

The important principles are:

```
keep the first health signal simple
use framework support when it fits
avoid exposing internal details
make public probes safe
distinguish process health from dependency health
verify the route through a real HTTP request
```

The endpoint is small, but it is a practical step toward deployment and monitoring.

Reader Exercise

Run the API and call the health endpoint without a JWT.

Confirm:

```
HTTP status is 200
response body is Healthy
content type is text/plain
request logging records the call
the response contains no configuration or private data
```

Then explain in one sentence what the endpoint proves and what it does not prove.

Next Step

Add an automated integration test for the health endpoint, or design separate liveness and readiness checks before introducing database dependency checks.

Day 84 - Review Security Basics

Lesson Goal

Review JobTrackr's password, JWT, authentication, and authorization security, then fix one concrete configuration problem.

The review found that the development JWT signing key was tracked in appsettings.json.

Day 84 rotates that key, stores the replacement in .NET User Secrets, and removes the key from tracked configuration.

Security Review Summary

Area	Result	Notes
Password hashing	Pass	Passwords are hashed before storage
Password verification	Pass	Login verifies the submitted password against the stored hash
Password exposure	Pass	Response DTOs do not return passwords or hashes
JWT validation	Pass	Issuer, audience, lifetime, signature, and signing key are validated
JWT key storage	Fixed	The local key was removed from tracked configuration
Task authentication	Pass	TasksController requires authentication
Task ownership	Partial	List, get, create, update, and delete use the authenticated user
Complete/Reopen ownership	Not complete	Authentication is required, but ownership is not checked
User endpoint authorization	Not complete	UsersController remains public
Health endpoint	Intentional	The public route returns only Healthy

An honest security review distinguishes verified protection from partial or planned protection.

Why A JWT Signing Key Is Sensitive

The signing key creates and validates JWT signatures.

Anyone who obtains it may be able to create tokens that the API trusts. A signing key must therefore never be committed to a public repository.

Once a key has been tracked, removing it from the latest file is not enough. Earlier Git history may still contain it.

The safe response is:

```
treat the old value as compromised
generate a new value
store the replacement outside Git
stop using tokens signed with the old key
```

Initialize User Secrets

The API project is initialized for .NET User Secrets.

The project file contains a UserSecretsId:

```
<UserSecretsId>generated-identifier</UserSecretsId>
```

The identifier is safe to commit. It is a lookup identifier, not the signing key.

The actual secret value is stored outside the repository in the current developer's user profile.

Generate And Store A New Key

A new random local key is generated and stored under:

```
Jwt:Key
```

The value must not be copied into:

```
source code
appsettings files
documentation
commit messages
screenshots
videos
chat messages
```

After storage, temporary shell variables containing the value should be removed.

Remove The Tracked Key

Tracked appsettings now keeps only non-secret JWT configuration:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr"
  }
}
```

The Key property is absent.

Verification confirmed:

```
appsettings contains only Issuer and Audience under Jwt
the project has a UserSecretsId
the local Jwt:Key secret name exists
the old development key is absent from current tracked files
```

The secret value itself was not printed or copied during verification.

Configuration Precedence

During local development, ASP.NET Core loads User Secrets and combines them with the other configuration sources.

The application still reads:

```
var key = configuration["Jwt:Key"];
```

The authentication and token services do not need a different interface. Only the configuration source changes.

This is a useful design property: application code requests a configuration key without needing to know whether its value came from JSON, User Secrets, an environment variable, or a production secret manager.

Old Tokens Become Invalid

Tokens signed with the removed key cannot be validated with the new key.

After rotation, users must log in again to receive newly signed tokens.

This is expected security behavior:

```
old signing key removed
new signing key loaded
old token signature no longer matches
new login produces a valid token
```

Key rotation can invalidate active sessions, so production rotations should be planned deliberately.

Password Hashing Review

JobTrackr uses ASP.NET Core's PasswordHasher.

Registration calls HashPassword before saving a user:

```
plain password
    -> PasswordHasher
    -> PasswordHash stored on User
```

The User entity stores PasswordHash rather than Password.

Login calls VerifyPassword with the submitted password and stored hash.

The password-hashing implementation is appropriate for the current project and does not need replacement during this review.

Password Exposure Review

Authentication and user response DTOs do not include:

```
Password
PasswordHash
```

Request logging records method, path, status, and elapsed time rather than request bodies.

This keeps passwords out of:

```
API response JSON
request logs
normal application output
```

Hashing protects stored passwords, while careful DTO and logging design prevents accidental disclosure through other channels.

JWT Creation And Validation

JwtTokenService reads issuer, audience, and key from configuration.

It creates identity claims for:

```
user identifier
email
full name
```

Tokens are signed with HMAC SHA-256 and currently expire after one hour.

The API validates:

```
issuer
audience
lifetime
signature
```

```
signing key
```

If the key is missing, token generation fails rather than silently creating an unsigned token.

Task Authentication

TasksController uses the Authorize attribute.

Anonymous access to task routes returns:

```
401 Unauthorized
```

For list, get, create, update, and delete operations, the authenticated user identifier is extracted from the token and passed into the application service.

These operations are designed around the current user's ownership boundary.

Known Task Ownership Gap

Complete and Reopen require authentication because the controller is protected.

However, those actions currently call service methods without the authenticated user identifier.

Therefore:

```
authentication: complete
ownership verification: not complete
```

The distinction matters. Requiring any valid user token is not the same as proving that the caller owns the requested task.

This gap should be fixed through separate focused implementation and tests.

Public User Endpoints

UsersController is currently not protected by authorization.

That means its CRUD routes should not be described as secured simply because task routes use JWT authentication.

Future work must define who is allowed to:

```
list users
retrieve user details
create users outside registration
update a user
delete a user
view a user's task route
```

Authorization policy should be designed before adding attributes mechanically.

Public Health Endpoint

The health route remains intentionally public:

```
GET /health
```

It returns only:

```
Healthy
```

This allows deployment and monitoring systems to check process health without a user token while revealing no database, configuration, token, or user information.

Local And Production Secrets

.NET User Secrets are intended for local development.

A deployed environment should supply the signing key through a secure platform setting or secret manager.

ASP.NET Core maps:

```
Jwt__Key
```

to:

```
Jwt:Key
```

The double underscore allows hierarchical configuration keys to be represented in environment variables.

No deployment configuration is added during Day 84.

Scope Of The Commit

Commit cc23ece changes two tracked files:

```
JobTrackr.Api.csproj
appsettings.json
```

The project file adds the User Secrets identifier. The appsettings file removes the tracked key while preserving issuer and audience.

The local secret storage is outside the repository and is never committed.

No controller, service, entity, middleware, database, or workflow file changes.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The local secret name exists, and the old tracked development key is absent from current repository files.

Remaining Test Gaps

The current tests verify password hashing and authentication service behavior, but future integration coverage could verify:

```
API startup fails clearly when the JWT key is absent
login creates a token using runtime configuration
anonymous task requests return 401
newly issued tokens can access protected routes
old tokens fail after key rotation
```

Secrets used by tests should also come from isolated test configuration, not a developer's personal User Secrets.

What This Day Teaches

Security reviews should produce specific evidence and scoped fixes.

The main lessons are:

```
hash passwords before storage
do not expose hashes in responses
validate JWT signatures and lifetime
never commit signing keys
rotate a key after exposure
separate local and production secret storage
distinguish authentication from ownership authorization
state known security limitations honestly
```

Moving a secret out of Git is a small code change with meaningful security value.

Reader Exercise

Audit another project for secret-bearing configuration.

For every sensitive setting, identify:

```
where the application reads it
where local development stores it
where production should store it
whether it has ever entered Git history
how it would be rotated
```

Do not print or copy actual secret values while performing the audit.

Next Step

Add ownership enforcement to Complete and Reopen, or design authorization rules for the currently public user endpoints with focused automated tests.

Day 85 - Clean Up Application Settings

Lesson Goal

Organize JobTrackr configuration by purpose, environment, and sensitivity.

After Day 85:

```
appsettings.json contains shared non-secret settings
appsettings.Development.json contains local development database configuration
.NET User Secrets contains the local JWT signing key
launchSettings.json starts with the HTTPS profile
tracked JSON contains no signing key or database password
```

No C# code needs to change because ASP.NET Core already combines these configuration sources through IConfiguration.

Why Configuration Organization Matters

Configuration often starts in one file because that is convenient during early development.

As an application grows, settings begin to differ in important ways:

```
some settings are shared across environments
some belong only to local development
some are secrets and must stay outside Git
some control local tooling rather than application behavior
```

Treating every value the same eventually creates security and deployment problems.

The Configuration Categories

JobTrackr now separates settings into four categories.

Shared Settings

Shared, non-secret values belong in appsettings.json.

Examples:

```
JWT issuer
JWT audience
logging levels
allowed hosts
```

Development Settings

Values specific to the local Development environment belong in appsettings.Development.json.

The local SQL Server connection string fits this category because it targets the developer's machine.

Local Secrets

Sensitive local values belong in .NET User Secrets.

The JWT signing key remains outside the repository under:

```
Jwt:Key
```

Launch Tooling

Local launch-profile behavior belongs in launchSettings.json.

This file controls which profile and URLs are convenient during local development. It is not production hosting configuration.

Shared appsettings.json

The shared settings file now contains:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

It intentionally does not contain:

```
JWT signing key
database connection string
database password
user credentials
issued JWT token
```

This file can be shared by Development and future deployed environments without assuming a local database location.

Development appsettings

appsettings.Development.json now contains the local SQL Server connection:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

The connection uses Windows authentication and does not contain a username or password.

That makes it acceptable for this tracked development file.

If a future connection string includes a password, the entire connection string should move to User Secrets or another secure provider.

Why Move The Connection String

The local connection string is not a universal application setting.

It assumes:

```
SQL Server is available on localhost
the database is named JobTrackrDb
Windows authentication is available
the local certificate should be trusted
```

These assumptions belong to the Development environment, not every environment where JobTrackr may run.

Moving the value makes future deployment configuration clearer.

Keep The JWT Key In User Secrets

Day 84 removed the tracked signing key and created a local replacement in .NET User Secrets.

Day 85 preserves that boundary.

Verification confirms:

```
the local Jwt:Key secret name exists
no tracked JSON file contains a JWT key
the real secret value was not printed
```

The key does not move into appsettings.Development.json merely because that file is development-specific. It remains a tracked file, so it is not a secret store.

HTTPS As The Default Profile

The launch profiles now appear in this order:

```
https
http
IIS Express
```

The HTTPS profile listens on:

```
https://localhost:7024
http://localhost:5123
```

Program.cs uses HTTPS redirection.

When only the HTTP profile is selected, ASP.NET Core may not have an HTTPS address to use and can log:

```
Failed to determine the https port for redirect.
```

Making the existing HTTPS profile first gives local startup a valid redirect target while preserving the optional HTTP profile.

What Launch Settings Do Not Control

launchSettings.json is used by local development tooling.

It does not define:

```
production reverse proxy configuration
container port mappings
cloud platform health probes
production TLS certificates
deployment environment variables
```

Reordering a local profile improves development startup without changing production authentication or middleware behavior.

Configuration Loading Order

In Development, ASP.NET Core combines several sources.

For JobTrackr, the important progression is:

```
appsettings.json
appsettings.Development.json
User Secrets
environment variables
command-line values
```

Later sources can override values from earlier sources.

The resulting configuration includes:

```
Jwt:Issuer from appsettings.json
Jwt:Audience from appsettings.json
ConnectionStrings:DefaultConnection from appsettings.Development.json
Jwt:Key from User Secrets
```

Program.cs continues reading these values through builder.Configuration.

Why Program.cs Does Not Change

The application asks configuration for logical keys:

```
builder.Configuration.GetConnectionString("DefaultConnection")
builder.Configuration["Jwt:Issuer"]
builder.Configuration["Jwt:Audience"]
builder.Configuration["Jwt:Key"]
```

The code does not need to know which provider supplied each value.

This separates application behavior from environment-specific storage.

The same code can later use environment variables or a managed secret store without rewriting authentication or persistence services.

JSON Validation

All three edited JSON files were parsed successfully:

```
appsettings.json
appsettings.Development.json
launchSettings.json
```

Parsing configuration during review catches missing commas, braces, or quotation marks before runtime startup.

Secret Scan

Current tracked JSON was searched for:

```
JWT key properties
database Password values
database Pwd values
```

The result:

```
no JWT key found in tracked JSON
no database password found in tracked JSON
```

The development connection string is present, but it uses password-free Windows authentication.

Development Runtime Behavior

When the API runs with the HTTPS launch profile and the Development environment:

```
shared JWT metadata loads from appsettings.json
the local database connection loads from appsettings.Development.json
the signing key loads from User Secrets
HTTPS and HTTP addresses are available
```

This configuration supports:

```
health checks
SQL Server-backed login
JWT token creation
authenticated task requests
HTTPS redirection
```

Old JWT tokens from before the Day 84 key rotation remain invalid and users must log in again.

Production Configuration

appsettings.Development.json is loaded only when the environment is Development.

A future deployment should provide its own values through environment variables or a secret manager.

ASP.NET Core maps:

```
ConnectionStrings__DefaultConnection
Jwt__Key
```

to:

```
ConnectionStrings:DefaultConnection
Jwt:Key
```

The double underscore represents a configuration hierarchy in environment-variable form.

Production configuration is deliberately outside Day 85.

Scope Of The Commit

Commit cd2bde5 changes three files:

```
appsettings.json
appsettings.Development.json
Properties/launchSettings.json
```

The commit contains 6 additions and 12 removals.

No changes are made to:

```
Program.cs
controllers
services
entities
migrations
packages
GitHub Actions
```

This is a configuration organization change rather than an application behavior change.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The repository working tree is clean after commit cd2bde5.

Operational Boundaries

This cleanup improves organization, but it does not create production configuration.

Before deployment, the project still needs decisions about:

```
production SQL Server location
production database credentials
JWT key storage and rotation
allowed hosts
logging destinations and retention
HTTPS termination
```

```
environment-specific health checks
```

Keeping those concerns out of local files is the first useful step.

What This Day Teaches

Configuration should be grouped by meaning rather than convenience.

The main lessons are:

```
shared non-secret settings belong in shared configuration
local environment values belong in environment configuration
secrets belong outside Git
launch profiles are local tooling
configuration providers let code remain unchanged
tracked files should be scanned for accidental secrets
```

Good configuration organization makes both security and deployment easier to reason about.

Reader Exercise

Audit the configuration sources in another ASP.NET Core project.

Classify each setting as:

```
shared and non-secret
environment-specific and non-secret
local secret
production secret
local launch tooling
```

Move only one category at a time, validate every JSON file, scan tracked files for secrets, and run the build and tests afterward.

Next Step

Add configuration validation with clear startup errors, or begin designing production-ready environment variables and secret-provider requirements without committing real deployment credentials.

Day 86 - Document The Local Environment

Lesson Goal

Document everything another developer needs to configure and run JobTrackr locally.

The repository README now covers:

```
required development software
dependency restoration
local SQL Server configuration
EF Core migrations
JWT signing-key setup
development HTTPS trust
build and test commands
API startup
health verification
registration and login
Bearer-token requests
local secret safety
```

Day 86 changes documentation only. It does not change application behavior.

Why Local Setup Documentation Matters

A repository is not fully usable when only its original developer knows how to run it.

Source code may compile, but a new developer can still become blocked by:

```
missing SDKs
missing command-line tools
database configuration
unapplied migrations
missing secret values
untrusted HTTPS certificates
unknown ports
authentication requirements
```

A good setup guide turns those hidden assumptions into explicit, repeatable steps.

The Configuration Model

JobTrackr now loads local configuration from several places:

Purpose	Source
Shared non-secret settings	appsettings.json
Local SQL Server settings	appsettings.Development.json
Local JWT signing key	.NET User Secrets
Development URLs and profiles	launchSettings.json

The README explains each source without publishing the real JWT key.

Prerequisites

The local guide lists:

```
.NET SDK capable of building net8.0
SQL Server
SQL Server Management Studio or another SQL client
EF Core command-line tools 8.x
Git
```

Developers can verify their tools:

```
dotnet --version
dotnet ef --version
```

If the EF Core tool is absent, the documented installation command pins the existing project-compatible version:

```
dotnet tool install --global dotnet-ef --version 8.0.27
```

Matching the major EF Core version reduces tooling and migration compatibility surprises.

Restore Dependencies

From the repository root, restore the production and test projects:

```
dotnet restore src\JobTrackr.Api\JobTrackr.Api.csproj
dotnet restore tests\JobTrackr.Tests\JobTrackr.Tests.csproj
```

The guide uses a placeholder repository path rather than the original developer's local path.

This keeps the instructions portable between machines.

Configure SQL Server

The Development environment reads its connection string from:

```
src/JobTrackr.Api/appsettings.Development.json
```

The default local connection targets:

```
Server: localhost
Database: JobTrackrDb
Authentication: Windows authentication
```

The tracked development connection string contains no database username or password.

The guide explains that SQL Server must be running and the current Windows user must be allowed to create or update the database.

Apply EF Core Migrations

The documented migration command is:

```
dotnet ef database update --project src\JobTrackr.Infrastructure --startup-project src\
JobTrackr.Api
```

The two project arguments have different responsibilities:

```
Infrastructure project: contains AppDbContext and migrations
API startup project: supplies runtime configuration and dependency setup
```

Applying migrations creates or updates JobTrackrDb using the migration history already stored in the repository.

Configure A Local JWT Signing Key

The repository intentionally contains no JWT signing key.

Each developer generates a unique local value:

```
$jwtKey = [Guid]::NewGuid().ToString("N") + [Guid]::NewGuid().ToString("N")
dotnet user-secrets set "Jwt:Key" "$jwtKey" --project src\JobTrackr.Api
Remove-Variable jwtKey
```

The command generates the value on the developer's machine instead of distributing a shared development secret through documentation.

The UserSecretsId is tracked, but the key value is stored outside the repository in the current user's profile.

Why The README Does Not Include A Sample Key

A copyable signing key in documentation tends to become a shared secret across multiple machines and environments.

Generating a fresh value provides better habits:

```
each developer has a different key
the key never enters Git
the key is not exposed in screenshots
the key is not copied through chat
tokens are tied to the developer's local environment
```

The setup guide documents the process, not the secret.

Trust Development HTTPS

The README includes:

```
dotnet dev-certs https --trust
```

This trusts the local ASP.NET Core development certificate after the user accepts the operating-system prompt.

The step prevents avoidable browser and PowerShell certificate errors while testing the HTTPS launch profile.

Development certificates are for local use and are not production TLS certificates.

Build And Test

The documented commands are:

```
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release
```

The verified result is:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 15
Failed: 0
Skipped: 0
```

The service tests use EF Core InMemory and do not connect to SQL Server.

This means developers can verify the existing automated behavior independently from local database availability.

Run The API

The README starts JobTrackr through the HTTPS profile:

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

The default local addresses are:

```
https://localhost:7024
http://localhost:5123
```

The HTTPS profile supplies a valid redirect target for UseHttpsRedirection.

Swagger And Health URLs

Swagger is available in the Development environment at:

```
https://localhost:7024/swagger
```

The public health endpoint is:

```
https://localhost:7024/health
```

Its expected response is:

```
Healthy
```

These URLs give a new developer two quick startup checks:

```
Swagger proves the development API documentation is available
health proves the application process can answer a request
```

Registration And Login

The setup guide describes the authentication sequence:

```
POST /api/auth/register
    -> create a local user

POST /api/auth/login
    -> authenticate with email and password
    -> receive a JWT
```

The token is then sent as:

```
Authorization: Bearer TOKEN
```

Without a valid token, protected task routes return 401 Unauthorized.

Accurate Swagger Limitation

The README does not claim that Swagger currently has a Bearer-token Authorize button.

The existing Swagger registration has not added a Bearer security definition.

Swagger can still be used for registration and login, while an authenticated task request can be sent through PowerShell:

```
$token = "PASTE_LOGIN_TOKEN_HERE"
Invoke-RestMethod -Uri "https://localhost:7024/api/tasks" -Headers @{ Authorization = "Bearer
$token" }
Remove-Variable token
```

The placeholder is intentionally not a real token.

Why Remove The Token Variable

The example removes the temporary shell variable after use.

This does not erase every possible command-history or process-memory trace, but it avoids leaving the token available in the active PowerShell session.

The larger safety rule remains:

```
do not commit tokens
do not paste tokens into documentation
do not publish terminal output containing tokens
```

Local Configuration Safety

The README explicitly prohibits committing:

```
JWT signing keys
database passwords
access tokens
user passwords
production connection strings
```

Local secrets belong in User Secrets.

Future deployment secrets should come from environment variables or a secure secret manager.

Technical Accuracy Checks

The documentation was checked against the repository:

```
API target framework: net8.0
EF Core command-line tools: 8.0.27
database: JobTrackrDb
local database authentication: Windows authentication
HTTPS port: 7024
HTTP port: 5123
Swagger route: /swagger
health route: /health
current test count: 15
task endpoints: Bearer authentication required
Swagger Bearer button: not configured
```

This prevents the guide from promising features or paths that do not exist.

Secret Scan

Tracked JSON and Markdown were checked for:

```
the removed development JWT key
database Password values
database Pwd values
```

No old key or database password was found.

The README contains only a command that generates a new random JWT key on each developer's machine.

Scope Of The Commit

Commit f60f13c changes only README.md:

```
174 lines added
6 lines removed
```

No changes are made to:

```
C# source code
configuration files
migrations
packages
database model
GitHub Actions
```

The narrow scope makes the documentation update easy to review.

What Was Verified

The local environment section was compared with:

```
project target framework
installed EF Core tool version
development connection settings
launch profile URLs
health route
Swagger configuration
authentication design
```

```
current test count
tracked secret scan
```

The documented Release build and test commands completed successfully.

The database-update command is intentionally an environment-changing operation. Developers should run it against their own intended local SQL Server instance after reviewing the connection configuration.

What This Day Teaches

Environment documentation is part of software delivery.

The main lessons are:

```
document prerequisites explicitly
separate restore, database, secret, certificate, and startup steps
use placeholders instead of personal machine paths
generate secrets locally instead of publishing samples
state tooling limitations accurately
verify documentation against the repository
run safe documented commands before publishing
```

A reliable setup guide reduces onboarding time and proves that the project is understandable beyond the machine where it was created.

Reader Exercise

Follow the README from a fresh PowerShell session.

Record every step that depends on knowledge not written in the guide.

For each gap, add:

```
the required prerequisite
the exact command
the expected result
a safe troubleshooting note
```

Never add a real credential merely to make setup appear easier.

Next Step

Add Swagger Bearer-token support so authenticated API testing can be completed directly in Swagger, or create an automated integration test that verifies startup and the health route using isolated configuration.

Day 87 - Manual API Regression Test

Lesson Goal

Manually test the current JobTrackr API from startup through its main success, validation, authentication, ownership, and error paths.

Day 87 verifies:

```
API health
registration and login
JWT authentication
request validation
task create, read, update, delete, complete, and reopen
task search and completion filters
two-user task ownership
user CRUD
structured 400 and 404 responses
anonymous 401 responses
safe request logging
the complete automated test suite
```

This is a testing day. No repository file changes when the API behaves as expected.

Why Manual Regression Testing Still Matters

Automated tests are repeatable and fast, but the current suite focuses on application and infrastructure services.

It does not yet exercise the complete HTTP pipeline:

```
routing
DTO model binding
automatic validation
JWT authentication
controller responses
ProblemDetails serialization
request logging
SQL Server persistence
HTTPS hosting
```

A focused manual regression run connects those pieces and verifies that the API behaves as a complete system.

Keep Known Limitations Visible

Task ownership currently protects:

```
Create
Get All
Get By Id
Update
Delete
```

Complete and Reopen require authentication but do not pass the current user identifier to their service methods.

UsersController also remains public.

The regression test records those facts instead of presenting the API as fully authorized.

Start From A Known Baseline

Before making requests, build and run the API through the HTTPS profile:

```
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Expected addresses:

```
https://localhost:7024
http://localhost:5123
```

The test uses the HTTPS address.

Use Unique Test Data

Two test users are created with timestamp-based email addresses:

```
Regression User A
Regression User B
```

Unique addresses prevent duplicate-email failures from earlier regression runs.

The data belongs to the local development database and should never contain a real person's credentials.

Health Check

The first request is:

```
GET /health
```

Expected result:

```
200 OK
Healthy
```

Testing health first confirms the API process is reachable before authentication and persistence behavior are tested.

Request logging should record the call without exposing configuration.

Register Two Users

The regression creates User A and User B through:

```
POST /api/auth/register
```

Each response should include safe identity information:

```
user id
full name
email
```

It must not include:

```
plain password
password hash
JWT signing key
```

Creating two accounts provides the identities needed to test ownership isolation.

Registration Validation

An invalid registration uses:

```
{
  "fullName": "",
  "email": "invalid-email",
  "password": "123"
}
```

Expected result:

```
400 Bad Request
ValidationProblemDetails with field errors
no user created
```

This checks ApiController validation before ordinary controller logic executes.

Login And Token Handling

Both users log in through:

```
POST /api/auth/login
```

Each successful response returns a non-empty JWT.

The tokens are assigned to temporary PowerShell variables and used only through Authorization headers:

```
Authorization: Bearer TOKEN
```

Token values should not be printed, included in screenshots, committed, or copied into documentation.

Anonymous Access

Calling the task collection without a token must return:

```
401 Unauthorized
```

This proves the TasksController authorization boundary is active at the HTTP layer.

An authenticated route that accidentally succeeds anonymously is a high-priority regression.

Create A User-Owned Task

User A creates a task through:

```
POST /api/tasks
```

The request includes:

```
title
description
due date
priority
```

It intentionally does not include UserId.

The API reads User A's identifier from the JWT claim and assigns ownership internally.

Expected result:

```
201 Created
submitted values returned
task belongs to User A
task begins incomplete
```

Invalid Task Validation

Creating a task with an empty title should return:

```
400 Bad Request
field-level Title validation error
```

The request still includes User A's valid token because authentication and DTO validation are separate concerns.

Task Query Regression

User A verifies:

```
Get All returns the new task
search finds the task by title
isCompleted=false includes the task
combined incomplete and search filters include it
Get By Id returns the correct task
```

This checks both query behavior and the authenticated-user scope.

Two-User Ownership Isolation

User B then attempts to observe User A's task.

Expected behavior:

```
User B's list does not include User A's task
User B Get By Id returns 404
ProblemDetails says Task not found
```

Returning 404 instead of 403 avoids confirming that another user's resource exists.

This is an important test because successful authentication alone does not prove correct authorization.

Update Ownership

User A updates:

```
title
description
due date
priority
```

Expected result:

```
200 OK
updated values returned
```

User B then attempts the same update and receives 404.

This verifies that TaskService receives the authenticated user identifier for updates.

Delete Ownership

User B attempts to delete User A's task and receives:

```
404 Not Found
```

User A can still retrieve the task afterward.

This confirms the rejected request did not mutate the resource.

Testing the final state is as important as checking the status code.

Complete And Reopen

User A completes the task:

```
PATCH /api/tasks/{id}/complete
```

The response should show:

```
isCompleted = true
```

The completed filter should include it.

User A then reopens the task:

```
PATCH /api/tasks/{id}/reopen
```

The response should show:

```
isCompleted = false
```

Why Cross-User Complete/Reopen Is Not Run

User B is deliberately not used to call Complete or Reopen on User A's task.

Those endpoints currently lack ownership checks and could mutate User A's data.

Running a known unsafe request merely to prove the vulnerability would create unwanted development data changes.

The correct regression note is:

```
authentication is required
ownership is not enforced
focused implementation and tests are still needed
```

User CRUD Regression

A separate temporary user is used for public user CRUD testing.

The flow verifies:

```
POST creates a user with 201
GET collection includes the user
GET by id returns the user
GET user tasks returns an empty collection
PUT updates the user
DELETE returns 204
GET after deletion returns structured 404
```

Separate temporary data prevents this portion of the test from deleting either authenticated regression account.

Public User Endpoint Limitation

These requests currently work without a JWT.

That is existing behavior, not a security success.

The regression test confirms functional CRUD while keeping the authorization limitation explicit:

```
user CRUD behavior works
user CRUD authorization remains incomplete
```

Future work must define who may list, update, and delete users.

Delete The Task

User A deletes their own task:

```
DELETE /api/tasks/{id}
```

Expected result:

```
204 No Content
```

A later Get By Id should return:

```
404 Not Found
ProblemDetails detail: Task not found.
```

This completes the task lifecycle and confirms deletion persists.

Review Request Logs

The API terminal should contain a mix of successful and failed status codes:

```
GET /health -> 200
POST /api/auth/register -> 200
GET /api/tasks without token -> 401
POST /api/tasks -> 201
cross-user task operation -> 404
DELETE /api/tasks/{id} -> 204
```

Logs should not contain:

```
passwords
JWT tokens
Authorization headers
request bodies
connection strings
JWT signing keys
```

The regression verifies both observability and confidentiality.

Clear Sensitive Shell Variables

After testing, temporary variables containing tokens, headers, passwords, and request bodies are removed from the active PowerShell session.

Examples include:

```
tokenA
tokenB
headersA
headersB
password
login request bodies
registration request bodies
```

Removing variables is a useful cleanup step, though developers should also avoid publishing command history or terminal output containing secrets.

Automated Test Result

After the manual flow, the complete test project is run in Release configuration.

The verified result is:

```
Passed: 15
Failed: 0
Skipped: 0
```

Manual and automated testing provide different evidence:

```
manual regression: complete HTTP and SQL Server workflow
automated tests: repeatable service-level behavior
```

Both are valuable until broader integration tests are added.

Regression Checklist

The Day 87 checklist covers:

```
health returns Healthy
two users register and log in
invalid registration returns 400
anonymous task access returns 401
task creation derives ownership from JWT
invalid task returns 400
task list and filters work
User B cannot list, get, update, or delete User A's task
User A can complete and reopen
```

```
temporary user CRUD works
missing resources return structured 404
task deletion works
logs contain metadata but no secrets
all automated tests pass
```

A written checklist makes the manual run reproducible rather than relying on memory.

No Repository Change

Day 87 does not change source code or tracked documentation in the JobTrackr repository.

The latest source commit remains:

```
f60f13c Day 86: Document local environment setup
```

Git remains clean, so no empty commit is created.

The manual test does create temporary rows in the local development database. Those rows are runtime test data, not repository changes.

What This Day Teaches

Regression testing should verify both success and failure behavior.

The main lessons are:

```
start with a known running baseline
use unique and disposable test data
test unauthenticated and authenticated paths
use two identities for ownership checks
verify state after rejected mutations
do not perform knowingly unsafe mutations
keep known security gaps explicit
review logs for both status coverage and secret safety
combine manual HTTP testing with automated tests
avoid empty commits on review-only days
```

The result is an evidence-based picture of what JobTrackr currently protects and what still needs work.

Reader Exercise

Turn the manual checklist into a reusable test worksheet.

For each request, record:

```
acting user
HTTP method and route
expected status
expected response shape
expected database state
whether ownership is involved
whether sensitive data could appear
```

Then identify the highest-value cases to convert into automated integration tests.

Next Step

Fix ownership checks for Complete and Reopen with two-user automated tests, or begin protecting UsersController through explicit authorization requirements.

Day 88 - Cleanup Before The Month Review

Lesson Goal

Perform a small, behavior-preserving cleanup before the month review.

The repository audit found no dead starter code that needed deletion. Day 88 therefore focuses on formatting consistency, whitespace cleanup, and verification.

The key principle is:

```
do not delete valid code merely to make a cleanup commit look substantial
```

Repository Audit

The source and test folders were searched while excluding generated bin and obj directories.

The audit found:

```
no WeatherForecast starter code
no Class1 placeholder files
no empty C# source files
no TODO placeholders
no HACK placeholders
no NotImplementedException
no untracked changes
no obvious dead application classes
```

This is a successful cleanup result. A repository does not require deletion when the files are already meaningful.

Why Generated Folders Are Excluded

The bin and obj directories contain compiler and build-tool output.

Searching them can produce misleading matches because generated files may contain:

```
copied source metadata
generated assembly information
compiler caches
intermediate files
restored package artifacts
```

Those folders are ignored by Git and are not part of the hand-maintained source audit.

Cleanup Scope

Day 88 changes only:

```
ProblemDetails call formatting
one unnecessary blank line
```

It does not change:

```
API routes
method parameters
status codes
error messages
authorization attributes
service calls
DTO properties
database entities
migrations
settings
tests
```

This narrow scope makes behavioral review straightforward.

Consistent ProblemDetails Formatting

Several controller responses were written on one long line:

```
return Problem(statusCode: StatusCodes.Status400BadRequest, title: "Bad Request", detail: ex.
Message);
```

They now use a consistent multi-line style:

```
return Problem(
    statusCode: StatusCodes.Status400BadRequest,
    title: "Bad Request",
    detail: ex.Message);
```

The same pattern is used for not-found errors:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.TaskNotFound);
```

Only presentation changes. The named arguments and their values remain identical.

AuthController Cleanup

The Register and Login actions both catch known argument errors.

Their 400 ProblemDetails calls are now formatted over multiple lines.

The following behavior remains unchanged:

```
status: 400 Bad Request
title: Bad Request
detail: existing safe service message
```

Registration and login logic are untouched.

TasksController Cleanup

ProblemDetails formatting is standardized in:

```
Create
Update
Delete
Complete
Reopen
```

The existing Get By Id response already used the desired layout.

The cleanup does not change:

```
authenticated user extraction
ownership filtering
task service calls
404 behavior
400 behavior
successful response types
```

Complete and Reopen also retain their current ownership limitation. Formatting does not improve authorization.

UsersController Cleanup

ProblemDetails calls are formatted consistently for:

```
Get By Id
Get Tasks By User Id
```

```
Create
Update
Delete
```

Both bad-request and user-not-found responses now follow the same visual structure.

UsersController remains publicly accessible. Day 88 does not quietly add authorization during a formatting cleanup.

DTO Whitespace Cleanup

CreateTaskRequest contained an extra blank line before the closing class brace.

The blank line was removed.

The Priority property remains:

```
[Required]
[MaxLength(20)]
public string Priority { get; set; } = "Medium";
```

Its type, validation attributes, and default value are unchanged.

Namespace And Using Review

The edited files retain their existing namespaces:

```
JobTrackr.Api.Controllers
JobTrackr.Application.Tasks
```

No duplicate or stale using directive was introduced.

The review avoids removing imports based on guesswork. A using directive should only be removed when tooling or the compiler confirms it is unused.

Preserve EF Core Migrations

Migration files are generated history used by Entity Framework Core.

Files under the migrations directory may not be referenced directly by a controller, but they remain essential for:

```
creating a database from scratch
upgrading an existing database
tracking schema history
generating migration scripts
understanding model changes
```

They are not dead code simply because normal request handling does not call them directly.

Preserve Architectural Files

The cleanup also keeps:

```
DTOs referenced by interfaces
service interfaces used by dependency injection
middleware registered in Program.cs
entities referenced by AppDbContext
GitHub Actions workflow files
test infrastructure
```

Repository cleanup requires understanding indirect usage, configuration-based usage, and framework conventions.

Deleting a valid file can create a delayed failure that does not appear in the edited location.

Review The Diff, Not Only The Intent

The commit diff was inspected to confirm every change was visual.

The review checked that the diff did not modify:

```
route templates
HTTP methods
status constants
message constants
authentication attributes
service method names
method arguments
DTO members
```

An author may intend a formatting-only change, but the diff is the evidence.

Whitespace Verification

Git's whitespace checker reported no errors.

This detects issues such as:

```
trailing whitespace
space-before-tab problems
invalid blank-line whitespace
conflict markers in some workflows
```

Clean formatting should not introduce a new category of formatting defect.

Scope Of The Commit

Commit e81e388 changes four files:

```
AuthController.cs
TasksController.cs
UsersController.cs
CreateTaskRequest.cs
```

The commit contains:

```
56 inserted formatting lines
15 removed lines
```

Most added lines come from wrapping long method calls. The semantic content remains the same.

Verified Build

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

A formatting cleanup should never leave the project with compiler warnings or errors.

Verified Tests

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Tests provide additional evidence that the cleanup did not alter the covered behavior.

They do not replace diff review, because the test suite does not cover every controller response format or route.

Final Audit Result

After the change:

```
no starter or placeholder patterns are present
no empty C# source files are present
no whitespace errors are present
Release build succeeds
all tests pass
Git working tree is clean
```

The repository is ready for the upcoming month review without unnecessary deletion or refactoring.

Why Small Cleanup Commits Are Valuable

A small cleanup commit can improve:

```
readability
review speed
future diff clarity
consistency between neighboring files
confidence before a larger review
```

Its value does not depend on changing many files.

Keeping formatting separate from behavior also makes future debugging easier because a reader can see that this commit was not intended to alter application logic.

What This Day Teaches

Cleanup is a technical judgment exercise, not a file-deletion contest.

The main lessons are:

```
audit before editing
exclude generated output from source searches
remove only genuinely unnecessary content
preserve framework and architecture files
keep formatting changes behavior-free
inspect the actual diff
run whitespace checks
build and test afterward
avoid unrelated refactoring
```

A disciplined cleanup leaves the project easier to read without making its behavior harder to trust.

Reader Exercise

Audit another repository for:

```
starter templates
empty source files
TODO and HACK placeholders
NotImplementedException
inconsistent formatting
stale namespaces
generated files mistaken for dead code
```

For every proposed deletion, identify how the file is referenced directly, through dependency injection, by configuration, by migrations, or by framework convention.

Delete nothing until its role is understood.

Next Step

Use the clean repository baseline for the month review, summarizing architecture, authentication, ownership, testing, CI, logging, error handling, configuration, documentation, and remaining security work.

Day 89 - Update README And Progress

Lesson Goal

Update JobTrackr's public documentation so it accurately reflects the API after Days 80 through 88.

The README now documents:

```
request logging
ProblemDetails error responses
the public health endpoint
User Secrets
environment-specific configuration
local environment setup
manual regression status
the remaining authorization work
```

The separate daily progress record also receives a Day 89 entry.

Why Documentation Needs Regular Reviews

Documentation can become inaccurate even when every individual lesson is documented somewhere.

Before Day 89, the health endpoint appeared only in local setup instructions. A reader scanning the primary feature and endpoint lists could miss it.

Request logging, health checks, and environment documentation were complete, but some still appeared as future work.

This creates two common documentation failures:

```
completed capabilities are difficult to discover
the roadmap lists work that is no longer unfinished
```

A periodic documentation review reconnects the README with the current product.

Opening Summary

The repository introduction now identifies the major current capabilities:

```
users
user-owned tasks
SQL Server persistence
JWT authentication
structured error handling
request logging
health checks
automated service tests
GitHub Actions CI
```

This gives a reader a useful first impression without requiring them to inspect the complete endpoint list.

Current Features

The feature list now includes the reliability and configuration work completed during the recent lessons:

```
safe global ProblemDetails exception responses
structured ProblemDetails for known API errors
field-level DTO validation
request logging for method, path, status, and elapsed time
public GET /health endpoint
environment-specific local configuration
JWT signing key stored through User Secrets
documented local database, migration, HTTPS, and authentication setup
xUnit service tests
```

```
GitHub Actions build and test execution
```

Only completed behavior belongs in this section.

Make Health Discoverable

The primary endpoint list now begins with:

```
GET /health
```

The README explains that it:

```
is public
returns 200 OK
returns Healthy
confirms the API process is responding
may not appear in Swagger because it is mapped directly
```

The guide does not claim that this basic endpoint checks SQL Server or every dependency.

Complete Endpoint Inventory

The README lists the current routes by category.

Health

```
GET /health
```

Authentication

```
POST /api/auth/register
POST /api/auth/login
```

Tasks

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The list also retains completion and title-search examples.

Users

```
GET /api/users
POST /api/users
GET /api/users/{id}
GET /api/users/{userId}/tasks
PUT /api/users/{id}
DELETE /api/users/{id}
```

Reviewing documentation against controllers and Program.cs prevents route drift.

API Reliability Behavior

A new section explains the API's operational and error behavior:

```
invalid DTOs return 400 with field-level errors
known bad requests use ProblemDetails
missing resources use ProblemDetails
unexpected exceptions return a safe 500
request logs record safe request metadata
request logs exclude bodies and credentials
GET /health provides basic process health
```

This brings several cross-cutting concerns together in one discoverable place.

Safe Error Documentation

The README explains what clients receive without documenting internal exception details.

Known errors use a structured response, while unexpected failures return a generic message.

It does not expose:

```
stack traces
database exception details
connection strings
signing keys
passwords
tokens
```

Public documentation should describe the contract, not reveal private diagnostics.

Request Logging Documentation

Request logging is now a current feature rather than a planned item.

The README records the fields:

```
HTTP method
request path
status code
elapsed time
```

It also records what is deliberately excluded:

```
request bodies
passwords
JWT tokens
Authorization headers
signing keys
```

This communicates both usefulness and privacy boundaries.

Configuration Section

The new Configuration section summarizes where settings belong:

```
shared non-secret settings: appsettings.json
local SQL connection: appsettings.Development.json
local JWT signing key: .NET User Secrets
default local launch: HTTPS profile
future production secrets: environment variables or secret manager
```

This short overview complements the longer local environment setup instructions.

Why Configuration Belongs In The README

Configuration location affects whether another developer can run the application safely.

A reader should not need to infer:

```
which settings are committed
which settings are environment-specific
which value must be generated locally
where production secrets will eventually come from
```

The README makes those boundaries explicit without containing real credentials.

Manual Regression Status

The Automated Tests section still reports fifteen xUnit service tests.

It now separately records that the API passed a manual regression flow covering:

```
health
registration and login
validation
task CRUD and filters
two-user ownership isolation
user CRUD
structured errors
request logging
```

The distinction matters:

```
automated suite: repeatable service tests
manual regression: broader HTTP and SQL Server verification
```

The README does not inflate the automated test count or imply that the manual flow runs in CI.

Keep Authorization Limitations Explicit

The documentation still states:

```
Complete and Reopen require authentication but do not check ownership
User CRUD endpoints are not protected by authorization
```

These limitations are not hidden by the larger feature list.

A project can have JWT authentication and still contain incomplete resource authorization.

Accurate documentation helps readers understand that difference.

Correct The Roadmap

Completed work is removed from Next Steps.

The roadmap now includes:

```
owner protection for Complete and Reopen
stronger authorization for user endpoints
Swagger Bearer-token configuration
broader controller and integration tests
database-aware health checks when useful
deployment fundamentals
```

Request logging, the basic health endpoint, and local environment documentation no longer appear as future work.

Why Roadmap Accuracy Matters

A stale roadmap makes project planning less useful.

It can cause:

```
duplicate implementation work
confusing project evaluations
incorrect interview explanations
unclear priorities
loss of confidence in documentation
```

Moving completed items into current capabilities keeps the next steps actionable.

Security Review

Tracked JSON and Markdown were searched for:

```
the removed development JWT key
database Password values
database Pwd values
```

No old key or database password was found.

The README includes secret-generation instructions but no actual JWT signing key, access token, user password, or production connection string.

Separate Progress Record

The daily learning record receives a Day 89 entry outside the application repository.

It records:

```
the README sections updated
the health endpoint becoming discoverable
the successful secret check
the healthy build and test result
the Day 90 month review as the next task
```

Keeping the learning journal separate from the repository avoids mixing personal daily notes with public project documentation.

Scope Of The Repository Commit

Commit fed948d changes only README.md:

```
41 lines added
6 lines removed
```

PROGRESS.md is stored outside the JobTrackr repository and is not part of the application commit.

No changes are made to:

```
C# code
configuration
migrations
packages
database model
tests
GitHub Actions
```

Verified Build

Documentation is published from a healthy source baseline.

The Release build result is:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

Verified Tests

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Git remains clean after the committed documentation update.

What This Day Teaches

Documentation maintenance is part of engineering work.

The main lessons are:

```
make important endpoints easy to discover
move completed work into current features
remove completed work from future plans
separate automated and manual test claims
```

```
keep security limitations visible
describe configuration without publishing secrets
compare endpoint documentation with actual routes
verify build and tests before publishing
keep personal progress records separate from public project files
```

A README should describe the project that exists today, not a mixture of old behavior and future intentions.

Reader Exercise

Audit another project's README using three columns:

```
documented as current
verified in code
still planned
```

For every item, move it into the correct category.

Then check:

```
endpoint discoverability
security limitations
test counts
configuration sources
secret exposure
roadmap accuracy
```

Run the documented build and test commands before publishing the update.

Next Step

Complete the Month 3 review by summarizing the architecture, authentication, ownership, testing, CI, reliability, configuration, documentation, and highest-priority remaining work.

Day 90 - Month 3 Review

Review Goal

Close the third 30-day backend phase by verifying that authentication, task ownership, testing, continuous integration, API reliability, configuration, security, and documentation form a stable foundation.

Day 90 is a review day.

It deliberately avoids:

```
adding a feature
redesigning architecture
expanding test scope
starting pagination early
creating an empty commit
```

The purpose is to establish what is genuinely complete, what evidence supports it, and what remains unfinished.

Month 3 Scope

Days 61 through 90 focused on:

```
authenticated user identity
user-owned tasks
automated service tests
GitHub Actions
request logging
structured error responses
health checks
secret handling
environment-specific configuration
local setup documentation
manual API regression testing
repository cleanup
project documentation
```

This phase moved JobTrackr from basic authentication toward a more dependable backend foundation.

Days 61-68: Authentication And Ownership

The first part of Month 3 connected JWT identity to task behavior.

Completed work includes:

```
reviewing registration and login
reviewing JWT generation and validation
protecting task endpoints
reading the authenticated user id from JWT claims
removing UserId from public task creation requests
assigning new tasks to the authenticated user
restricting task lists to the authenticated user's tasks
restricting Get By Id to the owner
restricting Update to the owner
restricting Delete to the owner
testing isolation with two authenticated users
```

This establishes an important rule:

```
task ownership comes from authenticated identity, not client-supplied ownership data
```

Current Ownership Matrix

Operation	Current behavior
Create	Uses authenticated user id
Get All	Returns only the authenticated user's tasks
Get By Id	Returns only an owned task
Update	Updates only an owned task
Delete	Deletes only an owned task
Complete	Requires authentication; ownership not checked
Reopen	Requires authentication; ownership not checked

For ownership-protected operations, another user's task appears as:

```
404 Not Found
```

This avoids confirming that another user's resource exists.

Known Ownership Gap

Complete and Reopen still call service methods without the authenticated user identifier.

Therefore:

```
authentication is complete for these routes
ownership authorization is not complete
```

The month review does not hide this distinction.

It is one of the highest-priority items for the next phase.

Days 69-76: Automated Service Tests

The second part of the month introduced repeatable automated testing.

Completed work includes:

```
xUnit test project
password hashing test
task creation tests
task Get By Id tests
task update tests
task deletion tests
user service tests
authentication service tests
Release build readiness review
```

The project reached fifteen automated tests.

Current Automated Coverage

Authentication

```
password hashing and verification
registration
valid login
invalid-password login
```

Tasks

```
valid creation
empty-title creation
existing task retrieval
missing task retrieval
valid update
missing-task update
empty-title update
successful deletion
missing-task deletion
```

Users

```
valid user creation
existing user retrieval
```

The service tests use EF Core InMemory and do not require SQL Server.

Verified Test Result

The complete Release test run reports:

```
Passed: 15
Failed: 0
Skipped: 0
```

This is the verified Day 90 result, not only an expected value copied from documentation.

Current Testing Gaps

The suite does not yet provide broad HTTP integration coverage.

Future automated tests should cover:

```
controller routes and response contracts
JWT authentication through the HTTP pipeline
ProblemDetails serialization
health endpoint behavior
two-user ownership through real API requests
Complete and Reopen ownership
user endpoint authorization
SQL Server integration where useful
```

The manual regression flow currently supplies evidence for several of these areas, but automation would make the evidence repeatable in CI.

Days 77-79: Continuous Integration

GitHub Actions was added to run on pushes and pull requests targeting main.

The workflow:

```
checks out the repository
installs .NET 8
restores API dependencies
restores test dependencies
builds production projects in Release configuration
runs all fifteen tests
```

It uses read-only repository permissions.

The workflow does not require SQL Server because the existing tests use EF Core InMemory.

Why CI Matters

Local success can depend on cached packages, installed tooling, or machine state.

GitHub Actions repeats the build on a clean runner and answers:

```
can the repository restore from a clean checkout?
does production code compile?
does the automated suite pass?
```

This provides a shared baseline for future changes.

README After CI

The repository README was updated to describe:

```
authentication behavior
task ownership
the fifteen tests
local test commands
GitHub Actions behavior
known authorization limitations
```

Documentation was treated as part of the deliverable rather than an end-of-project task.

Days 80-89: API Reliability

The final third of Month 3 focused on operational behavior and maintainability.

Completed reliability work includes:

```
safe request logging
error-response review
ProblemDetails
global exception logging
health endpoint
security review
configuration cleanup
local setup guide
manual regression flow
formatting cleanup
README and progress updates
```

Request Logging

RequestLoggingMiddleware records:

```
HTTP method
request path
response status
elapsed milliseconds
```

It deliberately avoids:

```
request bodies
passwords
JWT tokens
Authorization headers
connection strings
signing keys
```

The middleware uses structured placeholders so logging systems can later query individual fields.

Structured Error Responses

Known controller errors and unexpected failures now use ASP.NET Core ProblemDetails.

Current behavior includes:

```
invalid DTO -> 400 ValidationProblemDetails
known bad request -> 400 ProblemDetails
missing resource -> 404 ProblemDetails
unexpected exception -> safe 500 ProblemDetails
missing authentication -> standard 401 response
```

Unexpected exception details are logged on the server, while clients receive a generic message.

This preserves diagnostic value without exposing implementation details.

Health Endpoint

JobTrackr provides:

```
GET /health
```

The basic public endpoint returns:

```
200 OK
Healthy
```

It confirms:

```
the API process is running
ASP.NET Core can answer a request
```

It does not currently confirm:

```
SQL Server availability
migration state
external service health
every endpoint's behavior
```

Database-aware checks remain optional future work when operational requirements justify them.

Secret Handling

The development JWT signing key was removed from tracked appsettings.

A replacement key is stored through .NET User Secrets.

Current configuration sources are:

Setting	Source
JWT issuer and audience	appsettings.json
Local SQL Server connection	appsettings.Development.json
Local JWT signing key	.NET User Secrets

Tracked configuration was checked for the removed development key and database password patterns.

The review found:

```
old JWT key not present
database password not present in tracked JSON or Markdown
local Jwt:Key secret is configured
```

Secret values were not printed during verification.

Environment Configuration

The local SQL Server connection is separated from shared settings.

The HTTPS launch profile is the default development profile.

The local setup documentation explains:

```
.NET prerequisites
EF Core tools
dependency restoration
SQL Server requirements
migration command
User Secrets setup
HTTPS certificate trust
```

```
build and tests
API startup
health verification
registration and login
Bearer-token requests
```

Another developer can follow the setup without receiving a committed JWT key.

Manual Regression Testing

Day 87 exercised the application from start to finish.

The flow covered:

```
health
two-user registration
validation errors
two-user login
anonymous 401 behavior
task creation
task filters and search
task ownership isolation
task update and deletion
complete and reopen for the owner
public user CRUD
structured 404 responses
request log safety
```

The test deliberately avoided using User B to mutate User A's task through Complete or Reopen because the known ownership gap could change another user's data.

This is careful testing rather than pretending a known unsafe case is complete.

Repository Cleanup

The source audit found:

```
no WeatherForecast starter code
no Class1 placeholders
no empty C# files
no TODO or HACK placeholders
no NotImplementedException
no obvious dead application classes
```

Only formatting consistency and one unnecessary blank line required changes.

Valid migrations, interfaces, middleware, DTOs, and infrastructure files were preserved.

Architecture Review

The project retains five clear areas:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
JobTrackr.Tests
```

API

Contains controllers, middleware, authentication configuration, dependency registration, health routing, and startup.

Application

Contains DTOs, service contracts, shared messages, and authentication abstractions.

Domain

Contains the User and JobTask entities.

Infrastructure

Contains EF Core persistence, SQL Server services, authentication implementation, JWT generation, and migrations.

Tests

Contains xUnit service tests using isolated in-memory databases.

Controllers do not access AppDbContext directly.

That preserves the intended application and infrastructure boundary.

Verified Release Build

The final Month 3 Release build reports:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

All projects restore and compile from the current repository state.

Repository State

The latest source commit is:

```
fed948d Day 89: Update project documentation
```

The working tree is clean.

Day 90 does not create an empty commit because no source or repository documentation changed during the successful review.

Documentation Review

The README now includes:

```
current features
health, authentication, task, and user endpoints
authentication behavior
task ownership behavior
authorization limitations
reliability behavior
SQL Server and migration setup
User Secrets setup
HTTPS startup
automated tests
manual regression status
GitHub Actions behavior
accurate next steps
```

GET /health appears in the main endpoint list rather than only in setup instructions.

Known Security Limitations

Month 3 ends with three explicit gaps:

```
Complete does not verify task ownership
Reopen does not verify task ownership
UsersController is not protected by authorization
```

Swagger also does not yet include Bearer-token authorization configuration.

These are planned improvements, not hidden failures.

Month 3 Result

JobTrackr now has a stable backend foundation containing:

```
SQL Server persistence
JWT authentication
authenticated task ownership
structured API errors
safe request logging
public process health
secure local secret handling
environment-specific configuration
automated service tests
continuous integration
manual regression evidence
reproducible local setup documentation
```

The project is ready to continue, but it is not presented as finished.

Evidence Summary

Area	Result
Release build	Passed with 0 warnings and 0 errors
Automated tests	15 passed, 0 failed, 0 skipped
Git status	Clean
Latest commit	fed948d
Architecture boundary	Controllers do not use AppDbContext
CI configuration	Release build and tests configured
Tracked old JWT key	Not found
Tracked database password	Not found
Local JWT secret	Configured
README	Current and complete for Month 3

Next Phase: Days 91-120

The next phase will focus on:

```
task pagination
sorting by created and due dates
combining filters, sorting, and pagination
authenticated profile endpoints
change-password behavior
authorization tests
Swagger Bearer-token configuration
API integration tests
database reset and migration documentation
```

Day 91 Direction

Day 91 begins by planning query improvements.

Before implementing pagination, the project should define predictable parameters for:

```
page number
page size
sort field
```

```
sort direction
existing search filter
existing completion filter
```

Planning the contract first reduces accidental breaking changes.

What This Review Teaches

A phase review should be evidence-based.

The important habits are:

```
verify the repository state
run the exact Release build
run the complete test suite
inspect architecture boundaries
review configuration and secret storage
compare documentation with code
state known limitations plainly
avoid adding unrelated work during review
avoid empty milestone commits
define the next phase without starting it early
```

Stable software grows through verified increments, not through feature count alone.

Reader Exercise

Create a phase-review table for another backend.

Include:

```
features completed
behavioral evidence
test coverage
CI status
security boundaries
configuration sources
runtime health
documentation accuracy
known gaps
next-phase priorities
```

For every completion claim, attach a build result, test, route, configuration check, or code reference.

Next Step

Start Day 91 by planning pagination, sorting, and predictable task-query behavior before changing the API contract.

MONTH 4

QUERYING, SECURITY, AND DELIVERY

Build richer queries, secure account workflows, expand integration tests, and document the system for continued delivery.

Day 91 - Plan Task Query Improvements

Goal

Review the current `GET /api/tasks` query and define a predictable contract for pagination and sorting before writing implementation code.

Today is planning only.

Do not change controllers, DTOs, service interfaces, service implementations, entities, migrations, tests, or the database.

Current Endpoint

```
GET /api/tasks
```

Current optional query parameters:

```
isCompleted
search
```

Current examples:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=false&search=resume
```

Current Request Flow

TasksController

Current action:

```
[HttpGet]
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
```

The controller:

1. Reads the authenticated user id from JWT claims.
2. Returns `401 Unauthorized` if the user id cannot be read.
3. Passes filters and the authenticated user id to `ITaskService`.
4. Returns the resulting task list with `200 OK`.

ITaskService

Current method:

```
Task<List<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int userId);
```

TaskService

The current EF Core query performs:

```
Filter by authenticated UserId
Filter by IsCompleted when supplied
Filter Title by search text when supplied
Project entities to TaskResponse
Return every matching row
```

Current Strengths

- Tasks are restricted to the authenticated user.
- Completion filtering is optional.
- Title search is optional.
- EF Core performs filtering in SQL Server.
- Database entities are projected to `TaskResponse` DTOs.
- Controllers do not access `AppDbContext` directly.

These behaviors must remain intact.

Current Problems

1. Every Matching Task Is Returned

The query calls `ToListAsync()` without `Skip()` or `Take()`.

This is acceptable with a few tasks but becomes inefficient as a user's task count grows.

2. Ordering Is Not Explicit

The query has no `OrderBy`.

SQL Server does not guarantee row order unless the query explicitly specifies it. Results may appear stable during small tests but should not be treated as predictable.

3. The Response Has No Page Metadata

The endpoint currently returns only:

```
[
  {
    "id": 1,
    "title": "Example task"
  }
]
```

The client cannot tell:

- total matching items
- current page
- page size
- total pages
- whether another page exists

4. Sorting Cannot Be Selected

The client cannot currently request sorting by:

- creation date
- due date
- ascending direction
- descending direction

Planned Query Contract

The endpoint path will remain unchanged:

```
GET /api/tasks
```

The final planned optional query parameters are:

Parameter	Type	Default	Purpose
`isCompleted`	nullable boolean	none	Filter completed or incomplete tasks
`search`	string	none	Search task titles
`pageNumber`	integer	`1`	Select the requested page
`pageSize`	integer	`10`	Select the number of tasks per page
`sortBy`	string	`createdAt`	Select creation-date or due-date sorting
`sortDirection`	string	`desc`	Select ascending or descending order

These parameters will be introduced one day at a time according to the roadmap.

Planned Parameter Rules

isCompleted

Accepted behavior remains:

```
not supplied   return completed and incomplete tasks
true           return only completed tasks
false          return only incomplete tasks
```

search

Planned behavior remains simple:

- optional
- searches `Title`
- whitespace-only input behaves like no search filter
- existing `Contains` behavior remains for now

Do not expand search to description, priority, or user data during this phase.

pageNumber

Planned rules:

```
Default: 1
Minimum: 1
```

Page numbering will be one-based because it is easier for API clients and Swagger users to understand.

pageSize

Planned rules:

```
Default: 10
Minimum: 1
Maximum: 100
```

The maximum prevents a client from bypassing pagination by requesting an excessively large page.

sortBy

Planned accepted values:

```
createdAt
dueDate
```

Planned default:

```
createdAt
```

Do not support arbitrary property names. An allow-list keeps the contract predictable and prevents fragile dynamic query logic.

sortDirection

Planned accepted values:

```
asc
desc
```

Planned default:

```
desc
```

Therefore, a request without sorting parameters will eventually return the newest tasks first.

Planned Query Examples

Default First Page

```
GET /api/tasks
```

Planned meaning:

```
pageNumber=1
pageSize=10
sortBy=createdAt
sortDirection=desc
```

Second Page

```
GET /api/tasks?pageNumber=2&pageSize=10
```

Filter And Paginate

```
GET /api/tasks?isCompleted=false&pageNumber=1&pageSize=5
```

Search And Paginate

```
GET /api/tasks?search=resume&pageNumber=1&pageSize=10
```

Sort By Oldest Creation Date

```
GET /api/tasks?sortBy=createdAt&sortDirection=asc
```

Sort By Due Date

```
GET /api/tasks?sortBy=dueDate&sortDirection=asc
```

Combine Everything

```
GET /api/
tasks?isCompleted=false&search=resume&pageNumber=2&pageSize=5&sortBy=dueDate&sortDirection=asc
```

Planned Operation Order

The final EF Core query should apply operations in this logical order:

```
1. Restrict tasks to the authenticated user
2. Apply completion filter
3. Apply title search
4. Count all matching rows
5. Apply explicit sorting
6. Apply Skip and Take for pagination
7. Project to TaskResponse
8. Execute the query with ToListAsync
```

Conceptual example only:

```
var query = _dbContext.Tasks
    .Where(task => task.UserId == userId);

// Apply filters.

var totalCount = await query.CountAsync();

// Apply sorting.
// Apply Skip and Take.
// Project to TaskResponse.
```

Do not add this code today.

Why Count Happens Before Pagination

`totalCount` must represent all tasks matching the ownership and filter conditions.

If `CountAsync()` runs after `Skip()` and `Take()`, it counts only the current page and produces incorrect metadata.

Correct idea:

```
Filter -> Count -> Sort -> Skip/Take -> Return items
```

Stable Sorting Rule

Tasks can have equal creation dates or equal due dates.

The final query should use `Id` as a secondary sort value so pagination remains stable.

Conceptual examples:

```
query.OrderByDescending(task => task.CreatedAtUtc)
     .ThenByDescending(task => task.Id);

query.OrderBy(task => task.DueDateUtc)
     .ThenBy(task => task.Id);
```

Do not implement sorting today.

Planned Null Due-Date Behavior

`DueDateUtc` is nullable.

When due-date sorting is implemented, tasks without a due date should appear after tasks that have a due date. This keeps actionable deadlines visible first.

The exact EF Core expression will be implemented and tested on Day 96.

Planned Paged Response

The current endpoint returns `List<TaskResponse>`.

The final response will use a simple wrapper planned for Day 94:

```
{
  "items": [],
```

```
  "pageNumber": 1,
  "pageSize": 10,
  "totalCount": 0,
  "totalPages": 0
}
```

Possible later convenience fields such as `hasPreviousPage` and `hasNextPage` should be added only if they remain simple and useful.

Do not create the response DTO today.

Invalid Query Decisions

The planned behavior should be predictable:

Input	Planned result
`pageNumber=0`	`400 Bad Request`
`pageSize=0`	`400 Bad Request`
`pageSize=101`	`400 Bad Request`
unknown `sortBy`	`400 Bad Request`
unknown `sortDirection`	`400 Bad Request`
page beyond available data	`200 OK` with empty `items`

An empty valid page is not a missing resource, so it should not return `404`.

Validation implementation will be added only when the related parameters are introduced.

Compatibility Decision

Existing filters will continue to work:

```
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
```

However, once pagination is implemented, `GET /api/tasks` will return a paged object instead of a raw JSON array.

This is an intentional API contract change and must be documented in the README when completed.

Ownership Decision

The authenticated user id must never become a public query parameter.

Do not add:

```
GET /api/tasks?userId=1
```

The service must continue receiving the user id from trusted JWT claims through the controller.

Filtering, sorting, and pagination must operate only inside that user's task query.

Implementation Schedule

Day 92

Add `pageNumber` and `pageSize` request parameters with safe defaults and validation.

Day 93

Apply `Skip()` and `Take()` in the task query.

Pagination formula:

```
var skip = (pageNumber - 1) * pageSize;
```

Day 94

Add the paged response DTO containing items and metadata.

Day 95

Add creation-date sorting with newest-first default behavior.

Day 96

Add due-date sorting and clear null handling.

Day 97

Verify filtering, searching, sorting, and pagination work together.

Day 98

Run a focused query review and fix confusing behavior.

Review Checklist

```
[ ] Existing ownership filter remains mandatory
[ ] Existing isCompleted filter remains optional
[ ] Existing title search remains optional
[ ] pageNumber defaults to 1
[ ] pageSize defaults to 10
[ ] pageSize maximum is 100
[ ] createdAt is the default sort field
[ ] desc is the default sort direction
[ ] Id provides stable secondary ordering
[ ] null due dates appear after real due dates
[ ] invalid parameter values return 400
[ ] an empty valid page returns 200
[ ] the final response includes items and page metadata
[ ] implementation remains in the existing controller/service architecture
```

Interview Explanation

```
I reviewed the existing authenticated task query before adding pagination.
I defined safe defaults, validation limits, stable ordering, due-date null behavior, and a
future paged response contract while preserving ownership and existing filters.
I also defined the correct query order: filter, count, sort, paginate, project, and execute.
```

End Of Day Note Template

```
1. What I planned
I reviewed the current task list query and planned pagination, sorting, validation, stable
ordering, and paged response behavior.

2. What worked
The query contract is now clear: existing ownership and filters remain, pageNumber defaults to
1, pageSize defaults to 10 with a maximum of 100, and sorting will default to newest-created
tasks first.

3. What I will do tomorrow
Add pageNumber and pageSize request parameters with safe defaults and validation.
```

Day 92 - Add Pagination Request Parameters

Goal

Add validated pagination parameters to `GET /api/tasks`:

```
pageNumber
pageSize
```

Safe defaults:

```
pageNumber = 1
pageSize = 10
```

Validation rules:

```
pageNumber must be 1 or greater
pageSize must be between 1 and 100
```

Today adds the request contract only.

Do not apply `Skip()` or `Take()` today. Actual pagination is Day 93.

Why Use A Query Request DTO?

The current controller action receives separate parameters:

```
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
```

Adding pagination directly would make the method longer:

```
GetAll(bool? isCompleted, string? search, int pageNumber, int pageSize)
```

A small request DTO keeps related query values together and gives us one place for defaults and validation.

The DTO can later receive `sortBy` and `sortDirection` without repeatedly expanding the controller method.

Files To Create And Update

Create

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
```

Update

```
src\JobTrackr.Api\Controllers\TasksController.cs
```

Do not update today:

```
ITaskService.cs
TaskService.cs
TaskResponse.cs
AppDbContext.cs
JobTask.cs
```

Step 1 - Create TaskQueryRequest.cs

Create:

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
```

Add:

```
using System.ComponentModel.DataAnnotations;
```

```
namespace JobTrackr.Application.Tasks
{
    public class TaskQueryRequest
    {
        public bool? IsCompleted { get; set; }

        public string? Search { get; set; }

        [Range(1, int.MaxValue, ErrorMessage = "Page number must be at least 1.")]
        public int PageNumber { get; set; } = 1;

        [Range(1, 100, ErrorMessage = "Page size must be between 1 and 100.")]
        public int PageSize { get; set; } = 10;
    }
}
```

What Each Property Does

IsCompleted

```
public bool? IsCompleted { get; set; }
```

Preserves the existing optional completion filter:

```
null    return completed and incomplete tasks
true    return completed tasks
false   return incomplete tasks
```

Search

```
public string? Search { get; set; }
```

Preserves the existing optional title search.

PageNumber

```
[Range(1, int.MaxValue, ErrorMessage = "Page number must be at least 1.")]
public int PageNumber { get; set; } = 1;
```

The default is page 1. Zero and negative page numbers are invalid.

PageSize

```
[Range(1, 100, ErrorMessage = "Page size must be between 1 and 100.")]
public int PageSize { get; set; } = 10;
```

The default is 10 tasks. The maximum of 100 prevents excessively large page requests.

Step 2 - Update TasksController.GetAll

Open:

```
src\JobTrackr.Api\Controllers\TasksController.cs
```

Current action:

```
[HttpGet]
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var tasks = await _taskService.GetAllAsync(isCompleted, search, currentUserId.Value);

    return Ok(tasks);
}
```

Replace it with:

```
[HttpGet]
public async Task<IActionResult> GetAll([FromQuery] TaskQueryRequest request)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var tasks = await _taskService.GetAllAsync(
        request.IsCompleted,
        request.Search,
        currentUserId.Value);

    return Ok(tasks);
}
```

The controller already imports:

```
using JobTrackr.Application.Tasks;
using Microsoft.AspNetCore.Mvc;
```

Therefore, no new using directive is required.

How FromQuery Works

```
[FromQuery] TaskQueryRequest request
```

ASP.NET Core reads values from the URL query string and fills the DTO.

Example:

```
GET /api/tasks?isCompleted=false&search=resume&pageNumber=2&pageSize=5
```

Produces values equivalent to:

```
request.IsCompleted = false
request.Search = "resume"
request.PageNumber = 2
request.PageSize = 5
```

When pagination parameters are omitted:

```
GET /api/tasks
```

The property initializers provide:

```
request.PageNumber = 1
request.PageSize = 10
```

How Validation Works

`TasksController` already uses:

```
[ApiController]
```

Because `TaskQueryRequest` uses `[Range]`, ASP.NET Core automatically validates it before `GetAll` runs.

Invalid values return:

```
400 Bad Request
```

with field-level validation errors.

No manual `if` statements or `try/catch` blocks are needed in the action.

Why The Service Does Not Change Today

The service still receives only:

```
isCompleted
```

```
search
userId
```

Today the controller accepts and validates `PageNumber` and `PageSize`, but does not pass them to the service yet.

This is intentional:

- Day 92 defines and validates the HTTP request contract.
- Day 93 passes pagination values into the service and applies `Skip()` and `Take()`.
- Day 94 adds the paged response and metadata.

Therefore, valid pagination values appear in Swagger today but do not limit the returned task list until Day 93.

Do not claim pagination is complete today.

Step 3 - Build And Test

Run:

```
cd [repository root]
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-restore
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 15
Failed: 0
Skipped: 0
```

Step 4 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Open:

```
https://localhost:7024/swagger
```

Expand:

```
GET /api/tasks
```

Swagger should now show:

```
isCompleted
search
pageNumber
pageSize
```

Expected displayed defaults:

```
pageNumber = 1
pageSize = 10
```

API Test Examples

Use a valid Bearer token through PowerShell because Swagger Bearer authorization is not configured yet.

Prepare the values:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

Test Defaults

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks" `
    -Method Get `
    -Headers $headers
```

Expected: `200 OK`.

The request uses `PageNumber = 1` and `PageSize = 10` internally, but the list is not limited yet.

Test Valid Pagination Values

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=2&pageSize=5" `
    -Method Get `
    -Headers $headers
```

Expected: `200 OK`.

The values are accepted and validated, but Day 93 will make them affect the query results.

Test Existing Filters With Pagination Values

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?isCompleted=false&search=resume&pageNumber=1&pageSize=5" `
    -Method Get `
    -Headers $headers
```

Expected: `200 OK`, with the existing completion and title filters still working.

Test Invalid Page Number

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks?pageNumber=0&pageSize=10" `
        -Method Get `
        -Headers $headers
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
400
```

Validation message:

```
Page number must be at least 1.
```

Test Page Size Below Minimum

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=0" `
        -Method Get `
        -Headers $headers
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected: `400 Bad Request`.

Test Page Size Above Maximum

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=101" `
        -Method Get `
        -Headers $headers
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected: `400 Bad Request`.

Validation message:

```
Page size must be between 1 and 100.
```

Important Checks

Confirm:

```
[ ] Existing isCompleted filtering still works
[ ] Existing title search still works
[ ] pageNumber appears in Swagger
[ ] pageSize appears in Swagger
[ ] pageNumber defaults to 1
[ ] pageSize defaults to 10
[ ] pageNumber=0 returns 400
[ ] pageSize=0 returns 400
[ ] pageSize=101 returns 400
[ ] Anonymous requests still return 401
[ ] The service and database query are unchanged today
[ ] All 15 tests pass
```

Files Changed

TaskQueryRequest.cs

Groups task-list filters and pagination request values. It defines safe defaults and `[Range]` validation.

TasksController.cs

Binds `GET /api/tasks` query parameters into `TaskQueryRequest` while continuing to pass the existing filters and authenticated user id to the service.

Interview Explanation

```
I added a query request DTO for the task list endpoint with pageNumber and pageSize defaults
and range validation.
ASP.NET Core binds it from the query string and automatically returns 400 for invalid values
because the controller uses ApiController.
I kept actual Skip and Take query behavior separate for the next implementation step.
```

End Of Day Note Template

```
1. What I built
I added a TaskQueryRequest DTO with pageNumber and pageSize defaults and validation, then used
it in the task list controller action.

2. What worked
Swagger shows the existing filters plus pageNumber and pageSize, valid values are accepted,
invalid values return 400, and all 15 tests pass.

3. What I will do tomorrow
Pass the pagination values into TaskService and apply Skip and Take to the EF Core query.
```

Day 93 - Implement Task Pagination

Goal

Make `pageNumber` and `pageSize` affect the SQL task query.

Today will:

- pass pagination values from the controller to the service
- update the service interface
- apply stable ordering
- apply `Skip()` and `Take()`
- add focused pagination tests

The response remains a raw `List<TaskResponse>` today. Page metadata will be added on Day 94.

Current State

Day 92 added:

```
pageNumber default: 1
pageSize default: 10
pageSize maximum: 100
```

ASP.NET Core validates these values, but the controller currently does not pass them to the service. Therefore, every matching task is still returned.

Day 93 connects the request parameters to the EF Core query.

Files To Update And Create

Update

```
src\JobTrackr.Api\Controllers\TasksController.cs
src\JobTrackr.Application\Tasks\ITaskService.cs
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

Create

```
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

Do not change:

- `TaskQueryRequest`
- `TaskResponse`
- `JobTask`
- database migrations
- existing filters
- task ownership behavior

Step 1 - Update ITaskService

Open:

```
src\JobTrackr.Application\Tasks\ITaskService.cs
```

Current method:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int userId);
```

Replace it with:

```
Task<List<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int pageNumber,
    int pageSize,
    int userId);
```

The interface now describes the complete Day 93 use case.

Step 2 - Update TasksController

Open:

```
src\JobTrackr.Api\Controllers\TasksController.cs
```

Inside `GetAll`, find:

```
var tasks = await _taskService.GetAllAsync(
    request.IsCompleted,
    request.Search,
    currentUserId.Value);
```

Change it to:

```
var tasks = await _taskService.GetAllAsync(
    request.IsCompleted,
    request.Search,
    request.PageNumber,
    request.PageSize,
    currentUserId.Value);
```

The authenticated user id still comes from JWT claims. Never add `userId` to the public query DTO.

Step 3 - Update TaskService Signature

Open:

```
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

Current signature:

```
public async Task<List<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int userId)
```

Change it to:

```
public async Task<List<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int pageNumber,
    int pageSize,
    int userId)
```

Step 4 - Apply Pagination To The EF Core Query

Keep the current ownership and filter logic unchanged:

```
var query = _dbContext.Tasks.Where(task => task.UserId == userId);

if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}
```

```
if (!string.IsNullOrWhiteSpace(search))
{
    query = query.Where(task => task.Title.Contains(search));
}
```

Then update the final query from:

```
return await query
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();
```

to:

```
var skip = (pageNumber - 1) * pageSize;

return await query
    .OrderBy(task => task.Id)
    .Skip(skip)
    .Take(pageSize)
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();
```

Pagination Formula

```
var skip = (pageNumber - 1) * pageSize;
```

Examples:

Page	Size	Skip	Take
1	10	0	10
2	10	10	10
3	10	20	10
2	5	5	5

For page 2 with a page size of 5:

```
(2 - 1) * 5 = 5
```

EF Core skips the first five matching tasks and returns the next five.

Why OrderBy Is Required

Pagination must use a predictable order.

Without `OrderBy`, SQL Server does not guarantee which rows appear on each page. A task could move between pages across requests even when the data has not changed.

For Day 93, use:

```
.OrderBy(task => task.Id)
```

This gives stable temporary ordering by task id.

Day 95 will replace this temporary default with newest-created-first ordering while retaining a stable id tie-breaker.

Do not implement created-date sorting early today.

Why Filtering Happens Before Pagination

The logical order is:

```
1. Restrict by authenticated user
2. Apply completion filter
3. Apply title search
4. Order matching tasks
5. Skip previous pages
6. Take the requested page size
7. Project to TaskResponse
8. Execute in SQL Server
```

If pagination happened before filtering, pages could contain too few relevant results or miss matching tasks.

Step 5 - Add Pagination Tests

Create:

```
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

Add:

```
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Persistence;
using JobTrackr.Infrastructure.Tasks;
using Microsoft.EntityFrameworkCore;

namespace JobTrackr.Tests.Tasks
{
    public class TaskServicePaginationTests
    {
        [Fact]
        public async Task GetAllAsync_WithSmallPageSize_ReturnsRequestedPage()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddUserWithTasksAsync(dbContext, 5);
            var taskService = new TaskService(dbContext);

            var response = await taskService.GetAllAsync(
                null,
                null,
                2,
                2,
                user.Id);

            Assert.Equal(2, response.Count);
            Assert.Equal("Task 3", response[0].Title);
            Assert.Equal("Task 4", response[1].Title);
        }

        [Fact]
        public async Task GetAllAsync_WithLargePageSize_ReturnsAllMatchingTasks()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddUserWithTasksAsync(dbContext, 5);
            var taskService = new TaskService(dbContext);

            var response = await taskService.GetAllAsync(
                null,
                null,
                1,
```

```
                100,
                user.Id);

            Assert.Equal(5, response.Count);
        }

        private static AppDbContext CreateDbContext()
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;

            return new AppDbContext(options);
        }

        private static async Task<User> AddUserWithTasksAsync(
            AppDbContext dbContext,
            int taskCount)
        {
            var user = new User
            {
                FullName = "Pagination Test User",
                Email = "pagination.test@example.com",
                PasswordHash = "test-password-hash"
            };

            for (var number = 1; number <= taskCount; number++)
            {
                user.Tasks.Add(new JobTask
                {
                    Title = $"Task {number}",
                    Description = $"Pagination task {number}",
                    Priority = "Medium"
                });
            }

            dbContext.Users.Add(user);
            await dbContext.SaveChangesAsync();

            return user;
        }
    }
}
```

What The Tests Prove

Small Page Test

Five tasks are stored with ids in insertion order.

The request uses:

```
pageNumber = 2
pageSize = 2
```

It skips Tasks 1 and 2, then returns Tasks 3 and 4.

Large Page Test

The request uses the maximum valid page size:

```
pageNumber = 1
pageSize = 100
```

Only five tasks exist, so all five matching tasks are returned.

The query never creates extra rows to fill a page.

Important Test Note

The test calls `TaskService` directly, so `[Range]` validation from the API controller does not run in this test.

These service tests use valid pagination values. Day 92 already verified invalid query values through ASP.NET Core request validation.

Step 6 - Build And Test

Run:

```
cd
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-restore
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 17
Failed: 0
Skipped: 0
```

The test count increases from 15 to 17.

Step 7 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Obtain a valid JWT token through login and prepare PowerShell:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

API Test Examples

These examples are easiest to see when the authenticated user has more than five tasks.

First Page With Two Tasks

```
$page1 = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=2" `
    -Method Get `
    -Headers $headers

$page1
```

Expected: at most two tasks.

Second Page With Two Tasks

```
$page2 = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=2&pageSize=2" `
    -Method Get `
    -Headers $headers

$page2
```

Expected: the next two tasks, different from page 1.

Page Beyond Available Tasks

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=999&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Expected:

```
[]
```

The endpoint returns `200 OK` with an empty list, not `404 Not Found`.

Pagination With Existing Filters

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?isCompleted=false&search=task&pageNumber=1&pageSize=2" `
    -Method Get `
    -Headers $headers
```

Expected:

- only the authenticated user's tasks

- only incomplete tasks

- only titles matching the search

- at most two results

Current Response Shape

Day 93 still returns a JSON array:

```
[
  {
    "id": 1,
    "title": "Task 1"
  }
]
```

It does not yet return:

```
pageNumber
pageSize
totalCount
totalPages
```

Those fields belong to Day 94's paged response DTO.

Important Checks

```
[ ] Controller passes PageNumber and PageSize
[ ] ITaskService includes pagination parameters
[ ] TaskService includes pagination parameters
[ ] Ownership filtering happens before pagination
[ ] Completion filtering happens before pagination
[ ] Search happens before pagination
[ ] OrderBy(Id) provides temporary stable ordering
[ ] Skip uses (pageNumber - 1) * pageSize
[ ] Take uses pageSize
[ ] Page beyond available tasks returns an empty list
[ ] Small and large page tests pass
[ ] Total automated tests increase to 17
```

Files Changed

TasksController.cs

Passes validated page number and page size values to the task service.

ITaskService.cs

Updates the Get All use-case contract with pagination values.

TaskService.cs

Applies stable ordering, `Skip`, and `Take` to the filtered EF Core query.

TaskServicePaginationTests.cs

Verifies a small requested page and the maximum valid page size.

Interview Explanation

I implemented server-side task pagination by passing validated page parameters through the existing controller and service contract.
The EF Core query filters by owner, completion state, and search text before applying stable ordering, Skip, and Take.
I added tests for a small second page and the maximum allowed page size, while leaving page metadata for the next step.

End Of Day Note Template

1. What I built
I passed pageNumber and pageSize into TaskService and implemented server-side pagination with stable ordering, Skip, and Take.

2. What worked
Task pages now return the correct number of owned and filtered tasks, pages beyond available data return an empty list, and all 17 automated tests pass.

3. What I will do tomorrow
Add a paged response DTO with items, pageNumber, pageSize, totalCount, and totalPages.

Day 94 - Add Paged Response DTO

Goal

Return task page metadata together with the current page of tasks.

Today you will:

- create a reusable `PagedResponse<T>` DTO
- return task items and pagination metadata
- count filtered tasks before applying `Skip()` and `Take()`
- update the existing pagination tests

Do not add sorting options today. Created-date sorting belongs to Day 95.

Current State

Day 93 returns only a JSON array:

```
[
  {
    "id": 3,
    "title": "Task 3"
  }
]
```

The client receives the requested tasks but cannot tell how many matching tasks or pages exist.

Day 94 will return:

```
{
  "items": [
    {
      "id": 3,
      "title": "Task 3"
    }
  ],
  "pageNumber": 2,
  "pageSize": 2,
  "totalCount": 5,
  "totalPages": 3
}
```

Files To Create And Update

Create

```
src\JobTrackr.Application\Common\PagedResponse.cs
```

If the `Common` folder does not exist, create it with:

```
cd
mkdir src\JobTrackr.Application\Common
New-Item src\JobTrackr.Application\Common\PagedResponse.cs -ItemType File
```

Update

```
src\JobTrackr.Application\Tasks\ITaskService.cs
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

No controller or database migration change is required today.

Step 1 - Create PagedResponse

Open:

```
src\JobTrackr.Application\Common\PagedResponse.cs
```

Add:

```
namespace JobTrackr.Application.Common
{
    public class PagedResponse<T>
    {
        public List<T> Items { get; set; } = [];

        public int PageNumber { get; set; }

        public int PageSize { get; set; }

        public int TotalCount { get; set; }

        public int TotalPages { get; set; }
    }
}
```

What This File Does

`PagedResponse<T>` is a generic response wrapper. The `T` means the same class can later hold task DTOs, user DTOs, or another response type.

`Items` contains only the current page. The other properties describe that page and the complete filtered result.

Step 2 - Update ITaskService

Open:

```
src\JobTrackr.Application\Tasks\ITaskService.cs
```

Add this using statement:

```
using JobTrackr.Application.Common;
```

Change the `GetAllAsync` return type from:

```
Task<List<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int pageNumber,
    int pageSize,
    int userId);
```

to:

```
Task<PagedResponse<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    int pageNumber,
    int pageSize,
    int userId);
```

What Changed

The service no longer returns only `List<TaskResponse>`. It now promises to return the task list together with pagination metadata.

The method parameters remain unchanged.

Step 3 - Update TaskService Return Type

Open:

```
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

The file should already use the Application `Common` namespace. If it does not, add:

```
using JobTrackr.Application.Common;
```

Change the method signature from:

```
public async Task<List<TaskResponse>> GetAllAsync(
```

to:

```
public async Task<PagedResponse<TaskResponse>> GetAllAsync(
```

Keep all five existing parameters unchanged.

Step 4 - Count Before Pagination

Keep the existing ownership and filter code:

```
var query = _dbContext.Tasks.Where(task => task.UserId == userId);

if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}

if (!string.IsNullOrWhiteSpace(search))
{
    query = query.Where(task => task.Title.Contains(search));
}
```

Immediately after these filters, add:

```
var totalCount = await query.CountAsync();
```

This must happen before `Skip()` and `Take()`. `TotalCount` means all matching tasks owned by the authenticated user, not only the number on the current page.

Step 5 - Build Items And Metadata

Replace the current code beginning with `var skip` and ending with the returned list with:

```
var skip = (pageNumber - 1) * pageSize;

var items = await query
    .OrderBy(task => task.Id)
    .Skip(skip)
    .Take(pageSize)
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();

var totalPages = (int)Math.Ceiling(totalCount / (double)pageSize);

return new PagedResponse<TaskResponse>
{
    Items = items,
    PageNumber = pageNumber,
    PageSize = pageSize,
    TotalCount = totalCount,
    TotalPages = totalPages
};
```

Why Count Comes First

The query order is now:

```
1. Restrict tasks to the authenticated user
2. Apply completion and search filters
3. Count every matching task
4. Apply stable ordering
5. Skip previous pages
6. Take the requested page
7. Return items and metadata
```

If `CountAsync()` ran after `Take()`, `TotalCount` would only describe the current page.

TotalPages Formula

```
var totalPages = (int)Math.Ceiling(totalCount / (double)pageSize);
```

For five tasks with a page size of two:

```
5 / 2 = 2.5
Math.Ceiling(2.5) = 3 pages
```

The cast to `double` prevents integer division from turning `5 / 2` into `2`.

When no tasks match, `TotalCount` and `TotalPages` are both `0`, while `Items` is an empty array.

Step 6 - Controller Behavior

No controller edit is required.

The existing controller code:

```
var tasks = await _taskService.GetAllAsync(
    request.IsCompleted,
    request.Search,
    request.PageNumber,
    request.PageSize,
    currentUserId.Value);

return Ok(tasks);
```

will automatically serialize the new `PagedResponse<TaskResponse>` object.

Step 7 - Update Pagination Tests

Open:

```
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

In `GetAllAsync_WithSmallPageSize_ReturnsRequestedPage`, replace the current assertions with:

```
Assert.Equal(2, response.Items.Count);
Assert.Equal("Task 3", response.Items[0].Title);
Assert.Equal("Task 4", response.Items[1].Title);
Assert.Equal(2, response.PageNumber);
Assert.Equal(2, response.PageSize);
Assert.Equal(5, response.TotalCount);
Assert.Equal(3, response.TotalPages);
```

In `GetAllAsync_WithLargePageSize_ReturnsAllMatchingTasks`, replace the current assertion with:

```
Assert.Equal(5, response.Items.Count);
Assert.Equal(1, response.PageNumber);
Assert.Equal(100, response.PageSize);
Assert.Equal(5, response.TotalCount);
Assert.Equal(1, response.TotalPages);
```

What The Updated Tests Prove

The tests still verify the returned task items. They now also verify that the page number, page size, total task count, and calculated page count are correct.

You are updating two existing tests, so the total remains 17 tests.

Step 8 - Build And Test

Run:

```
cd
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-restore
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 17
Failed: 0
Skipped: 0
```

Step 9 - Run And Test The API

Start the API:

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Prepare a valid login token:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

Request page two:

```
$response = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=2&pageSize=2" `
    -Method Get `
    -Headers $headers

$response | ConvertTo-Json -Depth 5
```

Expected shape:

```
{
  "items": [],
  "pageNumber": 2,
  "pageSize": 2,
  "totalCount": 0,
  "totalPages": 0
}
```

Your actual items and counts depend on how many tasks the logged-in user owns.

Page Beyond Available Data

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=999&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Expected behavior:

- `200 OK`
- `items` is empty
- `totalCount` still describes all matching tasks

- `totalPages` still describes the real number of available pages

Pagination With Filters

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?isCompleted=false&search=api&pageNumber=1&pageSize=5" `
    -Method Get `
    -Headers $headers
```

`TotalCount` must count only tasks that belong to the authenticated user and match both filters.

Important Checks

```
[ ] PagedResponse<T> is in the Application project
[ ] Items is initialized to an empty list
[ ] ITaskService returns PagedResponse<TaskResponse>
[ ] TaskService returns PagedResponse<TaskResponse>
[ ] Ownership and filters run before CountAsync
[ ] CountAsync runs before Skip and Take
[ ] Items contains only the requested page
[ ] TotalPages uses Math.Ceiling
[ ] No controller change was needed
[ ] No migration was created
[ ] All 17 tests pass
```

Files Explained

PagedResponse.cs

Provides a reusable API response containing the current page of items and its pagination metadata. It belongs in Application because it describes an application response contract, not a database entity.

ITaskService.cs

Changes the Get All use-case contract from a plain task list to a paged task response. Its parameters and ownership rules remain unchanged.

TaskService.cs

Counts the complete filtered result, loads only the requested page, calculates total pages, and returns both items and metadata.

TaskServicePaginationTests.cs

Updates existing tests for the new response shape and verifies that the metadata is calculated correctly.

Interview Explanation

```
I changed the task list from a raw array to a generic paged response containing items and
pagination metadata.
The service counts the authenticated user's filtered tasks before applying Skip and Take, so
TotalCount and TotalPages describe the complete result rather than only the current page.
The API still returns an empty item list for pages beyond the available data.
```

End Of Day Note Template

```
1. What I built
I added a reusable PagedResponse DTO and changed the task list service to return items with
pageNumber, pageSize, totalCount, and totalPages.

2. What worked
GET /api/tasks now returns paged task items with accurate metadata, filters and ownership still
apply before counting, and all 17 tests pass.

3. What I will do tomorrow
Add task sorting by creation date with newest tasks first by default.
```

Day 95 - Sort Tasks By Created Date

Goal

Allow clients to sort paged tasks by creation date.

Today you will:

- add a `sortDirection` query parameter
- accept `asc` or `desc`
- use `desc` by default so newest tasks appear first
- keep pagination ordering stable with an id tie-breaker
- add focused sorting tests

Do not add due-date sorting today. That belongs to Day 96.

Current State

Day 94 returns paged tasks with metadata:

```
{
  "items": [],
  "pageNumber": 1,
  "pageSize": 10,
  "totalCount": 0,
  "totalPages": 0
}
```

The task query currently uses temporary ordering:

```
.OrderBy(task => task.Id)
```

Day 95 replaces this with created-date sorting.

API Behavior

Query	Result
`GET /api/tasks`	Newest tasks first
`GET /api/tasks?sortDirection=desc`	Newest tasks first
`GET /api/tasks?sortDirection=asc`	Oldest tasks first
`GET /api/tasks?sortDirection=random`	`400 Bad Request`

Sorting continues to work with ownership, completion, search, and pagination.

Files To Update And Create

Update

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
src\JobTrackr.Application\Tasks\ITaskService.cs
src\JobTrackr.Api\Controllers\TasksController.cs
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

Create

```
tests\JobTrackr.Tests\Tasks\TaskServiceSortingTests.cs
```

No entity or database migration change is required.

Step 1 - Add SortDirection To TaskQueryRequest

Open:

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
```

After `Search`, add:

```
[RegularExpression(
    "^(asc|desc)$",
    ErrorMessage = "Sort direction must be asc or desc.")]
public string SortDirection { get; set; } = "desc";
```

The complete DTO should be:

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Tasks
{
    public class TaskQueryRequest
    {
        public bool? IsCompleted { get; set; }

        public string? Search { get; set; }

        [RegularExpression(
            "^(asc|desc)$",
            ErrorMessage = "Sort direction must be asc or desc.")]
        public string SortDirection { get; set; } = "desc";

        [Range(1, int.MaxValue, ErrorMessage = "Page number must be at least 1.")]
        public int PageNumber { get; set; } = 1;

        [Range(1, 100, ErrorMessage = "Page size must be between 1 and 100.")]
        public int PageSize { get; set; } = 10;
    }
}
```

What This Does

`SortDirection` defaults to `desc`, which means descending order and therefore newest-created tasks first.

The validation attribute accepts only lowercase `asc` or `desc`. ASP.NET Core automatically returns `400 Bad Request` for another value before the controller calls the service.

Step 2 - Update ITaskService

Open:

```
src\JobTrackr.Application\Tasks\ITaskService.cs
```

Change `GetAllAsync` to:

```
Task<PagedResponse<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    string sortDirection,
    int pageNumber,
    int pageSize,
    int userId);
```

Only one parameter is new:

```
string sortDirection
```

Step 3 - Pass SortDirection From TasksController

Open:

```
src\JobTrackr.Api\Controllers\TasksController.cs
```

In `GetAll`, change the service call to:

```
var tasks = await _taskService.GetAllAsync(
    request.IsCompleted,
    request.Search,
    request.SortDirection,
    request.PageNumber,
    request.PageSize,
    currentUserId.Value);
```

The controller only passes the validated value. Sorting remains service/query logic.

Step 4 - Update TaskService Signature

Open:

```
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

Change the method signature to:

```
public async Task<PagedResponse<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    string sortDirection,
    int pageNumber,
    int pageSize,
    int userId)
```

Keep the ownership filters, search filters, `CountAsync()`, and page calculation unchanged.

Step 5 - Add Created-Date Ordering

After this existing line:

```
var skip = (pageNumber - 1) * pageSize;
```

add:

```
var orderedQuery = string.Equals(
    sortDirection,
    "asc",
    StringComparison.OrdinalIgnoreCase)
    ? query
        .OrderBy(task => task.CreatedAtUtc)
        .ThenBy(task => task.Id)
    : query
        .OrderByDescending(task => task.CreatedAtUtc)
        .ThenByDescending(task => task.Id);
```

Then change:

```
var items = await query
    .OrderBy(task => task.Id)
```

to:

```
var items = await orderedQuery
```

The remaining `Skip`, `Take`, `Select`, and `ToListAsync` code stays unchanged.

The relevant query section should now look like:

```
var totalCount = await query.CountAsync();

var skip = (pageNumber - 1) * pageSize;
```

```
var orderedQuery = string.Equals(
    sortDirection,
    "asc",
    StringComparison.OrdinalIgnoreCase)
    ? query
        .OrderBy(task => task.CreatedAtUtc)
        .ThenBy(task => task.Id)
    : query
        .OrderByDescending(task => task.CreatedAtUtc)
        .ThenByDescending(task => task.Id);

var items = await orderedQuery
    .Skip(skip)
    .Take(pageSize)
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();
```

Why ThenBy Uses Id

Multiple tasks can have the same `CreatedAtUtc` value. Pagination needs a predictable order even when dates are equal.

For ascending order:

```
.OrderBy(task => task.CreatedAtUtc)
.ThenBy(task => task.Id)
```

For descending order:

```
.OrderByDescending(task => task.CreatedAtUtc)
.ThenByDescending(task => task.Id)
```

The id is only a tie-breaker. Creation date remains the main sorting field.

Query Execution Order

```
1. Restrict tasks to the authenticated user
2. Apply completion filter
3. Apply title search
4. Count all matching tasks
5. Sort by CreatedAtUtc and Id
6. Skip previous pages
7. Take the requested page size
8. Map entities to TaskResponse DTOs
9. Execute the SQL query
```

Sorting must happen before `Skip()` and `Take()` so each page comes from one predictable ordered result.

Step 6 - Update Existing Pagination Tests

Open:

```
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

Both existing `GetAllAsync` calls need the default sorting argument after `search`:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "desc",
    2,
    2,
```

```
    user.Id);
```

and:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "desc",
    1,
    100,
    user.Id);
```

The test tasks currently share the same default creation date. Descending id is therefore used as the tie-breaker.

In the small-page test, change:

```
Assert.Equal("Task 4", response.Items[1].Title);
```

to:

```
Assert.Equal("Task 2", response.Items[1].Title);
```

With five tasks in descending order, pages are:

```
Page 1: Task 5, Task 4
Page 2: Task 3, Task 2
Page 3: Task 1
```

All other metadata assertions remain unchanged.

Step 7 - Add Sorting Tests

Create:

```
tests\JobTrackr.Tests\Tasks\TaskServiceSortingTests.cs
```

Add:

```
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Persistence;
using JobTrackr.Infrastructure.Tasks;
using Microsoft.EntityFrameworkCore;

namespace JobTrackr.Tests.Tasks
{
    public class TaskServiceSortingTests
    {
        [Fact]
        public async Task GetAllAsync_WithDescendingSort_ReturnsNewestTasksFirst()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddUserWithTasksAsync(dbContext);
            var taskService = new TaskService(dbContext);

            var response = await taskService.GetAllAsync(
                null,
                null,
                "desc",
                1,
                10,
                user.Id);

            Assert.Equal("Newest task", response.Items[0].Title);
            Assert.Equal("Middle task", response.Items[1].Title);
            Assert.Equal("Oldest task", response.Items[2].Title);
        }

        [Fact]
        public async Task GetAllAsync_WithAscendingSort_ReturnsOldestTasksFirst()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddUserWithTasksAsync(dbContext);
            var taskService = new TaskService(dbContext);
```

```
            var response = await taskService.GetAllAsync(
                null,
                null,
                "asc",
                1,
                10,
                user.Id);

            Assert.Equal("Oldest task", response.Items[0].Title);
            Assert.Equal("Middle task", response.Items[1].Title);
            Assert.Equal("Newest task", response.Items[2].Title);
        }

        private static AppDbContext CreateDbContext()
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;

            return new AppDbContext(options);
        }

        private static async Task<User> AddUserWithTasksAsync(AppDbContext dbContext)
        {
            var user = new User
            {
                FullName = "Sorting Test User",
                Email = "sorting.test@example.com",
                PasswordHash = "test-password-hash"
            };

            user.Tasks.Add(new JobTask
            {
                Title = "Middle task",
                Priority = "Medium",
                CreatedAtUtc = new DateTime(2026, 2, 1, 0, 0, 0, DateTimeKind.Utc)
            });

            user.Tasks.Add(new JobTask
            {
                Title = "Newest task",
                Priority = "Medium",
                CreatedAtUtc = new DateTime(2026, 3, 1, 0, 0, 0, DateTimeKind.Utc)
            });

            user.Tasks.Add(new JobTask
            {
                Title = "Oldest task",
                Priority = "Medium",
                CreatedAtUtc = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)
            });

            dbContext.Users.Add(user);
            await dbContext.SaveChangesAsync();

            return user;
        }
    }
}
```

What These Tests Prove

The tasks are deliberately inserted in a different order from their creation dates. This proves the service sorts by `CreatedAtUtc`, not insertion order or id alone.

One test verifies newest first and the other verifies oldest first.

Step 8 - Build And Test

Run:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 19
Failed: 0
Skipped: 0
```

The test count increases from 17 to 19.

Step 9 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Prepare a valid JWT token:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

API Test Examples

Default Newest First

```
$response = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers

$response.items | Select-Object id, title, createdAtUtc
```

Expected: newest creation dates appear first.

Explicit Newest First

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortDirection=desc&pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Oldest First

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortDirection=asc&pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Invalid Direction

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks?sortDirection=random" `
        -Method Get `
        -Headers $headers
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

Sorting With Existing Filters And Pagination

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/
tasks?isCompleted=false&search=api&sortDirection=desc&pageNumber=1&pageSize=5" `
    -Method Get `
    -Headers $headers
```

Expected:

- only the authenticated user's tasks
- only incomplete tasks matching `api`
- newest matching tasks first
- no more than five items
- accurate pagination metadata

Important Checks

```
[ ] TaskQueryRequest includes SortDirection
[ ] SortDirection defaults to desc
[ ] Validation accepts only asc or desc
[ ] Controller passes SortDirection
[ ] ITaskService includes sortDirection
[ ] TaskService includes sortDirection
[ ] CreatedAtUtc is the primary sort field
[ ] Id is used as a stable tie-breaker
[ ] Sorting happens before Skip and Take
[ ] CountAsync remains before pagination
[ ] Existing ownership and filters remain unchanged
[ ] No migration was created
[ ] All 19 tests pass
```

Files Explained

TaskQueryRequest.cs

Adds the public `sortDirection` query parameter, its default value, and input validation. It keeps all task-list query inputs in one request DTO.

ITaskService.cs

Adds sorting direction to the Get All use-case contract. It still returns the same `PagedResponse<TaskResponse>`.

TasksController.cs

Passes the validated sorting direction from the query DTO to the task service. It does not contain sorting logic.

TaskService.cs

Orders the filtered task query by creation date before applying pagination. It uses task id to make ordering stable when dates are equal.

TaskServicePaginationTests.cs

Passes the new default sort argument and updates the expected second-page order.

TaskServiceSortingTests.cs

Verifies both newest-first and oldest-first created-date ordering with deliberately unordered test data.

Interview Explanation

```
I added ascending and descending created-date sorting to the paged task query, with newest
first as the default.
The query applies ownership and filters first, counts the complete matching result, sorts by
CreatedAtUtc with Id as a stable tie-breaker, and then applies Skip and Take.
Invalid sort directions are rejected through request DTO validation.
```

End Of Day Note Template

```
1. What I built
I added created-date sorting to the task list endpoint with asc and desc directions, using
newest first by default.

2. What worked
GET /api/tasks sorts owned and filtered tasks by CreatedAtUtc before pagination, invalid
directions return 400, and all 19 tests pass.

3. What I will do tomorrow
Add task sorting by due date and define how tasks without due dates are ordered.
```

Day 96 - Sort Tasks By Due Date

Goal

Allow the task list endpoint to sort by either creation date or due date.

Today you will:

- add a `sortBy` query parameter
- keep `createdAt` as the default sort field
- support `dueDate` sorting
- support ascending and descending directions
- always place tasks without a due date last
- add focused due-date sorting tests

Do not add new filters or redesign the query system today.

Current State

Day 95 added:

```
sortDirection=desc  -> newest-created tasks first
sortDirection=asc   -> oldest-created tasks first
```

The API currently knows the direction but has only one sorting field: `CreatedAtUtc`.

Day 96 adds a separate field selector:

```
sortBy=createdAt
sortBy=dueDate
```

Final API Behavior

Query	Result
`GET /api/tasks`	Created newest first
`?sortBy=createdAt&sortDirection=asc`	Created oldest first
`?sortBy=dueDate&sortDirection=asc`	Earliest due date first; no due date last
`?sortBy=dueDate&sortDirection=desc`	Latest due date first; no due date last
`?sortBy=random`	`400 Bad Request`

Null Due-Date Rule

`DueDateUtc` is nullable because a task may not have a deadline.

For this API, tasks without a due date will always appear after tasks that have a due date:

```
Ascending:  earliest due -> latest due -> no due date
Descending: latest due -> earliest due -> no due date
```

This is predictable and keeps scheduled tasks together before unscheduled tasks.

Files To Update

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
src\JobTrackr.Application\Tasks\ITaskService.cs
src\JobTrackr.Api\Controllers\TasksController.cs
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
tests\JobTrackr.Tests\Tasks\TaskServiceSortingTests.cs
```

No new file, entity change, or database migration is required.

Step 1 - Add SortBy To TaskQueryRequest

Open:

```
src\JobTrackr.Application\Tasks\TaskQueryRequest.cs
```

Add `SortBy` before `SortDirection`:

```
[RegularExpression(
    "^(createdAt|dueDate)$",
    ErrorMessage = "Sort field must be createdAt or dueDate.")]
public string SortBy { get; set; } = "createdAt";
```

The complete request DTO should be:

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Tasks
{
    public class TaskQueryRequest
    {
        public bool? IsCompleted { get; set; }

        public string? Search { get; set; }

        [RegularExpression(
            "^(createdAt|dueDate)$",
            ErrorMessage = "Sort field must be createdAt or dueDate.")]
        public string SortBy { get; set; } = "createdAt";

        [RegularExpression(
            "^(asc|desc)$",
            ErrorMessage = "Sort direction must be asc or desc.")]
        public string SortDirection { get; set; } = "desc";

        [Range(1, int.MaxValue, ErrorMessage = "Page number must be at least 1.")]
        public int PageNumber { get; set; } = 1;

        [Range(1, 100, ErrorMessage = "Page size must be between 1 and 100.")]
        public int PageSize { get; set; } = 10;
    }
}
```

Why SortBy And SortDirection Are Separate

`SortBy` answers which field should be used:

```
createdAt or dueDate
```

`SortDirection` answers which direction should be used:

```
asc or desc
```

This avoids unclear combined values such as `dueDateDescending`.

Step 2 - Update ITaskService

Open:

```
src\JobTrackr.Application\Tasks\ITaskService.cs
```

Change `GetAllAsync` to:

```
Task<PagedResponse<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    string sortBy,
    string sortDirection,
    int pageNumber,
    int pageSize,
    int userId);
```

The only new parameter is:

```
string sortBy
```

Step 3 - Update TasksController

Open:

```
src\JobTrackr.Api\Controllers\TasksController.cs
```

In `GetAll`, pass `SortBy` before `SortDirection`:

```
var tasks = await _taskService.GetAllAsync(
    request.IsCompleted,
    request.Search,
    request.SortBy,
    request.SortDirection,
    request.PageNumber,
    request.PageSize,
    currentUserId.Value);
```

The controller continues to stay thin. It passes validated query values but does not sort data itself.

Step 4 - Update TaskService Signature

Open:

```
src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

Change the method signature to:

```
public async Task<PagedResponse<TaskResponse>> GetAllAsync(
    bool? isCompleted,
    string? search,
    string sortBy,
    string sortDirection,
    int pageNumber,
    int pageSize,
    int userId)
```

Do not change the ownership, completion, search, count, mapping, or response metadata logic.

Step 5 - Replace The Ordering Logic

Find the Day 95 ordering block beginning with:

```
var orderedQuery = string.Equals(
```

Replace the complete block with:

```
IOrderedQueryable<JobTask> orderedQuery;

if (string.Equals(sortBy, "dueDate", StringComparison.OrdinalIgnoreCase))
{
    if (string.Equals(sortDirection, "asc", StringComparison.OrdinalIgnoreCase))
    {
        orderedQuery = query
            .OrderBy(task => task.DueDateUtc == null)
            .ThenBy(task => task.DueDateUtc)
            .ThenBy(task => task.Id);
    }
```

```
    else
    {
        orderedQuery = query
            .OrderBy(task => task.DueDateUtc == null)
            .ThenByDescending(task => task.DueDateUtc)
            .ThenByDescending(task => task.Id);
    }
}
else if (string.Equals(sortDirection, "asc", StringComparison.OrdinalIgnoreCase))
{
    orderedQuery = query
        .OrderBy(task => task.CreatedAtUtc)
        .ThenBy(task => task.Id);
}
else
{
    orderedQuery = query
        .OrderByDescending(task => task.CreatedAtUtc)
        .ThenByDescending(task => task.Id);
}
```

Keep this existing items query immediately afterward:

```
var items = await orderedQuery
    .Skip(skip)
    .Take(pageSize)
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();
```

How Nulls Are Kept Last

This is the first due-date ordering expression:

```
.OrderBy(task => task.DueDateUtc == null)
```

For a task with a due date, the expression is `false`. For a task without one, it is `true`.

Ascending boolean order places `false` before `true`:

```
false -> tasks with due dates
true  -> tasks without due dates
```

The next `ThenBy` or `ThenByDescending` sorts only within those groups.

Why IOrderedQueryable Is Used

Each branch creates an ordered EF Core query, but execution should happen only once after the branch.

```
IOrderedQueryable<JobTask> orderedQuery;
```

stores whichever ordering was selected. `ToListAsync()` later executes the final SQL query.

This does not load all tasks into memory.

Stable Tie-Breaking

Two tasks may share the same due date. Task id keeps their order stable:

```
.ThenBy(task => task.Id)
```

or:

```
.ThenByDescending(task => task.Id)
```

Stable ordering is important because `Skip()` and `Take()` depend on rows staying in a predictable order.

Step 6 - Update Existing Pagination Test Calls

Open:

```
tests\JobTrackr.Tests\Tasks\TaskServicePaginationTests.cs
```

Both existing calls need `"createdAt"` before `"desc"`.

First call:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "createdAt",
    "desc",
    2,
    2,
    user.Id);
```

Second call:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "createdAt",
    "desc",
    1,
    100,
    user.Id);
```

Do not change the existing assertions.

Step 7 - Update Created-Date Sorting Test Calls

Open:

```
tests\JobTrackr.Tests\Tasks\TaskServiceSortingTests.cs
```

In both existing created-date tests, add `"createdAt"` before the direction.

Descending call:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "createdAt",
    "desc",
    1,
    10,
    user.Id);
```

Ascending call:

```
var response = await taskService.GetAllAsync(
    null,
    null,
    "createdAt",
    "asc",
    1,
    10,
    user.Id);
```

Step 8 - Add Due-Date Sorting Tests

In `TaskServiceSortingTests`, add these two tests after the created-date tests:

```
[Fact]
public async Task GetAllAsync_WithDueDateAscending_ReturnsEarliestDueDateFirstAndNullLast()
```

```
{
    await using var dbContext = CreateDbContext();
    var user = await AddUserWithDueDatesAsync(dbContext);
    var taskService = new TaskService(dbContext);

    var response = await taskService.GetAllAsync(
        null,
        null,
        "dueDate",
        "asc",
        1,
        10,
        user.Id);

    Assert.Equal("Due soon", response.Items[0].Title);
    Assert.Equal("Due later", response.Items[1].Title);
    Assert.Equal("No due date", response.Items[2].Title);
}

[Fact]
public async Task GetAllAsync_WithDueDateDescending_ReturnsLatestDueDateFirstAndNullLast()
{
    await using var dbContext = CreateDbContext();
    var user = await AddUserWithDueDatesAsync(dbContext);
    var taskService = new TaskService(dbContext);

    var response = await taskService.GetAllAsync(
        null,
        null,
        "dueDate",
        "desc",
        1,
        10,
        user.Id);

    Assert.Equal("Due later", response.Items[0].Title);
    Assert.Equal("Due soon", response.Items[1].Title);
    Assert.Equal("No due date", response.Items[2].Title);
}
```

Add this helper below the existing `AddUserWithTasksAsync` helper:

```
private static async Task<User> AddUserWithDueDatesAsync(AppDbContext dbContext)
{
    var user = new User
    {
        FullName = "Due Date Sorting User",
        Email = "due.date.sorting@example.com",
        PasswordHash = "test-password-hash"
    };

    user.Tasks.Add(new JobTask
    {
        Title = "No due date",
        Priority = "Medium",
        DueDateUtc = null
    });

    user.Tasks.Add(new JobTask
    {
        Title = "Due later",
        Priority = "Medium",
        DueDateUtc = new DateTime(2026, 8, 20, 0, 0, 0, DateTimeKind.Utc)
    });

    user.Tasks.Add(new JobTask
    {
        Title = "Due soon",
        Priority = "Medium",
        DueDateUtc = new DateTime(2026, 8, 18, 0, 0, 0, DateTimeKind.Utc)
    });

    dbContext.Users.Add(user);
    await dbContext.SaveChangesAsync();

    return user;
}
```

What These Tests Prove

The tasks are inserted in an order that differs from their due dates. The tests prove that:

- ascending means earliest due date first
- descending means latest due date first
- a null due date remains last in both directions

The total test count increases from 19 to 21.

Step 9 - Check Every GetAllAsync Call

Use this command to find any call that still has the old parameter list:

```
cd
rg -n "GetAllAsync\(" src tests
```

Every task-service `GetAllAsync` call must now pass:

```
isCompleted
search
sortBy
sortDirection
pageNumber
pageSize
userId
```

User-service `GetAllAsync()` methods are unrelated and should not be changed.

Step 10 - Build And Test

Run:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 21
Failed: 0
Skipped: 0
```

Step 11 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Prepare a valid JWT token:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

API Test Examples

Default Created Date Sorting

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Expected: created-date descending remains the default, so existing clients keep the Day 95 behavior.

Earliest Due Date First

```
$response = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortBy=dueDate&sortDirection=asc&pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers

$response.items | Select-Object id, title, dueDateUtc
```

Expected: the earliest non-null due dates appear first and null due dates appear last.

Latest Due Date First

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortBy=dueDate&sortDirection=desc&pageNumber=1&pageSize=10" `
    -Method Get `
    -Headers $headers
```

Expected: the latest non-null due dates appear first and null due dates still appear last.

Invalid Sort Field

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks?sortBy=priority" `
        -Method Get `
        -Headers $headers
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

Due-Date Sorting With Filters And Pagination

```
Invoke-RestMethod `
    -Uri "$baseUrl/api/
tasks?isCompleted=false&search=api&sortBy=dueDate&sortDirection=asc&pageNumber=1&pageSize=5" `
    -Method Get `
    -Headers $headers
```

Expected:

- only the authenticated user's tasks
- only incomplete tasks matching `api`
- earliest non-null due dates first
- null due dates last
- no more than five items
- accurate pagination metadata

Important Checks

```
[ ] TaskQueryRequest includes SortBy
[ ] SortBy defaults to createdAt
[ ] SortBy accepts only createdAt or dueDate
[ ] SortDirection still defaults to desc
[ ] Controller passes SortBy before SortDirection
[ ] Interface and implementation signatures match
[ ] Created-date sorting still works
[ ] Due-date ascending works
[ ] Due-date descending works
[ ] Null due dates are last in both directions
[ ] Id remains the stable tie-breaker
[ ] Sorting happens before Skip and Take
[ ] Ownership and filters remain unchanged
[ ] No migration was created
[ ] All 21 tests pass
```

Files Explained

TaskQueryRequest.cs

Adds the `sortBy` query parameter and validates the two supported fields. It keeps `createdAt` as the default to preserve existing behavior.

ITaskService.cs

Adds the selected sort field to the task-list use-case contract. The paged response and other query parameters remain unchanged.

TasksController.cs

Passes the validated sort field and direction into the service. No query logic is placed in the controller.

TaskService.cs

Selects created-date or due-date ordering before pagination. Its due-date ordering deliberately places null values last in both directions.

TaskServicePaginationTests.cs

Adds the default `createdAt` argument to existing direct service calls without changing their behavior.

TaskServiceSortingTests.cs

Keeps the created-date tests and adds ascending and descending due-date tests, including null handling.

Interview Explanation

```
I extended the paged task query to support sorting by either creation date or due date.
For due-date sorting, I first group non-null dates before null dates, then apply the requested
date direction and an id tie-breaker before pagination.
The default remains created-date descending so existing API behavior stays compatible.
```

End Of Day Note Template

```
1. What I built
I added due-date sorting to the task list endpoint while keeping created-date sorting as the
default.

2. What worked
Tasks can be sorted by due date in ascending or descending order, tasks without due dates
appear last, and all 21 tests pass.

3. What I will do tomorrow
Verify that filtering, sorting, and pagination work correctly together.
```

Day 97 - Verify Filtering, Sorting, And Pagination Together

Goal

Prove that the complete task-list query works correctly when all query features are combined.

Today you will verify:

- authenticated-user ownership filtering
- completion filtering
- title search
- due-date sorting
- null due-date placement
- page one and page two results
- accurate pagination metadata

This is primarily a test day. Do not change production code unless a test reveals a real defect.

Current Query Features

The task list now supports:

```
isCompleted
search
sortBy
sortDirection
pageNumber
pageSize
```

Each feature has been implemented separately. Day 97 verifies that they still behave correctly when used in the same request.

Why Combined Tests Matter

A pagination test can pass while filtering is wrong. A sorting test can pass while another user's tasks leak into the result.

The combined tests create data that must be excluded for different reasons:

```
another user's matching task -> excluded by ownership
completed matching task      -> excluded by completion filter
incomplete nonmatching task  -> excluded by title search
four valid matching tasks    -> sorted and split across two pages
```

Expected Query Order

The service should process the query in this logical order:

```
1. Restrict tasks to the authenticated user
2. Apply isCompleted filter
3. Apply title search
4. Count all remaining matching tasks
5. Apply requested stable sorting
6. Skip tasks from earlier pages
7. Take the requested page size
8. Map entities to TaskResponse DTOs
```

```
9. Execute the query
```

`TotalCount` must be calculated after filters but before `Skip()` and `Take()`.

File To Create

```
tests\JobTrackr.Tests\Tasks\TaskServiceCombinedQueryTests.cs
```

Create it with:

```
cd
New-Item tests\JobTrackr.Tests\Tasks\TaskServiceCombinedQueryTests.cs -ItemType File
```

No Application, Domain, Infrastructure, controller, or migration file should need to change today.

Step 1 - Create The Combined Query Test File

Open:

```
tests\JobTrackr.Tests\Tasks\TaskServiceCombinedQueryTests.cs
```

Add:

```
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Persistence;
using JobTrackr.Infrastructure.Tasks;
using Microsoft.EntityFrameworkCore;

namespace JobTrackr.Tests.Tasks
{
    public class TaskServiceCombinedQueryTests
    {
        [Fact]
        public async Task GetAllAsync_WithCombinedQuery_ReturnsFilteredSortedFirstPage()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddQueryTestDataAsync(dbContext);
            var taskService = new TaskService(dbContext);

            var response = await taskService.GetAllAsync(
                false,
                "API",
                "dueDate",
                "asc",
                1,
                2,
                user.Id);

            Assert.Equal(2, response.Items.Count);
            Assert.Equal("API due soon", response.Items[0].Title);
            Assert.Equal("API due middle", response.Items[1].Title);
            Assert.All(response.Items, task => Assert.Equal(user.Id, task.UserId));
            Assert.Equal(1, response.PageNumber);
            Assert.Equal(2, response.PageSize);
            Assert.Equal(4, response.TotalCount);
            Assert.Equal(2, response.TotalPages);
        }

        [Fact]
        public async Task GetAllAsync_WithCombinedQuery_ReturnsCorrectSecondPageAndMetadata()
        {
            await using var dbContext = CreateDbContext();
            var user = await AddQueryTestDataAsync(dbContext);
            var taskService = new TaskService(dbContext);

            var response = await taskService.GetAllAsync(
                false,
                "API",
                "dueDate",
                "asc",
                2,
                2,
                user.Id);

            Assert.Equal(2, response.Items.Count);
```

```
    Assert.Equal("API due later", response.Items[0].Title);
    Assert.Equal("API no due date", response.Items[1].Title);
    Assert.All(response.Items, task => Assert.Equal(user.Id, task.UserId));
    Assert.Equal(2, response.PageNumber);
    Assert.Equal(2, response.PageSize);
    Assert.Equal(4, response.TotalCount);
    Assert.Equal(2, response.TotalPages);
}

private static AppDbContext CreateDbContext()
{
    var options = new DbContextOptionsBuilder<AppDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    return new AppDbContext(options);
}

private static async Task<User> AddQueryTestDataAsync(AppDbContext dbContext)
{
    var user = new User
    {
        FullName = "Combined Query User",
        Email = "combined.query@example.com",
        PasswordHash = "test-password-hash"
    };

    user.Tasks.Add(new JobTask
    {
        Title = "API due later",
        Priority = "Medium",
        IsCompleted = false,
        DueDateUtc = new DateTime(2026, 8, 22, 0, 0, 0, DateTimeKind.Utc)
    });

    user.Tasks.Add(new JobTask
    {
        Title = "API due soon",
        Priority = "Medium",
        IsCompleted = false,
        DueDateUtc = new DateTime(2026, 8, 18, 0, 0, 0, DateTimeKind.Utc)
    });

    user.Tasks.Add(new JobTask
    {
        Title = "API no due date",
        Priority = "Medium",
        IsCompleted = false,
        DueDateUtc = null
    });

    user.Tasks.Add(new JobTask
    {
        Title = "API due middle",
        Priority = "Medium",
        IsCompleted = false,
        DueDateUtc = new DateTime(2026, 8, 20, 0, 0, 0, DateTimeKind.Utc)
    });

    user.Tasks.Add(new JobTask
    {
        Title = "API completed",
        Priority = "Medium",
        IsCompleted = true,
        DueDateUtc = new DateTime(2026, 8, 17, 0, 0, 0, DateTimeKind.Utc)
    });

    user.Tasks.Add(new JobTask
    {
        Title = "Database task",
        Priority = "Medium",
        IsCompleted = false,
        DueDateUtc = new DateTime(2026, 8, 16, 0, 0, 0, DateTimeKind.Utc)
    });

    var otherUser = new User
    {
        FullName = "Other Query User",
```

```
                Email = "other.query@example.com",
                PasswordHash = "test-password-hash"
            };

            otherUser.Tasks.Add(new JobTask
            {
                Title = "API other user task",
                Priority = "Medium",
                IsCompleted = false,
                DueDateUtc = new DateTime(2026, 8, 15, 0, 0, 0, DateTimeKind.Utc)
            });

            dbContext.Users.AddRange(user, otherUser);
            await dbContext.SaveChangesAsync();

            return user;
        }
    }
}
```

Step 2 - Understand The Test Data

The authenticated test user owns six tasks:

Task	Completed	Matches `API`	Due date	Expected
API due soon	No	Yes	Aug 18	Included
API due middle	No	Yes	Aug 20	Included
API due later	No	Yes	Aug 22	Included
API no due date	No	Yes	Null	Included last
API completed	Yes	Yes	Aug 17	Excluded
Database task	No	No	Aug 16	Excluded

Another user owns `API other user task`. It must be excluded even though its title, completion state, and due date match the query.

Step 3 - Understand The Combined Request

Both tests call:

```
await taskService.GetAllAsync(
    false,
    "API",
    "dueDate",
    "asc",
    pageNumber,
    2,
    user.Id);
```

Parameter meaning:

```
isCompleted = false
search = API
sortBy = dueDate
sortDirection = asc
pageNumber = 1 or 2
pageSize = 2
userId = authenticated test user
```

After ownership and filters, exactly four tasks remain.

Step 4 - Understand The Expected Pages

Due-date ascending order with null last gives:

```
1. API due soon
2. API due middle
3. API due later
4. API no due date
```

With `pageSize = 2`:

```
Page 1: API due soon, API due middle
Page 2: API due later, API no due date
```

Metadata for both pages should be:

```
pageSize = 2
totalCount = 4
totalPages = 2
```

Only `pageNumber` and `items` change between the responses.

What The First Test Proves

The first test proves that:

- another user's earlier task is not returned
- the completed API task is not returned
- the incomplete Database task is not returned
- the remaining tasks are sorted before pagination
- page one contains the first two sorted matches
- metadata counts all four filtered matches

What The Second Test Proves

The second test proves that:

- `Skip()` happens after filtering and sorting
- page two contains the next two tasks
- the null due date remains last
- `TotalCount` does not become the current page count
- metadata remains stable across pages

Why No Production Code Is Added

Days 92 through 96 already implemented the query features. Day 97 combines them in tests to protect their behavior.

If the tests pass, do not refactor `TaskService` merely to create a code change. A focused verification commit is meaningful engineering work.

If a test fails, inspect this order in `TaskService`:

```
Where owner
Where completion
Where search
CountAsync
OrderBy / ThenBy
Skip
Take
Select
ToListAsync
```

Fix only the demonstrated defect.

Step 5 - Build And Test

Run:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 23
Failed: 0
Skipped: 0
```

The test count increases from 21 to 23.

Step 6 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Prepare a valid JWT token:

```
$baseUrl = "https://localhost:7024"
$token = "PASTE_LOGIN_TOKEN_HERE"
$headers = @{ Authorization = "Bearer $token" }
```

Manual API Test

Use a logged-in user that already has several tasks, then run:

```
$page1 = Invoke-RestMethod `
    -Uri "$baseUrl/api/
tasks?isCompleted=false&search=API&sortBy=dueDate&sortDirection=asc&pageNumber=1&pageSize=2" `
    -Method Get `
    -Headers $headers

$page1 | ConvertTo-Json -Depth 5
```

Request page two:

```
$page2 = Invoke-RestMethod `
    -Uri "$baseUrl/api/
tasks?isCompleted=false&search=API&sortBy=dueDate&sortDirection=asc&pageNumber=2&pageSize=2" `
    -Method Get `
    -Headers $headers

$page2 | ConvertTo-Json -Depth 5
```

Check:

```
page 1 and page 2 contain different items
each page contains at most two items
only incomplete matching tasks appear
due dates increase across the pages
tasks without a due date appear last
totalCount and totalPages are identical on both pages
```

The exact item count depends on your local SQL Server data.

Important Checks

```
[ ] New combined-query test file exists
[ ] Test data contains two users
[ ] Other user's matching task is excluded
[ ] Completed matching task is excluded
[ ] Incomplete nonmatching task is excluded
[ ] Four valid matching tasks remain
[ ] Due-date ascending sorting is correct
[ ] Null due date is last
[ ] Page one contains the first two matches
[ ] Page two contains the next two matches
[ ] TotalCount is 4 on both pages
[ ] TotalPages is 2 on both pages
[ ] No production code changed unless a defect was found
[ ] No migration was created
[ ] All 23 tests pass
```

File Explained

TaskServiceCombinedQueryTests.cs

Creates realistic mixed task data and verifies the complete task-list query over two pages. It protects ownership, filters, sorting, null handling, pagination, and metadata from future regressions.

Interview Explanation

```
I added combined query tests for the authenticated task list instead of testing each query
feature only in isolation.
The tests verify ownership, completion filtering, title search, due-date sorting, null
placement, pagination, and metadata across two pages.
This confirms filters and counting happen before stable sorting and pagination.
```

End Of Day Note Template

```
1. What I built
I added combined TaskService tests for ownership, completion filtering, title search, due-date
sorting, and pagination.

2. What worked
The query returns the correct first and second pages, excludes another user's and nonmatching
tasks, keeps null due dates last, returns accurate metadata, and all 23 tests pass.

3. What I will do tomorrow
Review and manually test search, filtering, sorting, and pagination combinations for confusing
behavior.
```

Day 98 - Task Query Review

Goal

Manually review the complete `GET /api/tasks` query through the real API and SQL Server data.

Today you will test:

- default query behavior
- completion filtering
- title search
- created-date sorting
- due-date sorting and null placement
- pagination and metadata
- combined query parameters
- invalid query values
- authenticated-user ownership

Do not add a new feature today. Change code only if manual testing reveals incorrect or confusing behavior.

Current Query Contract

The endpoint is:

```
GET /api/tasks
```

Supported query parameters:

Parameter	Values	Default
`isCompleted`	`true`, `false`, or omitted	omitted
`search`	title text or omitted	omitted
`sortBy`	`createdAt`, `dueDate`	`createdAt`
`sortDirection`	`asc`, `desc`	`desc`
`pageNumber`	1 or greater	1
`pageSize`	1 through 100	10

The sort values are lowercase and case-sensitive at the request-validation boundary.

Expected Response Shape

```
{
  "items": [],
  "pageNumber": 1,
  "pageSize": 10,
  "totalCount": 0,
  "totalPages": 0
}
```

`items` contains only the requested page. `totalCount` describes every owned task that matches the filters before pagination.

Expected Processing Order

```
1. Restrict tasks to the authenticated user
2. Apply isCompleted when supplied
3. Apply title search when supplied
4. Count all matching tasks
5. Apply stable sorting
6. Skip earlier pages
7. Take the requested page size
8. Return items and metadata
```

Files To Change

Expected:

```
No code files
No migration files
```

This is a review day. Record what you tested and fix only demonstrated defects.

Step 1 - Build And Run Automated Tests

Before manual API testing, verify the current baseline:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 23
Failed: 0
Skipped: 0
```

Step 2 - Start The API

```
cd
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Keep this terminal running. Open another PowerShell terminal for the API requests.

Step 3 - Login And Prepare Authorization

In the second PowerShell terminal:

```
$baseUrl = "https://localhost:7024"

$loginBody = @{
    email = "YOUR_EXISTING_EMAIL"
    password = "YOUR_EXISTING_PASSWORD"
} | ConvertTo-Json

$login = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $loginBody

$token = $login.token
$headers = @{ Authorization = "Bearer $token" }
```

Confirm a token was returned:

```
$token.Length
```

Expected: a number greater than zero.

Optional - Add Review Tasks

If the logged-in user has too few tasks, create several through Swagger or use this helper:

```
function New-ReviewTask {
    param(
        [string]$Title,
        [object]$DueDateUtc = $null
    )

    $bodyData = @{
        title = $Title
        description = "Day 98 query review"
        priority = "Medium"
    }

    if ($null -ne $DueDateUtc) {
        $bodyData.dueDateUtc = $DueDateUtc
    }

    $body = $bodyData | ConvertTo-Json

    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks" `
        -Method Post `
        -Headers $headers `
        -ContentType "application/json" `
        -Body $body
}
```

Example calls:

```
New-ReviewTask -Title "API review early" -DueDateUtc "2026-08-20T00:00:00Z"
New-ReviewTask -Title "API review later" -DueDateUtc "2026-08-25T00:00:00Z"
New-ReviewTask -Title "API review no date"
New-ReviewTask -Title "Database review" -DueDateUtc "2026-08-22T00:00:00Z"
```

These tasks are stored for the logged-in user because task creation uses the user id from the JWT.

For the task without a due date, the helper omits `dueDateUtc` from the JSON. ASP.NET Core therefore binds the nullable `DueDateUtc` property as `null` instead of trying to convert an empty string into a date.

Test 1 - Default Query

```
$response = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks" `
    -Method Get `
    -Headers $headers

$response | ConvertTo-Json -Depth 5
```

Check:

```
[ ] Status is 200
[ ] PageNumber is 1
[ ] PageSize is 10
[ ] Items contains at most 10 tasks
[ ] Tasks belong only to the logged-in user
[ ] Newest CreatedAtUtc values appear first
[ ] TotalCount is at least Items.Count
[ ] TotalPages matches TotalCount and PageSize
```

Useful display:

```
$response.items | Select-Object id, title, createdAtUtc, dueDateUtc, isCompleted
```

Test 2 - Completion Filters

Incomplete tasks:

```
$incomplete = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?isCompleted=false&pageSize=100" `
    -Method Get `
```

```
    -Headers $headers

$incomplete.items | Select-Object title, isCompleted
```

Check that every returned `isCompleted` value is `False`.

Completed tasks:

```
$completed = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?isCompleted=true&pageSize=100" `
    -Method Get `
    -Headers $headers

$completed.items | Select-Object title, isCompleted
```

Check that every returned `isCompleted` value is `True`.

An empty `items` array is valid when the user has no tasks in one state.

Test 3 - Title Search

```
$searchResult = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?search=API&pageSize=100" `
    -Method Get `
    -Headers $headers

$searchResult.items | Select-Object title
```

Check that every returned title contains the search text according to your SQL Server database collation.

Search currently applies only to `Title`, not `Description`.

No matches should return:

```
{
  "items": [],
  "pageNumber": 1,
  "pageSize": 10,
  "totalCount": 0,
  "totalPages": 0
}
```

It should return `200 OK`, not `404 Not Found`.

Test 4 - Created-Date Sorting

Newest first:

```
$createdDesc = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortBy=createdAt&sortDirection=desc&pageSize=100" `
    -Method Get `
    -Headers $headers

$createdDesc.items | Select-Object title, createdAtUtc
```

Oldest first:

```
$createdAsc = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortBy=createdAt&sortDirection=asc&pageSize=100" `
    -Method Get `
    -Headers $headers

$createdAsc.items | Select-Object title, createdAtUtc
```

Check that the two responses use opposite creation-date directions.

Test 5 - Due-Date Sorting

Earliest due dates first:

```
$dueAsc = Invoke-RestMethod `
```

```
    -Uri "$baseUrl/api/tasks?sortBy=dueDate&sortDirection=asc&pageSize=100" `
    -Method Get `
    -Headers $headers

$dueAsc.items | Select-Object title, dueDateUtc
```

Latest due dates first:

```
$dueDesc = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?sortBy=dueDate&sortDirection=desc&pageSize=100" `
    -Method Get `
    -Headers $headers

$dueDesc.items | Select-Object title, dueDateUtc
```

Check:

```
[ ] Ascending non-null dates go from earliest to latest
[ ] Descending non-null dates go from latest to earliest
[ ] Null due dates appear last in both responses
```

Test 6 - Pagination

Request two small pages:

```
$page1 = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=2" `
    -Method Get `
    -Headers $headers

$page2 = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=2&pageSize=2" `
    -Method Get `
    -Headers $headers
```

Display both pages:

```
$page1.items | Select-Object id, title
$page2.items | Select-Object id, title
```

Check:

```
[ ] Each page has at most two items
[ ] Page 1 and page 2 do not repeat task ids
[ ] TotalCount is the same on both pages
[ ] TotalPages is the same on both pages
[ ] PageNumber reflects the requested page
```

Test 7 - Page Beyond Available Data

```
$beyond = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks?pageNumber=999&pageSize=10" `
    -Method Get `
    -Headers $headers

$beyond | ConvertTo-Json -Depth 5
```

Expected:

```
Status: 200 OK
Items: empty array
PageNumber: 999
PageSize: 10
TotalCount: unchanged total matching count
TotalPages: unchanged real page count
```

An out-of-range page is not a missing resource, so it should not return `404`.

Test 8 - Combined Query

```
$combined = Invoke-RestMethod `
    -Uri "$baseUrl/api/
tasks?isCompleted=false&search=API&sortBy=dueDate&sortDirection=asc&pageNumber=1&pageSize=2" `
    -Method Get `
    -Headers $headers

$combined | ConvertTo-Json -Depth 5
```

Check:

```
[ ] Only incomplete tasks appear
[ ] Every title matches API
[ ] Due dates are ascending
[ ] Null due dates appear after non-null dates
[ ] Items contains at most two tasks
[ ] Metadata describes the complete filtered result
```

This manually confirms the behavior protected by Day 97's combined automated tests.

Test 9 - Invalid Query Values

Use this helper to display the HTTP status and validation response:

```
function Test-InvalidTaskQuery {
    param([string]$Query)

    try {
        Invoke-RestMethod `
            -Uri "$baseUrl/api/tasks?$Query" `
            -Method Get `
            -Headers $headers
    } catch {
        [int]$_.Exception.Response.StatusCode
        $_.ErrorDetails.Message
    }
}
```

Run:

```
Test-InvalidTaskQuery "pageNumber=0"
Test-InvalidTaskQuery "pageSize=0"
Test-InvalidTaskQuery "pageSize=101"
Test-InvalidTaskQuery "sortBy=priority"
Test-InvalidTaskQuery "sortDirection=random"
```

Each request should return:

```
400 Bad Request
```

The validation response should identify the invalid field.

Test 10 - Authentication And Ownership

Without an authorization header:

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks" `
        -Method Get
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

If you have tokens for two users, request `GET /api/tasks?pageSize=100` with each token. Each response must contain only tasks created by that authenticated user.

Day 97 already protects ownership filtering with an automated test, so a second account is optional for today's manual review.

Review For Confusing Behavior

After the tests, answer these questions:

```
[ ] Are parameter names clear in Swagger?
[ ] Are accepted sort values obvious?
[ ] Do default results show newest-created tasks first?
[ ] Are null due dates consistently last?
[ ] Does an empty result return a useful paged shape?
[ ] Does a page beyond available data behave predictably?
[ ] Do validation errors identify the invalid field?
[ ] Do filters preserve accurate TotalCount and TotalPages?
```

If every answer is yes, do not change code.

If You Find A Problem

Before editing, write down:

```
Request:
Expected result:
Actual result:
```

Then fix only the demonstrated issue and add or update a test that would fail without the fix.

Do not redesign the query system during review.

Final Verification

After manual testing or any necessary fix:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
git status
```

Expected when no issue was found:

```
Build succeeds with 0 warnings and 0 errors
All 23 tests pass
Git working tree is clean
```

Important Checks

```
[ ] Default query works
[ ] Completion filtering works
[ ] Title search works
[ ] Created-date asc and desc work
[ ] Due-date asc and desc work
[ ] Null due dates stay last
[ ] Pagination pages do not overlap
[ ] Page metadata stays accurate
[ ] Combined query works
[ ] Invalid values return 400
[ ] Anonymous request returns 401
[ ] Ownership remains isolated
[ ] All 23 automated tests pass
[ ] No unnecessary code change was made
```

Interview Explanation

I manually reviewed the authenticated task-list contract against real HTTP requests and SQL Server data.
I checked defaults, filters, both sorting fields, null handling, pagination boundaries, validation, and ownership.
The existing automated tests remained green, and I avoided changing production code when no defect was demonstrated.

End Of Day Note Template

1. What I reviewed
I manually reviewed task search, completion filtering, created-date and due-date sorting, pagination, validation, and ownership behavior.

2. What worked
The query combinations returned predictable paged results and metadata, invalid values returned 400, anonymous access returned 401, and all 23 automated tests passed.

3. What I will do tomorrow
Add GET /api/users/me so an authenticated user can retrieve their own profile.

Day 99 - Add Current User Profile Endpoint

Goal

Add an authenticated endpoint that returns the currently logged-in user's profile.

The new endpoint will be:

```
GET /api/users/me
```

Today you will:

- protect the endpoint with JWT authentication
- read the user id from the JWT claims
- reuse the existing `IUserService.GetByIdAsync` method
- return the existing safe `UserResponse` DTO
- test authenticated and anonymous requests

Do not create a new profile service or DTO today. The existing user use case already provides the required behavior.

Expected Behavior

Request	Expected result
Valid Bearer token	`200 OK` with the logged-in user's profile
Missing Bearer token	`401 Unauthorized`
Invalid or expired token	`401 Unauthorized`
Valid token whose user no longer exists	`404 Not Found`

Expected Response

```
{
  "id": 1,
  "fullName": "Example User",
  "email": "user@example.com",
  "createdAtUtc": "2026-08-20T00:00:00Z"
}
```

The response must not include:

```
password
passwordHash
JWT token
```

Existing Code You Will Reuse

The Application layer already has:

```
Task<UserResponse?> GetByIdAsync(int id);
```

The Infrastructure implementation already reads the user from the database and maps it to:

```
public class UserResponse
{
    public int Id { get; set; }
```

```
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
}
```

Therefore, no service, DTO, entity, database, or migration change is needed.

File To Update

```
src\JobTrackr.Api\Controllers\UsersController.cs
```

Only this file should change today.

Step 1 - Add Required Using Statements

Open:

```
src\JobTrackr.Api\Controllers\UsersController.cs
```

Add:

```
using Microsoft.AspNetCore.Authorization;
using System.Security.Claims;
```

The top of the file should include:

```
using JobTrackr.Application.Common;
using JobTrackr.Application.Tasks;
using JobTrackr.Application.Users;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
```

Why These Usings Are Needed

`Microsoft.AspNetCore.Authorization` provides the `[Authorize]` attribute.

`System.Security.Claims` provides `ClaimTypes.NameIdentifier`, which identifies the logged-in user from the JWT.

Step 2 - Add GET /api/users/me

Inside `UsersController`, place this action after `GetAll` and before `GetById`:

```
[HttpGet("me")]
[Authorize]
public async Task<IActionResult> GetCurrentUser()
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var user = await _userService.GetByIdAsync(currentUserId.Value);

    if (user is null)
    {
        return Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Not Found",
            detail: ErrorMessages.UserNotFound);
    }

    return Ok(user);
}
```

What This Action Does

1. `[Authorize]` requires a valid JWT.
2. `GetCurrentUserId()` reads the user id claim.
3. The existing user service retrieves that user from SQL Server.
4. A missing user returns the existing safe ProblemDetails response.
5. A found user returns `200 OK` with `UserResponse`.

Step 3 - Add The Claim Helper

At the bottom of `UsersController`, before the final class closing brace, add:

```
private int? GetCurrentUserId()
{
    var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (int.TryParse(userIdValue, out var userId))
    {
        return userId;
    }

    return null;
}
```

This is the same simple claim-reading approach already used by `TasksController`.

Why The Return Type Is int?

```
private int? GetCurrentUserId()
```

The method returns an integer when the claim is valid. It returns `null` when the claim is missing or cannot be converted to an integer.

The controller then safely returns:

```
return Unauthorized();
```

Relevant Final Controller Structure

Your controller should have this general structure:

```
[Route("api/[controller]")]
[ApiController]
public class UsersController : ControllerBase
{
    private readonly IUserService _userService;
    private readonly ITaskService _taskService;

    public UsersController(IUserService userService, ITaskService taskService)
    {
        _userService = userService;
        _taskService = taskService;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var users = await _userService.GetAllAsync();

        return Ok(users);
    }

    [HttpGet("me")]
    [Authorize]
    public async Task<IActionResult> GetCurrentUser()
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
```

```
        {
            return Unauthorized();
        }

        var user = await _userService.GetByIdAsync(currentUserId.Value);

        if (user is null)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.UserNotFound);
        }

        return Ok(user);
    }

    // Keep all existing user actions unchanged.

    private int? GetCurrentUserId()
    {
        var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);

        if (int.TryParse(userIdValue, out var userId))
        {
            return userId;
        }

        return null;
    }
}
```

Do not replace the controller with this shortened example because it intentionally omits existing actions. Add only the new action, usings, and helper to your current file.

Why /me Is Better Than A Request User Id

The client should call:

```
GET /api/users/me
```

It should not send:

```
GET /api/users/me?userId=5
```

The JWT already identifies the authenticated user. Accepting a profile user id from the client would be unnecessary and easier to misuse.

Route Behavior

The controller already has:

```
[HttpGet("{id}")]
```

The new route is:

```
[HttpGet("me")]
```

ASP.NET Core gives the literal `me` route precedence over the `{id}` parameter route. Therefore:

```
/api/users/me -> GetCurrentUser
/api/users/5  -> GetById
```

Placing `GetCurrentUser` before `GetById` also makes the controller easier to read.

Authorization Scope

Use `[Authorize]` on the new action only:

```
[HttpGet("me")]
[Authorize]
```

Do not add `[Authorize]` to the entire `UsersController` today. That would change every existing user endpoint and exceed the Day 99 scope.

The broader authorization limitations of the existing user endpoints will be handled separately.

Why No New Service Method Is Needed

Avoid adding:

```
GetCurrentUserAsync()
```

The service does not know about HTTP JWT claims. The API controller reads the claim and passes the resulting id to the existing application use case:

```
await _userService.GetByIdAsync(currentUserId.Value);
```

This keeps HTTP authentication details in the API layer and database access in Infrastructure.

Why UserResponse Is Safe

The database entity contains `PasswordHash`, but the endpoint returns `UserResponse`, not the entity.

`UserResponse` contains only:

```
Id
FullName
Email
CreatedAtUtc
```

The password hash never reaches the API response.

Step 4 - Build And Test

Run:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 23
Failed: 0
Skipped: 0
```

The test count remains 23 because today reuses already-tested service behavior and does not add a new test project layer.

Step 5 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Keep the API running and open another PowerShell terminal.

Step 6 - Login

```
$baseUrl = "https://localhost:7024"

$loginBody = @{
    email = "YOUR_EXISTING_EMAIL"
    password = "YOUR_EXISTING_PASSWORD"
} | ConvertTo-Json

$login = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $loginBody

$token = $login.token
$headers = @{ Authorization = "Bearer $token" }
```

API Test 1 - Authenticated Profile

```
$profile = Invoke-RestMethod `
    -Uri "$baseUrl/api/users/me" `
    -Method Get `
    -Headers $headers

$profile | ConvertTo-Json -Depth 5
```

Expected:

```
Status: 200 OK
Id: logged-in user's id
FullName: logged-in user's name
Email: logged-in user's email
CreatedAtUtc: account creation time
```

Compare the profile id with the login response:

```
$login.userId
$profile.id
```

The two values should be equal.

API Test 2 - Check Safe Fields

```
$profile.PSObject.Properties.Name
```

Expected property names:

```
id
fullName
email
createdAtUtc
```

The output must not contain `passwordHash` or `token`.

API Test 3 - Missing Token

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/users/me" `
        -Method Get
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

ASP.NET Core authentication middleware should reject the request before the action runs.

API Test 4 - Invalid Token

```
$invalidHeaders = @{ Authorization = "Bearer invalid-token" }

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/users/me" `
        -Method Get `
        -Headers $invalidHeaders
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

Swagger Test

1. Run the API and open Swagger.
2. Login and obtain a JWT token.
3. Use Swagger's Authorize control with the token.
4. Call `GET /api/users/me`.
5. Confirm it returns the same user represented by the token.
6. Remove authorization and confirm the endpoint returns `401`.

Important Checks

```
[ ] UsersController imports Microsoft.AspNetCore.Authorization
[ ] UsersController imports System.Security.Claims
[ ] GET /api/users/me exists
[ ] The endpoint has [Authorize]
[ ] User id comes from ClaimTypes.NameIdentifier
[ ] No user id is accepted from the request
[ ] Existing IUserService.GetByIdAsync is reused
[ ] Existing UserResponse is returned
[ ] PasswordHash is not returned
[ ] Missing token returns 401
[ ] Valid token returns 200
[ ] Profile id matches login userId
[ ] Existing user endpoints remain unchanged
[ ] No migration was created
[ ] All 23 tests pass
```

File Explained

UsersController.cs

Adds an authenticated profile endpoint and reads the logged-in user id from JWT claims. It delegates profile retrieval to the existing user service and returns the existing safe response DTO.

Interview Explanation

```
I added GET /api/users/me as an authenticated profile endpoint.
The API reads the user id from the JWT NameIdentifier claim and reuses the existing
GetByIdAsync use case instead of accepting a user id from the client.
It returns UserResponse, so password data is not exposed.
```

End Of Day Note Template

```
1. What I built
I added GET /api/users/me so an authenticated user can retrieve their own profile using the
user id stored in the JWT.

2. What worked
The endpoint returns the logged-in user's safe UserResponse with a valid token, returns 401
without a valid token, and all 23 tests pass.

3. What I will do tomorrow
Add PUT /api/users/me so an authenticated user can safely update their own profile without
changing their password.
```

Day 100 - Update Current User Profile

Goal

Allow an authenticated user to update their own profile without accepting a user id from the request.

The new endpoint will be:

```
PUT /api/users/me
```

Today you will:

- require JWT authentication
- read the current user id from JWT claims
- reuse the existing `UpdateUserRequest`
- reuse the existing `IUserService.UpdateUserAsync` method
- update only full name and email
- keep password changes outside this endpoint

Do not add password fields or password-changing logic today. Password changes will be planned separately on Day 101.

Expected Request

```
{
  "fullName": "Updated User Name",
  "email": "updated@example.com"
}
```

Expected Response

```
{
  "id": 1,
  "fullName": "Updated User Name",
  "email": "updated@example.com",
  "createdAtUtc": "2026-08-20T00:00:00Z"
}
```

The response must not contain password data.

Expected Status Codes

Situation	Status
Valid token and valid request	`200 OK`
Missing or invalid token	`401 Unauthorized`
Invalid full name or email	`400 Bad Request`
Valid token whose user no longer exists	`404 Not Found`

Existing Code You Will Reuse

The request DTO already exists:

```
public class UpdateUserRequest
{
```

```
    [Required]
    [MaxLength(100)]
    public string FullName { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    [MaxLength(200)]
    public string Email { get; set; } = string.Empty;
}
```

The service contract already exists:

```
Task<UserResponse?> UpdateUserAsync(int id, UpdateUserRequest request);
```

The service already:

- validates required profile values
- finds the user in SQL Server
- updates `FullName` and `Email`
- saves the changes
- returns `UserResponse`

Therefore, do not create a second DTO or service method.

File To Update

```
src\JobTrackr.Api\Controllers\UsersController.cs
```

Only this file should change today.

Step 1 - Add PUT /api/users/me

Open:

```
src\JobTrackr.Api\Controllers\UsersController.cs
```

The controller already imports authorization and claims namespaces from Day 99, and it already has `GetCurrentUserId()`.

Add this action after `GetCurrentUser`:

```
[HttpPut("me")]
[Authorize]
public async Task<IActionResult> UpdateCurrentUser(UpdateUserRequest request)
{
    try
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
        {
            return Unauthorized();
        }

        var response = await _userService.UpdateUserAsync(
            currentUserId.Value,
            request);

        if (response is null)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.UserNotFound);
        }

        return Ok(response);
    }
    catch (ArgumentException ex)
```

```
    {
        return Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Bad Request",
            detail: ex.Message);
    }
}
```

What The Action Does

1. [Authorize] requires a valid JWT
2. GetCurrentUserId reads NameIdentifier from the token
3. The authenticated id is passed to UpdateUserAsync
4. The service updates only that user's FullName and Email
5. A safe UserResponse is returned

The request never supplies the user id.

Relevant Controller Structure

The profile actions should now appear together:

```
[HttpGet("me")]
[Authorize]
public async Task<IActionResult> GetCurrentUser()
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var user = await _userService.GetByIdAsync(currentUserId.Value);

    if (user is null)
    {
        return Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Not Found",
            detail: ErrorMessages.UserNotFound);
    }

    return Ok(user);
}

[HttpPut("me")]
[Authorize]
public async Task<IActionResult> UpdateCurrentUser(UpdateUserRequest request)
{
    try
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
        {
            return Unauthorized();
        }

        var response = await _userService.UpdateUserAsync(
            currentUserId.Value,
            request);

        if (response is null)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.UserNotFound);
        }

        return Ok(response);
    }
    catch (ArgumentException ex)
    {
```

```
            return Problem(
                statusCode: StatusCodes.Status400BadRequest,
                title: "Bad Request",
                detail: ex.Message);
        }
    }
```

Keep every existing controller action and the `GetCurrentUserId()` helper unchanged.

Why The Endpoint Does Not Accept An Id

Use:

```
PUT /api/users/me
```

Do not use a body or query parameter such as:

```
{
  "userId": 5,
  "fullName": "Changed Name"
}
```

The JWT is the trusted source of the current user identity. A client-provided user id would create an unnecessary authorization risk.

Why The Existing Service Is Reused

The API layer understands JWT claims. The service layer understands the profile update use case and database operation.

The controller connects them:

```
var currentUserId = GetCurrentUserId();

var response = await _userService.UpdateUserAsync(
    currentUserId.Value,
    request);
```

No JWT or `HttpContext` logic is moved into Infrastructure.

Why Password Is Not Updated

`UpdateUserRequest` contains only:

```
FullName
Email
```

It does not contain:

```
Password
PasswordHash
OldPassword
NewPassword
```

A password change requires old-password verification and new-password hashing. Mixing that workflow into a general profile endpoint would make security behavior unclear.

Day 101 will define the password-change contract before it is implemented.

Validation Behavior

Because `UsersController` uses `[ApiController]`, data annotations run before the action for invalid request shapes.

Examples that return `400 Bad Request`:

```
{
  "fullName": "",
```

```
  "email": "updated@example.com"
}

{
  "fullName": "Updated User",
  "email": "not-an-email"
}
```

The service also protects against whitespace-only values.

Route Behavior

The controller already has:

```
[HttpPut("{id}")]
```

The new literal route is:

```
[HttpPut("me")]
```

ASP.NET Core gives the literal route precedence:

```
PUT /api/users/me -> UpdateCurrentUser
PUT /api/users/5  -> existing Update action
```

Do not remove or redesign the older route today.

JWT Behavior After Updating Email Or Name

The current JWT contains the user id, email, and full name that existed when the user logged in.

After the update:

- the database and returned `UserResponse` contain the new profile values
- `GET /api/users/me` immediately returns the new database values
- the current token remains valid until it expires
- the token's embedded name and email claims remain old until the user logs in again
- the user id claim remains correct and continues to identify the account

Login again after changing the email to receive a token containing the updated email and full-name claims.

Current Scope Limitation

The existing `PUT /api/users/{id}` endpoint still exists. Day 100 adds a safe self-service profile route but does not redesign or remove older user administration endpoints.

Keep today's change limited to `PUT /api/users/me`.

Step 2 - Build And Test

Run:

```
cd
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 23
Failed: 0
Skipped: 0
```

The test count remains 23 because this endpoint reuses the existing tested user update service behavior.

Step 3 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Keep the API running and open another PowerShell terminal.

Step 4 - Login And Get The Existing Profile

```
$baseUrl = "https://localhost:7024"

$loginBody = @{
    email = "YOUR_EXISTING_EMAIL"
    password = "YOUR_EXISTING_PASSWORD"
} | ConvertTo-Json

$login = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $loginBody

$token = $login.token
$headers = @{ Authorization = "Bearer $token" }

$before = Invoke-RestMethod `
    -Uri "$baseUrl/api/users/me" `
    -Method Get `
    -Headers $headers

$before | ConvertTo-Json
```

Keep the original email and name so you can restore them after testing:

```
$originalName = $before.fullName
$originalEmail = $before.email
```

API Test 1 - Update Current Profile

Choose an email that is not used by another account:

```
$updateBody = @{
    fullName = "Updated Profile Name"
    email = "updated.profile@example.com"
} | ConvertTo-Json

$updated = Invoke-RestMethod `
    -Uri "$baseUrl/api/users/me" `
    -Method Put `
    -Headers $headers `
    -ContentType "application/json" `
    -Body $updateBody

$updated | ConvertTo-Json
```

Expected:

```
Status: 200 OK
Id: unchanged
FullName: Updated Profile Name
Email: updated.profile@example.com
CreatedAtUtc: unchanged
```

API Test 2 - Confirm Database-Backed Result

Use the same existing token:

```
$after = Invoke-RestMethod `
    -Uri "$baseUrl/api/users/me" `
    -Method Get `
    -Headers $headers
```

```
$after | ConvertTo-Json
```

Expected: `GET /api/users/me` returns the updated full name and email because it reads the current user from SQL Server using the token's user id.

API Test 3 - Invalid Email

```
$invalidEmailBody = @{
    fullName = "Updated Profile Name"
    email = "not-an-email"
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/users/me" `
        -Method Put `
        -Headers $headers `
        -ContentType "application/json" `
        -Body $invalidEmailBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

API Test 4 - Missing Name

```
$missingNameBody = @{
    email = "updated.profile@example.com"
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/users/me" `
        -Method Put `
        -Headers $headers `
        -ContentType "application/json" `
        -Body $missingNameBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

API Test 5 - Missing Token

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/users/me" `
        -Method Put `
        -ContentType "application/json" `
        -Body $updateBody
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

Step 5 - Restore Your Original Profile

Use the same valid token to restore the original values:

```
$restoreBody = @{
    fullName = $originalName
```

```
    email = $originalEmail
} | ConvertTo-Json

Invoke-RestMethod `
    -Uri "$baseUrl/api/users/me" `
    -Method Put `
    -Headers $headers `
    -ContentType "application/json" `
    -Body $restoreBody
```

Login again with the final email after testing if you need a JWT containing the latest name and email claims.

Swagger Test

1. Login and obtain a JWT.
2. Authorize Swagger with the token.
3. Call `GET /api/users/me` and note the current values.
4. Call `PUT /api/users/me` with a valid full name and email.
5. Call `GET /api/users/me` again and confirm the update.
6. Confirm the response contains no password fields.
7. Remove authorization and confirm PUT returns `401`.

Important Checks

```
[ ] PUT /api/users/me exists
[ ] The endpoint has [Authorize]
[ ] User id comes only from the JWT
[ ] Request body has no user id
[ ] Existing UpdateUserRequest is reused
[ ] Existing UpdateUserAsync is reused
[ ] Only FullName and Email are updated
[ ] PasswordHash is unchanged
[ ] Valid request returns 200
[ ] Invalid profile input returns 400
[ ] Missing token returns 401
[ ] Missing database user returns 404
[ ] CreatedAtUtc remains unchanged
[ ] Response contains no password data
[ ] No migration was created
[ ] All 23 tests pass
```

File Explained

UsersController.cs

Adds an authenticated self-service profile update action. It reads identity from JWT claims, delegates validation and persistence to the existing user service, and returns the safe user response DTO.

Interview Explanation

```
I added PUT /api/users/me for authenticated profile updates.
The endpoint takes the user id only from the JWT NameIdentifier claim and reuses the existing
update use case for full name and email.
Password changes are intentionally separated because they require old-password verification and
hashing.
```

End Of Day Note Template

```
1. What I built
I added PUT /api/users/me so an authenticated user can update their own full name and email
using the user id stored in the JWT.

2. What worked
The endpoint updates only the logged-in user's safe profile fields, returns validation errors
for invalid input, rejects anonymous requests, and all 23 tests pass.

3. What I will do tomorrow
Plan the change-password request, validation rules, security behavior, and response contract
before implementing it.
```

Day 101 - Plan The Change Password Flow

Goal

Define the change-password contract and security rules before implementing it.

Today is a planning day. You will decide:

- the endpoint route and HTTP method
- the authenticated user identity source
- the request DTO fields
- password validation rules
- service behavior
- response status codes
- security limitations
- tests required after implementation

Do not create or modify application code today. Day 102 will implement the agreed plan.

Why This Needs A Separate Plan

Changing a password is more sensitive than updating a name or email.

The API must:

```
verify the user is authenticated
verify the current password
validate the new password
prevent accidental password mismatch
hash the new password
never store or return plain text passwords
never expose the password hash
avoid logging password values
```

Planning these rules first keeps the implementation small and predictable.

Existing Authentication Components

JobTrackr already has:

```
AuthController
IAuthService
AuthService
IPasswordHasherService
PasswordHasherService
JWT authentication
NameIdentifier user-id claim
User.PasswordHash
```

The password hasher already provides:

```
string HashPassword(string password);

bool VerifyPassword(string password, string passwordHash);
```

Day 102 must reuse these methods. Do not write custom password hashing or compare password hashes as strings.

Planned Endpoint

```
PUT /api/auth/change-password
```

Why PUT

The endpoint updates the authenticated user's existing password credential.

The route does not contain a user id because the JWT identifies the account.

Do not use:

```
PUT /api/auth/users/5/change-password
```

or:

```
{
  "userId": 5
}
```

The client must never choose which user's password is changed.

Authentication Rule

The endpoint will use:

```
[Authorize]
```

The controller will read:

```
ClaimTypes.NameIdentifier
```

The parsed user id will be passed into `IAuthService`.

Expected identity behavior:

Situation	Result
Valid token and valid user claim	Continue to password verification
No token	`401 Unauthorized`
Invalid or expired token	`401 Unauthorized`
Missing or invalid user-id claim	`401 Unauthorized`
Valid token but database user was deleted	`404 Not Found`

Planned Request DTO

Create this file on Day 102:

```
[repository root]\src\JobTrackr.Application\Auth\ChangePasswordRequest.cs
```

Planned request shape:

```
{
  "currentPassword": "CurrentPassword123",
  "newPassword": "NewPassword456",
  "confirmNewPassword": "NewPassword456"
}
```

Planned DTO:

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Auth
{
    public class ChangePasswordRequest
```

```
    {
        [Required(ErrorMessage = "Current password is required.")]
        [MaxLength(100, ErrorMessage = "Current password cannot be longer than 100 characters.
")]
        public string CurrentPassword { get; set; } = string.Empty;

        [Required(ErrorMessage = "New password is required.")]
        [MinLength(6, ErrorMessage = "New password must be at least 6 characters.")]
        [MaxLength(100, ErrorMessage = "New password cannot be longer than 100 characters.")]
        public string NewPassword { get; set; } = string.Empty;

        [Required(ErrorMessage = "New password confirmation is required.")]
        [Compare(
            nameof(NewPassword),
            ErrorMessage = "New password and confirmation do not match.")]
        public string ConfirmNewPassword { get; set; } = string.Empty;
    }
}
```

Do not create this file today. This is the agreed contract for tomorrow.

Why Three Fields Are Needed

CurrentPassword

Proves that the authenticated person knows the account's existing password.

A valid JWT alone is not enough because a token may have been copied or left active on another device.

NewPassword

Contains the replacement password that will be hashed and stored.

ConfirmNewPassword

Protects the user from accidentally setting a mistyped password.

The confirmation is validated but is never stored.

Password Validation Rules

Day 102 will enforce:

```
CurrentPassword is required
CurrentPassword maximum length is 100
NewPassword is required
NewPassword minimum length is 6
NewPassword maximum length is 100
ConfirmNewPassword is required
ConfirmNewPassword must equal NewPassword
CurrentPassword must verify against PasswordHash
NewPassword must differ from CurrentPassword
```

The six-character minimum matches the current registration rule. Do not introduce a different password policy only for change-password requests.

Stronger password rules can be introduced later as one consistent policy for registration and password changes.

Planned Service Contract

Update this file on Day 102:

```
[repository root]\src\JobTrackr.Application\Auth\IAuthService.cs
```

Planned method:

```
Task<bool> ChangePasswordAsync(
    int userId,
```

```
    ChangePasswordRequest request);
```

Return Meaning

```
true  -> password changed successfully
false -> the database user does not exist
```

Invalid password input will use `ArgumentException`, matching the current AuthService/controller pattern.

Planned Service Flow

Day 102's `AuthService.ChangePasswordAsync` will perform these steps in order:

```
1. Find the user by authenticated userId
2. Return false if the user does not exist
3. Verify CurrentPassword against the stored PasswordHash
4. Reject an incorrect current password
5. Confirm NewPassword and ConfirmNewPassword match
6. Reject a new password equal to the current password
7. Hash NewPassword with PasswordHasherService
8. Replace User.PasswordHash
9. Save changes to SQL Server
10. Return true
```

Planned Service Pseudocode

This is behavior planning, not code to add today:

```
find user by userId

if user does not exist
    return false

verify current password using VerifyPassword

if current password is wrong
    throw safe validation error

if new password and confirmation differ
    throw safe validation error

if new password verifies against the current hash
    throw safe validation error

hash new password using HashPassword
save the new PasswordHash
return true
```

Planned Safe Error Messages

Use client-safe messages:

```
Current password is incorrect.
New password and confirmation do not match.
New password must be different from the current password.
User not found.
```

Do not return:

```
stored password hash
hashing algorithm details
exception stack trace
database details
plain text passwords
```

Planned Status Codes

Situation	Status
Password changed	`204 No Content`
Missing or invalid token	`401 Unauthorized`
Invalid DTO values	`400 Bad Request`
Incorrect current password	`400 Bad Request`
New password equals current password	`400 Bad Request`
Token user no longer exists	`404 Not Found`
Unexpected server error	safe `500 Internal Server Error`

Why Success Returns 204

The client does not need the password, password hash, or another response DTO after success.

```
204 No Content
```

clearly communicates that the update succeeded without returning sensitive or unnecessary data.

Do not return `AuthResponse` or generate a new token in the first implementation.

Planned Controller Flow

Update this file on Day 102:

```
[repository root]\src\JobTrackr.Api\Controllers\AuthController.cs
```

The endpoint will:

```
require [Authorize]
read NameIdentifier from JWT claims
return 401 if the claim is invalid
call ChangePasswordAsync
return 404 if the user does not exist
return 204 when successful
map safe ArgumentException messages to 400
```

The controller should not:

```
read AppDbContext directly
verify passwords itself
hash passwords itself
accept userId from the client
return password information
```

Files Planned For Day 102

Create

```
[repository root]\src\JobTrackr.Application\Auth\ChangePasswordRequest.cs
```

Update

```
[repository root]\src\JobTrackr.Application\Auth\IAuthService.cs
[repository root]\src\JobTrackr.Infrastructure\Auth\AuthService.cs
[repository root]\src\JobTrackr.Api\Controllers\AuthController.cs
```

Do Not Change

```
User entity
PasswordHasherService
JWT token generation
database migrations
UsersController profile endpoints
```

No database migration is needed because `User.PasswordHash` already exists.

JWT Limitation After Password Change

JobTrackr currently uses stateless JWT tokens and does not store token versions or a revocation list.

Therefore, after changing the password:

```
the current JWT remains valid until it expires
other previously issued JWTs also remain valid until they expire
future login requires the new password
the old password no longer works for login
```

Invalidating every existing token would require a separate token-revocation design. Do not add that complexity during Day 102.

This limitation should be understood and documented rather than hidden.

Security Rules

```
[ ] User identity comes only from JWT NameIdentifier
[ ] Current password is verified with PasswordHasherService
[ ] New password is hashed with PasswordHasherService
[ ] Plain text passwords are never saved
[ ] PasswordHash is never returned
[ ] Password values are never logged
[ ] Request body is not written to logs
[ ] New password differs from current password
[ ] No new JWT is returned
[ ] No user id is accepted from the client
```

The current request logging middleware records method, path, status, and elapsed time, not request bodies. Keep it that way.

Planned Tests

Day 103 is the roadmap's auth-test day. Plan these tests:

```
Successful password change updates the hash
New hash is not equal to the plain text password
New password verifies against the new hash
Old password no longer verifies
Incorrect current password is rejected
Matching confirmation is required
New password equal to current password is rejected
Missing user returns false
Login succeeds with the new password
Login fails with the old password
```

Day 102 may implement the endpoint first. Day 103 will add the focused automated coverage without mixing too much work into one day.

Manual Test Plan For Day 102

Successful Request

```
PUT /api/auth/change-password
Authorization: Bearer VALID_TOKEN
Content-Type: application/json

{
  "currentPassword": "CurrentPassword123",
  "newPassword": "NewPassword456",
  "confirmNewPassword": "NewPassword456"
}
```

Expected:

```
204 No Content
old password login fails
new password login succeeds
```

Incorrect Current Password

Expected:

```
400 Bad Request
password remains unchanged
```

Mismatched Confirmation

Expected:

```
400 Bad Request
password remains unchanged
```

Missing Token

Expected:

```
401 Unauthorized
```

Decision Summary

```
Route: PUT /api/auth/change-password
Authentication: required JWT
Identity source: ClaimTypes.NameIdentifier
Request fields: CurrentPassword, NewPassword, ConfirmNewPassword
Minimum password length: 6
Maximum password length: 100
Success response: 204 No Content
Wrong current password: 400 Bad Request
Missing user: 404 Not Found
Password hashing: existing PasswordHasherService
New token after success: no
Existing token revocation: not implemented
Migration: none
```

Today's Verification

Because today changes no application code:

```
cd [repository root]
git status
```

Expected:

```
Working tree clean
```

No build, migration, code commit, or Git push is required for a planning-only lesson.

Important Checks

```
[ ] Route is decided
[ ] Request fields are decided
[ ] Validation rules are decided
[ ] Authenticated identity source is decided
[ ] Service method contract is decided
[ ] Success status is decided
[ ] Safe error responses are decided
[ ] Password hash is never exposed
[ ] Token revocation limitation is understood
[ ] Implementation files are identified
[ ] Test cases are planned
[ ] No application code was changed today
```

Interview Explanation

```
Before implementing password changes, I defined the authenticated route, request validation,
service flow, safe errors, and token limitations.
The design verifies the current password with the existing hasher, hashes the replacement,
takes identity only from JWT claims, and returns no password data.
I kept token revocation outside the first implementation and documented that existing JWTs
remain valid until expiry.
```

End Of Day Note Template

```
1. What I planned
I planned the authenticated change-password flow, including the request DTO, validation rules,
current-password verification, new-password hashing, status codes, and JWT limitations.

2. What worked
The implementation scope is clear: the API will use PUT /api/auth/change-password, identify the
user from JWT claims, return 204 on success, never expose password data, and require no
database migration.

3. What I will do tomorrow
Implement the change-password DTO, service method, and authenticated controller endpoint one
step at a time.
```

Day 102 - Implement Change Password

Goal

Implement the authenticated change-password flow planned on Day 101.

The new endpoint will be:

```
PUT /api/auth/change-password
```

Today you will:

- create `ChangePasswordRequest`
- extend `IAuthService`
- verify the current password
- reject mismatched or unchanged new passwords
- hash and save the new password
- add an authenticated controller endpoint
- return `204 No Content` on success

Do not add JWT revocation, refresh tokens, password reset email, or a database migration today.

Expected Request

```
{
  "currentPassword": "CurrentPassword123",
  "newPassword": "NewPassword456",
  "confirmNewPassword": "NewPassword456"
}
```

Expected Status Codes

Situation	Status
Password changed successfully	`204 No Content`
Missing, invalid, or expired JWT	`401 Unauthorized`
Invalid request values	`400 Bad Request`
Incorrect current password	`400 Bad Request`
New password equals current password	`400 Bad Request`
JWT user no longer exists	`404 Not Found`

Files To Create And Update

Create

```
[repository root]\src\JobTrackr.Application\Auth\ChangePasswordRequest.cs
```

Update

```
[repository root]\src\JobTrackr.Application\Common\ErrorMessages.cs
[repository root]\src\JobTrackr.Application\Auth\IAuthService.cs
[repository root]\src\JobTrackr.Infrastructure\Auth\AuthService.cs
[repository root]\src\JobTrackr.Api\Controllers\AuthController.cs
```

Do Not Change

```
User entity
PasswordHasherService
JWT token generation
database migrations
UsersController
```

Step 1 - Create ChangePasswordRequest

Create:

```
cd [repository root]
New-Item src\JobTrackr.Application\Auth\ChangePasswordRequest.cs -ItemType File
```

Open:

```
[repository root]\src\JobTrackr.Application\Auth\ChangePasswordRequest.cs
```

Add:

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Auth
{
    public class ChangePasswordRequest
    {
        [Required(ErrorMessage = "Current password is required.")]
        [MaxLength(
            100,
            ErrorMessage = "Current password cannot be longer than 100 characters.")]
        public string CurrentPassword { get; set; } = string.Empty;

        [Required(ErrorMessage = "New password is required.")]
        [MinLength(6, ErrorMessage = "New password must be at least 6 characters.")]
        [MaxLength(
            100,
            ErrorMessage = "New password cannot be longer than 100 characters.")]
        public string NewPassword { get; set; } = string.Empty;

        [Required(ErrorMessage = "New password confirmation is required.")]
        [Compare(
            nameof(NewPassword),
            ErrorMessage = "New password and confirmation do not match.")]
        public string ConfirmNewPassword { get; set; } = string.Empty;
    }
}
```

What This File Does

The DTO defines only the values needed for a password change. Data annotations make ASP.NET Core reject missing, short, long, or mismatched values before the controller calls the service.

It does not contain `UserId`; the authenticated JWT supplies identity.

Step 2 - Add Centralized Business Errors

Open:

```
[repository root]\src\JobTrackr.Application\Common\ErrorMessages.cs
```

Add these constants inside `ErrorMessages`:

```
public const string CurrentPasswordIncorrect = "Current password is incorrect.";
public const string NewPasswordMismatch = "New password and confirmation do not match.";
public const string NewPasswordMustBeDifferent =
    "New password must be different from the current password.";
```

The complete new section should look like:

```
public const string UserEmailRequired = "User email is required.";
public const string CurrentPasswordIncorrect = "Current password is incorrect.";
public const string NewPasswordMismatch = "New password and confirmation do not match.";
```

```
public const string NewPasswordMustBeDifferent =
    "New password must be different from the current password.";
```

These are service/business errors that may be used in more than one place. Keep the DTO's field-level validation messages in the DTO.

Step 3 - Update IAuthService

Open:

```
[repository root]\src\JobTrackr.Application\Auth\IAuthService.cs
```

Add:

```
Task<bool> ChangePasswordAsync(
    int userId,
    ChangePasswordRequest request);
```

The complete interface should be:

```
namespace JobTrackr.Application.Auth
{
    public interface IAuthService
    {
        Task<AuthResponse> RegisterAsync(RegisterRequest request);

        Task<AuthResponse> LoginAsync(LoginRequest request);

        Task<bool> ChangePasswordAsync(
            int userId,
            ChangePasswordRequest request);
    }
}
```

Return Meaning

```
true  -> password changed
false -> user was not found
```

Invalid password conditions throw safe `ArgumentException` messages, following the existing auth pattern.

Step 4 - Import ErrorMessages In AuthService

Open:

```
[repository root]\src\JobTrackr.Infrastructure\Auth\AuthService.cs
```

Add:

```
using JobTrackr.Application.Common;
```

The top should begin with:

```
using JobTrackr.Application.Auth;
using JobTrackr.Application.Common;
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
```

Step 5 - Implement ChangePasswordAsync

Inside `AuthService`, add:

```
public async Task<bool> ChangePasswordAsync(
    int userId,
    ChangePasswordRequest request)
{
    var user = await _dbContext.Users.FindAsync(userId);

    if (user is null)
    {
        return false;
```

```
    }

    var currentPasswordIsValid = _passwordHasherService.VerifyPassword(
        request.CurrentPassword,
        user.PasswordHash);

    if (!currentPasswordIsValid)
    {
        throw new ArgumentException(ErrorMessages.CurrentPasswordIncorrect);
    }

    if (request.NewPassword != request.ConfirmNewPassword)
    {
        throw new ArgumentException(ErrorMessages.NewPasswordMismatch);
    }

    var newPasswordMatchesCurrent = _passwordHasherService.VerifyPassword(
        request.NewPassword,
        user.PasswordHash);

    if (newPasswordMatchesCurrent)
    {
        throw new ArgumentException(ErrorMessages.NewPasswordMustBeDifferent);
    }

    user.PasswordHash = _passwordHasherService.HashPassword(request.NewPassword);

    await _dbContext.SaveChangesAsync();

    return true;
}
```

Place it with the other public auth methods, before the class closing brace.

Service Flow Explained

Find The Authenticated User

```
var user = await _dbContext.Users.FindAsync(userId);
```

The id came from the JWT, not the request body.

Verify The Current Password

```
_passwordHasherService.VerifyPassword(
    request.CurrentPassword,
    user.PasswordHash);
```

The plain current password is checked using ASP.NET Core Identity's password hasher. Password hashes must not be compared as ordinary strings because hashing uses salts.

Recheck Confirmation In The Service

`[Compare]` handles HTTP model validation, but the service may also be called directly from tests or another application entry point.

```
if (request.NewPassword != request.ConfirmNewPassword)
```

keeps the use case safe outside the controller.

Prevent Reusing The Same Password

The service verifies the proposed password against the current hash:

```
var newPasswordMatchesCurrent = _passwordHasherService.VerifyPassword(
    request.NewPassword,
    user.PasswordHash);
```

If it verifies successfully, the password has not actually changed, so the request is rejected.

Hash Before Saving

```
user.PasswordHash = _passwordHasherService.HashPassword(request.NewPassword);
```

Only the generated hash is saved to SQL Server. `CurrentPassword`, `NewPassword`, and `ConfirmNewPassword` are never stored.

Step 6 - Update AuthController Usings

Open:

```
[repository root]\src\JobTrackr.Api\Controllers\AuthController.cs
```

Add:

```
using JobTrackr.Application.Common;
using Microsoft.AspNetCore.Authorization;
using System.Security.Claims;
```

The top should be:

```
using JobTrackr.Application.Auth;
using JobTrackr.Application.Common;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
```

Step 7 - Add The Controller Endpoint

Inside `AuthController`, after `Login`, add:

```
[HttpPut("change-password")]
[Authorize]
public async Task<IActionResult> ChangePassword(ChangePasswordRequest request)
{
    try
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
        {
            return Unauthorized();
        }

        var passwordChanged = await _authService.ChangePasswordAsync(
            currentUserId.Value,
            request);

        if (!passwordChanged)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.UserNotFound);
        }

        return NoContent();
    }
    catch (ArgumentException ex)
    {
        return Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Bad Request",
            detail: ex.Message);
    }
}
```

What The Endpoint Does

```
[Authorize] validates the JWT
GetCurrentUserId reads NameIdentifier
AuthService performs password security logic
false becomes 404 Not Found
ArgumentException becomes safe 400 Bad Request
success becomes 204 No Content
```

Step 8 - Add The Claim Helper

At the bottom of `AuthController`, before its final closing brace, add:

```
private int? GetCurrentUserId()
{
    var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (int.TryParse(userIdValue, out var userId))
    {
        return userId;
    }

    return null;
}
```

This matches the simple helper already used by `TasksController` and `UsersController`.

Do not create a new claim abstraction today.

Complete New Controller Section

```
[HttpPut("change-password")]
[Authorize]
public async Task<IActionResult> ChangePassword(ChangePasswordRequest request)
{
    try
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
        {
            return Unauthorized();
        }

        var passwordChanged = await _authService.ChangePasswordAsync(
            currentUserId.Value,
            request);

        if (!passwordChanged)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.UserNotFound);
        }

        return NoContent();
    }
    catch (ArgumentException ex)
    {
        return Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Bad Request",
            detail: ex.Message);
    }
}

private int? GetCurrentUserId()
{
    var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (int.TryParse(userIdValue, out var userId))
    {
        return userId;
    }

    return null;
}
```

Keep the existing register and login endpoints unchanged.

Why No Password Response Is Returned

On success, the controller uses:

```
return NoContent();
```

The API does not return:

```
plain password
password confirmation
password hash
new JWT
```

The absence of output from PowerShell after a successful `Invoke-RestMethod` call is normal for a `204 No Content` response.

Why No Migration Is Needed

The `Users` table already contains `PasswordHash`.

Day 102 updates the value in that existing column. It does not add or change the database schema.

Do not run:

```
dotnet ef migrations add ...
```

Step 9 - Build And Test

Run:

```
cd [repository root]
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 23
Failed: 0
Skipped: 0
```

The existing test count remains 23 today. Day 103 will add focused change-password tests.

Step 10 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Keep the API running and open another PowerShell terminal.

Step 11 - Login With The Current Password

```
$baseUrl = "https://localhost:7024"
$email = "YOUR_EXISTING_EMAIL"
$oldPassword = "YOUR_CURRENT_PASSWORD"
$newPassword = "NewPassword456"

$loginBody = @{
    email = $email
    password = $oldPassword
} | ConvertTo-Json

$login = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $loginBody

$token = $login.token
$headers = @{ Authorization = "Bearer $token" }
```

Choose a temporary new password that differs from the current password.

API Test 1 - Successful Password Change

```
$changeBody = @{
    currentPassword = $oldPassword
    newPassword = $newPassword
    confirmNewPassword = $newPassword
} | ConvertTo-Json

Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/change-password" `
    -Method Put `
    -Headers $headers `
    -ContentType "application/json" `
    -Body $changeBody

"Password changed successfully."
```

Expected:

```
204 No Content
Password changed successfully.
```

`Invoke-RestMethod` itself prints no response body for `204`.

API Test 2 - Old Password Login Fails

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/login" `
        -Method Post `
        -ContentType "application/json" `
        -Body $loginBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

API Test 3 - New Password Login Works

```
$newLoginBody = @{
    email = $email
    password = $newPassword
} | ConvertTo-Json

$newLogin = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $newLoginBody

$newLogin.token.Length
```

Expected: a token length greater than zero.

API Test 4 - Incorrect Current Password

Use the new token and an incorrect current password:

```
$newHeaders = @{ Authorization = "Bearer $($newLogin.token)" }

$wrongCurrentBody = @{
    currentPassword = "WrongPassword"
    newPassword = "AnotherPassword789"
    confirmNewPassword = "AnotherPassword789"
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/change-password" `
        -Method Put `
        -Headers $newHeaders `
        -ContentType "application/json" `
        -Body $wrongCurrentBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected:

```
400
Current password is incorrect.
```

API Test 5 - Mismatched Confirmation

```
$mismatchBody = @{
    currentPassword = $newPassword
    newPassword = "AnotherPassword789"
    confirmNewPassword = "DifferentPassword789"
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/change-password" `
        -Method Put `
        -Headers $newHeaders `
        -ContentType "application/json" `
        -Body $mismatchBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected status:

```
400
```

ASP.NET Core data annotation validation should reject the request before service logic.

API Test 6 - Same Password

```
$samePasswordBody = @{
    currentPassword = $newPassword
    newPassword = $newPassword
    confirmNewPassword = $newPassword
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/change-password" `
        -Method Put `
        -Headers $newHeaders `
        -ContentType "application/json" `
        -Body $samePasswordBody
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected:

```
400
New password must be different from the current password.
```

API Test 7 - Missing Token

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/change-password" `
        -Method Put `
        -ContentType "application/json" `
        -Body $changeBody
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

Step 12 - Restore The Original Password

Use the valid new token. The current password is now `$newPassword`:

```
$restoreBody = @{
    currentPassword = $newPassword
```

```
    newPassword = $oldPassword
    confirmNewPassword = $oldPassword
} | ConvertTo-Json

Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/change-password" `
    -Method Put `
    -Headers $newHeaders `
    -ContentType "application/json" `
    -Body $restoreBody

"Original password restored."
```

Login again with the original password to verify restoration.

Existing JWT Limitation

Changing the password does not revoke JWTs that were already issued.

The original token and new token remain valid until their expiration time because JobTrackr does not yet have token-version or revocation storage.

This is a known limitation, not a Day 102 bug.

Security Checks

```
[ ] Endpoint requires [Authorize]
[ ] User id comes only from NameIdentifier
[ ] Current password is verified through PasswordHasherService
[ ] New password differs from current password
[ ] Confirmation is checked
[ ] New password is hashed before saving
[ ] Plain passwords are never stored
[ ] PasswordHash is never returned
[ ] Request body is not logged
[ ] Success returns no response body
[ ] No new token is returned
[ ] No migration was created
```

Files Explained

ChangePasswordRequest.cs

Defines current password, new password, confirmation, and request validation. It contains no user id and no response data.

ErrorMessages.cs

Centralizes safe reusable password-change business errors.

IAuthService.cs

Adds the application contract for changing the authenticated user's password.

AuthService.cs

Finds the user, verifies the current password, validates the replacement, hashes it, and saves only the new hash.

AuthController.cs

Adds the protected HTTP endpoint, reads identity from JWT claims, and maps service results to `204`, `400`, `401`, or `404`.

Interview Explanation

```
I implemented an authenticated change-password endpoint that takes identity only from the JWT.
The service verifies the current password with ASP.NET Core's password hasher, rejects
mismatched or unchanged passwords, hashes the replacement, and stores only the new hash.
The endpoint returns 204 without sensitive data, while previously issued JWTs remain valid
until expiry as a documented limitation.
```

End Of Day Note Template

```
1. What I built
I added an authenticated change-password endpoint with a validated request DTO, current-
password verification, new-password hashing, and safe error responses.

2. What worked
The correct current password can be replaced, the old password stops working for login, the new
password works, invalid requests are rejected, no password data is returned, and all 23 tests
pass.

3. What I will do tomorrow
Add focused AuthService tests for successful password changes and important failure cases.
```

Day 103 - Add Change Password Tests

Goal

Add focused automated tests for the change-password use case implemented on Day 102.

Today you will test:

- a successful password change
- the old password no longer working
- the new password working for login
- an incorrect current password
- mismatched password confirmation
- reusing the current password
- a missing database user
- password hashes remaining unchanged after failed attempts

Do not change production behavior unless a test reveals a real defect.

Current Test Coverage

`AuthServiceTests` already covers:

```
successful registration
successful login
invalid-password login
```

The complete project currently has:

```
23 tests
```

Day 103 adds five change-password tests, increasing the total to:

```
28 tests
```

File To Update

```
[repository root]\tests\JobTrackr.Tests\Auth\AuthServiceTests.cs
```

No Application, Domain, Infrastructure, controller, or migration file should need to change today.

Behaviors To Protect

The tests must prove these service rules:

```
authenticated user id selects the account
current password must verify
new password and confirmation must match
new password must differ from current password
only a hash is stored
old password stops verifying after success
new password verifies after success
failed requests do not change PasswordHash
missing user returns false
```

Step 1 - Add ErrorMessages Using

Open:

```
[repository root]\tests\JobTrackr.Tests\Auth\AuthServiceTests.cs
```

Add:

```
using JobTrackr.Application.Common;
```

The top should include:

```
using JobTrackr.Application.Auth;
using JobTrackr.Application.Common;
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Auth;
using JobTrackr.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
```

The tests will compare exceptions against centralized safe messages instead of repeating message strings.

Step 2 - Add Successful Change Test

Add this test after the existing login tests and before the private helper methods:

```
[Fact]
public async Task ChangePasswordAsync_WithValidRequest_ChangesPassword()
{
    await using var dbContext = CreateDbContext();
    var passwordHasher = new PasswordHasherService();
    var authService = new AuthService(
        dbContext,
        passwordHasher,
        new FakeJwtTokenService());
    var registeredUser = await RegisterChangePasswordUserAsync(authService);
    var savedUser = await dbContext.Users.SingleAsync();
    var originalPasswordHash = savedUser.PasswordHash;

    var request = new ChangePasswordRequest
    {
        CurrentPassword = "Password123",
        NewPassword = "NewPassword456",
        ConfirmNewPassword = "NewPassword456"
    };

    var passwordChanged = await authService.ChangePasswordAsync(
        registeredUser.UserId,
        request);

    Assert.True(passwordChanged);
    Assert.NotEqual(originalPasswordHash, savedUser.PasswordHash);
    Assert.False(passwordHasher.VerifyPassword(
        request.CurrentPassword,
        savedUser.PasswordHash));
    Assert.True(passwordHasher.VerifyPassword(
        request.NewPassword,
        savedUser.PasswordHash));

    var newPasswordLogin = await authService.LoginAsync(new LoginRequest
    {
        Email = registeredUser.Email,
        Password = request.NewPassword
    });

    Assert.Equal(registeredUser.UserId, newPasswordLogin.UserId);
    Assert.Equal(
        $"test-token-{registeredUser.UserId}",
        newPasswordLogin.Token);

    var oldPasswordException = await Assert.ThrowsAsync<ArgumentException>(
        () => authService.LoginAsync(new LoginRequest
        {
            Email = registeredUser.Email,
            Password = request.CurrentPassword
        }));

    Assert.Equal("Invalid email or password.", oldPasswordException.Message);
}
```

What The Success Test Proves

The test verifies the complete service result:

```
ChangePasswordAsync returns true
the stored hash changes
the old password does not verify against the new hash
the new password verifies against the new hash
login succeeds with the new password
login fails with the old password
```

Checking login after the change proves that the change works through the existing authentication flow, not only through direct hash inspection.

Why The Original Hash Is Captured

```
var originalPasswordHash = savedUser.PasswordHash;
```

The test stores the original string before calling the service. After success:

```
Assert.NotEqual(originalPasswordHash, savedUser.PasswordHash);
```

proves the database entity now contains a replacement hash.

It does not compare the hash with the plain password because a secure stored hash should never equal the password.

Step 3 - Add Incorrect Current Password Test

Add:

```
[Fact]
public async Task ChangePasswordAsync_WithIncorrectCurrentPassword_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var authService = CreateAuthService(dbContext);
    var registeredUser = await RegisterChangePasswordUserAsync(authService);
    var savedUser = await dbContext.Users.SingleAsync();
    var originalPasswordHash = savedUser.PasswordHash;

    var request = new ChangePasswordRequest
    {
        CurrentPassword = "WrongPassword",
        NewPassword = "NewPassword456",
        ConfirmNewPassword = "NewPassword456"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => authService.ChangePasswordAsync(
            registeredUser.UserId,
            request));

    Assert.Equal(ErrorMessages.CurrentPasswordIncorrect, exception.Message);
    Assert.Equal(originalPasswordHash, savedUser.PasswordHash);
}
```

The final assertion is important. The service must reject the request before modifying the hash.

Step 4 - Add Confirmation Mismatch Test

Data annotations handle this through the HTTP API, but service tests call `AuthService` directly. The service includes its own confirmation check for this reason.

Add:

```
[Fact]
public async Task ChangePasswordAsync_WithMismatchedConfirmation_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var authService = CreateAuthService(dbContext);
```

```
    var registeredUser = await RegisterChangePasswordUserAsync(authService);
    var savedUser = await dbContext.Users.SingleAsync();
    var originalPasswordHash = savedUser.PasswordHash;

    var request = new ChangePasswordRequest
    {
        CurrentPassword = "Password123",
        NewPassword = "NewPassword456",
        ConfirmNewPassword = "DifferentPassword789"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => authService.ChangePasswordAsync(
            registeredUser.UserId,
            request));

    Assert.Equal(ErrorMessages.NewPasswordMismatch, exception.Message);
    Assert.Equal(originalPasswordHash, savedUser.PasswordHash);
}
```

Step 5 - Add Same Password Test

Add:

```
[Fact]
public async Task ChangePasswordAsync_WithSamePassword_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var authService = CreateAuthService(dbContext);
    var registeredUser = await RegisterChangePasswordUserAsync(authService);
    var savedUser = await dbContext.Users.SingleAsync();
    var originalPasswordHash = savedUser.PasswordHash;

    var request = new ChangePasswordRequest
    {
        CurrentPassword = "Password123",
        NewPassword = "Password123",
        ConfirmNewPassword = "Password123"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => authService.ChangePasswordAsync(
            registeredUser.UserId,
            request));

    Assert.Equal(ErrorMessages.NewPasswordMustBeDifferent, exception.Message);
    Assert.Equal(originalPasswordHash, savedUser.PasswordHash);
}
```

The service checks password reuse through `VerifyPassword`. It cannot compare the plain password with the stored hash as ordinary strings.

Step 6 - Add Missing User Test

Add:

```
[Fact]
public async Task ChangePasswordAsync_WithMissingUser_ReturnsFalse()
{
    await using var dbContext = CreateDbContext();
    var authService = CreateAuthService(dbContext);

    var request = new ChangePasswordRequest
    {
        CurrentPassword = "Password123",
        NewPassword = "NewPassword456",
        ConfirmNewPassword = "NewPassword456"
    };

    var passwordChanged = await authService.ChangePasswordAsync(999, request);

    Assert.False(passwordChanged);
    Assert.Empty(dbContext.Users);
}
```

The controller converts this `false` result into `404 Not Found`.

Step 7 - Add Registration Helper

Add this helper with the other private helper methods:

```
private static Task<AuthResponse> RegisterChangePasswordUserAsync(
    AuthService authService)
{
    return authService.RegisterAsync(new RegisterRequest
    {
        FullName = "Change Password Test User",
        Email = "change.password@example.com",
        Password = "Password123"
    });
}
```

Each test uses a different in-memory database, so the same test email can safely be reused.

The helper removes repetitive registration setup while leaving each test's important request and assertions visible.

Where To Place The New Code

Use this order inside `AuthServiceTests`:

```
existing registration test
existing valid login test
existing invalid login test
successful password-change test
incorrect-current-password test
mismatched-confirmation test
same-password test
missing-user test
RegisterChangePasswordUserAsync helper
CreateAuthService helper
CreateDbContext helper
FakeJwtTokenService
```

Complete New Test Count

```
Before Day 103: 23
New tests: 5
After Day 103: 28
```

The five new tests are:

```
ChangePasswordAsync_WithValidRequest_ChangesPassword
ChangePasswordAsync_WithIncorrectCurrentPassword_ThrowsArgumentException
ChangePasswordAsync_WithMismatchedConfirmation_ThrowsArgumentException
ChangePasswordAsync_WithSamePassword_ThrowsArgumentException
ChangePasswordAsync_WithMissingUser_ReturnsFalse
```

Why No Controller Test Is Added Today

Day 103 focuses on authentication use-case behavior:

```
verification
hash replacement
failure safety
login behavior after change
```

Day 104 is the roadmap's authorization-test day. Keep JWT endpoint access and cross-user authorization tests for that lesson.

Step 8 - Run The Auth Tests First

Run only the auth test class while developing:

```
cd [repository root]
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj `
    --configuration Release `
    --filter "FullyQualifiedName~AuthServiceTests"
```

Expected:

```
Passed: 8
Failed: 0
Skipped: 0
```

The class has three existing tests and five new tests.

Step 9 - Build And Run All Tests

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 28
Failed: 0
Skipped: 0
```

If A Test Fails

Read the failed assertion before editing production code.

Check:

```
Did the test register a user first?
Does userId match the registered user?
Is CurrentPassword exactly Password123?
Do NewPassword and ConfirmNewPassword match where expected?
Was originalPasswordHash captured before the service call?
Does each test create a unique in-memory database?
```

Fix production code only when the test demonstrates a real behavior defect.

Important Checks

```
[ ] ErrorMessages using was added
[ ] Successful change returns true
[ ] Stored hash changes after success
[ ] Old password no longer verifies
[ ] New password verifies
[ ] Login succeeds with new password
[ ] Login fails with old password
[ ] Incorrect current password returns the safe error
[ ] Mismatched confirmation returns the safe error
[ ] Same password returns the safe error
[ ] Failed attempts preserve the original hash
[ ] Missing user returns false
[ ] No production code changed unless a defect was found
[ ] No migration was created
[ ] AuthServiceTests has 8 passing tests
[ ] Complete suite has 28 passing tests
```

File Explained

AuthServiceTests.cs

Extends the existing authentication test suite with successful and failed change-password scenarios. The tests verify stored hash behavior and confirm that login moves from the old password to the new password after a successful change.

Interview Explanation

I added service tests for successful and failed password changes using an EF Core in-memory database and the real password hasher.
The tests verify that the stored hash changes only on success, the old password stops working, the new password works for login, and invalid attempts preserve the existing hash.
I also covered password reuse, confirmation mismatch, incorrect current password, and missing-user behavior.

End Of Day Note Template

1. What I built
I added AuthService tests for successful password changes, incorrect current passwords, mismatched confirmation, password reuse, and missing users.

2. What worked
The tests confirm the hash changes only on success, old-password login fails, new-password login works, failed attempts preserve the original hash, and all 28 tests pass.

3. What I will do tomorrow
Add authorization tests that prove protected endpoints reject anonymous users and prevent access to another user's tasks.

Day 104 - Strengthen Task Authorization Tests

Goal

Verify that authenticated users cannot access or modify another user's tasks, and fix the remaining ownership gap in the complete and reopen endpoints.

Today you will:

- review protected task endpoints
- add `userId` ownership checks to complete and reopen
- keep unauthorized ownership results hidden as `404 Not Found`
- add cross-user tests for get, update, delete, complete, and reopen
- verify the owner can still complete and reopen a task
- manually confirm anonymous task requests return `401 Unauthorized`

Important Finding

The current service already scopes these operations by user id:

```
list tasks
get task by id
update task
delete task
```

However, these methods currently accept only a task id:

```
Task<TaskResponse?> CompleteTaskAsync(int id);

Task<TaskResponse?> ReopenTaskAsync(int id);
```

Their service implementation uses:

```
_dbContext.Tasks.FindAsync(id)
```

That checks whether the task exists but does not check who owns it.

Because `TasksController` has `[Authorize]`, anonymous users are blocked, but one authenticated user could still change another user's completion state if they know the task id.

Day 104 closes this authorization gap.

Authorization Rules

```
No JWT                        -> 401 Unauthorized
Valid JWT and owned task      -> operation succeeds
Valid JWT and another's task  -> 404 Not Found
Valid JWT and missing task    -> 404 Not Found
```

Why Another User Receives 404

Returning `403 Forbidden` would confirm that the task exists but belongs to someone else.

Returning the same `404 Not Found` response for missing and non-owned tasks avoids leaking another user's task existence.

The service implements this with one query:

```
task.Id == id && task.UserId == userId
```

Files To Update And Create

Update

```
[repository root]\src\JobTrackr.Application\Tasks\ITaskService.cs
[repository root]\src\JobTrackr.Infrastructure\Tasks\TaskService.cs
[repository root]\src\JobTrackr.Api\Controllers\TasksController.cs
```

Create

```
[repository root]\tests\JobTrackr.Tests\Tasks\TaskServiceAuthorizationTests.cs
```

No DTO, entity, database, migration, or package change is required.

Step 1 - Update ITaskService

Open:

```
[repository root]\src\JobTrackr.Application\Tasks\ITaskService.cs
```

Change:

```
Task<TaskResponse?> CompleteTaskAsync(int id);

Task<TaskResponse?> ReopenTaskAsync(int id);
```

to:

```
Task<TaskResponse?> CompleteTaskAsync(int id, int userId);

Task<TaskResponse?> ReopenTaskAsync(int id, int userId);
```

The interface now requires ownership context for every task-by-id mutation.

Step 2 - Secure CompleteTaskAsync

Open:

```
[repository root]\src\JobTrackr.Infrastructure\Tasks\TaskService.cs
```

Replace the current complete method with:

```
public async Task<TaskResponse?> CompleteTaskAsync(int id, int userId)
{
    var task = await _dbContext.Tasks
        .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);

    if (task is null)
    {
        return null;
    }

    task.IsCompleted = true;

    await _dbContext.SaveChangesAsync();

    return MapToResponse(task);
}
```

What Changed

Before:

```
FindAsync(id)
```

After:

```
FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId)
```

The task is found only when both its id and owner id match.

Step 3 - Secure ReopenTaskAsync

Replace the current reopen method with:

```
public async Task<TaskResponse?> ReopenTaskAsync(int id, int userId)
{
    var task = await _dbContext.Tasks
        .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);

    if (task is null)
    {
        return null;
    }

    task.IsCompleted = false;

    await _dbContext.SaveChangesAsync();

    return MapToResponse(task);
}
```

Complete and reopen now use the same ownership rule as get, update, and delete.

Step 4 - Update Complete Controller Action

Open:

```
[repository root]\src\JobTrackr.Api\Controllers\TasksController.cs
```

Replace `Complete` with:

```
[HttpPatch("{id}/complete")]
public async Task<IActionResult> Complete(int id)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var response = await _taskService.CompleteTaskAsync(
        id,
        currentUserId.Value);

    if (response is null)
    {
        return Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Not Found",
            detail: ErrorMessages.TaskNotFound);
    }

    return Ok(response);
}
```

The controller class already has `[Authorize]`. The action reads the authenticated user id from the existing claim helper and passes it to the service.

Step 5 - Update Reopen Controller Action

Replace `Reopen` with:

```
[HttpPatch("{id}/reopen")]
public async Task<IActionResult> Reopen(int id)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
```

```
            return Unauthorized();
        }

        var response = await _taskService.ReopenTaskAsync(
            id,
            currentUserId.Value);

        if (response is null)
        {
            return Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Not Found",
                detail: ErrorMessages.TaskNotFound);
        }

        return Ok(response);
    }
```

Controller Authorization Flow

```
JWT middleware rejects anonymous request
controller reads NameIdentifier
controller passes userId to service
service queries by task id and owner id
owned task returns response
missing or non-owned task returns null
controller maps null to 404
```

Step 6 - Find Every Call Site

Run:

```
cd [repository root]
rg -n "CompleteTaskAsync\(|ReopenTaskAsync\(" src tests
```

Every call must now provide:

```
task id
authenticated user id
```

Expected production call sites:

```
ITaskService
TaskService
TasksController
```

The new tests will also call these methods with owner and non-owner ids.

Step 7 - Create Authorization Test File

Create:

```
New-Item tests\JobTrackr.Tests\Tasks\TaskServiceAuthorizationTests.cs -ItemType File
```

Open:

```
[repository root]\tests\JobTrackr.Tests\Tasks\TaskServiceAuthorizationTests.cs
```

Add:

```
using JobTrackr.Application.Tasks;
using JobTrackr.Domain.Entities;
using JobTrackr.Infrastructure.Persistence;
using JobTrackr.Infrastructure.Tasks;
using Microsoft.EntityFrameworkCore;

namespace JobTrackr.Tests.Tasks
{
    public class TaskServiceAuthorizationTests
    {
        [Fact]
        public async Task GetByIdAsync_WithAnotherUsersTask_ReturnsNull()
        {
```

```
        await using var dbContext = CreateDbContext();
        var testData = await AddUsersAndTaskAsync(dbContext);
        var taskService = new TaskService(dbContext);

        var response = await taskService.GetByIdAsync(
            testData.Task.Id,
            testData.OtherUser.Id);

        Assert.Null(response);
    }

    [Fact]
    public async Task UpdateTaskAsync_WithAnotherUsersTask_ReturnsNullAndPreservesTask()
    {
        await using var dbContext = CreateDbContext();
        var testData = await AddUsersAndTaskAsync(dbContext);
        var taskService = new TaskService(dbContext);
        var originalTitle = testData.Task.Title;
        var request = new UpdateTaskRequest
        {
            Title = "Unauthorized update",
            Description = "This must not be saved",
            Priority = "High"
        };

        var response = await taskService.UpdateTaskAsync(
            testData.Task.Id,
            request,
            testData.OtherUser.Id);

        Assert.Null(response);
        Assert.Equal(originalTitle, testData.Task.Title);
    }

    [Fact]
    public async Task DeleteTaskAsync_WithAnotherUsersTask_ReturnsFalseAndPreservesTask()
    {
        await using var dbContext = CreateDbContext();
        var testData = await AddUsersAndTaskAsync(dbContext);
        var taskService = new TaskService(dbContext);

        var isDeleted = await taskService.DeleteTaskAsync(
            testData.Task.Id,
            testData.OtherUser.Id);

        Assert.False(isDeleted);
        Assert.True(await dbContext.Tasks.AnyAsync(
            task => task.Id == testData.Task.Id));
    }

    [Fact]
    public async Task CompleteTaskAsync_WithAnotherUsersTask_ReturnsNullAndPreservesState()
    {
        await using var dbContext = CreateDbContext();
        var testData = await AddUsersAndTaskAsync(dbContext);
        var taskService = new TaskService(dbContext);

        var response = await taskService.CompleteTaskAsync(
            testData.Task.Id,
            testData.OtherUser.Id);

        Assert.Null(response);
        Assert.False(testData.Task.IsCompleted);
    }

    [Fact]
    public async Task ReopenTaskAsync_WithAnotherUsersTask_ReturnsNullAndPreservesState()
    {
        await using var dbContext = CreateDbContext();
        var testData = await AddUsersAndTaskAsync(dbContext, isCompleted: true);
        var taskService = new TaskService(dbContext);

        var response = await taskService.ReopenTaskAsync(
            testData.Task.Id,
            testData.OtherUser.Id);

        Assert.Null(response);
        Assert.True(testData.Task.IsCompleted);
```

```
        }

        [Fact]
        public async Task CompleteAndReopenTaskAsync_WithOwnedTask_UpdatesCompletionState()
        {
            await using var dbContext = CreateDbContext();
            var testData = await AddUsersAndTaskAsync(dbContext);
            var taskService = new TaskService(dbContext);

            var completedTask = await taskService.CompleteTaskAsync(
                testData.Task.Id,
                testData.Owner.Id);

            Assert.NotNull(completedTask);
            Assert.True(completedTask.IsCompleted);
            Assert.True(testData.Task.IsCompleted);

            var reopenedTask = await taskService.ReopenTaskAsync(
                testData.Task.Id,
                testData.Owner.Id);

            Assert.NotNull(reopenedTask);
            Assert.False(reopenedTask.IsCompleted);
            Assert.False(testData.Task.IsCompleted);
        }

        private static AppDbContext CreateDbContext()
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;

            return new AppDbContext(options);
        }

        private static async Task<AuthorizationTestData> AddUsersAndTaskAsync(
            AppDbContext dbContext,
            bool isCompleted = false)
        {
            var owner = new User
            {
                FullName = "Task Owner",
                Email = "task.owner@example.com",
                PasswordHash = "test-password-hash"
            };

            var otherUser = new User
            {
                FullName = "Other User",
                Email = "other.user@example.com",
                PasswordHash = "test-password-hash"
            };

            var task = new JobTask
            {
                Title = "Owner's private task",
                Description = "Only the owner may access this task",
                Priority = "Medium",
                IsCompleted = isCompleted,
                User = owner
            };

            dbContext.Tasks.Add(task);
            dbContext.Users.Add(otherUser);
            await dbContext.SaveChangesAsync();

            return new AuthorizationTestData(task, owner, otherUser);
        }

        private record AuthorizationTestData(
            JobTask Task,
            User Owner,
            User OtherUser);
    }
}
```

Why A Small Test Record Is Used

The helper creates three related objects:

```
the task
the owner
the other authenticated user
```

The private record gives those returned values clear names:

```
testData.Task
testData.Owner
testData.OtherUser
```

It is local test organization, not a production abstraction.

What The Tests Prove

Get By Id

Another user's id cannot retrieve the task.

Update

Another user's id receives `null`, and the original task title remains unchanged.

Delete

Another user's id receives `false`, and the task remains in the database.

Complete

Another user's id receives `null`, and an incomplete task remains incomplete.

Reopen

Another user's id receives `null`, and a completed task remains completed.

Owner Complete And Reopen

The ownership fix does not block the real owner. The owner can still complete and reopen their own task.

Why These Are Service Authorization Tests

Task ownership is enforced in `TaskService` database queries. Testing the service directly proves that a caller cannot bypass ownership merely by supplying another task id.

Full HTTP integration testing is planned later in the roadmap. Today also includes a manual check that `[Authorize]` rejects anonymous requests.

Step 8 - Run The Authorization Tests

```
cd [repository root]
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj `
    --configuration Release `
    --filter "FullyQualifiedName~TaskServiceAuthorizationTests"
```

Expected:

```
Passed: 6
Failed: 0
Skipped: 0
```

Step 9 - Build And Run All Tests

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --no-build
```

Expected:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 34
Failed: 0
Skipped: 0
```

The test count increases from 28 to 34.

Step 10 - Run The API

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Manual Test 1 - Anonymous Request

Without a JWT:

```
$baseUrl = "https://localhost:7024"

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks" `
        -Method Get
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

Also test a mutation without a token:

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks/1/complete" `
        -Method Patch
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
401
```

Manual Test 2 - Two-User Ownership

Use two registered users and obtain `$tokenA` and `$tokenB`.

Prepare headers:

```
$headersA = @{ Authorization = "Bearer $tokenA" }
$headersB = @{ Authorization = "Bearer $tokenB" }
```

Create a task as User A and keep its id:

```
$taskBody = @{
    title = "User A private task"
    description = "Authorization test"
    priority = "Medium"
} | ConvertTo-Json

$task = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks" `
```

```
    -Method Post `
    -Headers $headersA `
    -ContentType "application/json" `
    -Body $taskBody

$taskId = $task.id
```

Try to complete User A's task as User B:

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks/$taskId/complete" `
        -Method Patch `
        -Headers $headersB
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected:

```
404
Task not found.
```

Complete the same task as User A:

```
$completed = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks/$taskId/complete" `
    -Method Patch `
    -Headers $headersA

$completed.isCompleted
```

Expected:

```
True
```

Try to reopen it as User B:

```
try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/tasks/$taskId/reopen" `
        -Method Patch `
        -Headers $headersB
} catch {
    [int]$_.Exception.Response.StatusCode
}
```

Expected:

```
404
```

Reopen it as User A:

```
$reopened = Invoke-RestMethod `
    -Uri "$baseUrl/api/tasks/$taskId/reopen" `
    -Method Patch `
    -Headers $headersA

$reopened.isCompleted
```

Expected:

```
False
```

Important Security Checks

```
[ ] TasksController remains protected with [Authorize]
[ ] Anonymous GET tasks returns 401
[ ] Anonymous PATCH complete returns 401
[ ] Complete service requires userId
[ ] Reopen service requires userId
[ ] Controller reads userId from JWT claims
[ ] Controller never accepts userId from the request
[ ] Get another user's task returns null/404
[ ] Update another user's task is blocked
[ ] Delete another user's task is blocked
[ ] Complete another user's task is blocked
[ ] Reopen another user's task is blocked
[ ] Failed cross-user operations preserve task state
[ ] Owner complete and reopen still work
[ ] Non-owned and missing tasks use the same 404 behavior
[ ] No migration was created
[ ] All 34 tests pass
```

Files Explained

ITaskService.cs

Adds authenticated user context to complete and reopen use-case contracts.

TaskService.cs

Queries complete and reopen targets by both task id and owner id, closing the remaining cross-user mutation gap.

TasksController.cs

Reads the logged-in user id from JWT claims and passes it into complete and reopen operations.

TaskServiceAuthorizationTests.cs

Verifies that another user cannot get, update, delete, complete, or reopen the owner's task, while confirming the owner can still change completion state.

Interview Explanation

```
I reviewed task authorization and found that complete and reopen required authentication but
did not enforce ownership.
I added the authenticated user id to those service contracts and queried by both task id and
owner id, returning the same not-found behavior for missing and non-owned tasks.
I added cross-user tests for every task-by-id operation and confirmed the real owner can still
complete and reopen their task.
```

End Of Day Note Template

```
1. What I built
I added ownership checks to task complete and reopen operations and added authorization tests
for cross-user task access.

2. What worked
Anonymous task requests return 401, another user's get, update, delete, complete, and reopen
operations are blocked, owners can still change their own tasks, and all 34 tests pass.

3. What I will do tomorrow
Review API contracts, DTO names, route names, and status codes for consistency.
```

Day 105 - API Contract Review

Today's Goal

Review the existing JobTrackr API contract without adding a new feature.

Today you will check:

- Route names
- Request and response DTO names
- HTTP status codes
- Authentication requirements
- Task ownership behavior
- Known contract limitations

This is a review day. Do not redesign the architecture or change working endpoints unless the review reveals an immediate build error.

Why This Review Matters

An API contract is the agreement between the backend and its clients. It includes the URL, HTTP method, request body, response body, status codes, and authentication rules.

A stable contract makes the API easier to use, document, test, and change safely.

Files to Review

```
src/JobTrackr.Api/Controllers/AuthController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Application/DTOs/
```

No production code file is expected to change today.

1. Route Contract Review

Health

Method	Route	Authentication	Success
GET	`/health`	Public	200 OK

Authentication

Method	Route	Authentication	Success	Common errors
POST	`/api/auth/register`	Public	200 OK	400 Bad Request
POST	`/api/auth/login`	Public	200 OK	400 Bad Request
PUT	`/api/auth/change-password`	JWT	204 No Content	400, 401, 404

Tasks

All task endpoints require a valid JWT token.

Method	Route	Purpose	Success	Common errors
GET	`/api/tasks`	List the current user's tasks	200 OK	400, 401
GET	`/api/tasks/{id}`	Get one owned task	200 OK	401, 404
POST	`/api/tasks`	Create an owned task	201 Created	400, 401
PUT	`/api/tasks/{id}`	Update an owned task	200 OK	400, 401, 404
DELETE	`/api/tasks/{id}`	Delete an owned task	204 No Content	401, 404
PATCH	`/api/tasks/{id}/complete`	Mark an owned task complete	200 OK	401, 404
PATCH	`/api/tasks/{id}/reopen`	Reopen an owned task	200 OK	401, 404

The task list supports:

```
isCompleted
search
sortBy
sortDirection
pageNumber
pageSize
```

Current User Profile

Method	Route	Authentication	Success	Common errors
GET	`/api/users/me`	JWT	200 OK	401, 404
PUT	`/api/users/me`	JWT	200 OK	400, 401, 404

The `/me` route is appropriate because the API reads the user identity from JWT claims instead of accepting a user id from the client.

Legacy User Routes

Method	Route	Current concern
GET	`/api/users`	Currently public
GET	`/api/users/{id}`	Currently public
GET	`/api/users/{userId}/tasks`	Can expose another user's tasks
POST	`/api/users`	Currently public
PUT	`/api/users/{id}`	Currently public
DELETE	`/api/users/{id}`	Currently public

Do not fix all these routes today. Record them as authorization work for a future lesson.

2. DTO Naming Review

The current DTO naming style is clear and production-friendly:

DTO pattern	Purpose	Examples
`Create...Request`	Data accepted when creating a resource	`CreateTaskRequest`, `CreateUserRequest`
`Update...Request`	Data accepted when updating a resource	`UpdateTaskRequest`, `UpdateUserRequest`
`...QueryRequest`	Query-string filtering and paging values	`TaskQueryRequest`
`...Response`	Data returned to the client	`TaskResponse`, `UserResponse`, `AuthResponse`
`PagedResponse<T>`	Items and pagination metadata	`PagedResponse<TaskResponse>`
Action-specific request	Input for a specific use case	`LoginRequest`, `RegisterRequest`, `ChangePasswordRequest`

Database entities are not returned directly from controllers. Password hashes are never included in response DTOs.

3. Route Naming Decisions

Keep these conventions:

- Use plural resource routes: `/api/tasks` and `/api/users`.
- Use route parameters for a specific resource: `/api/tasks/{id}`.
- Use `/me` when the identity comes from the authenticated token.
- Use action words only for clear state transitions such as `complete`, `reopen`, and `change-password`.
- Keep route text lowercase and use a hyphen for multiword route segments.

4. Status Code Decisions

The API should continue using:

Status	Meaning in JobTrackr
200 OK	Successful read, login, registration, update, or task state change
201 Created	A task or user was created and a location can be returned
204 No Content	Successful delete or password change with no response body
400 Bad Request	Validation or safe business-rule failure
401 Unauthorized	JWT is missing, invalid, or does not contain a usable user id
404 Not Found	Resource is missing or the authenticated user does not own the task
500 Internal Server Error	Unexpected server failure handled safely

Returning 404 for another user's task is intentional. It does not reveal whether that task exists.

5. Response Shape Review

Check these behaviors in Swagger:

- Validation errors use a structured `ProblemDetails` response.

- Missing resources return a safe not-found response.
- Unexpected exceptions do not expose stack traces or secrets.
- Task lists return `items`, `pageNumber`, `pageSize`, `totalCount`, and `totalPages`.
- Create Task returns 201 and a `Location` header.
- Delete and Change Password return 204 with no response body.

6. Contract Issue Register

These issues are recorded for future lessons and should not be hidden by the contract review.

Priority	Issue	Decision for Day 105
High	Legacy user CRUD routes are not protected	Record for future authorization work
High	`/api/users/{userId}/tasks` can bypass task ownership isolation	Record for a focused security fix
Medium	Profile email updates may need duplicate-email protection	Add later validation and tests
Low	Registration returns `AuthResponse` with an empty token	Keep the current contract for now and revisit deliberately
Future security	Existing JWTs remain valid after a password change	Handle later with token expiry or revocation design

7. Manual Review Checklist

Open Swagger and confirm:

1. Route names match the tables above.
2. Task routes display the lock icon and require JWT authentication.
3. Request bodies use request DTOs rather than entities.
4. Responses do not expose `PasswordHash`.
5. Task list paging metadata is present.
6. Create Task returns 201 with a `Location` header.
7. Missing or non-owned tasks return 404.
8. Invalid request data returns 400.
9. Anonymous access to task routes returns 401.
10. The legacy user-route limitations are understood and recorded.

8. Build and Test

Run these commands from the solution folder:

```
cd [repository root]
dotnet build JobTrackr.sln -c Release
dotnet test JobTrackr.sln -c Release --no-build
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
```

Interview Explanation

> An API contract defines routes, HTTP methods, request and response models, authentication requirements, and status codes. I reviewed JobTrackr to ensure its main task, authentication, and profile APIs use consistent DTOs and predictable HTTP behavior. I also recorded legacy user authorization risks separately so they can be fixed in focused changes without redesigning the application.

End-of-Day Progress Note

```
1. What I reviewed
I reviewed API route names, DTO names, response shapes, status codes, authentication
requirements, and task ownership behavior.

2. What worked
The main task, authentication, and self-profile contracts are consistent, all 34 tests pass,
and I documented the remaining legacy user authorization, duplicate-email, registration-
response, and JWT-revocation limitations.

3. What I will do tomorrow
Add useful XML comments or Swagger descriptions to important endpoints without overdocumenting
the API.
```

Day 106 - Add XML Comments to Swagger

Today's Goal

Add useful endpoint descriptions to Swagger by:

1. Generating an XML documentation file during the build.
2. Configuring Swagger to read that file.
3. Adding short XML summaries to endpoints whose behavior is not obvious.

Do not document every line or repeat information already clear from the route name.

What You Will Change

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/Controllers/AuthController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
```

No new package is required. `Swashbuckle.AspNetCore` is already installed.

1. Generate the XML Documentation File

Open:

```
[repository root]\src\JobTrackr.Api\JobTrackr.Api.csproj
```

Add these two properties inside the existing `<PropertyGroup>`:

```
<GenerateDocumentationFile>true</GenerateDocumentationFile>
<NoWarn>$(NoWarn);1591</NoWarn>
```

The property group should look like this:

```
<PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <UserSecretsId>44d05343-2676-4046-b55f-43a90e2b8002</UserSecretsId>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
  <NoWarn>$(NoWarn);1591</NoWarn>
</PropertyGroup>
```

What These Properties Do

- `GenerateDocumentationFile` creates `JobTrackr.Api.xml` during the build.
- Warning `1591` means a public member has no XML comment.
- Suppressing only `1591` lets us document useful endpoints gradually without hiding other compiler warnings.

Do not add the generated XML file manually to Git. It is created inside `bin` during each build, and `bin` should already be ignored.

2. Configure Swagger to Read the XML File

Open:

```
[repository root]\src\JobTrackr.Api\Program.cs
```

Add this using statement:

```
using System.Reflection;
```

Replace:

```
builder.Services.AddSwaggerGen();
```

with:

```
builder.Services.AddSwaggerGen(options =>
{
    var xmlFileName = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlFilePath = Path.Combine(AppContext.BaseDirectory, xmlFileName);

    options.IncludeXmlComments(xmlFilePath);
});
```

How This Works

- `Assembly.GetExecutingAssembly()` gets the running API assembly.
- Its name is `JobTrackr.Api`, so the documentation filename is `JobTrackr.Api.xml`.
- `AppContext.BaseDirectory` points to the API build-output folder.
- `IncludeXmlComments` tells Swagger to use the summaries from that XML file.

Avoid writing an absolute XML path because build-output paths differ between Debug, Release, Windows, and GitHub Actions.

3. Document Important Authentication Endpoints

Open `AuthController.cs` and add each comment directly above its endpoint attribute.

Login

```
/// <summary>
/// Authenticates a user and returns a JWT access token.
/// </summary>
[HttpPost("login")]
public async Task<IActionResult> Login(LoginRequest request)
```

This description is useful because it tells the client what successful login returns.

Change Password

```
/// <summary>
/// Changes the authenticated user's password after verifying the current password.
/// </summary>
[HttpPut("change-password")]
[Authorize]
public async Task<IActionResult> ChangePassword(ChangePasswordRequest request)
```

This explains both authentication and current-password verification without describing the implementation.

Registration is already clear from `POST /api/auth/register`, so it does not need a comment today.

4. Document Important Task Endpoints

Open `TasksController.cs` and add these comments directly above the matching attributes.

Get All Tasks

```
/// <summary>
/// Returns the authenticated user's tasks with optional filtering, sorting, and pagination.
/// </summary>
[HttpGet]
public async Task<IActionResult> GetAll([FromQuery] TaskQueryRequest request)
```

This comment explains that the endpoint is owner-scoped and supports query behavior.

Get Task by Id

```
/// <summary>
/// Returns an owned task by id, or not found when it is missing or belongs to another user.
/// </summary>
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
```

This records the intentional ownership rule from the Day 105 contract review.

Create Task

```
/// <summary>
/// Creates a task for the authenticated user.
/// </summary>
[HttpPost]
public async Task<IActionResult> Create(CreateTaskRequest request)
```

This makes it clear that task ownership comes from the JWT token, not from a `UserId` in the request body.

Do not add comments such as `Deletes a task` above `DELETE /api/tasks/{id}`. That simply repeats information Swagger already communicates.

5. Run and Check Swagger

From the solution folder, run:

```
cd [repository root]
dotnet run --project src\JobTrackr.Api
```

Open the HTTPS Swagger URL shown in the terminal and expand the documented endpoints.

Confirm that Swagger displays descriptions for:

```
POST  /api/auth/login
PUT   /api/auth/change-password
GET   /api/tasks
GET   /api/tasks/{id}
POST  /api/tasks
```

Also confirm that undocumented endpoints still appear and work normally.

6. Build and Test

Stop the running API, then run:

```
dotnet build JobTrackr.slnx -c Release
dotnet test JobTrackr.slnx -c Release --no-build
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
```

You can confirm the XML file was generated with:

```
Test-Path .\src\JobTrackr.Api\bin\Release\net8.0\JobTrackr.Api.xml
```

Expected output:

```
True
```

What Each Changed File Does

`JobTrackr.Api.csproj`

Enables XML documentation generation during the API build. It suppresses only missing XML-comment warnings so partial, useful documentation remains warning-free.

`Program.cs`

Finds the generated XML documentation file and registers it with Swagger. This makes endpoint summaries visible in Swagger UI.

`AuthController.cs`

Documents the non-obvious behavior of login and password changes. It does not change authentication logic or API responses.

`TasksController.cs`

Documents ownership, filtering, sorting, pagination, and task creation behavior. It does not change task logic.

Interview Explanation

> I enabled XML documentation generation in the API project and configured Swagger to read the generated file. I added concise summaries only where they explain important behavior such as JWT authentication, task ownership, filtering, and pagination. XML comments improve the API documentation without changing runtime behavior.

End-of-Day Progress Note

```
1. What I built
I enabled XML documentation generation, configured Swagger to read the generated file, and
added concise descriptions to important authentication and task endpoints.

2. What worked
Swagger displays the new endpoint descriptions, undocumented endpoints still work normally, the
Release build has no warnings or errors, and all 34 tests pass.

3. What I will do tomorrow
Configure JWT authorization in Swagger so protected endpoints can be tested with a Bearer token.
```

Day 107 - Configure JWT Authentication in Swagger

Today's Goal

Configure Swagger so you can:

1. Log in and receive a JWT token.
2. Enter the token once using Swagger's Authorize button.
3. Test protected endpoints without manually editing request headers.
4. Show lock icons only on endpoints that use `[Authorize]`.

This changes Swagger documentation and testing behavior only. It does not change JWT validation or application authorization rules.

What You Will Change

```
src/JobTrackr.Api/Swagger/AuthorizeCheckOperationFilter.cs
src/JobTrackr.Api/Program.cs
```

No new NuGet package is required. The existing `Swashbuckle.AspNetCore` package contains the Swagger types needed today.

1. Create the Swagger Folder and File

Run these commands from the solution folder:

```
cd [repository root]
New-Item -ItemType Directory -Path .\src\JobTrackr.Api\Swagger -Force
New-Item -ItemType File -Path .\src\JobTrackr.Api\Swagger\AuthorizeCheckOperationFilter.cs
```

The filter will inspect controller and action attributes while Swagger generates the API document.

2. Add the Authorization Operation Filter

Open:

```
[repository root]\src\JobTrackr.Api\Swagger\AuthorizeCheckOperationFilter.cs
```

Add this code:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;

namespace JobTrackr.Api.Swagger
{
    public class AuthorizeCheckOperationFilter : IOperationFilter
    {
        public void Apply(OpenApiOperation operation, OperationFilterContext context)
        {
            var controllerType = context.MethodInfo.DeclaringType;

            var hasAuthorizeAttribute =
                context.MethodInfo.GetCustomAttributes(true)
                    .OfType<AuthorizeAttribute>()
                    .Any()
                || controllerType?.GetCustomAttributes(true)
                    .OfType<AuthorizeAttribute>()
                    .Any() == true;
```

```
            var hasAllowAnonymousAttribute =
                context.MethodInfo.GetCustomAttributes(true)
                    .OfType<AllowAnonymousAttribute>()
                    .Any()
                || controllerType?.GetCustomAttributes(true)
                    .OfType<AllowAnonymousAttribute>()
                    .Any() == true;

            if (!hasAuthorizeAttribute || hasAllowAnonymousAttribute)
            {
                return;
            }

            operation.Security = new List<OpenApiSecurityRequirement>
            {
                new()
                {
                    [new OpenApiSecurityScheme
                    {
                        Reference = new OpenApiReference
                        {
                            Type = ReferenceType.SecurityScheme,
                            Id = "Bearer"
                        }
                    }] = Array.Empty<string>()
                }
            };
        }
    }
}
```

How the Filter Works

The filter runs once for each Swagger operation.

- It checks the action method for `[Authorize]`.
- It also checks the controller class for `[Authorize]`.
- It respects `[AllowAnonymous]` if that attribute is added later.
- It attaches the `Bearer` security requirement only to protected operations.

This matters because `TasksController` has `[Authorize]` on the class, while change-password and `/me` use `[Authorize]` on individual methods.

Without this filter, a global Swagger security requirement could incorrectly show public registration and login endpoints as protected.

3. Register Bearer Authentication with Swagger

Open:

```
[repository root]\src\JobTrackr.Api\Program.cs
```

Add these using statements:

```
using JobTrackr.Api.Swagger;
using Microsoft.OpenApi.Models;
```

Keep the XML documentation configuration from Day 106. Update the existing `AddSwaggerGen` block to this:

```
builder.Services.AddSwaggerGen(options =>
{
    var xmlFileName = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlFilePath = Path.Combine(AppContext.BaseDirectory, xmlFileName);

    options.IncludeXmlComments(xmlFilePath);

    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
```

```
        Description = "Enter the JWT token returned by the login endpoint.",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT"
    });

    options.OperationFilter<AuthorizeCheckOperationFilter>();
});
```

Understanding the Security Definition

```
Type = SecuritySchemeType.Http
Scheme = "bearer"
```

These settings tell Swagger UI to send the header in the standard format:

```
Authorization: Bearer your-jwt-token
```

`BearerFormat = "JWT"` is documentation for API users. It does not validate the token. Actual token validation remains in `AddJwtBearer`.

4. Test Authentication in Swagger

Run the API:

```
dotnet run --project .\src\JobTrackr.Api
```

Open the HTTPS Swagger URL shown in the terminal.

Step A: Confirm the Locks

Protected operations should show a lock icon:

```
PUT    /api/auth/change-password
GET    /api/tasks
GET    /api/tasks/{id}
POST   /api/tasks
PUT    /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
GET    /api/users/me
PUT    /api/users/me
```

Public register and login operations should not show a lock.

Step B: Login

Use:

```
POST /api/auth/login
```

Enter the email and password of an existing user, execute the request, and copy only the value of the returned `token` property.

Step C: Authorize Swagger

1. Select the Authorize button near the top of Swagger UI.
2. Paste only the JWT token.
3. Do not type `Bearer` before it because Swagger adds that prefix automatically.
4. Select Authorize, then close the dialog.

Step D: Call a Protected Endpoint

Execute:

```
GET /api/tasks
```

Expected result:

```
200 OK
```

Swagger should send a request header similar to:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Step E: Test Without Authorization

Select Authorize, choose Logout, and call `GET /api/tasks` again.

Expected result:

```
401 Unauthorized
```

This confirms Swagger is only supplying the token; the API still enforces authentication.

5. Build and Test

Stop the running API, then run:

```
cd [repository root]
dotnet build JobTrackr.slnx -c Release
dotnet test JobTrackr.slnx -c Release --no-build
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
```

No new automated business-logic test is required today because the change affects generated Swagger documentation and its browser interface. The authorization behavior already has automated tests; today's Swagger flow is verified manually.

What Each Changed File Does

`AuthorizeCheckOperationFilter.cs`

Checks whether each controller action requires authorization. It adds Swagger's Bearer security requirement only to protected operations.

`Program.cs`

Defines the JWT Bearer scheme for Swagger and registers the authorization filter. The existing ASP.NET Core JWT validation configuration remains unchanged.

Common Mistakes

Typing `Bearer` in the Dialog

With an HTTP Bearer security scheme, paste only the JWT. Swagger adds `Bearer` to the header.

Using the Registration Response Token

The current registration response does not provide a usable token. Use `POST /api/auth/login` and copy its token.

Changing `AddJwtBearer`

Do not change issuer, audience, signing key, or token validation today. Swagger configuration is separate from backend authentication configuration.

Applying Security to Every Endpoint

Do not add a global security requirement that marks registration and login as protected. The operation filter keeps the Swagger contract accurate.

Interview Explanation

> I configured Swagger with an HTTP Bearer security scheme so authenticated endpoints can be tested using a JWT. I also added an operation filter that reads `[Authorize]` and `[AllowAnonymous]` attributes, which keeps public endpoints unlocked and marks only protected operations as requiring authentication. Swagger sends the token, while ASP.NET Core remains responsible for validating it.

End-of-Day Progress Note

```
1. What I built
I configured JWT Bearer authorization in Swagger and added an operation filter that marks only
protected endpoints as requiring authentication.

2. What worked
Swagger accepts the login token through its Authorize button, protected requests include the
Bearer header, authenticated task requests succeed, and requests without the token still return
401 Unauthorized.

3. What I will do tomorrow
Review and update the README setup section with database, authentication testing, and automated
test commands.
```

Day 108 - Update the README Setup Guide

Today's Goal

Review and update the repository README so a developer can correctly:

1. Configure SQL Server.
2. Apply EF Core migrations.
3. Configure the local JWT key.
4. Build and test the complete solution.
5. Log in and test protected endpoints through Swagger.

Today is a documentation day. Do not change application behavior.

File to Change

```
[repository root]\README.md
```

The current README already contains useful setup instructions, so do not rewrite the entire file. Update only information that is missing or outdated.

1. Update Current Features

In the `Current Features` list, confirm these features are included:

```
- Task pagination with page metadata
- Task sorting by creation date and due date
- Authenticated user profile retrieval and update
- Authenticated password changes
- XML endpoint descriptions in Swagger
- Swagger JWT Bearer authorization support
```

Do not duplicate a bullet if the same feature is already described clearly.

2. Update the Authentication Endpoints

Find the `### Authentication` section and replace its endpoint block with:

POST /api/auth/register POST /api/auth/login PUT /api/auth/change-password

Add this short explanation below it:

```
Registration and login are public. Changing a password requires a valid Bearer token for the
authenticated user.
```

3. Update the Task Query Examples

The task endpoint section should show the current filtering, sorting, and pagination contract.

Use this endpoint block:

GET /api/tasks GET /api/tasks?isCompleted=true GET /api/tasks?search=resume GET /api/tasks?sortBy=createdAt&sortDirection=desc GET /api/tasks?sortBy=dueDate&sortDirection=asc GET /api/tasks?pageNumber=1&pageSize=10 GET /api/tasks?isCompleted=false&search=resume&sortBy=dueDa

te&sortDirection=asc&pageNumber=1&pageSize=10 POST /api/tasks GET /api/tasks/{id} PUT /api/tasks/{id} DELETE /api/tasks/{id} PATCH /api/tasks/{id}/complete PATCH /api/tasks/{id}/reopen

You do not need to list every possible query combination. These examples explain the available behavior without making the README noisy.

4. Add the Current User Endpoints

At the beginning of the `### Users` endpoint block, add:

```
GET /api/users/me
PUT /api/users/me
```

Add this explanation below the block:

```
The `/api/users/me` endpoints require a valid Bearer token and use the authenticated user id
from JWT claims. The remaining user-id-based CRUD routes are legacy endpoints and are not yet
authorization-protected.
```

5. Update Authentication Behavior

Update the `Authentication Behavior` list so it includes the current behavior:

```
- Registration rejects an email that is already registered.
- Passwords are hashed before they are stored.
- Authentication responses do not expose passwords or password hashes.
- Registration currently returns an empty token.
- Successful login returns a JWT token.
- Invalid login credentials return a safe authentication error.
- Protected endpoints require a valid `Authorization: Bearer TOKEN` header.
- An authenticated user can change their password after providing the correct current password.
- Password confirmation must match, and the new password must differ from the current password.
- Existing JWTs remain valid until they expire after a password change; token revocation is not
implemented yet.
```

The last item is important because documentation should state current security limitations honestly.

6. Update Authorization Limitations

Replace the current `Current Authorization Limitations` list with:

```
- Legacy user CRUD endpoints are not yet protected by authorization.
- `GET /api/users/{userId}/tasks` accepts a user id from the route and is not yet ownership-
protected.
- Existing JWTs are not revoked after a password change and remain valid until expiration.
```

Keep this sentence below the list:

```
These limitations are planned future improvements and should not be treated as completed
security behavior.
```

This does not fix the limitations. It prevents the README from claiming stronger security than the API currently provides.

7. Review the Database Setup

The README already contains the correct local SQL Server connection string:

```
Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;TrustServerCertificate=True
```

Confirm that the database section still explains:

- SQL Server must be running.
- The project uses Windows authentication locally.
- Database passwords must not be committed.

- The connection string is read from `appsettings.Development.json`.
- EF Core migrations are stored in the Infrastructure project.

Confirm this migration command remains in the README:

```
dotnet ef database update --project src\JobTrackr.Infrastructure --startup-project src\
JobTrackr.Api
```

No new migration is required today.

8. Simplify Restore, Build, and Test Commands

The repository uses `JobTrackr.slnx`, so the README should use the real solution filename.

Restore Dependencies

Replace the two project-specific restore commands in `Restore Dependencies` with:

```
cd C:\path\to\JobTrackr
dotnet restore JobTrackr.slnx
```

Keep the explanation that the placeholder path must be replaced with the developer's actual repository location.

Build and Test

Replace the current local build and test commands with:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Using the solution verifies all five projects together.

Expected test result:

```
Passed: 34
Failed: 0
Skipped: 0
```

9. Update the Automated Tests Section

Replace:

```
JobTrackr currently has 15 xUnit service tests.
```

with:

```
JobTrackr currently has 34 xUnit service tests.
```

Update the coverage list to:

```
- password hashing and verification
- registration and login behavior
- valid and invalid password changes
- task creation, retrieval, update, and deletion
- task ownership authorization
- task completion and reopening
- task pagination and page metadata
- created-date and due-date sorting
- combined ownership, filtering, search, sorting, and pagination
- user creation and retrieval
```

Replace the `Run all tests` commands with:

```
dotnet restore JobTrackr.slnx
dotnet build JobTrackr.slnx --configuration Release --no-restore
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

Update its expected result from 15 to 34.

10. Replace the Swagger Authentication Instructions

The README currently says Swagger does not provide a Bearer-token Authorize button. That became outdated on Day 107.

In `### Test Authentication`, replace the old Swagger statement and PowerShell-only instructions with:

```
1. Call `POST /api/auth/register` to create a user if needed.
2. Call `POST /api/auth/login` with the registered email and password.
3. Copy only the JWT value returned in the `token` property.
4. Select **Authorize** at the top of Swagger UI.
5. Paste only the JWT token; Swagger adds the `Bearer` prefix automatically.
6. Select **Authorize** and close the dialog.
7. Call `GET /api/tasks` or another protected endpoint.

With a valid token, the request succeeds for the authenticated user. After using **Logout** in
the Authorize dialog, protected requests return `401 Unauthorized`.
```

Keep the PowerShell example as an optional alternative:

```
$token = "PASTE_LOGIN_TOKEN_HERE"
Invoke-RestMethod `
    -Uri "https://localhost:7024/api/tasks" `
    -Headers @{ Authorization = "Bearer $token" }
Remove-Variable token
```

Never place a real token in the README.

11. Update Next Steps

Remove this item from `Next Steps` because it is now complete:

```
- Swagger Bearer-token configuration
```

Add the next planned improvement:

```
- optional development-only seed data
```

Keep stronger user authorization, broader tests, health checks, and deployment work in the future list.

12. Review the Final Diff

Run:

```
git diff -- README.md
git diff --check
```

Confirm:

- The README says 34 tests everywhere.
- `JobTrackr.slnx` is used for solution commands.
- Swagger authentication instructions use the Authorize button.
- Database setup still uses SQL Server and EF Core migrations.
- No real JWT, password, token, or secret was added.
- Swagger Bearer configuration is no longer listed as future work.
- Application source files were not changed.

You can search for stale statements with:

```
Select-String -Path .\README.md -Pattern "15|does not yet provide|Swagger Bearer-token
configuration|JobTrackr.sln"
```

The search should not find stale setup claims. `JobTrackr.slnx` may appear because the pattern `JobTrackr.sln` is also part of that filename, so inspect any matching solution lines instead of deleting them.

13. Verify the Documented Commands

Run the same commands documented in the README:

```
dotnet restore JobTrackr.slnx
dotnet build JobTrackr.slnx --configuration Release --no-restore
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
```

What the Changed File Does

`README.md`

Explains the project's current capabilities and gives another developer the commands needed to configure SQL Server, create the database, configure JWT secrets, run the API, authenticate through Swagger, and execute all tests.

Interview Explanation

> I reviewed the README against the actual API and corrected outdated setup information. The documentation now uses the real solution file, explains SQL Server migrations and secret configuration, documents Swagger JWT testing, lists current endpoints and authorization limitations, and provides verified commands for all 34 automated tests.

End-of-Day Progress Note

```
1. What I updated
I updated the README with current endpoints, SQL Server setup, JWT authentication testing
through Swagger, solution-level build and test commands, and the current automated test
coverage.

2. What worked
The README now uses JobTrackr.slnx, documents all 34 tests, explains the Swagger Authorize flow,
 preserves safe database and User Secrets guidance, and matches the current API behavior and
limitations.

3. What I will do tomorrow
Add optional development-only seed data without affecting production environments.
```

Day 109 - Add Optional Development Seed Data

Today's Goal

Add optional sample data that makes local API testing faster.

The seed process must:

- Run only in the `Development` environment.
- Run only when `SeedData:Enabled` is true.
- Hash the sample user's password.
- Avoid creating duplicate data after repeated API restarts.
- Never run migrations or reset the database automatically.

What Is Seed Data?

Seed data is predictable sample data inserted into a development database.

Instead of manually registering a user and creating several tasks every time you need useful local data, the application can create them for you when development seeding is enabled.

Seed data is not a replacement for migrations. Migrations create the database structure; seeding adds sample rows after that structure already exists.

Files to Change

```
src/JobTrackr.Infrastructure/Persistence/DevelopmentDataSeeder.cs
src/JobTrackr.Api/appsettings.Development.json
src/JobTrackr.Api/Program.cs
README.md
```

No new NuGet package or EF Core migration is required because no entity or database column is changing.

1. Create the Development Seeder

Run from the solution folder:

```
cd [repository root]
New-Item -ItemType File -Path .\src\JobTrackr.Infrastructure\Persistence\DevelopmentDataSeeder.
cs
```

Open the new file and add:

```
using JobTrackr.Application.Auth;
using JobTrackr.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace JobTrackr.Infrastructure.Persistence;

public static class DevelopmentDataSeeder
{
    private const string SeedEmail = "developer@jobtrackr.local";
    private const string SeedPassword = "Development123!";

    public static async Task SeedAsync(
        AppDbContext dbContext,
        IPasswordHasherService passwordHasherService)
    {
```

```
        var seedUserExists = await dbContext.Users
            .AnyAsync(user => user.Email == SeedEmail);

        if (seedUserExists)
        {
            return;
        }

        var now = DateTime.UtcNow;

        var user = new User
        {
            FullName = "Development User",
            Email = SeedEmail,
            PasswordHash = passwordHasherService.HashPassword(SeedPassword),
            CreatedAtUtc = now,
            Tasks =
            [
                new JobTask
                {
                    Title = "Review JobTrackr API",
                    Description = "Test filtering, sorting, and pagination.",
                    Priority = "High",
                    DueDateUtc = now.AddDays(3),
                    CreatedAtUtc = now.AddMinutes(-2)
                },
                new JobTask
                {
                    Title = "Update backend notes",
                    Description = "Record the latest development progress.",
                    Priority = "Medium",
                    DueDateUtc = now.AddDays(7),
                    CreatedAtUtc = now.AddMinutes(-1)
                },
                new JobTask
                {
                    Title = "Completed sample task",
                    Description = "Use this task to test completion filtering.",
                    IsCompleted = true,
                    Priority = "Low",
                    CreatedAtUtc = now
                }
            ]
        };

        dbContext.Users.Add(user);
        await dbContext.SaveChangesAsync();
    }
}
```

How the Seeder Works

Fixed Development Identity

```
private const string SeedEmail = "developer@jobtrackr.local";
```

The dedicated email identifies rows owned by this seeder. Do not use a real person's email address.

Password Hashing

```
PasswordHash = passwordHasherService.HashPassword(SeedPassword)
```

The database receives a password hash, not the plain development password. The visible password is only a known local demo credential and must never be reused for a real account.

Duplicate Protection

```
var seedUserExists = await dbContext.Users
    .AnyAsync(user => user.Email == SeedEmail);

if (seedUserExists)
{
    return;
}
```

The first run creates the user and tasks. Later runs find the same seed email and stop without adding duplicates.

This is called idempotent behavior: running the operation again does not keep changing the result.

Related Tasks

The tasks are assigned through the user's `Tasks` navigation property. EF Core inserts the user first and automatically uses the new user id as each task's `UserId`.

The three examples support useful testing:

- Different priorities
- Completed and incomplete tasks
- Tasks with and without due dates
- Different creation times for sorting

2. Add the Disabled-by-Default Setting

Open:

```
[repository root]\src\JobTrackr.Api\appsettings.Development.json
```

Replace its contents with:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  },
  "SeedData": {
    "Enabled": false
  }
}
```

Keeping the tracked value `false` means cloning and starting the API does not silently insert sample records.

Do not place this setting in `appsettings.json`. The setting belongs to local development behavior.

3. Call the Seeder from `Program.cs`

The required namespaces are already imported in `Program.cs`:

```
using JobTrackr.Application.Auth;
using JobTrackr.Infrastructure.Persistence;
```

After:

```
var app = builder.Build();
```

add:

```
if (app.Environment.IsDevelopment()
    && app.Configuration.GetValue<bool>("SeedData:Enabled"))
{
    using var scope = app.Services.CreateScope();

    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    var passwordHasherService = scope.ServiceProvider
        .GetRequiredService<IPasswordHasherService>();

    await DevelopmentDataSeeder.SeedAsync(
        dbContext,
        passwordHasherService);
}
```

Keep the existing middleware and Swagger code below this block unchanged.

Why There Are Two Safety Checks

```
app.Environment.IsDevelopment()
```

This prevents the seeder from running in Production, even if somebody accidentally provides the setting there.

```
app.Configuration.GetValue<bool>("SeedData:Enabled")
```

This makes seeding optional even in Development.

Both conditions must be true.

Why We Create a Scope

`AppDbContext` and `IPasswordHasherService` are registered as scoped services.

```
using var scope = app.Services.CreateScope();
```

The scope gives those services a correct lifetime during application startup and disposes the database context afterward.

Why We Do Not Run Migrations Here

Do not add either of these calls today:

```
await dbContext.Database.MigrateAsync();
dbContext.Database.EnsureCreated();
```

JobTrackr already uses explicit EF Core migrations. Developers should apply migrations deliberately with `dotnet ef database update`. Startup seeding should not unexpectedly change database structure.

4. Document the Development Seed Data

In the README `Current Features` list, add:

```
- Optional development-only seed data
```

In `Local Environment Setup`, add this section after `Configure The JWT Signing Key`:

```
### Optional Development Seed Data

Development seeding is disabled by default. To enable it locally without changing tracked
configuration:
```

dotnet user-secrets set "SeedData:Enabled" "true" --project src\JobTrackr.Api dotnet run --project src\JobTrackr.Api --launch-profile https

```
When the API starts in Development, it creates one sample user and three sample tasks if the
seed user does not already exist.
```

Email: developer@jobtrackr.local Password: Development123!

```
These credentials are for local development only. Do not reuse them for a real account or
production environment.

Disable the local override with:
```

dotnet user-secrets remove "SeedData:Enabled" --project src\JobTrackr.Api

Remove `optional development-only seed data` from the README `Next Steps` list after the feature is complete.

5. Apply Migrations Before Testing

Make sure SQL Server is running, then apply existing migrations:

```
dotnet ef database update `
    --project src\JobTrackr.Infrastructure `
    --startup-project src\JobTrackr.Api
```

The seeder expects the database schema to already exist.

6. Enable Seeding Locally

Keep the tracked `appsettings.Development.json` value set to `false`.

Enable a local override through User Secrets:

```
dotnet user-secrets set "SeedData:Enabled" "true" `
    --project src\JobTrackr.Api
```

Although this setting is not sensitive, User Secrets is useful here because it provides an untracked local override.

7. Run and Verify

Start the API:

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

Confirm the following in SQL Server:

```
Users:
- One row with Email = developer@jobtrackr.local

Tasks:
- Three rows belonging to the development user
```

Stop and start the API again. The counts should remain one seed user and three seed tasks.

8. Test the Seeded Login

In Swagger, call:

```
POST /api/auth/login
```

Use:

```
{
  "email": "developer@jobtrackr.local",
  "password": "Development123!"
}
```

Copy the returned token, use Swagger's Authorize button, and call:

```
GET /api/tasks?pageNumber=1&pageSize=10
```

Expected result:

- The login succeeds.
- The response contains the three sample tasks.
- The paged response reports `totalCount` as 3.

9. Disable the Local Override

After testing, remove the local setting:

```
dotnet user-secrets remove "SeedData:Enabled" `
    --project src\JobTrackr.Api
```

The seeded database rows remain until you delete them manually. Removing the setting only prevents the seeder from running during future startups.

10. Build and Test

Stop the running API, then run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
```

The automated tests continue using EF Core InMemory and do not run startup seeding.

11. Final Safety Check

Run:

```
git diff --check
git status
```

Confirm:

- `SeedData:Enabled` is `false` in tracked configuration.
- The seed call checks `app.Environment.IsDevelopment()`.
- The sample password is hashed before database storage.
- The seeder checks the dedicated email before insertion.
- No migration was created.
- No real password, JWT, connection password, or production secret was added.

What Each Changed File Does

`DevelopmentDataSeeder.cs`

Creates one known local user and three related tasks. It hashes the password and returns without inserting anything when the seed user already exists.

`appsettings.Development.json`

Defines the development-only seed switch and keeps it disabled by default in tracked configuration.

`Program.cs`

Runs the seeder during startup only when the environment is Development and the seed switch is enabled. It creates a dependency injection scope for the database context and password hasher.

`README.md`

Explains how to enable, test, and disable local seed data and clearly identifies the sample credentials as development-only.

Interview Explanation

> I added idempotent development seed data behind both an environment check and a disabled-by-default configuration flag. The seeder uses the existing password hashing service, creates related entities through EF Core, and checks a dedicated seed identity before inserting. It does not run migrations or execute in production.

End-of-Day Progress Note

1. What I built
I added optional development-only seed data with one sample user and three related tasks, protected by environment and configuration checks.

2. What worked
When enabled locally, the API creates hashed-password sample data once, repeated startups do not create duplicates, disabled or non-development environments do not run the seeder, and all 34 tests pass.

3. What I will do tomorrow
Review EF Core migration names and confirm the database can be recreated from migrations.

Day 110 - Review EF Core Migrations

Today's Goal

Review the existing EF Core migration history and confirm that JobTrackr can create a fresh SQL Server database from migrations.

Today you will:

1. Review migration names and order.
2. Confirm the model snapshot matches the current entities.
3. Apply all migrations to a separate temporary database.
4. Verify the recreated schema and migration history.
5. Avoid changing or deleting the real `JobTrackrDb` database.

Important Safety Rule

Do not drop, reset, rename, or recreate `JobTrackrDb` today.

Use this separate database only:

```
JobTrackrMigrationReviewDb
```

The different name makes the recreation test independent from your normal development data.

Expected Code Changes

No application code or migration file should change today.

This is a migration verification day. Do not create an empty migration just to produce a commit.

1. Understand the Migration Files

The migration folder is:

```
[repository root]\src\JobTrackr.Infrastructure\Persistence\Migrations
```

JobTrackr currently has these five migrations:

Order	Migration	Purpose
1	`20260607060440_InitialCreate`	Creates the initial `Users` and `Tasks` tables
2	`20260614072653_AddTaskUserRelationship`	Adds the `UserId` index and task-user foreign key with cascade delete
3	`20260617071002_AddTaskDueDate`	Adds nullable `DueDateUtc` to tasks
4	`20260618083307_AddTaskPriority`	Adds `Priority` to tasks with a default value
5	`20260706153247_AddUserPasswordHash`	Adds `PasswordHash` to users

The timestamp prefix controls migration order. The descriptive suffix explains the schema change.

These names are clear enough and should not be renamed now. Renaming old migrations after they have been applied can break migration-history matching between databases and source code.

2. Understand the Three Migration File Types

Main Migration File

Example:

```
20260617071002_AddTaskDueDate.cs
```

It contains:

- `Up`, which applies the schema change.
- `Down`, which reverses that migration when possible.

Designer File

Example:

```
20260617071002_AddTaskDueDate.Designer.cs
```

EF Core generates this metadata file for the migration. Do not edit it manually.

Model Snapshot

```
AppDbContextModelSnapshot.cs
```

The snapshot represents EF Core's latest known database model. EF compares the current entity model with this snapshot when deciding whether a new migration is needed.

Do not edit the snapshot manually.

3. Disable Development Seeding for This Review

Day 109 allowed a User Secrets override for development seed data. Remove that local override before running migration commands:

```
cd [repository root]
dotnet user-secrets remove "SeedData:Enabled" --project src\JobTrackr.Api
```

The tracked `appsettings.Development.json` value should remain:

```
"SeedData": {
  "Enabled": false
}
```

This keeps migration review separate from sample-data insertion.

4. Build Before Reviewing Migrations

Run:

```
dotnet build JobTrackr.slnx --configuration Release
```

Expected result:

```
Build succeeded with no warnings or errors.
```

5. List the Existing Migrations

Run:

```
dotnet ef migrations list `
    --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj `
    --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

Expected migration order:

```
20260607060440_InitialCreate
20260614072653_AddTaskUserRelationship
20260617071002_AddTaskDueDate
20260618083307_AddTaskPriority
20260706153247_AddUserPasswordHash
```

If the command shows all five in this order, EF Core can discover the migration assembly correctly.

6. Check for Pending Model Changes

Run:

```
dotnet ef migrations has-pending-model-changes `
    --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj `
    --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

Expected result:

```
No changes have been made to the model since the last migration.
```

This means the current entities and `AppDbContext` match `AppDbContextModelSnapshot`.

If EF reports pending changes, stop and review them. Do not immediately create a migration until you understand why the model differs.

7. Create a Temporary Review Database

Make sure SQL Server is running.

Set a PowerShell variable containing a connection string for the temporary database:

```
$reviewConnection = "Server=localhost;Database=JobTrackrMigrationReviewDb;
Trusted_Connection=True;TrustServerCertificate=True"
```

Confirm the variable contains the temporary database name:

```
$reviewConnection
```

You must see:

```
Database=JobTrackrMigrationReviewDb
```

Do not continue if it says `Database=JobTrackrDb`.

8. Apply All Migrations to the Temporary Database

Run:

```
dotnet ef database update `
    --connection "$reviewConnection" `
    --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj `
    --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

Because `JobTrackrMigrationReviewDb` does not exist yet, SQL Server creates it and EF Core applies all five migrations in order.

Expected final message:

```
Done.
```

This command does not change the connection string stored in `appsettings.Development.json`.

9. Verify the Recreated Database in SQL Server

Refresh the Databases list in SQL Server Management Studio and open:

```
JobTrackrMigrationReviewDb
```

Confirm these tables exist:

```
dbo.Users
dbo.Tasks
dbo.__EFMigrationsHistory
```

Verify Migration History

Run this read-only query in SSMS:

```
SELECT MigrationId, ProductVersion
FROM JobTrackrMigrationReviewDb.dbo.__EFMigrationsHistory
ORDER BY MigrationId;
```

Expected result: five rows, one for each migration.

Verify Users Columns

Expand `dbo.Users > Columns` and confirm:

```
Id
FullName
Email
CreatedAtUtc
PasswordHash
```

Verify Tasks Columns

Expand `dbo.Tasks > Columns` and confirm:

```
Id
Title
Description
IsCompleted
CreatedAtUtc
UserId
DueDateUtc
Priority
```

Verify the Relationship

Under `dbo.Tasks`, confirm:

```
Index: IX_Tasks_UserId
Foreign key: FK_Tasks_Users_UserId
```

The foreign key should reference `Users.Id` and use cascade delete.

10. Check the Database Is Fresh

Run these read-only queries:

```
SELECT COUNT(*) AS UserCount
FROM JobTrackrMigrationReviewDb.dbo.Users;

SELECT COUNT(*) AS TaskCount
FROM JobTrackrMigrationReviewDb.dbo.Tasks;
```

Expected result:

```
UserCount: 0
TaskCount: 0
```

The schema exists, but no seed data was inserted because development seeding was disabled.

11. What This Test Proves

The successful temporary database creation proves:

- The migration files can run from the beginning.
- Migration ordering is valid.
- The snapshot matches the current EF Core model.
- The final tables contain all expected columns.
- The task-user foreign key and index are created.
- A new developer or deployment can build the schema without an existing database copy.

It does not prove that production deployment is ready. Production still needs environment-specific connection strings, backups, permissions, and a migration deployment strategy.

12. Clean Up Safely

Close any query window connected to `JobTrackrMigrationReviewDb`.

In SQL Server Management Studio:

1. Confirm the selected database is exactly `JobTrackrMigrationReviewDb`.
2. Confirm it is not `JobTrackrDb`.
3. Right-click `JobTrackrMigrationReviewDb`.
4. Select Delete.
5. Enable closing existing connections only if SQL Server reports an active connection.
6. Confirm deletion.

Do not delete the normal `JobTrackrDb` database.

After cleanup, remove the PowerShell variable:

```
Remove-Variable reviewConnection
```

Day 111 will document reusable local database reset instructions with additional safety checks.

13. Run Automated Tests

Run:

```
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
All 34 tests pass.
```

The tests use EF Core InMemory. The fresh SQL Server recreation is a separate manual migration verification.

14. Check Git Status

Run:

```
git status
```

Expected result:

```
nothing to commit, working tree clean
```

EF migration review should not modify migration files, the snapshot, entities, or configuration.

Do not create an empty commit when no repository files changed.

Migration Review Checklist

- [ ] All five migrations have clear names.
- [ ] Migration order is correct.
- [ ] EF reports no pending model changes.
- [ ] The Release build succeeds.
- [ ] `JobTrackrMigrationReviewDb` is created successfully.
- [ ] `Users`, `Tasks`, and `__EFMigrationsHistory` exist.
- [ ] The migration history contains five rows.
- [ ] Current columns, index, and foreign key exist.
- [ ] The temporary tables contain zero rows.
- [ ] The real `JobTrackrDb` was not changed.
- [ ] The temporary database was removed after verification.
- [ ] All 34 tests pass.
- [ ] Git remains clean.

Interview Explanation

> I reviewed the ordered EF Core migration chain, verified that the current model has no pending changes, and applied every migration to a separate empty SQL Server database. I then checked the migration history, final tables, columns, index, and foreign key. This confirmed that the database schema can be recreated from source without touching the normal development database.

End-of-Day Progress Note

```
1. What I reviewed
I reviewed all five EF Core migrations, their names and order, and the current model snapshot.

2. What worked
EF Core reported no pending model changes, all migrations created a fresh temporary SQL Server
database successfully, the final schema and migration history were correct, and all 34 tests
passed.

3. What I will do tomorrow
Document safe local database reset commands and precautions.
```

Day 111 - Document a Safe Local Database Reset

Today's Goal

Add clear README instructions for resetting the local JobTrackr development database safely.

The documented process will:

1. Warn that all local rows will be deleted.
2. Stop the API before resetting.
3. Disable optional seed data.
4. Verify the tracked connection points to local `JobTrackrDb`.
5. Preview the EF Core drop target and answer `N` to cancel.
6. Rerun the interactive command and answer `Y` only for the confirmed local database.
7. Recreate the schema from all migrations.
8. Verify the empty database and run all tests.

Today is a documentation task. Do not create a new migration or application service.

File to Change

```
[repository root]\README.md
```

Important Warning

A database reset permanently deletes local users, tasks, password hashes, and migration-history rows before recreating the schema.

Use these instructions only for the local development database:

```
Server: localhost
Database: JobTrackrDb
```

Never use them with a shared, staging, testing-team, or production database.

Why a Reset Is Sometimes Useful

A local reset can help when:

- You want a clean database for manual testing.
- Local sample data is no longer useful.
- You want to prove migrations can recreate the current schema.
- Your disposable development database has become inconsistent.

A reset should not be the first response to every database error. Read the error and understand the cause before deleting data.

1. Add the README Section

In `README.md`, add the following section after `Optional Development Seed Data` and before `Trust The Development HTTPS Certificate`.

Use this content:

```
### Reset The Local Development Database

> **Warning:** This permanently deletes every user and task in the local `JobTrackrDb` database.
 Use it only for disposable local development data. Never run these commands against a shared
or production database.

Stop the running API before resetting the database. Back up any local data you need to keep.

From the repository root, disable optional seed data:
```

cd C:\path\to\JobTrackr dotnet user-secrets remove "SeedData:Enabled" --project .\src\JobTrackr.Api\JobTrackr.Api.csproj

```
Inspect the tracked Development connection string:
```

$settings = Get-Content .\src\JobTrackr.Api\appsettings.Development.json | ConvertFrom-Json $connectionString = $settings.ConnectionStrings.DefaultConnection $connectionString

$isLocalServer = $connectionString -match '(?i)(?:^|;)\s*(?:Server|Data Source)\s*=\s*localhost\s*(?:;|$)' $isJobTrackrDatabase = $connectionString -match '(?i)(?:^|;)\s*(?:Database|Initial Catalog)\s*=\s*JobTrackrDb\s*(?:;|$)'

if (!$isLocalServer -or !$isJobTrackrDatabase) { throw "Reset stopped: expected the local JobTrackrDb connection string." }

```
The displayed connection string must contain both:
```

Server=localhost Database=JobTrackrDb

```
Preview the database deletion using EF Core's interactive confirmation:
```

dotnet ef database drop --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj

```
EF Core displays the target and asks for confirmation. Confirm that it identifies `JobTrackrDb`
on `localhost`, then type `N` and press Enter to cancel the preview safely.

When you intentionally want to reset the confirmed local database, run the same command again:
```

dotnet ef database drop --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj

```
Type `Y` only when the prompt names `JobTrackrDb` on `localhost`. Otherwise, type `N`.

Recreate the empty database by applying all migrations:
```

dotnet ef database update --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj

```
Refresh SQL Server Management Studio and confirm `JobTrackrDb` contains:
```

dbo.Users dbo.Tasks dbo.__EFMigrationsHistory

```
The `Users` and `Tasks` tables should be empty. The migration-history table should contain all
five migrations.

Run the Release build and automated tests:
```

dotnet build JobTrackr.slnx --configuration Release dotnet test JobTrackr.slnx --configuration Release --no-build

```
Development seeding remains disabled unless it is explicitly enabled again through User Secrets.
```

Replace `C:\path\to\JobTrackr` in the actual README example with a generic placeholder, as shown. Do not put a machine-specific user folder into repository documentation.

2. Understand the Connection Check

```
$settings = Get-Content .\src\JobTrackr.Api\appsettings.Development.json | ConvertFrom-Json
```

This reads the JSON configuration as structured PowerShell data.

```
$connectionString = $settings.ConnectionStrings.DefaultConnection
```

This selects the exact Development connection string used by JobTrackr.

The two regular-expression checks require:

- `Server=localhost` or `Data Source=localhost`
- `Database=JobTrackrDb` or `Initial Catalog=JobTrackrDb`

If either value differs, the script throws an error before any destructive EF command is shown as safe to continue.

The check is a guard, not permission to ignore EF Core's confirmation prompt. You must still read the displayed target.

3. Preview with Interactive Confirmation

```
dotnet ef database drop --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.
csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

EF Core reports which database it is about to delete and waits for your answer.

Expected preview should identify:

```
JobTrackrDb
localhost
```

Stop immediately if it identifies any other server or database.

For the first check, type `N` and press Enter. EF Core cancels the deletion, so you can verify the target without changing the database.

4. Understand the Destructive Command

```
dotnet ef database drop --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.
csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

- `database drop` deletes the selected database.
- EF Core requires an interactive `Y` or `N` answer.
- Type `Y` only after the connection check and target preview are correct.

Do not automate or bypass this confirmation in the local reset instructions.

5. Understand Database Recreation

```
dotnet ef database update --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.
csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

When the database does not exist, EF Core:

1. Creates `JobTrackrDb`.
2. Creates `__EFMigrationsHistory`.

3. Applies all five migrations in timestamp order.
4. Produces the current Users and Tasks schema.

This is the same migration chain verified against the temporary database on Day 110.

6. Why Seeding Is Disabled First

Day 109 introduced optional startup seeding.

```
dotnet user-secrets remove "SeedData:Enabled" --project .\src\JobTrackr.Api\JobTrackr.Api.csproj
```

Removing the local override prevents sample rows from appearing unexpectedly when the API starts after the reset.

The tracked value remains `false`, so a freshly recreated database should contain:

```
Users: 0 rows
Tasks: 0 rows
```

Seeding can be enabled deliberately later by following the README's development seed-data section.

7. Manual Verification Queries

After recreation, these read-only SQL queries can verify the result:

```
SELECT MigrationId, ProductVersion
FROM JobTrackrDb.dbo.__EFMigrationsHistory
ORDER BY MigrationId;

SELECT COUNT(*) AS UserCount
FROM JobTrackrDb.dbo.Users;

SELECT COUNT(*) AS TaskCount
FROM JobTrackrDb.dbo.Tasks;
```

Expected result:

```
Migration rows: 5
UserCount: 0
TaskCount: 0
```

8. Cases Where You Must Not Reset

Do not reset when:

- The connection points anywhere except local `JobTrackrDb`.
- Another developer uses the database.
- The database contains information you need and have not backed up.
- You are investigating a migration failure and need the current state as evidence.
- The API or another process is actively using the database.
- You do not understand which configuration environment EF Core loaded.

Ask for help or inspect configuration before continuing when the target is unclear.

9. Review the README Diff

Run:

```
git diff -- README.md
git diff --check
```

Confirm the new documentation:

- Clearly says that data will be deleted.
- Names only local `JobTrackrDb` as the allowed target.
- Requires the API to be stopped.
- Disables seed data first.
- Checks both local server and database name.
- Requires an interactive target preview and cancellation before deletion.
- Uses explicit `.csproj` paths.
- Reapplies migrations after deletion.
- Contains no password, JWT, or production connection string.

10. Verify Non-Destructive Commands Today

The README is the feature today. You do not need to delete your current database merely to edit documentation.

You can safely verify:

```
dotnet build JobTrackr.slnx --configuration Release

dotnet ef database drop --project .\src\JobTrackr.Infrastructure\JobTrackr.Infrastructure.
csproj --startup-project .\src\JobTrackr.Api\JobTrackr.Api.csproj

dotnet test JobTrackr.slnx --configuration Release --no-build
```

Read the displayed target carefully and type `N` to cancel this non-destructive check. Type `Y` only when you intentionally want to erase the confirmed local development database.

What the Changed File Does

`README.md`

Documents a repeatable local database reset with explicit warnings, connection checks, interactive confirmation, migration-based recreation, schema verification, and optional seed-data behavior.

Interview Explanation

> I documented a safe local database reset process using EF Core migrations. The procedure checks the exact local connection, previews the drop target, requires an explicit destructive step, and recreates the schema from migration history. It also disables optional seeding first and clearly prohibits using the procedure against shared or production databases.

End-of-Day Progress Note

```
1. What I documented
I added safe local database reset instructions with connection checks, interactive EF Core
confirmation, migration-based recreation, and verification steps.

2. What worked
The documented process identifies local JobTrackrDb before deletion, keeps seed data disabled,
recreates the schema from all five migrations, and preserves clear warnings against shared or
production use.

3. What I will do tomorrow
Plan the first API integration testing approach without overbuilding the test infrastructure.
```

Day 112 - Plan API Integration Testing

Today's Goal

Plan the first JobTrackr API integration tests before adding test infrastructure.

Today you will decide:

- What an integration test should cover.
- Which test tools to use.
- How the test API will start.
- How to avoid using the real SQL Server database.
- Which endpoints to test first.
- What should not be built yet.

This is a planning day. Do not install packages or change application code today.

Current Test Position

JobTrackr currently has 34 automated tests.

Those tests mainly create services directly and use EF Core InMemory. They are useful unit/service tests because they verify business behavior such as:

- Password hashing and authentication
- Task CRUD logic
- Task ownership
- Filtering, sorting, and pagination
- Password changes
- User creation and retrieval

They do not currently send real HTTP requests through the complete ASP.NET Core pipeline.

Unit Test vs Integration Test

Current Service Test

A service test calls a class directly:

```
Test -> TaskService -> EF Core InMemory
```

It checks application behavior quickly without starting the API.

Planned Integration Test

An API integration test sends an HTTP request:

```
HttpClient
    -> ASP.NET Core middleware
    -> authentication and authorization
    -> controller
    -> service
    -> EF Core InMemory
    -> HTTP response
```

This verifies that multiple parts work together correctly.

Main Testing Decision

Use:

```
xUnit
Microsoft.AspNetCore.Mvc.Testing
WebApplicationFactory<Program>
EF Core InMemory
```

Keep integration tests inside the existing project:

```
tests/JobTrackr.Tests
```

Create an `Integration` folder instead of creating another test project now.

Why This Approach Fits JobTrackr

- The existing xUnit project and test command remain unchanged.
- `WebApplicationFactory` starts JobTrackr in memory without opening a real network port.
- `HttpClient` sends realistic API requests.
- Controllers, middleware, routing, JSON serialization, validation, services, and JWT behavior can be tested together.
- EF Core InMemory keeps tests independent from local SQL Server and suitable for GitHub Actions.
- The setup is small enough for the current learning stage.

Planned File Structure

Day 113 will begin with this structure:

```
tests/JobTrackr.Tests/
  Integration/
    JobTrackrApiFactory.cs
    HealthEndpointTests.cs
```

The existing service tests stay in their current folders.

Later days can add:

```
Integration/
  AuthEndpointTests.cs
  TaskEndpointTests.cs
```

Do not create all future files today.

Required Setup for Day 113

1. Add the ASP.NET Core Testing Package

Planned package:

```
Microsoft.AspNetCore.Mvc.Testing 8.0.27
```

The major version must match the API's .NET 8 and ASP.NET Core package family.

Planned command:

```
dotnet add .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj package Microsoft.AspNetCore.Mvc.
Testing --version 8.0.27
```

Do not run it today. It belongs to Day 113 implementation.

2. Reference the API Project

The test project currently references Application and Infrastructure. Integration tests will also need a reference to the API project so they can use its generated `Program` type.

Planned command:

```
dotnet add .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj reference .\src\JobTrackr.Api\
JobTrackr.Api.csproj
```

Do not run it today.

3. Make Top-Level `Program` Accessible

`Program.cs` uses top-level statements. The compiler generates a `Program` class, but integration tests need public access to it.

Day 113 will add this after `app.Run();`:

```
public partial class Program
{
}
```

This does not change how the API runs. It only makes the entry point available to `WebApplicationFactory<Program>`.

Planned Test Factory Responsibilities

`JobTrackrApiFactory` will inherit from:

```
WebApplicationFactory<Program>
```

It will have four focused responsibilities.

1. Use the Testing Environment

```
ASPNETCORE_ENVIRONMENT = Testing
```

This prevents Development-only Swagger and seed-data startup behavior from running during tests.

2. Supply Test-Only Configuration

The API needs a JWT signing key during startup. The factory will provide a fixed test-only key through in-memory configuration.

It will not read or expose the developer's User Secrets.

The test key is not a production secret and must never be reused outside automated tests.

3. Replace SQL Server

The factory will remove JobTrackr's SQL Server `AppDbContext` registration and replace it with EF Core InMemory.

The integration tests must never read, create, reset, or delete:

```
JobTrackrDb
```

4. Give Each Factory an Isolated Database

The InMemory database will use a unique name, such as a new GUID.

This prevents data created by one test class from leaking into another test class.

What Remains Real in Integration Tests

The test should use the real:

- ASP.NET Core routing
- Request logging middleware
- Exception middleware
- Model validation
- Authentication and authorization middleware
- Controllers
- Service implementations
- Password hashing
- JWT generation and validation
- JSON request and response serialization
- HTTP status-code handling

Only external infrastructure is replaced:

```
SQL Server -> EF Core InMemory
Local User Secrets -> in-memory test configuration
```

This provides useful integration coverage without requiring a running database server.

First Integration Test

Day 113 will add one small test:

```
GET /health
```

Expected result:

```
Status: 200 OK
Body: Healthy
```

The health endpoint is the correct first test because it:

- Is public.
- Does not require test data.
- Does not require JWT authentication.
- Proves the API can start in the test host.
- Proves routing and the HTTP pipeline work.

Planned Client Configuration

The factory will create an in-memory test server. Tests will use its `HttpClient` rather than `https://localhost:7024`.

The client should use an HTTPS base address:

```
https://localhost
```

This avoids HTTPS-redirection noise while still keeping requests inside the in-memory test server. No local port is opened.

Day-by-Day Integration Test Scope

Day 113

Add the test package, API project reference, test factory, and one health endpoint test.

Day 114

Send real HTTP requests to:

```
POST /api/auth/register
POST /api/auth/login
```

Verify registration succeeds and login returns a non-empty JWT token.

Day 115

Use the real JWT from login to:

```
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
```

Verify an authenticated task can be created, retrieved, and updated.

Day 116

Delete the authenticated user's task and verify a later GET returns 404.

Day 117

Confirm the integration tests are stable in GitHub Actions before changing CI behavior.

Test Data Rules

Use test-created data, not Development seed data.

Each authentication test should register its own unique email, for example:

```
integration-{unique-value}@example.com
```

This prevents duplicate-email failures when tests share a factory during one test class.

Never depend on:

- An existing SQL Server user
- `developer@jobtrackr.local`
- A token copied from Swagger
- A hard-coded database row id
- The order in which test classes happen to run

Assertion Rules

Integration tests should assert behavior visible to an API client:

- HTTP status code
- Response body
- Important response headers
- Public DTO fields

- Authorization result

Avoid asserting private service fields or EF Core tracking details in an HTTP integration test. Those details belong in service tests.

What We Will Not Build Yet

Do not add these today:

- A separate integration-test project
- Testcontainers
- A real SQL Server test database
- Docker Compose
- Respawn database-reset tooling
- Custom authentication bypasses
- Mock controllers or mock services
- A large shared test framework
- Integration tests for every endpoint at once

These tools may become useful later, but they would add more setup than the current tests require.

Known Limitation of This Plan

EF Core InMemory does not behave exactly like SQL Server. It cannot fully verify SQL constraints, SQL translation, indexes, or provider-specific behavior.

The integration tests will verify the ASP.NET Core application pipeline. Day 110 separately confirmed that the real SQL Server migration chain creates the correct schema.

A real-database testing strategy can be introduced later when its additional cost is justified.

Success Criteria

The integration-test approach is ready when:

- The existing test project remains the single automated test project.
- Tests start the API through `WebApplicationFactory<Program>`.
- The environment is `Testing`, so Development seeding does not run.
- The JWT test key comes from in-memory test configuration.
- SQL Server is replaced with an isolated InMemory database.
- Tests communicate through `HttpClient`.
- The first test checks only `GET /health`.
- Existing 34 tests continue to pass.
- No local database or User Secrets are required in CI.

Today's Verification

Because today changes no application or test code, run the existing suite:

```
cd [repository root]
```

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
git status
```

Expected result:

```
Build succeeded with no warnings or errors.
All 34 tests pass.
Git working tree is clean.
```

Do not create an empty commit for a planning-only day.

Interview Explanation

> I planned API integration tests using xUnit and WebApplicationFactory. The tests will send HTTP requests through the real ASP.NET Core pipeline while replacing SQL Server with an isolated EF Core InMemory database and supplying test-only JWT configuration. I chose the public health endpoint as the first test and deferred heavier infrastructure until the project needs real SQL Server integration coverage.

End-of-Day Progress Note

```
1. What I planned
I planned API integration testing with xUnit, WebApplicationFactory, a Testing environment,
test-only JWT configuration, and an isolated EF Core InMemory database.

2. What worked
The approach keeps the real HTTP pipeline, controllers, services, authentication, and
serialization while avoiding local SQL Server, User Secrets, development seed data, and
unnecessary test infrastructure.

3. What I will do tomorrow
Add the integration-test setup and the first GET /health API integration test.
```

Day 113 - Add the First API Integration Test

Today's Goal

Add the first HTTP integration test for JobTrackr.

The test will start the API in memory and verify:

```
GET /health
```

Expected result:

```
Status: 200 OK
Body: Healthy
```

Keep today's scope small. Do not add authentication or task integration tests yet.

What You Will Change

```
tests/JobTrackr.Tests/JobTrackr.Tests.csproj
src/JobTrackr.Api/Program.cs
tests/JobTrackr.Tests/Integration/JobTrackrApiFactory.cs
tests/JobTrackr.Tests/Integration/HealthEndpointTests.cs
```

1. Install the ASP.NET Core Testing Package

Open PowerShell in the solution folder:

```
cd [repository root]
```

Install the .NET 8 testing package:

```
dotnet add .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj package Microsoft.AspNetCore.Mvc.
Testing --version 8.0.27
```

Why This Package Is Needed

`Microsoft.AspNetCore.Mvc.Testing` provides `WebApplicationFactory<TEntryPoint>`.

The factory starts the real API pipeline inside the test process. It does not open port 7024 and does not require you to run the API separately.

Use version `8.0.27` because JobTrackr targets .NET 8. Do not install version 9 or 10.

2. Reference the API Project

Run:

```
dotnet add .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj reference .\src\JobTrackr.Api\
JobTrackr.Api.csproj
```

The test project needs this reference to access JobTrackr's generated `Program` class and start the API.

After both commands, `JobTrackr.Tests.csproj` should contain the new package:

```
<PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.27" />
```

It should also contain this project reference:

```
<ProjectReference Include="..\..\src\JobTrackr.Api\JobTrackr.Api.csproj" />
```

Keep the existing Application and Infrastructure references.

3. Prepare `Program` for the Test Host

Open:

```
[repository root]\src\JobTrackr.Api\Program.cs
```

Move JWT Configuration Reads into `AddJwtBearer`

Delete these three lines from immediately above `builder.Services.AddAuthentication`:

```
var jwtIssuer = builder.Configuration["Jwt:Issuer"];
var jwtAudience = builder.Configuration["Jwt:Audience"];
var jwtKey = builder.Configuration["Jwt:Key"];
```

Add the same declarations at the beginning of the existing `AddJwtBearer` callback:

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        var jwtIssuer = builder.Configuration["Jwt:Issuer"];
        var jwtAudience = builder.Configuration["Jwt:Audience"];
        var jwtKey = builder.Configuration["Jwt:Key"];

        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtIssuer,
            ValidAudience = jwtAudience,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtKey!))
        };
    });
```

The token-validation properties remain unchanged. Only the location of the three configuration reads moves.

`WebApplicationFactory` adds test configuration after the application builder is created. Reading these values inside the options callback allows the callback to see the test JWT key when authentication is first used. Normal API authentication continues using the same configuration values.

Make `Program` Public

Add this code after `app.Run();` at the end of the file:

```
public partial class Program
{
}
```

The end of the file should look like:

```
app.MapHealthChecks("/health");
app.MapControllers();

app.Run();

public partial class Program
{
}
```

Why `partial` Is Used

Top-level statements cause the compiler to generate a hidden part of the `Program` class.

This public partial declaration adds another part of that same class and makes it accessible to:

```
WebApplicationFactory<Program>
```

It does not change normal API startup behavior.

4. Create the Integration Folder and Files

Run:

```
New-Item -ItemType Directory -Path .\tests\JobTrackr.Tests\Integration -Force
New-Item -ItemType File -Path .\tests\JobTrackr.Tests\Integration\JobTrackrApiFactory.cs
New-Item -ItemType File -Path .\tests\JobTrackr.Tests\Integration\HealthEndpointTests.cs
```

5. Create the Test API Factory

Open:

```
[repository root]\tests\JobTrackr.Tests\Integration\JobTrackrApiFactory.cs
```

Add:

```
using JobTrackr.Infrastructure.Persistence;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;

namespace JobTrackr.Tests.Integration;

public sealed class JobTrackrApiFactory : WebApplicationFactory<Program>
{
    private readonly string _databaseName =
        $"JobTrackrIntegrationTests-{Guid.NewGuid()}";

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");

        builder.ConfigureAppConfiguration((context, configuration) =>
        {
            configuration.AddInMemoryCollection(
                new Dictionary<string, string?>
                {
                    ["ConnectionStrings:DefaultConnection"] =
                        "Server=localhost;Database=JobTrackrIntegrationTests;
Trusted_Connection=True;TrustServerCertificate=True",
                    ["Jwt:Key"] =
                        "JobTrackr-integration-test-signing-key-for-automated-tests-only",
                    ["SeedData:Enabled"] = "false"
                });
        });

        builder.ConfigureServices(services =>
        {
            services.RemoveAll<DbContextOptions<AppDbContext>>();

            services.AddDbContext<AppDbContext>(options =>
                options.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

6. Understand the Test Factory

`WebApplicationFactory<Program>`

```
public sealed class JobTrackrApiFactory : WebApplicationFactory<Program>
```

This tells ASP.NET Core to use JobTrackr's real application entry point when creating the test server.

Testing Environment

```
builder.UseEnvironment("Testing");
```

The API does not run Development-only Swagger or seed-data startup code during integration tests.

Test Configuration

```
configuration.AddInMemoryCollection(...);
```

This supplies configuration only inside the test process.

- The JWT key is a test value, not a production secret.
- The SQL connection string allows normal startup configuration to be created.
- The SQL Server registration is replaced before any test uses the database.
- Seed data remains disabled.

The test does not read your local User Secrets.

Remove SQL Server Registration

```
services.RemoveAll<DbContextOptions<AppDbContext>>();
```

This removes the options that configured `AppDbContext` for SQL Server.

Add EF Core InMemory

```
services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase(_databaseName));
```

The API services now receive an in-memory `AppDbContext` during integration tests.

The unique database name prevents test data from being shared with another factory instance.

7. Add the Health Endpoint Test

Open:

```
[repository root]\tests\JobTrackr.Tests\Integration\HealthEndpointTests.cs
```

Add:

```
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;

namespace JobTrackr.Tests.Integration;

public class HealthEndpointTests : IClassFixture<JobTrackrApiFactory>
{
    private readonly JobTrackrApiFactory _factory;

    public HealthEndpointTests(JobTrackrApiFactory factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task GetHealth_ReturnsOkWithHealthyBody()
    {
        using var client = _factory.CreateClient(
            new WebApplicationFactoryClientOptions
            {
                BaseAddress = new Uri("https://localhost")
            });

        var response = await client.GetAsync("/health");
        var body = await response.Content.ReadAsStringAsync();

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
        Assert.Equal("Healthy", body);
    }
}
```

8. Understand the Health Test

Shared Factory

```
IClassFixture<JobTrackrApiFactory>
```

xUnit creates the factory for the test class and disposes it after the class finishes.

In-Memory HTTP Client

```
using var client = _factory.CreateClient(...);
```

This client talks to the in-memory JobTrackr test server. It does not send requests to a separately running API.

HTTPS Base Address

```
BaseAddress = new Uri("https://localhost")
```

The HTTPS address avoids redirection noise from `UseHttpsRedirection`. It still uses the in-memory test server and does not open a network port.

Real HTTP Request

```
var response = await client.GetAsync("/health");
```

This request passes through ASP.NET Core middleware, routing, and health-check handling.

Client-Visible Assertions

```
Assert.Equal(HttpStatusCode.OK, response.StatusCode);
Assert.Equal("Healthy", body);
```

The test checks exactly what an API client receives: the HTTP status and response body.

9. Run Only the New Test

Run:

```
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~HealthEndpointTests"
```

Expected result:

```
Passed: 1
Failed: 0
```

You do not need to run the API separately.

You do not need SQL Server to be running for this integration test.

10. Run the Full Test Suite

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build succeeded with no warnings or errors.
Passed: 35
Failed: 0
Skipped: 0
```

The count increases from 34 to 35 because the health integration test is new.

11. Check That SQL Server Was Not Used

The test factory uses a unique in-memory database and the health endpoint does not perform a database query.

Confirm:

- The test passes when the API is not running.
- No `JobTrackrIntegrationTests` SQL Server database appears in SSMS.
- `JobTrackrDb` remains unchanged.
- Development seed data does not run.
- No local JWT key is required by the test.

12. Common Errors

`Program is inaccessible due to its protection level`

Confirm this appears after `app.Run();`:

```
public partial class Program
{
}
```

Also confirm the test project references `JobTrackr.Api.csproj`.

Package Is Incompatible with `net8.0`

Confirm the package version is `8.0.27`, not version 9 or 10.

Multiple EF Core Providers Are Registered

Confirm the factory removes:

```
DbContextOptions<AppDbContext>
```

before adding `UseInMemoryDatabase`.

JWT Key Is Missing

Confirm the factory adds a test value for:

```
Jwt:Key
```

Also confirm `Program.cs` reads `Jwt:Issuer`, `Jwt:Audience`, and `Jwt:Key` inside the `AddJwtBearer` callback rather than before `AddAuthentication`.

Do not solve this by adding a JWT key to tracked application settings.

HTTPS Redirection Warning

Confirm the test client uses:

```
https://localhost
```

What Each Changed File Does

`JobTrackr.Tests.csproj`

Adds the ASP.NET Core integration-testing package and references the API project.

`Program.cs`

Reads JWT configuration when authentication options are created and exposes the generated top-level `Program` class to `WebApplicationFactory`, without changing normal API behavior.

`JobTrackrApiFactory.cs`

Starts JobTrackr under the Testing environment, supplies test-only configuration, and replaces SQL Server with an isolated EF Core InMemory database.

`HealthEndpointTests.cs`

Sends a real in-memory HTTP request to `/health` and verifies the client receives `200 OK` with `Healthy`.

Interview Explanation

> I added the first API integration test using WebApplicationFactory. The test starts the real ASP.NET Core pipeline in memory, while a custom factory uses a Testing environment, test-only JWT configuration, and EF Core InMemory instead of SQL Server. The first test verifies the public health endpoint returns 200 OK and the expected response body.

End-of-Day Progress Note

```
1. What I built
I added the API integration-test setup with WebApplicationFactory and created the first HTTP
test for GET /health.

2. What worked
The test starts JobTrackr in memory under the Testing environment, avoids SQL Server and local
secrets, verifies 200 OK with Healthy, and all 35 tests pass.

3. What I will do tomorrow
Add an authentication integration test that registers and logs in through the API and verifies
a JWT token is returned.
```

Day 114 - Add an Authentication Integration Test

Today's Goal

Follow the existing backend roadmap: register and log in through the API, then verify that login returns a token.

Add one test only. Reuse Day 113's factory, test package, and API project reference. No new NuGet package, migration, controller, or service is required.

Files

Create:

```
[repository root]\tests\JobTrackr.Tests\Integration\AuthEndpointTests.cs
```

After the test passes, update the test count and coverage in:

```
[repository root]\README.md
```

Keep `JobTrackrApiFactory.cs` and `Program.cs` unchanged today.

1. Create the File

Run from PowerShell:

```
cd [repository root]
New-Item -ItemType File -Path .\tests\JobTrackr.Tests\Integration\AuthEndpointTests.cs
```

If the file already exists, open it instead of recreating it.

2. Full Test Code

Add this code to `AuthEndpointTests.cs`:

```
using System.Net;
using System.Net.Http.Json;
using JobTrackr.Application.Auth;
using Microsoft.AspNetCore.Mvc.Testing;

namespace JobTrackr.Tests.Integration;

public class AuthEndpointTests : IClassFixture<JobTrackrApiFactory>
{
    private readonly JobTrackrApiFactory _factory;

    public AuthEndpointTests(JobTrackrApiFactory factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task RegisterAndLogin_WithValidCredentials_ReturnsUserAndToken()
    {
        using var client = _factory.CreateClient(
            new WebApplicationFactoryClientOptions
            {
                BaseAddress = new Uri("https://localhost")
            });

        var registerRequest = new RegisterRequest
        {
            FullName = "Authentication Integration User",
```

```
            Email = $"auth-{Guid.NewGuid():N}@example.com",
            Password = "IntegrationTest123!"
        };

        using var registerResponse = await client.PostAsJsonAsync(
            "/api/auth/register",
            registerRequest);

        Assert.Equal(HttpStatusCode.OK, registerResponse.StatusCode);

        var registeredUser = await registerResponse.Content
            .ReadFromJsonAsync<AuthResponse>();

        Assert.NotNull(registeredUser);
        Assert.True(registeredUser.UserId > 0);
        Assert.Equal(registerRequest.FullName, registeredUser.FullName);
        Assert.Equal(registerRequest.Email, registeredUser.Email);
        Assert.Equal(string.Empty, registeredUser.Token);

        var loginRequest = new LoginRequest
        {
            Email = registerRequest.Email,
            Password = registerRequest.Password
        };

        using var loginResponse = await client.PostAsJsonAsync(
            "/api/auth/login",
            loginRequest);

        Assert.Equal(HttpStatusCode.OK, loginResponse.StatusCode);

        var loggedInUser = await loginResponse.Content
            .ReadFromJsonAsync<AuthResponse>();

        Assert.NotNull(loggedInUser);
        Assert.Equal(registeredUser.UserId, loggedInUser.UserId);
        Assert.Equal(registerRequest.FullName, loggedInUser.FullName);
        Assert.Equal(registerRequest.Email, loggedInUser.Email);
        Assert.False(string.IsNullOrWhiteSpace(loggedInUser.Token));
    }
}
```

3. What the File Does

`AuthEndpointTests.cs` sends registration and login requests through the real API pipeline. It verifies the response status codes, user identity, and returned token using the existing isolated test host.

The two requests belong to one test because they form one workflow: the user created by registration must be able to log in.

4. Code Explained Simply

Reuse the Factory

```
public class AuthEndpointTests : IClassFixture<JobTrackrApiFactory>
```

xUnit creates the factory for this test class and disposes it afterward. The factory uses the Testing environment, test configuration, and an isolated EF Core InMemory database.

Create the Client

```
using var client = _factory.CreateClient(
    new WebApplicationFactoryClientOptions
    {
        BaseAddress = new Uri("https://localhost")
    });
```

This client talks to the in-memory API, not an API running on port 7024. HTTPS avoids redirection, and `using` disposes the client afterward.

Use a Unique Email

```
Email = $"auth-{Guid.NewGuid():N}@example.com"
```

The GUID makes the email different for each run. The `:N` format removes GUID hyphens. This avoids duplicate-email failures from reused test data.

The password is fictional test data, not a real account credential.

Send JSON

```
using var registerResponse = await client.PostAsJsonAsync(
    "/api/auth/register",
    registerRequest);
```

`PostAsJsonAsync` converts the request DTO into JSON and sends an HTTP POST with the appropriate content type.

The request goes through routing, validation, the controller, AuthService, and the InMemory database.

Check Status Before Reading the DTO

```
Assert.Equal(HttpStatusCode.OK, registerResponse.StatusCode);
```

Registration currently returns 200 in JobTrackr. Do not change the controller to 201 for this lesson.

Checking the status first means an error response is not mistaken for a successful `AuthResponse`.

Read JSON as a DTO

```
var registeredUser = await registerResponse.Content
    .ReadFromJsonAsync<AuthResponse>();

Assert.NotNull(registeredUser);
```

The client receives JSON. This converts it into an `AuthResponse` so the test can check its properties.

Confirm the Current Registration Contract

```
Assert.True(registeredUser.UserId > 0);
Assert.Equal(registerRequest.FullName, registeredUser.FullName);
Assert.Equal(registerRequest.Email, registeredUser.Email);
Assert.Equal(string.Empty, registeredUser.Token);
```

The user must have a generated id and the expected name/email. Registration currently returns an empty token; only login returns a usable token.

Log In with the Same Credentials

```
var loginRequest = new LoginRequest
{
    Email = registerRequest.Email,
    Password = registerRequest.Password
};
```

This proves a user registered through HTTP can subsequently log in through HTTP. Do not seed a database user directly for this test.

Confirm Identity and Token

```
Assert.Equal(registeredUser.UserId, loggedInUser.UserId);
Assert.False(string.IsNullOrWhiteSpace(loggedInUser.Token));
```

The id proves login returned the same user. The token check rejects null, empty, and whitespace-only values.

This verifies that login returns a token; it does not independently prove its signature, expiry, or acceptance by a protected endpoint. Day 115 will use the returned token for authenticated task requests.

5. Run Only Today's Test

No separate API process or SQL Server is needed.

```
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~AuthEndpointTests"
```

Expected result:

```
Passed: 1
Failed: 0
```

6. Run the Complete Suite

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build succeeded with 0 warnings and 0 errors.
Passed: 36
Failed: 0
Skipped: 0
```

There are now 34 service tests and two HTTP integration tests: health and registration/login.

7. Update README Test Coverage

After verification:

- Change the total test count from 35 to 36.
- Change each expected `Passed: 35` result to `Passed: 36`.
- Add registration/login HTTP integration coverage to the test section.
- Describe both the health and authentication integration tests, not only health.
- Keep the note about the Testing environment, InMemory database, and test-only configuration.

Suggested coverage sentence:

> HTTP integration tests cover the public health endpoint and a registration/login flow that returns the expected user and a non-empty JWT token.

Do not claim that authenticated task HTTP tests exist yet.

8. Troubleshooting

Registration Returns 400

Check the DTO values: non-empty name, valid email, and a password with at least six characters. Keep the unique GUID-based email.

For detailed test logs:

```
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~AuthEndpointTests" --logger "console;verbosity=detailed"
```

Response Returns 500

Read the test logs. Confirm Day 113's factory remains in use and the three JWT configuration reads remain inside the `AddJwtBearer` callback.

Do not add a real JWT key to tracked configuration to make a test pass.

JSON Methods Are Not Found

Confirm the file includes:

```
using System.Net.Http.Json;
```

No additional package is required for these methods in this .NET 8 test project.

Interview Explanation

> I tested authentication through HTTP instead of calling AuthService directly. The test registers a unique user, logs in with the same credentials, and checks status codes, response DTO fields, matching user identity, and a returned token. It reuses the existing WebApplicationFactory with isolated in-memory storage.

End-of-Day Progress Note

Use this note after you implement and verify the lesson:

```
1. What I built
I added an HTTP integration test that registers a user and logs in through the authentication
endpoints.

2. What worked
Both requests return 200 OK, registration returns the expected user, login returns the same
user with a non-empty JWT token, and all 36 tests pass.

3. What I will do tomorrow
Add an authenticated task integration test for creating, retrieving, and updating a task.
```

Lesson Validation

The exact test above was validated in a disposable detached worktree: the focused test passed, the Release build had zero warnings/errors, and the full suite passed 36 tests. This does not change your working checkout; you still implement the lesson there.

Day 115 - Add an Authenticated Task Integration Test

Today's Goal

Use a JWT returned by the real login endpoint to test this workflow through HTTP:

```
Register -> Login -> Create Task -> Get Task -> Update Task
```

Keep it as one test because these operations form one connected API workflow.

File to Create

```
[repository root]\tests\JobTrackr.Tests\Integration\TaskEndpointTests.cs
```

No new package, migration, controller, service, or factory change is required.

1. Create the File

Run from the repository root:

```
cd [repository root]
New-Item -ItemType File -Path .\tests\JobTrackr.Tests\Integration\TaskEndpointTests.cs
```

2. Full Code

```
using System.Net;
using System.Net.Http.Headers;
using System.Net.Http.Json;
using JobTrackr.Application.Auth;
using JobTrackr.Application.Tasks;
using Microsoft.AspNetCore.Mvc.Testing;

namespace JobTrackr.Tests.Integration;

public class TaskEndpointTests : IClassFixture<JobTrackrApiFactory>
{
    private readonly JobTrackrApiFactory _factory;

    public TaskEndpointTests(JobTrackrApiFactory factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task CreateGetAndUpdateTask_WithAuthenticatedUser_ReturnsExpectedResponses()
    {
        using var client = _factory.CreateClient(
            new WebApplicationFactoryClientOptions
            {
                BaseAddress = new Uri("https://localhost")
            });

        var authenticatedUser = await RegisterAndLoginAsync(client);

        client.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);

        var dueDateUtc = DateTime.UtcNow.AddDays(5);

        var createRequest = new CreateTaskRequest
        {
            Title = "Prepare integration test",
            Description = "Create and retrieve a task through HTTP.",
            DueDateUtc = dueDateUtc,
            Priority = "High"
        };

        using var createResponse = await client.PostAsJsonAsync(
            "/api/tasks",
            createRequest);

        Assert.Equal(HttpStatusCode.Created, createResponse.StatusCode);
        Assert.NotNull(createResponse.Headers.Location);

        var createdTask = await createResponse.Content
            .ReadFromJsonAsync<TaskResponse>();

        Assert.NotNull(createdTask);
        Assert.True(createdTask.Id > 0);
        Assert.Equal(authenticatedUser.UserId, createdTask.UserId);
        Assert.Equal(createRequest.Title, createdTask.Title);
        Assert.Equal(createRequest.Description, createdTask.Description);
        Assert.Equal(createRequest.DueDateUtc, createdTask.DueDateUtc);
        Assert.Equal(createRequest.Priority, createdTask.Priority);
        Assert.False(createdTask.IsCompleted);

        using var getResponse = await client.GetAsync(
            $"/api/tasks/{createdTask.Id}");

        Assert.Equal(HttpStatusCode.OK, getResponse.StatusCode);

        var fetchedTask = await getResponse.Content
            .ReadFromJsonAsync<TaskResponse>();

        Assert.NotNull(fetchedTask);
        Assert.Equal(createdTask.Id, fetchedTask.Id);
        Assert.Equal(createdTask.UserId, fetchedTask.UserId);
        Assert.Equal(createRequest.Title, fetchedTask.Title);

        var updateRequest = new UpdateTaskRequest
```

```
        {
            Title = "Updated integration task",
            Description = "The task was updated through HTTP.",
            DueDateUtc = dueDateUtc.AddDays(2),
            Priority = "Medium"
        };

        using var updateResponse = await client.PutAsJsonAsync(
            $"/api/tasks/{createdTask.Id}",
            updateRequest);

        Assert.Equal(HttpStatusCode.OK, updateResponse.StatusCode);

        var updatedTask = await updateResponse.Content
            .ReadFromJsonAsync<TaskResponse>();

        Assert.NotNull(updatedTask);
        Assert.Equal(createdTask.Id, updatedTask.Id);
        Assert.Equal(authenticatedUser.UserId, updatedTask.UserId);
        Assert.Equal(updateRequest.Title, updatedTask.Title);
        Assert.Equal(updateRequest.Description, updatedTask.Description);
        Assert.Equal(updateRequest.DueDateUtc, updatedTask.DueDateUtc);
        Assert.Equal(updateRequest.Priority, updatedTask.Priority);
        Assert.False(updatedTask.IsCompleted);
    }

    private static async Task<AuthResponse> RegisterAndLoginAsync(
        HttpClient client)
    {
        var email = $"task-{Guid.NewGuid():N}@example.com";
        const string password = "IntegrationTest123!";

        var registerRequest = new RegisterRequest
        {
            FullName = "Task Integration User",
            Email = email,
            Password = password
        };

        using var registerResponse = await client.PostAsJsonAsync(
            "/api/auth/register",
            registerRequest);

        Assert.Equal(HttpStatusCode.OK, registerResponse.StatusCode);

        var loginRequest = new LoginRequest
        {
            Email = email,
            Password = password
        };

        using var loginResponse = await client.PostAsJsonAsync(
            "/api/auth/login",
            loginRequest);

        Assert.Equal(HttpStatusCode.OK, loginResponse.StatusCode);

        var authenticatedUser = await loginResponse.Content
            .ReadFromJsonAsync<AuthResponse>();

        Assert.NotNull(authenticatedUser);
        Assert.False(string.IsNullOrWhiteSpace(authenticatedUser.Token));

        return authenticatedUser;
    }
}
```

3. What This Test Covers

This is more than a service test. Each request passes through:

```
HttpClient
-> authentication middleware
-> authorization middleware
-> controller
-> service
-> EF Core InMemory
```

```
-> HTTP response
```

It proves that a token produced by JobTrackr login can access the protected task endpoints.

4. Code Explained Simply

Register and Login

```
var authenticatedUser = await RegisterAndLoginAsync(client);
```

The helper creates a unique user and logs in with the same email and password. It returns the login `AuthResponse`, including the user ID and JWT.

The GUID email prevents duplicate-email failures:

```
var email = $"task-{Guid.NewGuid():N}@example.com";
```

Add the Bearer Token

```
client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);
```

Every later request from this client now sends:

```
Authorization: Bearer JWT_TOKEN
```

ASP.NET Core validates the token and creates the authenticated user's claims.

Create the Task

```
using var createResponse = await client.PostAsJsonAsync(
    "/api/tasks",
    createRequest);
```

The request does not contain `UserId`. The controller reads the authenticated user ID from JWT claims.

The important assertions are:

```
Assert.Equal(HttpStatusCode.Created, createResponse.StatusCode);
Assert.NotNull(createResponse.Headers.Location);
Assert.Equal(authenticatedUser.UserId, createdTask.UserId);
```

They prove that creation returns 201, provides a location, and assigns ownership to the logged-in user.

Fetch the Same Task

```
using var getResponse = await client.GetAsync(
    $"/api/tasks/{createdTask.Id}");
```

The created task ID is used in the URL. The test checks that the fetched task has the same ID, owner, and title.

Update the Same Task

```
using var updateResponse = await client.PutAsJsonAsync(
    $"/api/tasks/{createdTask.Id}",
    updateRequest);
```

The update changes the title, description, due date, and priority.

The assertions confirm:

- The response is 200 OK.
- The task ID did not change.
- The owner did not change.
- All editable values changed.
- The completion state remained false.

Why the Helper Is Private

`RegisterAndLoginAsync` is used only by this test class, so it stays private. It removes setup noise from the main workflow without creating a large shared testing framework.

Day 116 can add its deletion test to this same class and reuse the helper.

5. Why Dates Are Stored in Variables

```
var dueDateUtc = DateTime.UtcNow.AddDays(5);
```

Using one variable means the test compares the response with the exact value sent in the request. It avoids calling `DateTime.UtcNow` separately in the assertion and receiving a slightly different time.

6. Run Today's Test

```
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~TaskEndpointTests"
```

Expected:

```
Passed: 1
Failed: 0
```

The API does not need to be running, and SQL Server is not used.

7. Run Everything

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected:

```
Build: 0 warnings, 0 errors
Passed: 37
Failed: 0
Skipped: 0
```

8. README Update After Verification

Update the repository README after the test passes:

- Change the test total from 36 to 37.
- Change every expected `Passed: 36` result to `Passed: 37`.
- Add authenticated task create/get/update HTTP coverage.
- Replace the statement that using the token is only planned.
- Do not claim delete integration coverage yet.

Suggested sentence:

> API integration tests cover health, registration/login, and an authenticated task workflow for creation, retrieval, and update.

9. Common Problems

Create Returns 401

Confirm the header uses the login token:

```
new AuthenticationHeaderValue("Bearer", authenticatedUser.Token)
```

Do not include the word `Bearer` inside the token value itself.

Create Returns 400

Confirm the title, description, and priority satisfy DTO validation. Use only `Low`, `Medium`, or `High` for priority.

Create Returns 500

Run the test with detailed logging:

```
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~TaskEndpointTests" --logger "console;verbosity=detailed"
```

Confirm the existing factory still replaces SQL Server with EF Core InMemory and provides the test JWT key.

Date Assertion Fails

Use the same `dueDateUtc` variable for the create request and derive the update date from it. Do not generate a second unrelated time in the assertion.

What the File Does

`TaskEndpointTests.cs`

Starts the API through the existing test factory, creates an authenticated user through HTTP, sends the JWT as a Bearer token, and verifies task creation, retrieval, and update as an API client.

Interview Explanation

> I added an API integration test that obtains a real JWT through registration and login, sends it through the authorization header, and verifies owner-scoped task creation, retrieval, and update. The test exercises the real HTTP pipeline and services while keeping persistence isolated with EF Core InMemory.

End-of-Day Progress Note

Use this after implementation and verification:

```
1. What I built
I added an authenticated task integration test that creates, retrieves, and updates a task
through HTTP.

2. What worked
Registration and login provide a valid Bearer token, task creation returns 201 with the
authenticated owner, retrieval returns the created task, update returns the changed task, and
all 37 tests pass.

3. What I will do tomorrow
Add a task deletion integration test and confirm the deleted task returns 404 Not Found.
```

Lesson Validation

This exact test was validated in a disposable checkout. The focused test passed, the Release build completed with zero warnings/errors, and all 37 tests passed.

Day 116 - Add a Task Delete Integration Test

Today's Goal

Test this authenticated API workflow through HTTP:

```
Register -> Login -> Create Task -> Delete Task -> Get Deleted Task
```

The test should prove that:

- task deletion returns `204 No Content`

- requesting the deleted task returns `404 Not Found`

File to Update

```
[repository root]\tests\JobTrackr.Tests\Integration\TaskEndpointTests.cs
```

No new file, package, migration, controller, or service is required.

1. Add the Test Method

Open:

```
[repository root]\tests\JobTrackr.Tests\Integration\TaskEndpointTests.cs
```

Add this method inside `TaskEndpointTests`, directly before the existing `RegisterAndLoginAsync` helper:

```
[Fact]
public async Task DeleteTask_WithAuthenticatedUser_ReturnsNoContentAndTaskIsNotFound()
{
    using var client = _factory.CreateClient(
        new WebApplicationFactoryClientOptions
        {
            BaseAddress = new Uri("https://localhost")
        });

    var authenticatedUser = await RegisterAndLoginAsync(client);

    client.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);

    var createRequest = new CreateTaskRequest
    {
        Title = "Delete integration test task",
        Description = "This task will be deleted through HTTP.",
        Priority = "Medium"
    };

    using var createResponse = await client.PostAsJsonAsync(
        "/api/tasks",
        createRequest);

    Assert.Equal(HttpStatusCode.Created, createResponse.StatusCode);

    var createdTask = await createResponse.Content
        .ReadFromJsonAsync<TaskResponse>();

    Assert.NotNull(createdTask);

    using var deleteResponse = await client.DeleteAsync(
        $"/api/tasks/{createdTask.Id}");

    Assert.Equal(HttpStatusCode.NoContent, deleteResponse.StatusCode);

    using var getResponse = await client.GetAsync(
```

```
        $"/api/tasks/{createdTask.Id}");

    Assert.Equal(HttpStatusCode.NotFound, getResponse.StatusCode);
}
```

The existing `using` statements already provide everything this test needs. Do not add duplicate `using` statements.

2. Code Explained Simply

Create an Authenticated Client

```
var authenticatedUser = await RegisterAndLoginAsync(client);

client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);
```

The existing helper registers and logs in a unique user. The returned JWT is added to all later requests from this client.

Create a Task to Delete

```
using var createResponse = await client.PostAsJsonAsync(
    "/api/tasks",
    createRequest);
```

The test first creates a real task through the protected API. Its response contains the database-generated task ID needed for deletion.

```
var createdTask = await createResponse.Content
    .ReadFromJsonAsync<TaskResponse>();
```

`ReadFromJsonAsync<TaskResponse>()` converts the JSON response into a C# object, allowing the test to use `createdTask.Id`.

Delete the Task

```
using var deleteResponse = await client.DeleteAsync(
    $"/api/tasks/{createdTask.Id}");
```

This sends an authenticated `DELETE` request for the task created by the same user.

```
Assert.Equal(HttpStatusCode.NoContent, deleteResponse.StatusCode);
```

`204 No Content` means deletion succeeded and the API intentionally returned no response body.

Confirm It Is Gone

```
using var getResponse = await client.GetAsync(
    $"/api/tasks/{createdTask.Id}");

Assert.Equal(HttpStatusCode.NotFound, getResponse.StatusCode);
```

The final GET proves the task was actually removed. Checking only the DELETE response would not prove that the later read fails.

3. Why This Is an Integration Test

The test exercises the complete request path:

```
HttpClient
-> JWT authentication
-> authorization
-> TasksController
-> TaskService
-> EF Core InMemory database
-> HTTP response
```

It checks that the API components work together, not only that one service method works by itself.

4. Why We Reuse `RegisterAndLoginAsync`

Day 115 already added this helper:

```
var authenticatedUser = await RegisterAndLoginAsync(client);
```

Reusing it keeps registration and login setup out of the main deletion workflow. Do not create a second login helper.

There is still some task-creation duplication between the two tests. Keep it today; Day 118 is specifically planned for cleaning up query and test duplication.

5. Run Today's Test

Run from the repository root:

```
cd [repository root]
dotnet test .\tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~DeleteTask_WithAuthenticatedUser"
```

Expected:

```
Passed: 1
Failed: 0
```

The API does not need to be running, and this test does not use SQL Server.

6. Run Everything

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected after adding today's test:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

7. README Update After Verification

After all tests pass:

- change the test total from 37 to 38
- change every expected `Passed: 37` result to `Passed: 38`
- add authenticated task deletion and missing-after-delete coverage
- change the API integration test count from three to four

Suggested coverage bullet:

```
- authenticated task creation, retrieval, update, and deletion through HTTP
```

8. Common Problems

Delete Returns 401 Unauthorized

Confirm the bearer token is assigned before the create and delete requests:

```
client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);
```

Delete Returns 404 Not Found Immediately

Confirm the URL uses the ID returned by the create response:

```
$"/api/tasks/{createdTask.Id}"
```

Do not use a hard-coded task ID.

Create Returns 400 Bad Request

Use a non-empty title and a valid priority: `Low`, `Medium`, or `High`.

The Final GET Returns 200 OK

Check that the delete request completed before sending the GET request and that both URLs use the same `createdTask.Id`.

What the Updated File Does

`TaskEndpointTests.cs`

The new test authenticates a user, creates that user's task through HTTP, deletes it, and verifies that a later GET returns 404. The existing Day 115 workflow test remains unchanged.

Interview Explanation

> I added an authenticated API integration test for task deletion. It creates a task through HTTP, verifies DELETE returns 204 No Content, and then verifies GET returns 404, proving the resource was removed through the complete application pipeline.

Lesson Validation

The exact test method in this lesson was validated in a disposable checkout. The focused deletion test passed, and the complete suite passed with all 38 tests.

End-of-Day Progress Note

Use this after implementation and verification:

```
1. What I built
I added an authenticated task deletion integration test.

2. What worked
The authenticated user can create and delete a task through HTTP, deletion returns 204 No
Content, the deleted task returns 404 Not Found, and all 38 tests pass.

3. What I will do tomorrow
Review whether the integration tests run reliably in GitHub Actions.
```

Day 117 - Review Integration Tests in GitHub Actions

Today's Goal

Confirm that GitHub Actions runs both kinds of tests:

```
Service tests + API integration tests
```

Today is a review and verification day. The current workflow already runs the complete test project, so no code or YAML change is required unless the GitHub run fails.

Files to Review

```
[repository root]\.github\workflows\build.yml
[repository root]\tests\JobTrackr.Tests\JobTrackr.Tests.csproj
[repository root]\tests\JobTrackr.Tests\Integration\JobTrackrApiFactory.cs
```

1. Review the Workflow Test Command

Open:

```
[repository root]\.github\workflows\build.yml
```

The current test step is:

```
- name: Test
  run: dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --no-
restore
```

This command runs the entire `JobTrackr.Tests` project. It does not run only files named service tests.

xUnit discovers every `[Fact]` in that project, including:

```
Service tests
HealthEndpointTests
AuthEndpointTests
TaskEndpointTests
```

Therefore, the Day 113-116 API integration tests are already part of CI.

2. Understand Why CI Does Not Need SQL Server

Open:

```
[repository root]\tests\JobTrackr.Tests\Integration\JobTrackrApiFactory.cs
```

The factory removes the normal SQL Server database registration:

```
services.RemoveAll<DbContextOptions<AppDbContext>>();
```

It replaces SQL Server with a unique EF Core in-memory database:

```
private readonly string _databaseName =
    $"JobTrackrIntegrationTests-{Guid.NewGuid()}";

services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase(_databaseName));
```

This means the GitHub runner does not need:

- SQL Server

- a real connection string

- database migrations

- database cleanup scripts

The integration tests still exercise the API, controllers, services, authentication, and EF Core behavior.

3. Understand Why CI Does Not Need Local User Secrets

The factory supplies test-only JWT configuration:

```
["Jwt:Key"] =
    "JobTrackr-integration-test-signing-key-for-automated-tests-only"
```

This key exists only in test code and is not a production secret. It allows the in-memory API to generate and validate JWTs during automated tests.

The factory also uses the `Testing` environment:

```
builder.UseEnvironment("Testing");
```

Therefore, GitHub Actions does not need your local .NET User Secrets.

4. Run the CI Commands Locally

Run from the repository root:

```
cd [repository root]

dotnet restore tests/JobTrackr.Tests/JobTrackr.Tests.csproj
dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-restore
dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --no-restore
```

Expected result:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 38
Failed: 0
Skipped: 0
```

These are the same restore, build, and test operations used by the workflow.

5. Check GitHub Actions

Open the repository Actions page:

[JobTrackr GitHub Actions](https://github.com/BackendSoftwareJourney/jobtrackr-backend/actions)

Open the run named:

```
Day 116: Add task delete API integration test
```

Then inspect the `Test` step. Confirm that it reports:

```
Passed: 38
Failed: 0
Skipped: 0
```

This proves the integration tests work on a clean GitHub-hosted Ubuntu runner, not only on your Windows computer.

6. Final Decision

Keep the current workflow unchanged.

The existing command already runs the complete test project, and the integration test factory provides isolated persistence and test JWT configuration. Adding SQL Server or repository secrets to CI would introduce complexity without helping the current test suite.

7. What Not to Change Today

Do not:

- add SQL Server to GitHub Actions
- add your local connection string to GitHub
- add your real JWT key to GitHub
- split integration tests into another project
- change application architecture
- install another test framework

The current setup is appropriate for this stage of JobTrackr.

8. Do We Need a Git Commit?

No commit is required if the workflow succeeds and no repository file changes.

Check with:

```
git status
```

Expected:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Review work is still meaningful engineering work. The result is a verified decision that the current CI design is reliable enough to keep.

What Each File Does

`build.yml`

Defines the clean Ubuntu environment and runs restore, Release build, and the complete test project for pushes and pull requests to `main`.

`JobTrackr.Tests.csproj`

Contains all service and API integration tests in one xUnit project and references the packages needed by the test suite.

`JobTrackrApiFactory.cs`

Starts the real API pipeline in memory while replacing external SQL Server and local JWT configuration with isolated test-only equivalents.

Interview Explanation

> I verified that our CI command runs the complete xUnit project, including API integration tests. The tests are reliable on a clean runner because WebApplicationFactory replaces SQL Server with an isolated EF Core in-memory database and provides test-only JWT configuration, so CI does not depend on developer secrets or external infrastructure.

End-of-Day Progress Note

Use this after checking the GitHub Actions result:

```
1. What I reviewed
I reviewed how the service and API integration tests run in GitHub Actions.

2. What worked
The workflow runs the complete test project, all 38 tests pass on a clean runner, and
integration tests use an isolated in-memory database and test JWT configuration without SQL
Server or local secrets.

3. What I will do tomorrow
Clean up duplicated query and integration test setup code and review unclear method names.
```

Lesson Validation

The workflow-equivalent restore, Release build, and test commands were run locally. The build completed with zero warnings/errors, and all 38 tests passed.

Day 118 - Clean Up Task Query and Integration Tests

Today's Goal

Improve readability without changing behavior:

1. Extract task sorting into a clearly named private method.
2. Improve unclear query variable names.
3. Remove repeated setup from task integration tests.
4. Confirm all 38 tests still pass.

No endpoint, DTO, database, migration, or service contract changes today.

Files to Update

```
[repository root]\src\JobTrackr.Infrastructure\Tasks\TaskService.cs
[repository root]\tests\JobTrackr.Tests\Integration\TaskEndpointTests.cs
```

Part 1 - Clean Up Task Sorting

Open `TaskService.cs`.

Replace the Sorting Code in `GetAllAsync`

Keep this line:

```
var totalCount = await query.CountAsync();
```

Replace the existing `skip`, sorting, and `items` code with:

```
var tasksToSkip = (pageNumber - 1) * pageSize;
var orderedQuery = ApplySorting(query, sortBy, sortDirection);

var taskResponses = await orderedQuery
    .Skip(tasksToSkip)
    .Take(pageSize)
    .Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    })
    .ToListAsync();
```

Then change the paged response property from `Items = items` to:

```
Items = taskResponses,
```

The new names are clearer:

- `tasksToSkip` is the number of database rows skipped before the requested page.
- `taskResponses` is the final list of response DTOs.
- `orderedQuery` is the filtered query after sorting.

Add `ApplySorting`

Add this method directly before `MapToResponse`:

```
private static IOrderedQueryable<JobTask> ApplySorting(
    IQueryable<JobTask> query,
    string sortBy,
    string sortDirection)
{
    var isAscending = string.Equals(
        sortDirection,
        "asc",
        StringComparison.OrdinalIgnoreCase);

    if (string.Equals(sortBy, "dueDate", StringComparison.OrdinalIgnoreCase))
    {
        return isAscending
            ? query
                .OrderBy(task => task.DueDateUtc == null)
                .ThenBy(task => task.DueDateUtc)
                .ThenBy(task => task.Id)
            : query
                .OrderBy(task => task.DueDateUtc == null)
                .ThenByDescending(task => task.DueDateUtc)
                .ThenByDescending(task => task.Id);
    }

    return isAscending
        ? query
            .OrderBy(task => task.CreatedAtUtc)
            .ThenBy(task => task.Id)
        : query
            .OrderByDescending(task => task.CreatedAtUtc)
            .ThenByDescending(task => task.Id);
}
```

`ApplySorting` receives the filtered EF Core query and returns it in the requested order. Due-date sorting still keeps null dates last, created date remains the default, and ID remains the stable tie-breaker.

The `condition ? firstValue : secondValue` syntax is a conditional expression. It returns the first query for ascending order and the second query for descending order.

Part 2 - Remove Integration Test Duplication

Open `TaskEndpointTests.cs`.

Simplify the Create/Get/Update Test

Replace its repeated client creation, login, and authorization code with:

```
using var client = CreateClient();
var authenticatedUser = await AuthenticateClientAsync(client);
```

Keep the existing `dueDateUtc` variable and `createRequest` object. They contain the task data used by the helper and later assertions.

Replace its POST request, response checks, and JSON conversion with:

```
var createdTask = await CreateTaskAsync(client, createRequest);
```

Keep all later assertions unchanged.

Simplify the Delete Test

Replace its repeated client creation, login, and authorization code with:

```
using var client = CreateClient();
await AuthenticateClientAsync(client);
```

Keep the existing `createRequest` object. Only replace the POST request and response-handling lines that come after it.

Replace its repeated task POST and response conversion with:

```
var createdTask = await CreateTaskAsync(client, createRequest);
```

Keep the DELETE request and final GET assertion unchanged.

Add the Test Helpers

Add these directly before `RegisterAndLoginAsync`:

```
private HttpClient CreateClient()
{
    return _factory.CreateClient(
        new WebApplicationFactoryClientOptions
        {
            BaseAddress = new Uri("https://localhost")
        });
}

private static async Task<AuthResponse> AuthenticateClientAsync(
    HttpClient client)
{
    var authenticatedUser = await RegisterAndLoginAsync(client);

    client.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", authenticatedUser.Token);

    return authenticatedUser;
}

private static async Task<TaskResponse> CreateTaskAsync(
    HttpClient client,
    CreateTaskRequest request)
{
    using var response = await client.PostAsJsonAsync(
        "/api/tasks",
        request);

    Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    Assert.NotNull(response.Headers.Location);

    var createdTask = await response.Content
        .ReadFromJsonAsync<TaskResponse>();

    Assert.NotNull(createdTask);

    return createdTask;
}
```

What the Helpers Do

- `CreateClient` creates the in-memory HTTPS API client.
- `AuthenticateClientAsync` registers and logs in a user, then adds the JWT bearer token to the client.
- `CreateTaskAsync` creates a task, checks `201 Created` and the location header, and returns its `TaskResponse`.

The main test methods now focus on the behavior being tested instead of repeating setup details.

Why `ITaskService` Is Not Renamed

The public service method names already describe their use cases. Renaming them would force unrelated changes across the interface, controller, implementation, and tests.

Today we introduce the clear private name `ApplySorting` and improve local variable names without changing public contracts.

Format, Build, and Test

Run from the repository root:

```
cd [repository root]
dotnet format JobTrackr.slnx --no-restore
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The test count remains 38 because today refactors existing code instead of adding tests.

Check the Diff

```
git status
git diff --check
git diff
```

Only these files should change:

```
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
tests/JobTrackr.Tests/Integration/TaskEndpointTests.cs
```

What Each File Does

`TaskService.cs`

Keeps filtering, counting, pagination, and projection in `GetAllAsync`, while `ApplySorting` owns the sorting decisions.

`TaskEndpointTests.cs`

Keeps the same authenticated workflows but moves repeated authentication and task creation setup into reusable private helpers.

Interview Explanation

> I performed a behavior-preserving refactor. I extracted sorting into a focused private method, improved local names, and removed repeated integration-test setup with small helpers. I ran the complete Release build and test suite to confirm behavior remained unchanged.

End-of-Day Progress Note

```
1. What I cleaned up
I extracted task sorting into a private helper, improved query variable names, and removed
repeated authentication and task creation setup from the task integration tests.

2. What worked
Task filtering, sorting, pagination, and authenticated API integration tests still behave the
same, the Release build succeeds, and all 38 tests pass.

3. What I will do tomorrow
Update the README with task query and authentication examples, then review and push the
documentation changes.
```

Lesson Validation

The exact refactoring was validated in a disposable checkout. The Release build completed with zero warnings/errors, and all 38 tests passed.

Day 119 - Add Authentication and Task Query Examples to README

Today's Goal

Improve the project documentation with runnable PowerShell examples for:

1. Registering a user.
2. Logging in and receiving a JWT.
3. Creating an authorization header.
4. Querying owned tasks with filtering, searching, sorting, and pagination.
5. Reading paged response metadata and task items.

Today changes documentation only. Do not change controllers, services, DTOs, tests, or the database.

File to Update

```
[repository root]\README.md
```

1. Find the Authentication Section

In `README.md`, find:

```
### Test Authentication
```

Keep the existing Swagger instructions.

Find the current small PowerShell example that starts with:

```
$token = "PASTE_LOGIN_TOKEN_HERE"
```

Replace that complete PowerShell block with the example below.

2. Add the Complete Authentication Example

```
$baseUrl = "https://localhost:7024"
$email = "readme-$([Guid]::NewGuid().ToString('N'))@example.com"
$password = "ReadmeTest123!"

$registerBody = @{
    fullName = "README Test User"
    email = $email
    password = $password
} | ConvertTo-Json

$registeredUser = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/register" `
    -Method Post `
    -ContentType "application/json" `
    -Body $registerBody

$loginBody = @{
    email = $email
    password = $password
} | ConvertTo-Json

$authenticatedUser = Invoke-RestMethod `
    -Uri "$baseUrl/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $loginBody

$headers = @{
    Authorization = "Bearer $($authenticatedUser.token)"
}

$authenticatedUser | Select-Object userId, fullName, email
```

What This Example Does

```
$email = "readme-$([Guid]::NewGuid().ToString('N'))@example.com"
```

The GUID creates a different email every time, preventing duplicate-email errors during repeated testing.

```
$authenticatedUser = Invoke-RestMethod ...
```

The login response contains `userId`, `fullName`, `email`, and `token`.

```
$headers = @{
    Authorization = "Bearer $($authenticatedUser.token)"
}
```

The header stores the JWT in the format required by protected endpoints. Later requests can reuse `$headers`.

The password is only sample local test data. It is not an application signing key or production credential.

3. Add Task Query Examples

Directly after the authentication PowerShell block and its short explanation, add this new README subsection:

```
### Test Task Queries

The following examples reuse `$baseUrl` and `$headers` from the authentication example.

Get the first page using the default created-date descending order:
```

$tasks = Invoke-RestMethod `-Uri "$baseUrl/api/tasks?pageNumber=1&pageSize=10"` -Method Get ` -Headers $headers

```
Filter incomplete tasks, search titles, sort by due date ascending, and request the first page:
```

$tasks = Invoke-RestMethod `-Uri "$baseUrl/api/tasks?isCompleted=false&search=resume&sortBy=dueDate&sortDirection=asc&pageNumber=1&pageSize=10"` -Method Get ` -Headers $headers

```
Display pagination metadata:
```

$tasks | Select-Object pageNumber, pageSize, totalCount, totalPages

```
Display the returned task items:
```

$tasks.items | Select-Object id, title, isCompleted, priority, dueDateUtc

```
An empty page is valid. When no owned task matches the filters, `items` is empty and
`totalCount` is `0`.

Query rules:

- `pageNumber` must be at least `1`.
- `pageSize` must be between `1` and `100`.
- `sortBy` accepts `createdAt` or `dueDate`.
- `sortDirection` accepts `asc` or `desc`.
- Tasks without a due date appear last when sorting by due date.
- Every result is restricted to the authenticated user.
```

Important: the outer `markdown` fence above is only showing what belongs in the README. In the actual README, use normal headings and separate `powershell` code blocks exactly as displayed.

4. Add a Cleanup Command

After the task-query examples, add:

```
Remove the temporary authentication values from the PowerShell session when testing is complete:
```

Remove-Variable registeredUser, authenticatedUser, headers, registerBody, loginBody, password, email

This removes the JWT and example password from the current PowerShell variables.

5. Review the Documentation

Confirm the examples match the current API:

```
POST /api/auth/register
POST /api/auth/login
GET /api/tasks
```

Confirm the task query uses only supported parameters:

```
isCompleted
search
sortBy
sortDirection
pageNumber
pageSize
```

Also confirm:

- no real JWT is written in the README
- no JWT signing key is written in the README
- no real database password is written in the README
- the example uses `https://localhost:7024`
- the example sends the bearer token through `$headers`

6. Build and Test

Documentation does not compile, but run the normal verification before publishing:

```
cd [repository root]
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

7. Check the Git Diff

```
git status
git diff --check
git diff -- README.md
```

Only `README.md` should be changed today.

What the Updated File Does

`README.md`

Shows another developer how to register, log in, construct a bearer header, run task queries, and inspect paged results without guessing request formats.

Interview Explanation

> I improved the API documentation with runnable authentication and task-query examples. The examples demonstrate JWT usage, ownership-aware filtering, sorting, pagination, and response metadata while avoiding real secrets.

End-of-Day Progress Note

Use this after verification and pushing:

```
1. What I documented
I added runnable PowerShell examples for registration, login, JWT authorization, and task
filtering, searching, sorting, and pagination.

2. What worked
The README examples match the current API, contain no real secrets, explain paged task results,
and the Release build and all 38 tests pass.

3. What I will do tomorrow
Complete the Month 4 review and confirm pagination, authentication, tests, CI, and
documentation are stable.
```

Day 120 - Month 4 Review

Today's Goal

Close the fourth 30-day backend phase by confirming that these areas are stable:

- task filtering, searching, sorting, and pagination
- authenticated user profile and password changes
- task ownership authorization
- Swagger and local setup documentation
- development seed data and migration instructions
- API integration tests
- GitHub Actions CI
- repository security and cleanliness

Today is a review day. Do not add a new feature, redesign the architecture, or begin Day 121 early.

Month 4 Scope

Days 91-120 focused on stronger API queries, authenticated account behavior, integration testing, CI reliability, and portfolio documentation.

What Was Completed

Days 91-98 - Task Query Improvements

- Planned a predictable query contract.
- Added `pageNumber` and `pageSize` with safe defaults and validation.
- Added server-side pagination with `Skip` and `Take`.
- Added a reusable `PagedResponse<T>` with page metadata.
- Added created-date ascending and descending sorting.
- Added due-date sorting with null due dates last.
- Kept stable ID tie-breakers for predictable pages.
- Tested ownership, filtering, search, sorting, and pagination together.

Days 99-104 - Authenticated Account and Authorization

- Added `GET /api/users/me`.
- Added `PUT /api/users/me` for safe profile changes.
- Planned and implemented authenticated password changes.
- Verified the current password before saving a new hash.
- Added service tests for password-change behavior.
- Reviewed and strengthened task ownership authorization.

Days 105-111 - API Usability and Local Setup

- Reviewed API routes, DTOs, and status codes.
- Added useful XML endpoint descriptions.
- Added JWT Bearer authorization support to Swagger.
- Improved README setup instructions.
- Added optional development-only seed data.
- Reviewed EF Core migrations and database recreation.
- Documented safe local database reset commands.

Days 112-119 - Integration Tests, CI, and Documentation

- Planned a small `WebApplicationFactory` integration-test approach.
- Added a public health endpoint integration test.
- Added registration and login integration testing.
- Added authenticated task create, get, and update testing.
- Added authenticated task deletion and missing-after-delete testing.
- Confirmed service and integration tests run together in GitHub Actions.
- Cleaned task sorting and repeated integration-test setup.
- Added runnable authentication and task-query examples to the README.
- Reached 38 automated tests: 34 service tests and four API integration tests.

Step 1 - Confirm Repository State

Run:

```
cd [repository root]
git status --short
git log -1 --oneline
```

Expected current commit:

```
c04b2eb Day 119: Add API usage examples
```

`git status --short` should produce no output.

Do not create an empty Day 120 commit when no repository file changed.

Step 2 - Verify Task Query Behavior

Confirm `GET /api/tasks` supports:

```
isCompleted
search
sortBy
sortDirection
pageNumber
pageSize
```

Confirm the rules:

- only the authenticated user's tasks are queried
- filters apply before counting and pagination
- default page number is 1

- default page size is `10`
- maximum page size is `100`
- default sorting is newest created task first
- due-date sorting keeps null due dates last
- ID provides stable ordering when dates are equal
- pages beyond available data return an empty `items` list

The response shape should remain:

```
{
  "items": [],
  "pageNumber": 1,
  "pageSize": 10,
  "totalCount": 0,
  "totalPages": 0
}
```

Step 3 - Verify Authentication and Account Behavior

Confirm:

- registration stores a password hash instead of a plain password
- login returns a signed JWT
- `GET /api/users/me` reads the user ID from JWT claims
- `PUT /api/users/me` updates the authenticated user's profile
- changing a password requires the correct current password
- the new password is hashed before saving
- authentication responses do not expose password data
- task endpoints reject anonymous requests with `401 Unauthorized`

Current limitation: existing JWTs remain valid until expiration after a password change because token revocation is not implemented yet.

Step 4 - Verify Task Ownership

Task operation	Expected behavior
Create	Assigns the authenticated user ID
Get All	Returns only the authenticated user's tasks
Get By Id	Returns only an owned task
Update	Updates only an owned task
Delete	Deletes only an owned task
Complete	Completes only an owned task
Reopen	Reopens only an owned task

A missing task or another user's task should return `404 Not Found` for ownership-protected operations.

Step 5 - Review Integration Test Isolation

Confirm the integration test factory:

- starts the API with `WebApplicationFactory<Program>`
- uses the `Testing` environment
- replaces SQL Server with EF Core InMemory
- uses a unique database name
- provides test-only JWT configuration
- disables development seed data
- does not require a running API
- does not require local User Secrets

The current four API integration tests cover:

1. Public health response.
2. Registration and login through HTTP.
3. Authenticated task create, get, and update.
4. Authenticated task delete followed by `404 Not Found`.

Step 6 - Build and Run All Tests

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

Local SDK Note

This computer currently selects .NET SDK 10 while the project targets .NET 8. The local .NET 8 SDK is not currently listed, and the .NET 10 installation has shown missing .NET 8 reference-pack analyzer files.

Check installed SDKs with:

```
dotnet --list-sdks
```

Install or repair the supported .NET 8 SDK before relying on normal local builds. Do not commit a temporary `global.json` merely to hide an incomplete SDK installation.

The project itself remains valid: GitHub Actions builds it with the .NET 8 SDK successfully, and local verification also passed with the installed .NET 9 SDK.

Step 7 - Verify GitHub Actions

Open:

[Day 119 GitHub Actions run](https://github.com/BackendSoftwareJourney/jobtrackr-backend/actions/runs/34344261468)

Confirm:

```
Workflow: Build
Run: #35
Commit: c04b2eb
Status: Success
```

The workflow restores dependencies, builds in Release configuration, and runs the complete xUnit project, including service and API integration tests.

Step 8 - Review README Accuracy

Confirm the README documents:

- all current health, authentication, task, and user endpoints
- SQL Server and migration setup
- User Secrets and JWT configuration
- HTTPS startup and Swagger authorization
- registration and login PowerShell commands
- task filter, search, sort, and pagination examples
- paged response metadata
- all 38 tests
- GitHub Actions behavior
- current authorization limitations

The README must not claim unfinished features.

Step 9 - Check for Tracked Secrets

Run:

```
git grep -n "Bearer eyJ"
git grep -n "Password=" -- "*.json" "*.md"
git grep -n "Jwt:Key" -- "*.json"
```

Expected: no real JWT, database password, or signing key is tracked.

The README may contain an obvious example-only password and the command name `Jwt:Key`; these are documentation, not real secrets.

Step 10 - Final Month 4 Decision

Month 4 is stable enough to close.

Do not add more pagination or integration-test infrastructure now. The next phase follows:

```
[planning archive]\backend_121_150_day_plan.md
```

Days 121-150 focus on maintainability, cleaner application contracts, mapping, validation, and simple domain rules.

Day 121 begins with an architecture review. Problems will be listed before any restructuring, and the existing four-project architecture will remain in place.

Do We Need a Git Commit Today?

No commit or push is required if the review finds no repository change.

```
git status
```

Expected:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Interview Explanation

> During Month 4, I added predictable server-side pagination and sorting, authenticated profile and password workflows, isolated API integration tests, CI execution, and runnable API documentation. I closed the phase by verifying ownership, query behavior, test isolation, secret handling, and a successful clean CI build.

End-of-Day Progress Note

Use this after completing the review:

```
1. What I reviewed
I completed the Month 4 review of task queries, authentication, ownership, integration tests,
CI, documentation, and repository security.

2. What worked
Pagination and sorting are stable, authenticated workflows and ownership behave correctly, all
38 tests pass, GitHub Actions succeeds, documentation matches the API, and no real secrets are
tracked.

3. What I will do tomorrow
Start the next backend phase with an architecture responsibility review before making further
changes.
```

Review Validation

The repository is clean at commit `c04b2eb`. GitHub Actions Build #35 completed successfully for that commit, and the tracked-secret scan found no secret-like values.

MONTH 5

VALIDATION, COMMENTS, AND MAINTAINABILITY

Strengthen validation and error handling, build owner-protected task comments, expand tests, and review repository health.

Day 121 - Architecture Responsibility Review

Today's Goal

Review the current JobTrackr architecture before making further maintainability changes.

The review confirms:

- what each project owns
- whether project references point in the correct direction
- whether controllers remain separated from EF Core
- which maintainability problems are real enough to address later

This is not an architecture redesign. The existing four-project structure remains in place.

Current Dependency Direction

```
Domain         -> no project reference
Application    -> Domain
Infrastructure -> Application + Domain
Api            -> Application + Infrastructure
Tests          -> Api + Application + Infrastructure
```

This direction is valid for the current application. Dependencies point toward the inner projects, while the API project composes the application at startup.

Domain and Application must not reference API or Infrastructure.

Domain Responsibilities

The Domain project contains the core `User` and `JobTask` entities. It owns business data and relationships without depending on controllers, HTTP, JWT, EF Core, or ASP.NET Core.

The current design is appropriately simple. The entities contain mostly properties and little behavior, but behavior should be added only when a real domain rule needs a natural home. Complexity should not be introduced merely to make the model appear more sophisticated.

Application Responsibilities

The Application project owns:

- request and response DTOs
- service interfaces
- shared response types and error messages
- the password-hashing contract
- feature-level application behavior

Controllers depend on interfaces from this project, and response DTOs do not expose persistence details or password hashes.

The review identified several focused improvements for later lessons: inconsistent service method naming, validation responsibilities, and repeated response mapping.

Infrastructure Responsibilities

The Infrastructure project owns external and persistence concerns:

- EF Core and `AppDbContext`
- SQL Server configuration
- migrations
- database-backed service implementations
- JWT token generation
- optional development seed data

Services implement interfaces defined by Application, database queries enforce task ownership, and migrations remain beside persistence code.

Some response mapping is repeated, and `TaskService` continues to grow. Neither issue justifies an immediate large abstraction. Later lessons will address duplication only where the result is easier to understand.

API Responsibilities

The API project owns:

- HTTP routes and controllers
- status codes and `ProblemDetails` responses
- authentication and authorization configuration
- dependency injection registration
- middleware and application startup
- Swagger configuration
- the health endpoint

Controllers call application service interfaces instead of querying `AppDbContext` directly. `Program.cs` remains the composition root, and authentication runs before authorization.

The review found repeated authenticated-user claim handling and repeated controller error responses. These are candidates for focused cleanup, not permission for a broad rewrite.

Test Responsibilities

The test project covers service behavior, ownership, task queries, passwords, authentication, and HTTP integration behavior.

Service tests use EF Core InMemory, while integration tests use `WebApplicationFactory<Program>` with isolated JWT and database configuration. GitHub Actions runs the complete test project. At this point, the suite contains 38 passing tests.

Repeated test setup should be extracted only when a helper remains easier to understand than the original tests.

Architecture Boundary Checks

Run these checks from the repository root.

Confirm that Application and Domain do not depend on API or Infrastructure namespaces:

```
rg "using JobTrackr\.(Api|Infrastructure)" src\JobTrackr.Application src\JobTrackr.Domain
```

Expected result: no output.

Confirm that Domain does not use EF Core:

```
rg "Microsoft\.EntityFrameworkCore|DbContext" src\JobTrackr.Domain
```

Expected result: no output.

Confirm that controllers do not access EF Core directly:

```
rg "AppDbContext|Microsoft\.EntityFrameworkCore" src\JobTrackr.Api\Controllers
```

Expected result: no output.

Maintainability Backlog

Finding	Planned review
Service method naming is not fully consistent	Day 122
DTO organization should be confirmed before moving files	Day 123
Validation is split between attributes and service checks	Days 124-125
Response mapping is repeated	Days 126-129
Domain entities may or may not need behavior	Day 130
Business rules need focused review	Days 131-135
Controllers repeat some error and claims handling	Days 136 and 139
Service size should be measured before splitting	Day 140

A known limitation remains: legacy user CRUD routes are publicly accessible, and user creation can bypass the registration workflow. This must not be confused with completed authorization, but it is outside today's architecture review.

What Must Not Change Today

Do not:

- add another project
- move services between projects
- introduce repository or unit-of-work abstractions
- add MediatR, CQRS, AutoMapper, or FluentValidation
- move every DTO or mapping method immediately
- redesign authentication or API routes
- create an empty commit

The current architecture is understandable and working. Future changes should solve specific problems one lesson at a time.

Verification

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The repository review confirmed clean dependency boundaries. Domain and Application do not reference outer projects, Domain does not use EF Core, and controllers do not access the database context directly.

Architecture Decision

Keep the current structure:

```
Api -> Application + Infrastructure
Infrastructure -> Application + Domain
Application -> Domain
Domain -> no project dependencies
```

The next step is not a redesign. Day 122 will review service method names, apply a consistent asynchronous naming style, update callers, and verify the build and tests.

Interview Explanation

I reviewed the architecture before refactoring. Domain remains independent, Application owns contracts and DTOs, Infrastructure owns EF Core and external implementations, and API owns HTTP and composition. I documented specific maintainability issues and scheduled small changes instead of performing a broad redesign.

Exercise

Draw the dependency graph for the solution and explain why each project reference points inward. Then identify one maintainability issue that can be improved without changing the overall architecture.

Next Lesson

Day 122 reviews service method names and makes asynchronous naming consistent across interfaces, implementations, controllers, and tests.

Day 122 - Make Service Method Names Consistent

Today's Goal

Remove repeated feature names from service methods while keeping the `Async` suffix.

For example:

```
_taskService.CreateTaskAsync(...)
```

becomes:

```
_taskService.CreateAsync(...)
```

`TaskService` already tells the reader that the method works with tasks. Repeating `Task` makes the call longer without adding useful context.

This is a naming-only refactor. Routes, parameters, return types, DTOs, validation rules, and runtime behavior must remain unchanged.

Naming Convention

Use these names for CRUD-style service methods:

```
CreateAsync
GetAllAsync
GetByIdAsync
UpdateAsync
DeleteAsync
```

Task-specific state changes become:

```
CompleteAsync
ReopenAsync
```

Keep descriptive query and authentication names such as `GetByUserIdAsync`, `RegisterAsync`, `LoginAsync`, and `ChangePasswordAsync`. Those names communicate distinct use cases rather than repeating the service type.

Task Service Renames

Previous name	New name
`CreateTaskAsync`	`CreateAsync`
`UpdateTaskAsync`	`UpdateAsync`
`DeleteTaskAsync`	`DeleteAsync`
`CompleteTaskAsync`	`CompleteAsync`
`ReopenTaskAsync`	`ReopenAsync`

The existing `GetAllAsync`, `GetByIdAsync`, and `GetByUserIdAsync` names remain unchanged.

The resulting task service contract is concise while retaining meaningful query context:

```
public interface ITaskService
{
    Task<PagedResponse<TaskResponse>> GetAllAsync(
        bool? isCompleted,
```

```
        string? search,
        string sortBy,
        string sortDirection,
        int pageNumber,
        int pageSize,
        int userId);

    Task<TaskResponse?> GetByIdAsync(int id, int userId);
    Task<TaskResponse> CreateAsync(CreateTaskRequest request, int userId);
    Task<TaskResponse?> UpdateAsync(int id, UpdateTaskRequest request, int userId);
    Task<bool> DeleteAsync(int id, int userId);
    Task<TaskResponse?> CompleteAsync(int id, int userId);
    Task<TaskResponse?> ReopenAsync(int id, int userId);
    Task<List<TaskResponse>?> GetByUserIdAsync(int userId);
}
```

User Service Renames

Previous name	New name
`CreateUserAsync`	`CreateAsync`
`UpdateUserAsync`	`UpdateAsync`
`DeleteUserAsync`	`DeleteAsync`

The resulting user service contract follows the same convention:

```
public interface IUserService
{
    Task<UserResponse> CreateAsync(CreateUserRequest request);
    Task<List<UserResponse>> GetAllAsync();
    Task<UserResponse?> GetByIdAsync(int id);
    Task<UserResponse?> UpdateAsync(int id, UpdateUserRequest request);
    Task<bool> DeleteAsync(int id);
}
```

Use Symbol Rename

Use the IDE's symbol rename operation instead of unrestricted text replacement. Symbol-aware renaming updates the interface, implementation, controller calls, and service tests that refer to that exact member.

This distinction matters because an integration test has a private helper named `CreateTaskAsync`. It describes an HTTP testing operation and is not part of `ITaskService`, so it should retain its name.

Update Implementations

Only method declarations change in `TaskService` and `UserService`. Keep every method body unchanged.

Examples:

```
public async Task<TaskResponse> CreateAsync(
    CreateTaskRequest request,
    int userId)

public async Task<TaskResponse?> UpdateAsync(
    int id,
    UpdateTaskRequest request,
    int userId)

public async Task<bool> DeleteAsync(int id, int userId)
public async Task<TaskResponse?> CompleteAsync(int id, int userId)
public async Task<TaskResponse?> ReopenAsync(int id, int userId)
```

The user service follows the same pattern:

```
public async Task<UserResponse> CreateAsync(CreateUserRequest request)
public async Task<UserResponse?> UpdateAsync(int id, UpdateUserRequest request)
public async Task<bool> DeleteAsync(int id)
```

Update Controller Calls

Controller action names and HTTP routes remain unchanged. Only calls to the service contract use the new names.

```
var response = await _taskService.CreateAsync(request, currentUserId.Value);
var updated = await _taskService.UpdateAsync(id, request, currentUserId.Value);
var isDeleted = await _taskService.DeleteAsync(id, currentUserId.Value);
var completed = await _taskService.CompleteAsync(id, currentUserId.Value);
var reopened = await _taskService.ReopenAsync(id, currentUserId.Value);
```

User controller calls become:

```
await _userService.CreateAsync(request);
await _userService.UpdateAsync(id, request);
await _userService.DeleteAsync(id);
```

Update Service Tests

Update service calls and test method names so they describe the renamed methods. For example:

```
CreateTaskAsync_WithValidRequest_CreatesTask
```

becomes:

```
CreateAsync_WithValidRequest_CreatesTask
```

Similarly, update task update, delete, complete, reopen, and user service test names. Leave the private HTTP integration helper named `CreateTaskAsync` because it is not a service method.

Check for Old Names

Search the source and service tests for old task service names:

```
rg "CreateTaskAsync|UpdateTaskAsync|DeleteTaskAsync|CompleteTaskAsync|ReopenTaskAsync" src
tests\JobTrackr.Tests\Tasks
```

Search for old user service names:

```
rg "CreateUserAsync|UpdateUserAsync|DeleteUserAsync" src tests\JobTrackr.Tests\Users
```

Both commands should produce no output. A separate search of the integration-test folder may still find the intentionally unchanged `CreateTaskAsync` helper.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The test count remains unchanged because only names changed.

Common Errors

Interface Member Is Not Implemented

The interface member was renamed but the matching implementation was missed. Rename the declaration in the service implementation.

Method Does Not Exist on the Service

A controller or test still calls an old name. Search for the previous names and update the remaining service call.

Integration Helper Was Accidentally Renamed

Restore the private `CreateTaskAsync` integration helper. Symbol renaming avoids changing unrelated methods that happen to contain the same words.

What Changed

Service methods now use concise names within their service context. All asynchronous operations retain the `Async` suffix, and every interface, implementation, controller, and service-test caller uses the same terminology.

The Release build completed with zero warnings and errors, and all 38 tests passed. API behavior remains unchanged.

Interview Explanation

I standardized asynchronous service naming by removing redundant entity names from methods such as `TaskService.CreateTaskAsync`. The service type already provides that context, so `CreateAsync` is clearer. I used symbol renaming to update contracts and callers safely, then ran the complete test suite to confirm behavior was unchanged.

Exercise

Review another service interface and classify each method name as CRUD-style or use-case-specific. Remove duplicated service context while preserving words that explain meaningful query or business behavior.

Next Lesson

Day 123 reviews DTO folders by feature and moves files only if the current organization presents a real maintainability problem.

Day 123 - Review DTO Folder Organization

Today's Goal

Review the Application project and confirm that request and response DTOs are organized by feature with simple folder names.

This is a cleanup decision lesson. Files should move only when the current layout creates a real navigation, naming, or ownership problem. Extra folders should not be introduced merely because a larger project might use them.

Current Application Layout

The Application project currently uses this structure:

```
JobTrackr.Application
|
+-- Auth
|   +-- AuthResponse.cs
|   +-- ChangePasswordRequest.cs
|   +-- IAuthService.cs
|   +-- IJwtTokenService.cs
|   +-- IPasswordHasherService.cs
|   +-- LoginRequest.cs
|   +-- PasswordHasherService.cs
|   +-- RegisterRequest.cs
|
+-- Common
|   +-- ErrorMessages.cs
|   +-- PagedResponse.cs
|
+-- Tasks
|   +-- CreateTaskRequest.cs
|   +-- ITaskService.cs
|   +-- TaskQueryRequest.cs
|   +-- TaskResponse.cs
|   +-- UpdateTaskRequest.cs
|
+-- Users
    +-- CreateUserRequest.cs
    +-- IUserService.cs
    +-- UpdateUserRequest.cs
    +-- UserResponse.cs
```

Generated `bin` and `obj` directories are build artifacts rather than part of the source design.

What Is a DTO?

DTO means Data Transfer Object. A DTO defines data entering or leaving an application boundary. It is not the database entity itself.

Examples include:

```
CreateTaskRequest -> data accepted when creating a task
UpdateTaskRequest -> data accepted when updating a task
TaskQueryRequest  -> query values accepted by the task list endpoint
TaskResponse      -> task data returned to an API client
```

DTOs protect the public API contract from persistence details. For example, `UserResponse` returns safe profile information without exposing `PasswordHash` from the domain entity.

Review the Auth Feature

The Auth folder contains registration, login, password-change, and authentication response contracts. Its interfaces and password-hashing service also belong to the same feature area at the current project size.

Decision:

```
Keep the Auth folder unchanged.
```

Nested folders such as `Auth\Dtos`, `Auth\Requests`, `Auth\Responses`, `Auth\Services`, and `Auth\Interfaces` would add navigation and namespace overhead without solving a current problem.

Review the Tasks Feature

The Tasks folder contains:

- `CreateTaskRequest` for task creation input
- `UpdateTaskRequest` for task update input
- `TaskQueryRequest` for filtering, searching, sorting, and pagination input
- `TaskResponse` for returned task data
- `ITaskService` for task use cases

Decision:

```
Keep the Tasks folder unchanged.
```

The `Request` and `Response` suffixes already explain the direction and purpose of each contract.

Review the Users Feature

The Users folder contains creation and update requests, the safe user response, and `IUserService`.

Decision:

```
Keep the Users folder unchanged.
```

Keeping the service contract beside the request and response types it consumes makes this small feature easy to scan.

Review the Common Folder

The Common folder contains `ErrorMessages` and the generic `PagedResponse<T>` type. Both are shared across feature areas.

Decision:

```
Keep the Common folder unchanged.
```

Feature-specific request and response types should not move into Common. A shared folder should contain genuinely shared concepts, not become a default location for files that lack a clear owner.

Confirm File and Type Names

Each public type should live in a file with the same name:

```
CreateTaskRequest.cs -> CreateTaskRequest
TaskResponse.cs      -> TaskResponse
IUserService.cs      -> IUserService
AuthResponse.cs      -> AuthResponse
```

Names should also communicate direction:

```
Request  -> the client sends data into the API
Response -> the API sends data to the client
```

Avoid vague names such as `TaskDto`, `TaskModel`, `UserData`, or `RequestModel` when a more specific name describes the contract.

Confirm Namespaces Match Features

Expected namespaces are:

```
namespace JobTrackr.Application.Auth
namespace JobTrackr.Application.Tasks
namespace JobTrackr.Application.Users
namespace JobTrackr.Application.Common
```

Run this command from the repository root:

```
rg "^namespace JobTrackr.Application" src\JobTrackr.Application -g "*.cs"
```

Each source file should use the namespace matching its feature folder.

Confirm Generated Folders Are Not Tracked

Run:

```
git ls-files "src/**/bin/**" "src/**/obj/**"
```

Expected result: no output.

The `bin` directory contains compiled output, while `obj` contains temporary build files. Neither belongs in source control or in a discussion about application organization.

Why We Are Not Adding a Dtos Folder

Both of these structures can work:

```
Application\Dtos\Tasks\CreateTaskRequest.cs
Application\Tasks\CreateTaskRequest.cs
```

JobTrackr already uses the second structure. It is shorter, the feature folder provides sufficient context, and the type suffix explains whether the file is a request or response.

Adding `Dtos`, `Requests`, and `Responses` folders now would require file moves and namespace updates without improving behavior, discoverability, or maintainability.

Final Decision

Keep the existing organization:

```
Application
+-- Auth
+-- Common
+-- Tasks
+-- Users
```

No source file needs to move. The existing structure meets the goal of organizing contracts by feature while keeping folder names simple.

The repository remains clean, namespaces match their folders, generated files are not tracked, and the latest continuous integration run completed successfully with all 38 tests.

Interview Explanation

I reviewed the Application contracts and kept DTOs grouped by feature in Auth, Tasks, and Users. Request and response suffixes already make their direction clear, while Common contains only shared types. I avoided adding nested DTO folders because they would create namespace and navigation overhead without solving a current problem.

Exercise

Review the folder layout of another application project. For each proposed new folder, identify the concrete navigation or ownership problem it solves. Do not add the folder when its only justification is that another, larger codebase uses it.

Next Lesson

Day 124 reviews validation attributes and service checks across create and update requests, then adds only the validation that is genuinely missing.

Day 124 - Complete Basic Request Validation

Today's Goal

Review the existing request DTOs and add only the basic validation attributes that are currently missing.

This lesson adds maximum-length validation to three authentication inputs. Controllers, services, endpoints, packages, database structures, and business behavior remain unchanged.

Why Validate Request DTOs?

Request DTOs receive untrusted client input. Data annotations describe simple input rules that ASP.NET Core can check before a controller action executes.

Because the controllers use `[ApiController]`, an invalid request model automatically produces a `400 Bad Request` response with validation details. This keeps basic shape and boundary checks at the HTTP boundary instead of repeating them inside every controller action.

Existing Validation

The task and user request contracts already contain appropriate basic validation:

```
CreateTaskRequest -> required title, title length, description length, required priority
UpdateTaskRequest -> required title, title length, description length, required priority
CreateUserRequest -> required name and email, length limits, email format
UpdateUserRequest -> required name and email, length limits, email format
RegisterRequest   -> required fields, length limits, email format, password minimum
```

These rules do not need to be duplicated or reorganized.

Improve Login Validation

The login request already requires an email and password and validates the email format. Add maximum lengths so oversized authentication input is rejected before reaching service logic.

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Auth;

public class LoginRequest
{
    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Email is not valid.")]
    [MaxLength(150, ErrorMessage = "Email cannot be longer than 150 characters.")]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Password is required.")]
    [MaxLength(100, ErrorMessage = "Password cannot be longer than 100 characters.")]
    public string Password { get; set; } = string.Empty;
}
```

The email boundary now matches registration. The password maximum prevents unnecessarily large login payloads without changing the login algorithm or password policy.

Improve Change Password Validation

The current password and new password fields already have maximum lengths. Add the same boundary to `ConfirmNewPassword` while retaining the comparison rule.

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Auth;

public class ChangePasswordRequest
{
    [Required(ErrorMessage = "Current password is required.")]
    [MaxLength(100, ErrorMessage = "Current password cannot be longer than 100 characters.")]
    public string CurrentPassword { get; set; } = string.Empty;

    [Required(ErrorMessage = "New password is required.")]
    [MinLength(6, ErrorMessage = "New password must be at least 6 characters.")]
    [MaxLength(100, ErrorMessage = "New password cannot be longer than 100 characters.")]
    public string NewPassword { get; set; } = string.Empty;

    [Required(ErrorMessage = "New password confirmation is required.")]
    [MaxLength(
        100,
        ErrorMessage = "New password confirmation cannot be longer than 100 characters.")]
    [Compare(
        nameof(NewPassword),
        ErrorMessage = "New password and confirmation do not match.")]
    public string ConfirmNewPassword { get; set; } = string.Empty;
}
```

`ConfirmNewPassword` now has the same maximum boundary as the other password fields. `[Compare]` continues to ensure that it matches `NewPassword`.

Validation Boundaries

`[Required]` rejects `null` and empty strings. A value containing only whitespace may still require an application-level rule, depending on how the value is used. Existing service checks already handle important trimmed values such as task titles and user fields.

This lesson stays focused on basic missing attributes. More specific business validation belongs in dedicated lessons:

```
Validation library decision    -> Day 125
Due-date business validation   -> Day 132
Allowed priority validation    -> Day 133
Email consistency and duplicates -> Day 134
Password policy improvements   -> Day 135
```

Keeping these concerns separate prevents a small validation change from becoming several features at once.

Test Through the API

Run the API and submit a login request containing an email longer than 150 characters or a password longer than 100 characters.

```
$baseUrl = "https://localhost:7024"
$tooLongEmail = ("a" * 140) + "@example.com"
$body = @{
    email = $tooLongEmail
    password = "Example123!"
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Uri "$baseUrl/api/auth/login" `
        -Method Post `
        -ContentType "application/json" `
        -Body $body
} catch {
    [int]$_.Exception.Response.StatusCode
    $_.ErrorDetails.Message
}
```

Expected result:

```
400 Bad Request
```

The response should identify the violated maximum-length rule. The controller action and authentication service should not run for an invalid model.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The completed change was verified with a successful Release build and all 38 tests passing.

What Changed

Three missing input boundaries were added:

- login email has a maximum length of 150 characters
- login password has a maximum length of 100 characters
- new-password confirmation has a maximum length of 100 characters

No endpoint contract fields or application workflows changed. Invalid oversized input is now rejected consistently before service logic executes.

Interview Explanation

I reviewed the request DTOs and added missing maximum-length constraints to authentication input. With `[ApiController]`, ASP.NET Core validates these data annotations before the controller action and returns 400 for invalid input. I kept business-specific validation separate for later focused changes.

Exercise

Review each request DTO and classify its validation rules as basic input constraints or business rules. Keep required, format, and reasonable length checks at the request boundary, then document business decisions that need focused application or domain logic.

Next Lesson

Day 125 decides whether built-in DataAnnotations remain sufficient or whether FluentValidation solves a real current need.

Day 125 - Decide Whether JobTrackr Needs FluentValidation

Today's Goal

Decide whether JobTrackr should keep ASP.NET Core's built-in validation or install FluentValidation.

The decision is:

```
Keep DataAnnotations. Do not install FluentValidation yet.
```

This is an engineering decision lesson. No project source code needs to change.

Current Validation Flow

JobTrackr validates requests in two places:

```
Request DTO attributes -> reject invalid HTTP input before controller logic
Service checks         -> protect application behavior outside HTTP requests
```

An example request DTO rule is:

```
[Required(ErrorMessage = "Email is required.")]
[EmailAddress(ErrorMessage = "Email is not valid.")]
[MaxLength(150, ErrorMessage = "Email cannot be longer than 150 characters.")]
public string Email { get; set; } = string.Empty;
```

A service can independently protect an important use-case rule:

```
if (string.IsNullOrWhiteSpace(request.Title))
{
    throw new ArgumentException(ErrorMessages.TaskTitleRequired);
}
```

These checks operate at different boundaries and are not automatically unnecessary duplication.

What ApiController Provides

Each controller uses the `[ApiController]` attribute:

```
[ApiController]
public class TasksController : ControllerBase
```

When request DTO validation fails, ASP.NET Core automatically returns:

```
400 Bad Request
```

The controller action and service method do not run for that invalid HTTP request.

This already supports JobTrackr's current field-level rules:

- required values
- minimum and maximum lengths
- email format
- matching password confirmation
- numeric ranges for pagination

Why Service Checks Still Matter

DataAnnotations are applied automatically by the HTTP pipeline. Application services can also be called directly by:

- automated tests
- a future background process
- another application entry point

Those callers do not pass through `[ApiController]`. Important service checks therefore protect application behavior even when there is no HTTP request.

For example, `TaskService` should still reject a blank title when a unit test or another process calls it directly.

What FluentValidation Would Add

FluentValidation places rules in dedicated validator classes. A rule might look like this:

```
RuleFor(request => request.Email)
    .NotEmpty()
    .EmailAddress()
    .MaximumLength(150);
```

Adopting it would require:

```
1. Installing NuGet packages
2. Creating validator classes
3. Registering validators
4. Choosing how automatic validation runs
5. Maintaining two validation approaches during migration
```

That structure can be valuable when validation becomes complex. It is still additional complexity and should solve a visible problem before entering the project.

Decision Comparison

Question	Current answer
Are most rules simple property checks?	Yes
Are DataAnnotations difficult to understand?	No
Do requests need many conditional rules?	No
Do rules depend on several properties?	Only simple password comparison
Is complex validation duplicated across many DTOs?	No
Are automatic 400 responses working?	Yes
Would a new package simplify the project today?	No

Conclusion:

```
FluentValidation does not solve a real current problem.
```

Do Not Add Infrastructure Without a Need

Do not install FluentValidation or create validator classes merely to follow a popular architecture pattern. The existing request attributes remain the project standard.

Dependencies should enter the solution when their benefit is concrete. Producing more files is not, by itself, an improvement.

When to Reconsider FluentValidation

Revisit this decision if the application later develops rules such as:

- one field is required only when another field has a particular value
- different workflows require different validation rules for the same request
- nested request objects need substantial validation
- the same complex rule is repeated across many request types
- validators require focused unit tests independent of controllers
- DataAnnotations make request DTOs difficult to read

Adding the library later is entirely acceptable. The important point is to adopt it in response to actual complexity rather than anticipated complexity.

Review the Existing Setup

Run these commands from the repository root:

```
rg "\[ApiController\]" src\JobTrackr.Api\Controllers -g "*.cs"
rg "Required|MinLength|MaxLength|EmailAddress|Compare|Range" src\JobTrackr.Application -g "*.cs"
rg "FluentValidation" src tests -g "*.cs" -g "*.csproj"
```

Expected results:

```
All controllers use [ApiController].
Request DTOs show the existing DataAnnotations.
The FluentValidation search returns no output.
```

Confirm Automatic Validation

Run the API and submit an invalid login request:

```
{
  "email": "not-an-email",
  "password": "example"
}
```

Submit it to the login endpoint. The expected result is:

```
400 Bad Request
```

This demonstrates that built-in model validation is active before the controller action executes.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

No commit is required because the correct result of this review is a documented decision rather than a source change. Do not create an empty commit or install a package merely to produce one.

Architecture Decision

Continue using DataAnnotations for simple request-boundary validation and retain important service checks for application behavior. Reconsider a dedicated validation library only when conditional, cross-property, nested, or heavily repeated rules make the current approach difficult to maintain.

Interview Explanation

I evaluated FluentValidation against the project's actual requirements. JobTrackr currently uses simple request rules that DataAnnotations and `[ApiController]` handle clearly, while service checks protect direct service callers. I kept the built-in approach to avoid adding package and registration complexity without a concrete benefit, and documented the conditions that would justify revisiting the decision.

Exercise

Choose three current validation rules and identify which boundary should enforce each one: request DTO, application service, or domain entity. Explain whether a dedicated validator would make those rules easier to understand today.

Next Lesson

Day 126 reviews repeated entity-to-response mapping and centralizes it only when the duplication justifies a helper.

Day 126 - Review Repeated DTO Mapping

Today's Goal

Review entity-to-response mapping across JobTrackr and decide whether any mapping should be centralized.

The decision is:

```
The duplication is real.
Clean it one feature at a time on Days 127-129.
Use small feature-local helpers where they improve clarity.
Do not add AutoMapper or a global generic mapper.
```

This is a review and design lesson. Project source code does not change yet.

What Is Mapping?

Mapping copies safe fields from one object into another object.

```
return new UserResponse
{
    Id = user.Id,
    FullName = user.FullName,
    Email = user.Email,
    CreatedAtUtc = user.CreatedAtUtc
};
```

Here, a database `User` entity becomes a `UserResponse` DTO. This protects the API contract and, most importantly, prevents `PasswordHash` from being returned.

Mapping Inventory

The current services contain these response mappings:

Service	Response	Current pattern
`TaskService`	`TaskResponse`	One query projection and one helper used by several methods
`UserService`	`UserResponse`	Four repeated mapping blocks
`AuthService`	`AuthResponse`	Two similar mapping blocks

The repetition is large enough to address, but each feature has different requirements. A single global abstraction would hide those differences.

Task Mapping Review

`TaskService` already contains an in-memory helper:

```
private static TaskResponse MapToResponse(JobTask task)
```

It is used after an entity has already been loaded or saved by operations such as create, get by ID, update, complete, and reopen.

The paged task query uses a direct EF Core projection:

```
.Select(task => new TaskResponse
{
    // Explicitly mapped response fields
})
```

This is useful because EF Core can translate the expression and request only the response fields from SQL Server.

EF Core Translation Rule

A normal C# helper method is not automatically translatable to SQL. This pattern needs attention:

```
.Select(task => MapToResponse(task))
```

When the source is `IQueryable`, EF Core attempts to translate the helper call into SQL. An arbitrary method call may cause a query translation exception with a relational provider.

Day 127 will clean task mapping while keeping database queries SQL-translatable.

User Mapping Review

`UserService` repeats the same `UserResponse` construction in four operations:

- create
- get all
- get by ID
- update

Each block copies:

```
Id
FullName
Email
CreatedAtUtc
```

This is meaningful duplication. If a safe response field changes, multiple blocks must be updated together.

Day 128 will centralize user mapping while preserving the essential rule that password data never enters `UserResponse`.

Authentication Mapping Review

`AuthService` creates `AuthResponse` in both login and registration workflows.

The shared fields are:

```
UserId
FullName
Email
```

The `Token` value differs between workflows: login returns a generated JWT, while registration currently returns an empty token. A small helper may accept the token as an argument, but that decision belongs to Day 129.

Why Not Build One Global Mapper?

A global mapper for every entity and DTO would combine unrelated feature rules:

- task mapping must work with both EF Core queries and loaded entities
- user mapping must exclude password information
- authentication mapping includes a workflow-specific token

Keeping mapping close to the owning feature makes these decisions visible.

Avoid a generic method such as:

```
Map<TEntity, TResponse>(TEntity entity)
```

It hides which properties are copied without eliminating the actual mapping work.

Why Not Add AutoMapper?

AutoMapper can be valuable in systems containing many repetitive mappings. JobTrackr currently has only three small response types.

Adding it now would introduce:

- another NuGet package
- mapping profile classes
- dependency registration
- additional conventions to learn and test
- extra debugging when a mapping is misconfigured

Small explicit mappings are easier to inspect at the current project size.

Decision:

```
Do not install AutoMapper.
```

Mapping Rules

Use these rules during the focused cleanup lessons:

1. Keep mappings inside the feature that owns them.
2. Use explicit property assignments.
3. Introduce a helper when the same in-memory mapping is repeated.
4. Use an EF-translatable projection for database queries.
5. Never include `PasswordHash` in a response.
6. Do not change endpoint behavior during cleanup.
7. Keep mapping code obvious to a developer reading the service.

Cleanup Plan

```
Day 127 -> Task mapping cleanup
Day 128 -> User mapping cleanup
Day 129 -> Authentication mapping cleanup
```

One focused feature per day keeps the changes small and makes failures easier to understand.

Review Commands

Find direct response construction and existing helpers:

```
rg -n -C 5 "new (TaskResponse|UserResponse|AuthResponse)" src -g "*.cs"
rg -n "MapToResponse" src\JobTrackr.Infrastructure -g "*.cs"
```

Inspect the user and authentication response contracts to confirm they contain no password field:

```
Get-Content src\JobTrackr.Application\Users\UserResponse.cs
Get-Content src\JobTrackr.Application\Auth\AuthResponse.cs
```

Verify the Baseline

Although no source code changes in this lesson, preserve a healthy baseline:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

No commit is required for a review that produces no repository change. Do not create an empty commit merely to mark the day.

Architecture Decision

Use explicit, feature-local mapping. Reuse small helpers for repeated in-memory mappings, and retain explicit EF Core projections when database queries must translate to SQL. Do not introduce AutoMapper or a global generic mapping abstraction at the current scale.

Interview Explanation

I reviewed repeated entity-to-response mapping and found useful cleanup opportunities in the task, user, and authentication services. I chose explicit feature-local mapping instead of adding AutoMapper because the project has only a few small DTOs. I also kept EF Core query projections separate from normal in-memory helper methods because query expressions must be translatable to SQL.

Exercise

Inventory the response mapping in another feature. Separate mappings that operate on loaded objects from projections that operate on `IQueryable`, then choose the smallest readable reuse mechanism for each case.

Next Lesson

Day 127 cleans task entity-to-response mapping while keeping EF Core queries SQL-translatable and easy to read.

Day 127 - Clean Task Response Mapping

Today's Goal

Clean task entity-to-response mapping while keeping EF Core queries readable and SQL-translatable.

The focused change is:

1. Add one reusable query projection helper.
2. Use it in both task list queries.
3. Keep the existing single-entity mapper.
4. Preserve all API behavior.

Only `TaskService` changes. No new file, package, migration, controller, or DTO is required.

Two Mapping Situations

`TaskService` maps tasks in two different execution contexts.

A Loaded Entity

Methods such as create, get by ID, update, complete, and reopen already have a `JobTask` object in memory.

They should continue to use:

```
MapToResponse(task)
```

A Database Query

Methods such as get all and get by user ID still operate on `IQueryable<JobTask>`.

They should use an explicit `Select` projection so EF Core can translate the selected fields into SQL. The reusable helper is named:

```
ProjectToResponses(query)
```

Add the Query Projection Helper

Add a helper that extends the query with an explicit response projection:

```
private static IQueryable<TaskResponse> ProjectToResponses(IQueryable<JobTask> query)
{
    return query.Select(task => new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    });
}
```

The helper does not execute the query. It adds a `Select` expression and returns another `IQueryable`. Execution still happens later when an async terminal operation such as `ToListAsync()` runs.

Use the Projection in the Paged Query

The paged task query previously constructed `TaskResponse` inline. Replace that repeated block with the helper:

```
var taskResponses = await ProjectToResponses(
```

```
        orderedQuery
            .Skip(tasksToSkip)
            .Take(pageSize))
    .ToListAsync();
```

Filtering, sorting, `Skip`, and `Take` still happen before projection. Pagination behavior and query semantics remain unchanged.

Keep the Single-Entity Mapper

Retain the existing mapper for tasks that have already been loaded or saved:

```
private static TaskResponse MapToResponse(JobTask task)
{
    return new TaskResponse
    {
        Id = task.Id,
        Description = task.Description,
        Title = task.Title,
        CreatedAtUtc = task.CreatedAtUtc,
        DueDateUtc = task.DueDateUtc,
        Priority = task.Priority,
        IsCompleted = task.IsCompleted,
        UserId = task.UserId
    };
}
```

The two methods look similar because they map the same fields, but they serve different execution contexts:

```
ProjectToResponses -> builds a translatable database query for multiple tasks
MapToResponse      -> maps one JobTask that is already in memory
```

Update the User Task Query

Replace the custom method call inside `Select` with the shared projection:

```
return await ProjectToResponses(
        _dbContext.Tasks.Where(task => task.UserId == userId))
    .ToListAsync();
```

Both task list queries now use the same explicit SQL-translatable response shape.

Why Not Call MapToResponse Inside Select?

This expression appears convenient:

```
.Select(task => MapToResponse(task))
```

However, `MapToResponse` is an ordinary C# method rather than a SQL expression. EF Core may perform client work in a final projection, while custom methods in other query positions can fail translation entirely. The behavior can also cause entity data to be loaded when only response fields are required.

The projection helper instead builds this standard expression:

```
query.Select(task => new TaskResponse { /* explicit fields */ })
```

EF Core can translate the projection and request only the columns required by `TaskResponse`.

Behavior That Must Not Change

This refactor does not change:

- task response fields
- authenticated ownership filtering
- completion and search filters

- sorting behavior
- pagination
- total count or total pages
- endpoint routes
- HTTP status codes

The change improves internal readability and query shape without altering the API contract.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The completed refactor was verified with a successful Release build and all 38 tests passing.

Test the Endpoints

Run the API and verify:

```
GET /api/tasks
GET /api/tasks?pageNumber=1&pageSize=10&sortBy=createdAt&sortDirection=desc
GET /api/users/{userId}/tasks
```

Every returned task should retain these fields:

```
id
description
title
createdAtUtc
dueDateUtc
priority
isCompleted
userId
```

Review the Final Mapping Calls

Search the task service:

```
rg -n "ProjectToResponses|MapToResponse|new TaskResponse" src\JobTrackr.Infrastructure\Tasks\
TaskService.cs
```

The results should show:

- `ProjectToResponses` used by both list queries
- `MapToResponse` used for individual loaded entities
- explicit readable property assignments in each mapping definition

Interview Explanation

I separated task mapping into two clear paths. Loaded entities use a normal `MapToResponse` method, while database list queries use an `IQueryable` projection that EF Core can translate into SQL. This keeps mapping explicit, avoids a mapping library, and preserves filtering, sorting, and pagination behavior.

Exercise

Inspect an EF Core query that returns a DTO. Identify which operations must remain inside the SQL-translatable expression and which mapping work can safely happen after an entity is loaded.

Next Lesson

Day 128 cleans repeated `UserResponse` mapping while ensuring password data is never returned.

Day 128 - Clean User Response Mapping

Today's Goal

Remove repeated `UserResponse` construction while ensuring password data is never returned by the API.

The focused change is:

```
1. Add a mapper for one loaded User entity.
2. Add an EF Core projection for user list queries.
3. Replace four repeated mapping blocks.
4. Keep PasswordHash and Tasks out of UserResponse.
```

Only `UserService` changes. No DTO, controller, endpoint, package, or database migration is required.

Review the Safe Response Contract

The domain `User` entity contains:

```
Id
FullName
Email
CreatedAtUtc
PasswordHash
Tasks
```

The API `UserResponse` intentionally contains only:

```
Id
FullName
Email
CreatedAtUtc
```

This separation is an important security boundary. `PasswordHash` is sensitive internal data and must never be added to the response contract. The `Tasks` navigation property also stays out of this compact profile response.

Map a Newly Created User

After a user has been saved, the entity is already available in memory. Replace the repeated response construction with:

```
return MapToResponse(user);
```

The single-entity mapper is the correct tool for an object that has already been created or loaded.

Project the User List

The user list starts with an `IQueryable<User>`, so it should use an EF Core projection:

```
public async Task<List<UserResponse>> GetAllAsync()
{
    return await ProjectToResponses(_dbContext.Users)
        .ToListAsync();
}
```

The projection allows EF Core to request only the safe response columns from the database.

Map a User Found by ID

Keep the database lookup and null check unchanged. Once the entity exists in memory, return:

```
if (user is null)
{
```

```
    return null;
}

return MapToResponse(user);
```

Map an Updated User

Keep validation, lookup, property changes, and `SaveChangesAsync` unchanged. Replace only the final repeated response block:

```
return MapToResponse(user);
```

This keeps the refactor limited to response construction.

Add the Mapping Helpers

Add one projection helper for queries and one mapper for loaded entities:

```
private static IQueryable<UserResponse> ProjectToResponses(IQueryable<User> query)
{
    return query.Select(user => new UserResponse
    {
        Id = user.Id,
        FullName = user.FullName,
        Email = user.Email,
        CreatedAtUtc = user.CreatedAtUtc
    });
}

private static UserResponse MapToResponse(User user)
{
    return new UserResponse
    {
        Id = user.Id,
        FullName = user.FullName,
        Email = user.Email,
        CreatedAtUtc = user.CreatedAtUtc
    };
}
```

Neither helper maps:

```
PasswordHash
Tasks
```

The safe field list remains explicit and easy to review.

Why Use Two Helpers?

The methods serve different execution contexts:

```
ProjectToResponses -> builds an EF Core database projection for many users
MapToResponse      -> maps one User entity already loaded in memory
```

`ProjectToResponses` returns `IQueryable<UserResponse>`. EF Core can translate its `Select` expression into SQL and retrieve only the response columns.

`MapToResponse` accepts a normal `User` object after it has already been created, found, or updated.

Although the property assignments are similar, preserving these two paths keeps database translation predictable and in-memory mapping simple.

Behavior That Must Not Change

This cleanup does not change:

- user endpoint routes

- request DTOs
- response fields
- HTTP status codes
- database entities or columns
- create, retrieve, update, or delete behavior
- authentication behavior

It removes repeated mapping code and makes the safe response boundary easier to maintain.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

The completed refactor was verified with a successful Release build and all 38 tests passing.

Test User Responses

Run the API and verify these operations:

```
GET /api/users
GET /api/users/{id}
POST /api/users
PUT /api/users/{id}
```

Every returned user should contain only the safe response properties:

```
{
  "id": 1,
  "fullName": "Example User",
  "email": "user@example.com",
  "createdAtUtc": "2026-09-18T00:00:00Z"
}
```

The exact values vary, but the response must not contain `passwordHash` or `tasks`.

Optional Response Safety Check

Inspect the property names returned by the user list:

```
$baseUrl = "https://localhost:7024"
$users = Invoke-RestMethod -Uri "$baseUrl/api/users" -Method Get
$users[0].PSObject.Properties.Name
```

Expected property names:

```
id
fullName
email
createdAtUtc
```

Review the Final Mapping Calls

Search the service:

```
rg -n "ProjectToResponses|MapToResponse|new UserResponse|PasswordHash" src\JobTrackr.
```

```
Infrastructure\Users\UserService.cs
```

The results should show:

- create, get by ID, and update call `MapToResponse`
- get all calls `ProjectToResponses`
- both helpers explicitly construct `UserResponse`
- `PasswordHash` does not appear in response mapping

Interview Explanation

I centralized user entity-to-response mapping while preserving the difference between EF Core query projection and in-memory object mapping. Both paths explicitly expose only safe profile fields, so the domain entity's password hash and navigation properties cannot leak through the user API response.

Exercise

Review another response contract that maps from an entity containing sensitive or internal fields. Write an explicit safe-field list, then verify both query projections and in-memory mappings expose only those fields.

Next Lesson

Day 129 reviews and cleans `AuthResponse` mapping while keeping the token response minimal.

Day 129 - Clean Auth Response Mapping

Today's Goal

Remove duplicated `AuthResponse` construction in `AuthService` without changing registration or login behavior.

This lesson changes one service file. It does not require a new DTO, controller, package, migration, or endpoint.

Review the Response Contract

`AuthResponse` contains only four properties:

```
UserId
FullName
Email
Token
```

Login and registration repeat the same three user fields. Their only response difference is the token:

```
LoginAsync    -> generated JWT token
RegisterAsync -> empty token
```

The empty registration token is existing behavior. Do not generate a token during registration as part of a mapping cleanup. Never include a password, `PasswordHash`, or the entity's `Tasks` navigation property in this response.

Update Login Mapping

Keep the user lookup, password verification, and token generation unchanged. Replace only the final response construction:

```
var token = _jwtTokenService.GenerateToken(user);

return MapToResponse(user, token);
```

The generated token remains an explicit argument at the call site.

Update Registration Mapping

Keep the duplicate-email check, password hashing, user creation, and save unchanged. Replace only the response construction:

```
return MapToResponse(user, string.Empty);
```

Registration continues returning an empty token. The refactor must not alter when or how a JWT is issued.

Add a Feature-Local Helper

Add this method inside `AuthService`:

```
private static AuthResponse MapToResponse(User user, string token)
{
    return new AuthResponse
    {
        UserId = user.Id,
        FullName = user.FullName,
        Email = user.Email,
        Token = token
    };
}
```

The method accepts a `User` already loaded or saved in memory. It does not run inside an EF Core query, so it does not need the query projection helper used for task and user lists. Its explicit property assignments make the safe response boundary easy to inspect.

Why the Token Is an Argument

The helper owns only the common response shape. The login and registration workflows retain ownership of their token decisions. Passing `token` makes that difference visible:

```
MapToResponse(user, token)         // login
MapToResponse(user, string.Empty)  // registration
```

This is more honest than hiding token generation or an empty-token rule inside a generic mapper.

Verify the Behavior

From the repository root, run:

```
git diff --check
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 38
Failed: 0
Skipped: 0
```

Existing authentication service tests cover both token behaviors: valid login returns a token, while registration returns an empty token.

For a manual HTTP check, use the registration and login endpoints in Swagger. Registration should return the safe user fields and an empty token. Valid login should return those fields and a JWT. Neither response should expose a password or password hash.

Publication and Repository Status

The exact refactor was validated in an isolated copy of the repository: the Release build had no warnings or errors, and all 38 tests passed. The main JobTrackr source was not changed while preparing this lesson. Apply the focused change there and rerun verification before claiming the repository itself contains it.

Once applied and verified, review the diff and commit only the changed service file. The README need not change because the API contract remains the same.

Interview Explanation

I extracted a small auth response mapper because registration and login repeated the same user fields. The token remains an explicit argument, so login still returns a JWT and registration still returns an empty token. The mapper exposes no password data.

Exercise

Compare the login and registration responses. Identify which fields are shared and which value depends on the workflow. Explain why token generation belongs to the workflow rather than the mapping helper.

Next Lesson

Day 130 reviews whether domain entities need real behavior without adding unnecessary domain complexity.

Day 130 - Review Domain Entity Behavior

Today's Goal

Review `JobTask` and `User` and decide whether either entity needs a domain method now. Do not add methods solely to make the Domain project appear more sophisticated.

The result of this review is a deliberate decision to keep both entities unchanged.

When Does Behavior Belong in an Entity?

A domain method is valuable when an entity must protect a real business rule or coordinate a meaningful state change. It can make an invariant explicit and prevent callers from leaving the entity in an invalid state.

Imagine a future rule that a task cannot be reopened after a deadline. A `Reopen()` method could enforce that rule before changing completion state. This is an illustration, not a rule JobTrackr currently has.

A method that merely wraps one assignment adds little value:

```
public void Complete()
{
    IsCompleted = true;
}
```

Moving this line out of a service does not, by itself, create meaningful domain behavior. The method becomes worthwhile only if it owns a rule or transition that callers must respect.

Review JobTask

`JobTask` currently stores:

```
Id, Title, Description, IsCompleted, CreatedAtUtc,
DueDateUtc, Priority, UserId, User
```

`TaskService` handles ownership lookup, persistence, and response mapping. Its complete and reopen operations set `IsCompleted` to `true` or `false` and save the change.

The product has not yet defined whether repeated completion is an error, whether reopening depends on a due date, or whether another state must change at the same time. Without such a rule, adding a domain method would only hide a simple assignment.

Keep `JobTask` unchanged until the transition rules are decided. Day 131 will review them directly.

Review User

`User` stores profile fields, a password hash, and the task relationship. Registration and password changes use a password hasher through `AuthService`.

Do not move password hashing, JWT generation, EF Core calls, or HTTP responses into `User`. These concerns require application or infrastructure services. Pulling them into the entity would introduce the wrong dependencies.

There is no current user-specific invariant that requires a domain method. Keep `User` unchanged.

Architecture Decision

```
No source code changes.
No new domain methods.
No new domain service or abstraction.
```

This is useful engineering work. The review checked actual responsibilities and avoided an abstraction that would only rename a property assignment.

Inspect the Current Code

From the repository root, inspect the entities and the current assignments:

```
Get-Content src\JobTrackr.Domain\Entities\JobTask.cs
Get-Content src\JobTrackr.Domain\Entities\User.cs
rg -n "IsCompleted =|PasswordHash =" src\JobTrackr.Infrastructure -g "*.cs"
git status --short
```

The entities should contain the current properties, and the services should contain the current state assignments and hashing workflow. `git status --short` should be empty when the repository has no unrelated changes.

An optional baseline verification is:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

The preceding lesson's verified build had no warnings or errors and 38 passing tests. Those results establish the recent baseline; they are not a claim that a new Day 130 build was run.

Commit Decision

No repository file changes today, so there is nothing to commit or push. Do not create an empty commit to mark a review day. The README does not need an update because API behavior is unchanged.

Interview Explanation

I reviewed whether the entities needed behavior. The current completion operation is a simple state assignment without an additional business invariant, so I kept it simple. I would move a real rule into the domain once that rule is defined, while keeping persistence, hashing, JWT creation, and HTTP concerns outside the entities.

Exercise

Write down one proposed `JobTask` method and the precise invariant it would enforce. If the method only assigns a property, explain why it can remain in the current service for now. If it protects a real rule, identify tests for both allowed and rejected transitions.

Next Lesson

Day 131 decides the complete and reopen rules for tasks and implements only a rule the product actually needs.

Day 131 - Define Task Completion Rules

Today's Goal

Define clear behavior for completing and reopening tasks, then protect that behavior with automated tests.

Completion Rules

JobTrackr uses idempotent state-setting operations:

```
Complete an open task          -> task becomes completed
Complete a completed task      -> task remains completed
Reopen a completed task        -> task becomes open
Reopen an open task            -> task remains open
Missing or another user's task -> not found through the controller
```

Idempotent means that repeating the same operation produces the same final state rather than creating an error.

The production implementation already follows these rules. No change is required in `JobTask`, `TaskService`, `ITaskService`, or `TasksController`. Focused tests make the repeated-call behavior an explicit contract.

Why Repeated Calls Are Valid

The complete operation asks for a desired state: completed. When the task is already completed, that state has already been reached.

Likewise, reopening an open task already has the requested result. Returning the current task keeps clients simple and avoids unnecessary bad-request or conflict responses.

The endpoints continue to report not found when a task does not exist or belongs to another user. Treating both cases the same preserves task ownership privacy.

Test Repeated Completion

Add a test that begins with an owned task already marked as completed:

```
[Fact]
public async Task CompleteAsync_WithAlreadyCompletedOwnedTask_RemainsCompleted()
{
    await using var dbContext = CreateDbContext();
    var testData = await AddUsersAndTaskAsync(dbContext, isCompleted: true);
    var taskService = new TaskService(dbContext);

    var response = await taskService.CompleteAsync(
        testData.Task.Id,
        testData.Owner.Id);

    Assert.NotNull(response);
    Assert.True(response.IsCompleted);
    Assert.True(testData.Task.IsCompleted);
}
```

The test verifies both the returned DTO and stored entity remain completed.

Test Repeated Reopening

Add a second test that starts with an owned task already open:

```
[Fact]
public async Task ReopenAsync_WithAlreadyOpenOwnedTask_RemainsOpen()
{
    await using var dbContext = CreateDbContext();
```

```
    var testData = await AddUsersAndTaskAsync(dbContext);
    var taskService = new TaskService(dbContext);

    var response = await taskService.ReopenAsync(
        testData.Task.Id,
        testData.Owner.Id);

    Assert.NotNull(response);
    Assert.False(response.IsCompleted);
    Assert.False(testData.Task.IsCompleted);
}
```

The test-data helper defaults `isCompleted` to `false`, so the task is open before `ReopenAsync` executes.

Why Production Code Does Not Change

The existing methods directly set the desired state:

```
task.IsCompleted = true;
```

and:

```
task.IsCompleted = false;
```

These assignments naturally provide idempotent behavior. Adding conditionals or exceptions would make the implementation more complicated without a product requirement.

The tests document the rule and guard against a future refactor accidentally changing it.

Update the Test Count

The two new service tests change the suite totals to:

```
36 service tests
4 API integration tests
40 total tests
```

Update every relevant README statement and command-output example so documentation reports 40 passing tests rather than 38.

Build and Test

Run:

```
git diff --check
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 40
Failed: 0
Skipped: 0
```

Optional API Verification

With a valid bearer token and an owned task ID, call the complete endpoint twice. Both responses should report `isCompleted` as `true`.

Then call the reopen endpoint twice. Both responses should report `isCompleted` as `false`.

This confirms the HTTP behavior matches the service contract while ownership checks remain active.

Publication and Repository Status

The two tests and README count updates were validated in an isolated copy of the repository. The Release build completed with no warnings or errors, and all 40 tests passed there. Apply the focused changes to the main repository and rerun verification before claiming that its current branch contains the Day 131 updates.

Interview Explanation

I defined complete and reopen as idempotent state-setting operations. Repeating either operation returns the same desired state instead of an error. Ownership and not-found behavior remain unchanged. I added focused tests to make that API rule explicit without introducing unnecessary domain methods or production conditionals.

Exercise

Identify another state-setting API operation and specify its behavior when repeated. Decide whether idempotency simplifies the client and write tests for both the first and repeated calls.

Next Lesson

Day 132 defines and implements clear validation rules for task due dates.

Day 132 - Validate Task Due Dates

Today's Goal

Prevent clients from creating or updating a task with a due date that has already passed.

Due-Date Rules

```
DueDateUtc is null      -> valid because a due date is optional
DueDateUtc is future    -> valid
DueDateUtc is now/past  -> invalid and produces a bad request
```

Use UTC values, preferably ISO 8601 timestamps ending in `Z`, because the property is named `DueDateUtc`.

No entity, DTO shape, controller, package, or migration change is required.

Add a Shared Error Message

Add a common message so the service and tests use the same contract:

```
public const string TaskDueDateMustBeFuture =
    "Task due date must be in the future.";
```

Validate Task Creation

In `TaskService.CreateAsync`, validate the due date after required field checks and before creating the entity:

```
if (string.IsNullOrWhiteSpace(request.Priority))
{
    throw new ArgumentException(ErrorMessages.TaskPriorityRequired);
}

ValidateDueDate(request.DueDateUtc);

var task = new JobTask();
```

An invalid request is rejected before a task is added to the database context.

Validate Task Updates

Apply the same validation in `UpdateAsync` after locating the owned task and checking required values, but before changing any entity properties:

```
ValidateDueDate(request.DueDateUtc);
```

The task lookup remains first. A missing task or a task owned by another user still produces the existing not-found behavior without revealing ownership information.

Validating before assignments also prevents a failed update from partially changing the tracked entity.

Add the Validation Helper

Add one private method to `TaskService`:

```
private static void ValidateDueDate(DateTime? dueDateUtc)
{
    if (dueDateUtc.HasValue && dueDateUtc.Value <= DateTime.UtcNow)
    {
        throw new ArgumentException(ErrorMessages.TaskDueDateMustBeFuture);
    }
}
```

`HasValue` allows an omitted due date. A supplied value equal to or earlier than the current UTC time is rejected.

This is service validation rather than a fixed DataAnnotation because the rule depends on the current time and must also protect callers that invoke `TaskService` directly.

Test Invalid Creation

Add a test that submits a date in the past:

```
[Fact]
public async Task CreateAsync_WithPastDueDate_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var user = await AddUserAsync(dbContext);
    var taskService = new TaskService(dbContext);
    var request = new CreateTaskRequest
    {
        Title = "Task with past due date",
        Description = "This task should not be saved",
        DueDateUtc = DateTime.UtcNow.AddDays(-1),
        Priority = "Medium"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => taskService.CreateAsync(request, user.Id));

    Assert.Equal(ErrorMessages.TaskDueDateMustBeFuture, exception.Message);
    Assert.Empty(dbContext.Tasks);
}
```

The final assertion proves the rejected task was not persisted.

Test Invalid Updates

Add a test that attempts to replace an existing due date with a past value:

```
[Fact]
public async Task UpdateAsync_WithPastDueDate_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var task = await AddTaskAsync(dbContext);
    var originalDueDateUtc = task.DueDateUtc;
    var taskService = new TaskService(dbContext);
    var request = new UpdateTaskRequest
    {
        Title = "Updated task title",
        Description = "This update should fail",
        DueDateUtc = DateTime.UtcNow.AddDays(-1),
        Priority = "Medium"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => taskService.UpdateAsync(task.Id, request, task.UserId));

    Assert.Equal(ErrorMessages.TaskDueDateMustBeFuture, exception.Message);

    var savedTask = await dbContext.Tasks.SingleAsync();
    Assert.Equal(originalDueDateUtc, savedTask.DueDateUtc);
}
```

This test verifies that a rejected update preserves the original stored value.

Update Documentation

The two new service tests change the published totals to:

```
38 service tests
4 API integration tests
42 total tests
```

Update all README totals and describe the feature as supporting optional future task due dates. Add task due-date validation to the documented test coverage.

Build and Test

Run:

```
git diff --check
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 42
Failed: 0
Skipped: 0
```

Verify Through the API

With a valid JWT, create or update a task using a past UTC date. The API should return a bad request with the shared due-date message.

Then verify that `null` and a comfortably future UTC value both succeed. Choose a date that is genuinely in the future when running the lesson instead of relying on a permanently fixed example.

Publication and Repository Status

The implementation, tests, and README updates were validated in an isolated copy of the repository. The Release build completed without warnings or errors, and all 42 tests passed there. Apply the focused changes to the main repository and rerun verification before claiming that its current branch contains Day 132.

Interview Explanation

I defined task due dates as optional, but required supplied values to be in the future. I placed the rule in the service so it protects HTTP and direct service callers, centralized the error message, and validated before mutating an entity. Tests verify that invalid creates save nothing and invalid updates preserve existing data.

Exercise

Add test cases for a null due date and a date comfortably in the future. Explain why testing a timestamp extremely close to the current time can produce unreliable tests.

Next Lesson

Day 133 restricts task priority to the supported Low, Medium, and High values and tests invalid input.

Day 133 - Validate Task Priority

Today's Goal

Allow only the three priority values documented by JobTrackr:

```
Low
Medium
High
```

Invalid priorities must produce a bad request without creating or changing a task.

Priority Rules

```
null, empty, or spaces -> "Task priority is required."
Low                    -> valid
Medium                 -> valid
High                   -> valid
anything else          -> "Task priority must be Low, Medium, or High."
```

The values use exact casing. Inputs such as `low` and `URGENT` are invalid. Canonical values keep stored data consistent for future filtering, reporting, and sorting.

No entity, DTO shape, controller, package, or migration change is required.

Add the Error Message

Add a message for supplied but unsupported values:

```
public const string TaskPriorityInvalid =
    "Task priority must be Low, Medium, or High.";
```

Keep the required and invalid messages separate. Missing input and unsupported input are different validation failures.

Validate Priority During Creation

Call the new helper after the existing required check and before due-date validation:

```
if (string.IsNullOrWhiteSpace(request.Priority))
{
    throw new ArgumentException(ErrorMessages.TaskPriorityRequired);
}

ValidatePriority(request.Priority);
ValidateDueDate(request.DueDateUtc);
```

This ordering preserves the existing required message for null, empty, or whitespace input.

Validate Priority During Updates

Use the same sequence in `UpdateAsync` before changing any task property:

```
ValidatePriority(request.Priority);
ValidateDueDate(request.DueDateUtc);
```

An invalid update is rejected before the tracked entity is mutated.

Add the Validation Helper

The allow-list can remain explicit:

```
private static void ValidatePriority(string priority)
```

```
{
    if (priority != "Low" &&
        priority != "Medium" &&
        priority != "High")
    {
        throw new ArgumentException(ErrorMessages.TaskPriorityInvalid);
    }
}
```

With only three supported values, this rule is easy to understand and review. A mapping library, extra package, or generic validation abstraction would add more machinery than value.

Why Keep Priority as a String?

An enum may become useful later, but introducing one would affect entity and DTO types, JSON behavior, EF Core configuration, database compatibility, and migration planning.

Today's requirement is narrower: prevent unsupported strings from entering the system. A small allow-list makes the existing representation safe without expanding the scope.

Test Invalid Creation

Add a test that attempts to create a task with `Urgent`:

```
[Fact]
public async Task CreateAsync_WithInvalidPriority_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var user = await AddUserAsync(dbContext);
    var taskService = new TaskService(dbContext);
    var request = new CreateTaskRequest
    {
        Title = "Task with invalid priority",
        Description = "This task should not be saved",
        Priority = "Urgent"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => taskService.CreateAsync(request, user.Id));

    Assert.Equal(ErrorMessages.TaskPriorityInvalid, exception.Message);
    Assert.Empty(dbContext.Tasks);
}
```

The empty task collection proves invalid input did not create a row.

Test Invalid Updates

Add a test that attempts to replace a valid priority with an unsupported one:

```
[Fact]
public async Task UpdateAsync_WithInvalidPriority_ThrowsArgumentException()
{
    await using var dbContext = CreateDbContext();
    var task = await AddTaskAsync(dbContext);
    var originalPriority = task.Priority;
    var taskService = new TaskService(dbContext);
    var request = new UpdateTaskRequest
    {
        Title = "Updated task title",
        Description = "This update should fail",
        Priority = "Urgent"
    };

    var exception = await Assert.ThrowsAsync<ArgumentException>(
        () => taskService.UpdateAsync(task.Id, request, task.UserId));

    Assert.Equal(ErrorMessages.TaskPriorityInvalid, exception.Message);

    var savedTask = await dbContext.Tasks.SingleAsync();
    Assert.Equal(originalPriority, savedTask.Priority);
```

```
}
```

The final assertion confirms that a rejected update preserves the original priority.

Update Documentation

The two tests change the published totals to:

```
40 service tests
4 API integration tests
44 total tests
```

Update all README totals and add task priority validation to the documented test coverage. The existing feature description already lists Low, Medium, and High, so it does not need another change.

Build and Test

Run:

```
git diff --check
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 44
Failed: 0
Skipped: 0
```

Verify Through the API

With a valid JWT, submit a task containing `Urgent`. The API should return a bad request and the shared invalid-priority message.

Then verify that `Low`, `Medium`, and `High` succeed. Test lowercase `low` as well; it should be rejected so inconsistent casing is never stored.

Publication and Repository Status

The implementation, tests, and README updates were validated in an isolated copy of the repository. The Release build completed with no warnings or errors, and all 44 tests passed there. Apply the focused changes to the main repository and rerun verification before claiming that its current branch contains Day 133.

Interview Explanation

I treated priority as an allow-list: only the canonical Low, Medium, and High strings are accepted. Validation runs before entity creation or mutation, so invalid requests cannot create inconsistent rows or partially update tracked entities. Tests verify both create rejection and update preservation behavior.

Exercise

Add tests proving all three canonical priority values succeed and lowercase variants fail. Explain the tradeoff between strict canonical casing and case-insensitive input normalization.

Next Lesson

Day 134 reviews email format and duplicate-email validation across registration and user create and update workflows.

Day 134 - Validate User Email

Today's Goal

Apply one consistent email rule across registration, login, and legacy user create and update workflows.

Email Rules

```
Email is required.
Email must have a valid format.
Email comparison is case-insensitive.
New emails are stored in lowercase.
Registration and user creation reject an email already in use.
User update rejects another user's email but allows the user's own email.
```

No new package or database migration is required in this lesson.

Create a Shared Email Helper

Add a small helper to the Application Common area:

```
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Common;

public static class EmailValidation
{
    private static readonly EmailAddressAttribute _emailAddressAttribute = new();

    public static bool IsValid(string email)
    {
        return _emailAddressAttribute.IsValid(email);
    }

    public static string Normalize(string email)
    {
        return email.ToLowerInvariant();
    }
}
```

`IsValid` uses the same built-in validator as `[EmailAddress]`. `Normalize` makes identity comparisons consistent across SQL Server and in-memory tests.

Add Shared Messages

Add distinct messages for invalid format and duplicate identity:

```
public const string EmailInvalid = "Email is not valid.";
public const string EmailAlreadyInUse = "Email is already in use.";
```

Also give user request DTOs the same explicit format message already used by authentication requests:

```
[EmailAddress(ErrorMessage = "Email is not valid.")]
```

Normalize Login Lookup

Normalize the supplied email before querying:

```
var normalizedEmail = EmailValidation.Normalize(request.Email);
var user = await _dbContext.Users.FirstOrDefaultAsync(
    user => user.Email.ToLower() == normalizedEmail);
```

Login remains deliberately vague when credentials fail. It should continue returning the safe `Invalid email or password.` response instead of revealing whether an account exists.

Validate Registration

Registration must check required input, format, and case-insensitive duplication before creating the user:

```
if (string.IsNullOrWhiteSpace(request.Email))
{
    throw new ArgumentException(ErrorMessages.UserEmailRequired);
}

if (!EmailValidation.IsValid(request.Email))
{
    throw new ArgumentException(ErrorMessages.EmailInvalid);
}

var normalizedEmail = EmailValidation.Normalize(request.Email);
var emailExists = await _dbContext.Users.AnyAsync(
    user => user.Email.ToLower() == normalizedEmail);

if (emailExists)
{
    throw new ArgumentException(ErrorMessages.EmailAlreadyInUse);
}
```

Store `normalizedEmail` rather than the original casing.

Validate Legacy User Creation

Apply the same format, normalization, and duplicate rules in `UserService.CreateAsync`. Reject invalid input before constructing or adding an entity, then store the normalized value:

```
Email = normalizedEmail
```

This keeps registration and legacy user creation consistent.

Validate User Updates

Validate and normalize the requested email before mutating the user. After finding the target user, check whether another user already owns the normalized address:

```
var emailExists = await _dbContext.Users.AnyAsync(
    existingUser => existingUser.Id != id &&
        existingUser.Email.ToLower() == normalizedEmail);
```

The `existingUser.Id != id` condition is essential. It allows a user to retain their own email while preventing them from adopting another user's address.

Only assign the normalized email after all checks succeed.

Test Invalid Registration

Add an authentication service test using `not-an-email`. Assert the shared invalid-format message and confirm that no user was saved.

Test Duplicate Registration

Register `duplicate@example.com`, then attempt to register `DUPLICATE@example.com`. Assert the duplicate message and confirm only one user exists. The uppercase request proves comparison is case-insensitive.

Test Legacy User Creation

Add equivalent user service tests for:

- invalid email format
- duplicate email with different casing

Both tests should prove rejected requests do not create additional users.

Test User Updates

Create two users, then attempt to update the second user with an uppercase version of the first user's email. Assert the duplicate message and verify the second user's stored email remains unchanged.

This protects against partial mutation when validation fails.

Database Constraint Decision

Service checks protect normal application requests, but they cannot completely prevent two concurrent requests from racing between the duplicate check and insert.

A unique database index is the final concurrency-safe protection. That change belongs to the later database phase because existing email data should be reviewed before introducing the constraint and migration.

Do not confuse application-level duplicate validation with complete database-level uniqueness enforcement.

Update Documentation

Five new tests change the published totals to:

```
45 service tests
4 API integration tests
49 total tests
```

Document email format and case-insensitive duplicate validation in both the feature list and test coverage.

Build and Test

Run:

```
git diff --check
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 49
Failed: 0
Skipped: 0
```

Verify Through the API

Test these scenarios:

1. Register with `not-an-email`; expect a bad request.
2. Register `case@example.com`, then try `CASE@example.com`; expect the duplicate-email message.
3. Log in using different casing; the same account should be found.
4. Create or update an email with uppercase characters; the returned value should be lowercase.

Publication and Repository Status

The implementation, five tests, and README updates were validated in an isolated copy. The Release build completed with no warnings or errors, and all 49 tests passed there. Apply the focused changes to the main repository and rerun verification before claiming that its current branch contains Day 134.

Interview Explanation

I centralized email format checking and lowercase normalization, then applied case-insensitive duplicate detection across registration and user create and update workflows. Validation happens before persistence or mutation, and update checks exclude the current user's own ID. I also documented that a future unique database index is still required for concurrency-safe enforcement.

Exercise

Add tests proving registration stores mixed-case email input in lowercase and login finds that user with different casing. Then explain why normalization and comparison rules should be shared across every account-creation path.

Next Lesson

Day 135 defines a clear, beginner-friendly password policy and tests its validation messages.

Day 135 - Add Password Validation

Today's Goal

Add one clear password policy to registration and password changes without changing login behavior.

Password Rules

```
Password must be at least 8 characters.
Password must contain at least one uppercase letter.
Password must contain at least one lowercase letter.
Password must contain at least one number.
A special character is not required yet.
```

Examples:

```
Password123 -> valid
Pass1       -> invalid because it is too short
password123 -> invalid because it has no uppercase letter
PASSWORD123 -> invalid because it has no lowercase letter
Password    -> invalid because it has no number
```

Do not apply the new policy to login. Login must verify whether the supplied password matches the stored hash. It should not reject an existing password merely because policy requirements changed after the account was created.

No package, migration, or database change is required.

Create a Shared Password Helper

Add a small helper to Application Common:

```
namespace JobTrackr.Application.Common;

public static class PasswordValidation
{
    public const int MinimumLength = 8;

    public static bool HasMinimumLength(string password)
    {
        return password.Length >= MinimumLength;
    }

    public static bool HasBasicComplexity(string password)
    {
        return password.Any(char.IsUpper) &&
            password.Any(char.IsLower) &&
            password.Any(char.IsDigit);
    }
}
```

`MinimumLength` keeps the value eight in one location. `HasBasicComplexity` checks the three required character categories without introducing an unnecessarily difficult policy.

Add Shared Error Messages

Add messages that tell clients how to correct invalid input:

```
public const string PasswordMinimumLength =
    "Password must be at least 8 characters.";
public const string PasswordComplexity =
    "Password must contain an uppercase letter, a lowercase letter, and a number.";
```

Validate Registration Requests

Update the registration password attributes:

```
[Required(ErrorMessage = "Password is required.")]
[MinLength(
    PasswordValidation.MinimumLength,
    ErrorMessage = ErrorMessages.PasswordMinimumLength)]
[RegularExpression(
    @"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).+$",
    ErrorMessage = ErrorMessages.PasswordComplexity)]
[MaxLength(100, ErrorMessage = "Password cannot be longer than 100 characters.")]
public string Password { get; set; } = string.Empty;
```

The regular expression checks for lowercase, uppercase, and numeric characters. It validates the request without storing or transforming the password.

Validate New Password Requests

Apply the same policy to `NewPassword` in the password-change request:

```
[Required(ErrorMessage = "New password is required.")]
[MinLength(
    PasswordValidation.MinimumLength,
    ErrorMessage = ErrorMessages.PasswordMinimumLength)]
[RegularExpression(
    @"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).+$",
    ErrorMessage = ErrorMessages.PasswordComplexity)]
[MaxLength(100, ErrorMessage = "New password cannot be longer than 100 characters.")]
public string NewPassword { get; set; } = string.Empty;
```

Keep `CurrentPassword` unchanged. Keep the comparison attribute on `ConfirmNewPassword` so confirmation must still equal the new password.

Enforce the Policy in AuthService

DTO attributes protect HTTP requests, but important application rules should also protect tests and other direct service callers.

At the start of registration, call:

```
ValidatePassword(request.Password);
```

During password changes, retain the confirmation check and then validate the proposed password:

```
if (request.NewPassword != request.ConfirmNewPassword)
{
    throw new ArgumentException(ErrorMessages.NewPasswordMismatch);
}

ValidatePassword(request.NewPassword);
```

Add the shared service helper:

```
private static void ValidatePassword(string password)
{
    if (!PasswordValidation.HasMinimumLength(password))
    {
        throw new ArgumentException(ErrorMessages.PasswordMinimumLength);
    }

    if (!PasswordValidation.HasBasicComplexity(password))
    {
        throw new ArgumentException(ErrorMessages.PasswordComplexity);
    }
}
```

Keep current-password verification, confirmation matching, and same-password rejection. A weak new password must never replace the stored password hash.

Test Short Registration Passwords

Add a registration test using `Pass1`. Assert the minimum-length message and verify that no user was saved.

Test Missing Complexity

Add another registration test using `password`. Assert the complexity message and again verify that the database contains no user.

These tests prove that invalid registration data does not create partial authentication records.

Test Weak Password Changes

Register a valid user and preserve the original hash. Attempt to change the password to `newpassword`, then assert:

- the complexity message is returned
- the stored password hash remains unchanged

The hash assertion is important. It proves rejected credentials do not alter authentication data.

Update Documentation

Document the policy clearly:

```
Registration and new passwords require at least eight characters,
including uppercase, lowercase, and numeric characters.
```

The three new tests change the published totals to:

```
48 service tests
4 API integration tests
52 total tests
```

Add password length and basic complexity validation to the test coverage list.

Build and Test

Run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

Expected result:

```
Build: 0 warnings, 0 errors
Passed: 52
Failed: 0
Skipped: 0
```

Verify Through the API

Test registration with:

```
Pass1       -> minimum-length error
password123 -> complexity error
Password123 -> success
```

Then authenticate and attempt to change the password to `newpassword`. The API should return the complexity error. A valid new password such as `NewPassword456` should continue to work.

Publication and Repository Status

The implementation, three tests, and README updates were validated in an isolated copy. The Release build completed with no warnings or errors, and all 52 tests passed there. Apply the focused changes to the main repository and rerun verification before claiming that its current branch contains Day 135.

Interview Explanation

I introduced one shared password policy for registration and password changes: at least eight characters with uppercase, lowercase, and numeric characters. Request attributes provide immediate HTTP validation, while service checks protect direct callers. Login remains unchanged so existing credentials are verified against their stored hashes rather than re-evaluated against a newer policy.

Exercise

Add separate tests for a password missing lowercase characters and one missing a number. Verify that each failure leaves the database or stored hash unchanged.

Next Lesson

Day 136 reviews validation, not-found, and authentication error response shapes for consistency.

Day 136 - Review Error Response Shapes

Today's Goal

Confirm that JobTrackr returns predictable and safe responses for:

- request validation failures
- service validation failures
- missing resources
- invalid login credentials
- missing or invalid JWT tokens
- unexpected server errors

This is a review and testing lesson. Do not redesign error handling or introduce a new result pattern when the existing responses match the intended contract.

Current Response Contract

Situation	Status	Response body
Invalid DTO fields	`400 Bad Request`	`ValidationProblemDetails` with an `errors` object
Service validation failure	`400 Bad Request`	`ProblemDetails` with a safe `detail` message
Missing task or user	`404 Not Found`	`ProblemDetails` with a shared not-found message
Invalid login credentials	`400 Bad Request`	`ProblemDetails` with `Invalid email or password.`
Missing, invalid, or expired JWT	`401 Unauthorized`	Usually an empty body from JWT middleware
Unexpected exception	`500 Internal Server Error`	`ProblemDetails` with a safe generic message

The validation shapes differ intentionally. Data annotations can report several field errors in one request, while a service rule or missing resource normally describes one failure. JWT authentication runs before controller actions and therefore controls its own unauthorized response.

Automatic Field Validation

Submit an invalid registration request containing an empty name, malformed email, and weak password. Because the controller uses `[ApiController]`, ASP.NET Core should return `400 Bad Request` before the action executes.

The response should contain:

```
status = 400
title explaining validation failure
errors grouped by field name
one or more useful messages for each invalid field
no stack trace or internal exception details
```

The exact standards URL and request trace identifier can vary.

Service Validation

Register a valid user, then attempt registration again using the same email with different casing. The duplicate check should return a bad-request `ProblemDetails` response with:

```
{
  "title": "Bad Request",
  "status": 400,
  "detail": "Email is already in use."
}
```

This response has `detail` rather than an `errors` dictionary because duplicate identity is a service rule rather than a request-attribute failure.

Not-Found Responses

With a valid JWT, request a task ID that does not exist. The API should return `404 Not Found` and the shared task-not-found message.

The equivalent missing-user request should use the same structure with the shared user-not-found message. Centralized messages prevent small wording differences between controllers.

Ownership-protected task lookups should preserve the same not-found behavior for missing tasks and tasks owned by another user.

Invalid Login Credentials

Attempt login with an unknown email or an incorrect password. Both cases should return the same safe message:

```
Invalid email or password.
```

Using one response avoids revealing which email addresses have accounts.

JWT Authentication Failures

Call a protected task endpoint without a bearer token. Repeat with a changed or expired token.

Each request should return:

```
401 Unauthorized
```

The response may have an empty body because JWT middleware rejects the request before the controller runs. Do not force this response into the controller `ProblemDetails` shape unless a real client requirement justifies custom JWT event handling.

Unexpected Error Safety

The exception middleware should convert an unhandled exception into a safe response:

```
{
  "title": "Internal Server Error",
  "status": 500,
  "detail": "An unexpected error occurred."
}
```

The server logs the real exception. The client must not receive:

- stack traces
- SQL statements or connection strings
- JWT signing keys

- internal file paths
- raw exception messages

Reviewing the middleware is sufficient for this lesson. Do not deliberately break production code merely to trigger a 500 response.

Automated Verification

Run the existing test suite:

```
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

The expected baseline is:

```
Passed: 52
Failed: 0
Skipped: 0
```

Architecture Decision

Keep the current response behavior:

```
ValidationProblemDetails for field validation
ProblemDetails for service validation and not-found responses
JWT middleware responses for unauthorized requests
Safe ProblemDetails for unexpected server errors
```

Do not add another exception-handling package, custom validation middleware, a custom API error DTO, or a result pattern during this review.

Commit Decision

When the responses behave as documented, there is no project code change and nothing to commit. A clean review day does not require an empty commit.

Interview Explanation

I reviewed the API's failure responses and kept the existing behavior. Field validation uses `ValidationProblemDetails` because several fields can fail together. Service and not-found failures use `ProblemDetails` with safe messages. JWT middleware owns unauthorized responses, and exception middleware prevents internal details from leaking in server errors.

Exercise

Create a small response matrix for one API feature. Include invalid fields, a failed service rule, a missing resource, unauthorized access, and an unexpected failure. Identify which component creates each response.

Next Lesson

Day 137 decides whether JobTrackr needs a common service result pattern or whether the current approach should remain unchanged.

Day 137 - Decide Whether Services Need a Result Pattern

Today's Goal

Review how services communicate success and failure, then decide whether JobTrackr needs a common result object.

This is a design decision lesson. No package, migration, or application code change is required.

Current Service Outcomes

Service outcome	Current return behavior	API behavior
Successful create or update	Response DTO	Successful HTTP response
Task or user not found	`null`	`404` with `ProblemDetails`
Successful delete	`true`	`204 No Content`
Missing task during delete	`false`	`404` with `ProblemDetails`
Known invalid input	`ArgumentException`	Controller returns `400` with safe detail
Unexpected exception	Exception reaches middleware	Safe `500` response

JWT authentication and automatic DTO validation occur in the API pipeline. They do not need a service result object.

Review an Existing Workflow

Follow the task get-by-ID workflow from `TaskService` into `TasksController`.

The service returns a `TaskResponse` when the authenticated user owns the task. It returns `null` when the task is missing or belongs to another user.

The controller translates that application outcome into HTTP:

```
var response = await _taskService.GetByIdAsync(id, currentUserId.Value);

if (response is null)
{
    return Problem(
        statusCode: StatusCodes.Status404NotFound,
        title: "Not Found",
        detail: ErrorMessages.TaskNotFound);
}

return Ok(response);
```

This workflow has two clear outcomes. A common result class would add ceremony without improving the decision.

What Is a Result Object?

A result object explicitly carries either successful data or a known failure.

Conceptually:

```
Result<TaskResponse>
```

```
    Success: true
    Value: the task
    ErrorCode: none

Result<TaskResponse>
    Success: false
    Value: none
    ErrorCode: TaskNotFound
```

The service would return the expected failure rather than throw an exception for that case. The controller would still map the outcome to an HTTP status code.

`Result<T>` is an application convention or library type. It is not a replacement for asynchronous `Task<T>`:

```
Task<T>         -> describes an asynchronous operation
Result<T>       -> describes a success or failure outcome
Task<Result<T>> -> describes both together
```

Compare the Tradeoffs

Current approach	Possible result approach
Already understood and tested	Requires a new shared contract
`null` and `bool` cover simple missing cases	Can distinguish several expected failures
Known validation uses exceptions	Expected failures can be returned explicitly
Controllers repeat some catch blocks	Controllers still need outcome-to-HTTP mapping
No additional abstraction	Adds result types, error codes, and tests

Using exceptions for expected validation is a tradeoff. It can become awkward as workflows grow, but adding a result object does not automatically remove controller repetition or improve every endpoint.

Review Questions

For each task, user, and authentication service method, ask:

1. Can success and failure be understood from the method's return type and implementation?
2. Are known errors converted into safe responses by the controller?
3. Does the workflow have several distinct expected failures that `null` or `bool` cannot explain?
4. Is current error handling causing bugs or making changes difficult?

The reviewed workflows remain understandable. Repeated `Problem(...)` calls and catch blocks alone do not justify changing every service interface.

Architecture Decision

```
Decision: Keep the current service return types for now.
```

Reasons:

- DTOs, `null`, `bool`, and existing validation behavior clearly describe current workflows.
- `ProblemDetails` responses already provide useful client errors.
- A result type would require changes across interfaces, implementations, controllers, and tests.
- No demonstrated problem currently justifies that cost.

Reconsider the decision when:

- one workflow has several expected failures that callers must distinguish

- exception-based validation becomes difficult to maintain
- non-HTTP callers need structured error codes without parsing exception messages

If that point arrives, introduce a result object in one use case first. Keep HTTP status-code decisions in the API layer and leave unexpected exceptions to middleware.

Verification and Git Decision

The preceding review baseline had a successful Release build and 52 passing tests. Day 137 changes documentation and design understanding only, so another build is unnecessary unless application code changes.

Check repository status. When no project files changed, do not create an empty commit or push merely to mark the lesson.

Interview Explanation

I evaluated a common result pattern against the project's actual workflows. The current response DTOs, nullable returns, booleans, and validation handling still make outcomes clear. A shared result type would require broad contract changes without solving a demonstrated problem, so I documented the conditions that would justify introducing one later.

Exercise

Choose one service method and list every expected outcome. Decide whether its current return type communicates those outcomes clearly. If not, sketch the smallest result type that would improve only that use case.

Next Lesson

Day 138 confirms the result-pattern decision against one real workflow and retains the existing code unless a concrete need appears.

Day 138 - Confirm the Result Pattern Decision

Today's Goal

Follow one real workflow from interface to service to controller and confirm whether it needs a result object.

Day 137 recommended keeping the current service return types. The roadmap says to implement a result pattern only when it solves a demonstrated problem. This lesson applies that condition to task deletion.

Review the Interface

The task service contract contains:

```
Task<bool> DeleteAsync(int id, int userId);
```

`Task` represents asynchronous execution. The `bool` communicates whether a task owned by the authenticated user was found and deleted.

Follow the Service

The existing implementation queries by task ID and authenticated user ID together:

```
public async Task<bool> DeleteAsync(int id, int userId)
{
    var task = await _dbContext.Tasks
        .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);

    if (task is null)
    {
        return false;
    }

    _dbContext.Tasks.Remove(task);
    await _dbContext.SaveChangesAsync();

    return true;
}
```

A missing task and a task owned by another user deliberately produce the same result. This preserves ownership privacy.

The method returns `true` only after the deletion is saved successfully. An unexpected database failure must throw rather than return `false`; otherwise, the application would incorrectly present a server failure as a missing task.

Follow the Controller

The controller converts the service outcome into HTTP:

```
var isDeleted = await _taskService.DeleteAsync(id, currentUserId.Value);

if (!isDeleted)
{
    return Problem(
        statusCode: StatusCodes.Status404NotFound,
        title: "Not Found",
        detail: ErrorMessages.TaskNotFound);
}

return NoContent();
```

The service owns lookup and deletion. The controller owns the HTTP response.

Situation	Service behavior	API response
Owned task exists	Saves deletion and returns `true`	`204 No Content`
Task is missing	Returns `false`	`404 Not Found`
Task belongs to another user	Returns `false`	`404 Not Found`
No valid JWT	Service is not called	`401 Unauthorized`
Unexpected database failure	Throws	Middleware returns a safe `500`

Review Existing Tests

The delete service tests cover:

```
DeleteAsync_WithExistingOwnedTask_DeletesTask
DeleteAsync_WithMissingTask_ReturnsFalse
```

The first verifies that the task is removed while its user remains. The second verifies that a missing task returns `false`.

These service tests do not, by themselves, prove every ownership or HTTP response path. Manual API checks remain useful for this review.

Verify the Workflow Through the API

Use disposable tasks and perform these steps:

1. Register or log in as User A and authorize the API client.
2. Create a task and record its ID.
3. Delete it and expect `204 No Content` with an empty body.
4. Delete the same ID again and expect `404 Not Found`.
5. Create another task as User A.
6. Log in as User B and attempt to delete User A's task; expect `404`.
7. Switch back to User A and retrieve the task; it should still exist.
8. Remove authorization and attempt deletion; expect `401 Unauthorized`.

These checks verify deletion, repeat deletion, ownership privacy, and authentication boundaries.

Architecture Decision

```
Keep Task<bool> for task deletion.
```

The use case has two expected service outcomes: deleted or not found for this user. A result object would carry the same information without improving the workflow today.

Do not add a `Result<T>` class, package, interface change, or migration.

Reconsider the decision if a future business rule adds a distinct expected outcome, such as deletion being blocked for a specific reason. Evaluate that requirement when it exists.

Verification and Git Decision

This lesson changes no application code. The last verified baseline contains 52 passing tests, but those tests were not rerun while preparing Day 138.

To run the focused existing tests:

```
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release --filter
"FullyQualifiedName~TaskServiceDeleteTests"
```

The expected result is two passing tests.

When the repository has no project changes, do not create an empty commit or push merely to mark the review day.

Interview Explanation

I traced task deletion through its interface, implementation, controller, and tests. The boolean return clearly represents the two expected service outcomes, while the controller maps them to 204 or 404. Authentication remains in the API pipeline, and unexpected persistence failures remain exceptions handled by middleware. A result type would not improve this workflow today.

Exercise

Trace another boolean service operation into its controller. List its expected outcomes and verify that unexpected infrastructure failures cannot be mistaken for an ordinary `false` result.

Next Lesson

Day 139 reviews controller thinness and confirms that controllers handle HTTP concerns while delegating use-case logic to services.

Day 139 - Review Controller Thinness

Today's Goal

Review `TasksController`, `UsersController`, and `AuthController`. Confirm that they receive requests, call services, and translate outcomes into HTTP responses.

The current controllers already follow this boundary. This is a code-reading lesson; no application edit is required unless the review finds misplaced business logic.

Responsibility Boundaries

Responsibility	Appropriate owner
Route, HTTP method, and request binding	Controller and API framework
Read authenticated user ID from claims	API or controller
Choose `200`, `201`, `204`, `400`, or `404`	Controller
Field format and length checks	Request DTO attributes
Check task ownership and apply task rules	Service
Check duplicate email and password policy	Service with Application helpers
Query EF Core, create entities, and save changes	Infrastructure service
Hash passwords and generate tokens	Dedicated services called by `AuthService`
Handle unexpected exceptions	Exception middleware

A thin controller can contain `if` statements and `try/catch` blocks. Judge whether its responsibilities belong at the HTTP boundary rather than judging only its line count.

Trace Task Creation

The task creation action follows this shape:

```
[HttpPost]
public async Task<IActionResult> Create(CreateTaskRequest request)
{
    try
    {
        var currentUserId = GetCurrentUserId();

        if (currentUserId is null)
        {
            return Unauthorized();
        }

        var response = await _taskService.CreateAsync(request, currentUserId.Value);

        return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
    }
    catch (ArgumentException ex)
    {
        return Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Bad Request",
            detail: ex.Message);
    }
}
```

It performs four HTTP-facing jobs:

1. Receives a request DTO from the body.
2. Reads the authenticated user ID and rejects an unusable identity.
3. Calls the task service for that user.
4. Returns `201 Created` with a location, or maps a known validation failure to `400 Bad Request`.

The controller's authorization attribute normally rejects anonymous requests before the action runs. The null check additionally handles a principal whose user-ID claim cannot be read.

Follow the Service Call

`TaskService.CreateAsync` performs the use-case work:

1. Validates the supplied user ID and confirms that the user exists.
2. Applies task title, priority, and due-date rules.
3. Creates a `JobTask` with its owner and initial completion state.
4. Adds and saves the entity through `AppDbContext`.
5. Maps the saved entity to `TaskResponse`.

These operations belong in the service. The controller should not use `AppDbContext` or repeat task validation rules.

The service receives an integer user ID rather than reading `HttpContext`. This keeps it independent of HTTP and lets service tests invoke the use case directly.

Review All Controllers

Confirm these boundaries:

Controller	Expected responsibility
`TasksController`	Passes the current user ID and request values to `ITaskService`, then returns HTTP results
`UsersController`	Calls user or task services and maps missing records and known validation failures
`AuthController`	Calls `IAuthService`, returns authentication responses, and maps known errors

Controllers should not directly:

- query or save an EF Core `DbSet`
- construct entities for persistence
- hash or verify passwords
- generate JWTs
- implement task ownership queries
- build sorting or pagination queries

The inspected controllers contain none of these operations.

Security Is a Separate Question

Existing authorization limitations on legacy user routes remain a separate concern. Thin controllers prove that responsibilities are separated; they do not prove that every endpoint has the correct authorization policy.

Architecture and security reviews should inform each other without being treated as the same check.

Refactoring Decision

Record the current decision:

```
Controllers handle request binding, authenticated claims, service calls, and HTTP responses.
Use-case rules and database access remain in services.
No business logic needs moving during this review.
```

Repeated claim-reading helpers or `ProblemDetails` blocks can be reconsidered if they become difficult to maintain. Their presence does not justify a base controller or shared result abstraction today.

Interview Explanation

My controllers receive HTTP requests and call application service interfaces. They read the authenticated user ID and translate service outcomes into HTTP responses. Services handle task rules, ownership queries, database changes, and response mapping. This keeps persistence out of controllers and allows use cases to be tested directly.

Verification and Git Decision

No application code changed and no tests were rerun while preparing this lesson. The last verified baseline contains 52 passing tests.

When the review only reads code, there is nothing to commit. If actual business logic is found and moved, build and test that change before publishing it:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

Exercise

Choose one controller action and classify every line as request binding, identity handling, service delegation, or HTTP response mapping. Flag any line that performs persistence or business-rule work.

Next Lesson

Day 140 reviews service size and splits a service only if its responsibilities genuinely justify it.

Day 140 - Review Service Size

Today's Goal

Review `TaskService`, `UserService`, and `AuthService` and decide whether their responsibilities justify splitting a service.

The inspected services remain understandable. The decision is to retain their current structure and document the evidence that would justify a future extraction.

Describe Each Service

Service	Current responsibility
`TaskService`	Creates, queries, updates, deletes, completes, and reopens tasks with ownership rules
`UserService`	Manages user records and profile data, including email validation
`AuthService`	Coordinates registration, login, and password changes

`TaskService` is the longest because task querying includes filtering, sorting, pagination, ownership, and response mapping. File length alone is not a reason to split a class.

Walk Through the Largest Query

`TaskService.GetAllAsync` performs these steps in order:

```
Restrict the query to the authenticated owner.
Apply optional completion and title filters.
Count all matching tasks before pagination.
Apply stable sorting.
Skip earlier pages and take the requested page.
Project selected tasks into response DTOs.
Return items and pagination metadata.
```

Ownership establishes the first boundary:

```
var query = _dbContext.Tasks.Where(task => task.UserId == userId);
```

Counting and page selection have separate purposes:

```
var totalCount = await query.CountAsync();

var tasksToSkip = (pageNumber - 1) * pageSize;
var orderedQuery = ApplySorting(query, sortBy, sortDirection);

var taskResponses = await ProjectToResponses(
        orderedQuery
            .Skip(tasksToSkip)
            .Take(pageSize))
    .ToListAsync();
```

`totalCount` describes every task matching the owner's filters. The returned `items` describe only the requested page, so counting must occur before `Skip` and `Take`.

Review Existing Helpers

`TaskService` already gives distinct operations clear names:

Helper	Purpose
`ApplySorting`	Orders tasks by creation or due date with stable ID tie-breaking
`ProjectToResponses`	Builds an EF query projection into `TaskResponse`
`MapToResponse`	Maps one loaded entity into `TaskResponse`
`ValidatePriority`	Checks supported task priorities
`ValidateDueDate`	Rejects a supplied due date that is not in the future

Projection and single-entity mapping look similar but execute in different contexts. Keeping both remains appropriate for this project.

Repeated owner lookups are recognizable. Extract them only when a helper would make maintenance easier without obscuring operation-specific behavior.

Review AuthService Dependencies

`AuthService` receives:

```
AppDbContext dbContext,
IPasswordHasherService passwordHasherService,
IJwtTokenService jwtTokenService
```

It coordinates authentication use cases while dedicated services perform specialized cryptographic work:

```
var passwordIsValid = _passwordHasherService.VerifyPassword(
    request.Password,
    user.PasswordHash);

var token = _jwtTokenService.GenerateToken(user);
```

Registration, login, and password changes still form one coherent authentication responsibility. Hashing and token generation already have separate implementations where that separation matters.

Decide Whether Splitting Helps

Ask these questions before extracting a service:

1. Can its purpose be explained clearly?
2. Can a use case be found and understood without navigating many unrelated classes?
3. Are its dependencies relevant to that purpose?
4. Are unrelated features causing frequent changes to the same class?
5. Would extraction give a meaningful responsibility a clear owner?

The current answers support keeping the services together:

```
Keep TaskService, UserService, and AuthService as they are.
Their methods remain related to their features.
Existing helpers keep longer operations readable.
No new interface, repository layer, query service, package, or migration is required.
```

When to Reconsider

An extraction becomes useful when evidence appears:

- unrelated responsibilities accumulate
- query logic becomes difficult to navigate
- several callers need the same substantial query behavior

- a distinct workflow needs independent dependencies and tests
- changes in one area repeatedly disturb another

A future task-query service may become reasonable if filtering and reporting grow substantially. Record that possibility without creating the abstraction today.

Practical Review

Write and explain these statements in your own words:

```
TaskService stays together because ...
AuthService delegates hashing and token creation because ...
I would split a service when ...
```

Then explain `GetAllAsync`, including why ownership and filtering occur before counting and pagination.

Verification and Git Decision

No application code changed and no tests were run while preparing this lesson. The last verified baseline contains 52 passing tests.

When the work is only a review, no commit is required. If a justified refactor is made, verify it before publishing:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

Interview Explanation

I reviewed service cohesion rather than splitting classes based on line count. Task, user, and authentication methods remain grouped by clear feature responsibilities. Existing helpers separate validation, sorting, and mapping, while password hashing and token generation already use dedicated services. I would extract a new service only when a distinct responsibility, dependency set, or reusable workflow provides concrete evidence.

Exercise

Choose the longest service in another project and list its responsibilities, dependencies, and helpers. Propose an extraction only if the new class would own a coherent responsibility rather than merely reducing line count.

Next Lesson

Day 141 plans the task-comments feature, including its entity, relationship, request and response DTOs, endpoints, and ownership rules.

Day 141 - Plan Task Comments

Today's Goal

Define the smallest useful task-comments feature before implementing it across Days 142-146.

A comment is a short note attached to one task, such as `Waiting for the recruiter to reply.` Only the authenticated task owner can create, read, or delete its comments.

This is a planning lesson. The code samples describe the future contract; application files are not created yet.

Define the Scope

The first version supports:

- create a comment on an owned task
- list comments for an owned task
- delete a comment from an owned task

Editing, attachments, replies, mentions, shared tasks, and notifications remain outside the first version. Completed tasks may still receive comments because the feature adds no completion-state restriction.

Plan the Entity

The proposed entity is:

```
public class TaskComment
{
    public int Id { get; set; }
    public string Content { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
    public int TaskId { get; set; }
    public JobTask? Task { get; set; }
}
```

Field	Purpose
`Id`	Identifies the comment
`Content`	Stores the note text
`CreatedAtUtc`	Records creation time in UTC
`TaskId`	Required foreign key to `JobTask`
`Task`	Navigation property for the related task

Add a `List<TaskComment> Comments` navigation property to `JobTask` during Day 142. One task has many comments, and every comment belongs to exactly one task.

Ownership is derived through `Task.UserId`. This private-note feature does not need a separate comment `UserId` or another foreign key to `User`. If shared tasks become a future requirement, author identity will need a separate design.

Plan Persistence

The persistence model will include:

```
DbSet<TaskComment> TaskComments
```

```
TaskComment.TaskId -> JobTask.Id
Content is required with a maximum length of 1000
Deleting a task deletes its comments through cascade delete
```

There is one new foreign-key path from tasks to comments. Avoid a second User-to-Comment cascade path.

Create and inspect the EF Core migration during Day 142 before applying it. There is no migration command to run during planning.

Plan Request and Response DTOs

The create request contains only editable content:

```
public class CreateTaskCommentRequest
{
    [Required(ErrorMessage = "Comment content is required.")]
    [MaxLength(1000, ErrorMessage = "Comment content cannot exceed 1000 characters.")]
    public string Content { get; set; } = string.Empty;
}
```

The response exposes the stored comment contract:

```
public class TaskCommentResponse
{
    public int Id { get; set; }
    public int TaskId { get; set; }
    public string Content { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; }
}
```

`TaskId` comes from the route, the authenticated user ID comes from JWT claims, and the server supplies `Id` and `CreatedAtUtc`. The API returns a response DTO rather than exposing the entity or navigation properties.

Define Endpoints

All endpoints require a valid JWT:

Method	Route	Success response
`POST`	`/api/tasks/{taskId}/comments`	`201 Created` with the comment DTO
`GET`	`/api/tasks/{taskId}/comments`	`200 OK` with a list of comment DTOs
`DELETE`	`/api/tasks/{taskId}/comments/{commentId}`	`204 No Content`

Example create request:

```
{
  "content": "Waiting for the recruiter to reply."
}
```

Example response:

```
{
  "id": 1,
  "taskId": 12,
  "content": "Waiting for the recruiter to reply.",
  "createdAtUtc": "2026-10-23T10:00:00Z"
}
```

The creation location header can point to the task's comment collection route using an explicit `taskId` route value. A single-comment GET endpoint is not planned for this phase.

List comments oldest first by `CreatedAtUtc`, then `Id` for stable ordering. The first version returns a simple list; add pagination only if future comment volume requires it.

Define Validation and Ownership

Situation	Expected API behavior
No valid JWT	`401 Unauthorized`
Task missing or owned by another user	`404` with `Task not found.`
Owned task has no comments	`200` with `[]`
Missing or whitespace-only content	`400` with a useful error
Content longer than 1000 characters	`400` with a useful error
Delete target missing or unrelated to route task	`404` with `Comment not found.`
Owned task and matching comment exist	`204 No Content`

Check ownership before listing or changing comments:

```
task.Id == taskId && task.UserId == userId
```

For deletion, also require:

```
comment.Id == commentId && comment.TaskId == taskId
```

Never delete by `commentId` alone. The route task, comment relationship, and authenticated owner must all match.

DTO validation handles required and length checks at the HTTP boundary. The service also rejects whitespace-only and oversized content so direct callers cannot bypass the rules. Validate the supplied length before trimming, then store trimmed content.

Plan the Service Boundary

The proposed interface is:

```
public interface ITaskCommentService
{
    Task<TaskCommentResponse?> CreateAsync(
        int taskId,
        CreateTaskCommentRequest request,
        int userId);

    Task<List<TaskCommentResponse>?> GetAllAsync(int taskId, int userId);

    Task<bool> DeleteAsync(int taskId, int commentId, int userId);
}
```

Use existing result conventions:

- create returns `null` for a missing or unowned task and throws `ArgumentException` for invalid content
- get all returns `null` for a missing or unowned task and an empty list for an owned task without comments
- delete returns `false` when no owned matching target can be deleted

For deletion, `false` maps to the same safe comment-not-found response for every unavailable target. The controller does not need another query to distinguish them.

Use a new feature service rather than expanding `TaskService`. Register it as scoped when implementation begins.

Test Checklist

Verify during implementation:

1. Valid creation saves a comment under the correct owned task.
2. Empty, whitespace-only, and oversized content saves nothing.
3. Missing and unowned tasks cannot receive comments.
4. An owned task without comments returns an empty list.
5. Listing returns only that task's comments in stable order.
6. Another user cannot list or delete the owner's comments.
7. Deletion removes only the comment matching the route task.
8. Missing or mismatched comment IDs return not found.
9. HTTP tests confirm authentication and create, list, and delete status codes.

Check the foreign key and cascade behavior against SQL Server when applying the migration. EF Core InMemory tests do not prove relational constraints or SQL Server cascade behavior.

Implementation Roadmap

Day	Work
141	Agree on the contract
142	Entity, EF relationship, and migration
143	Create comment use case and endpoint
144	List comments use case and endpoint
145	Delete comment use case and endpoint
146	Service and integration tests
147	README endpoint documentation and examples

No application file changed and no tests ran while preparing this planning lesson. Planning alone does not require a project commit.

Interview Explanation

I planned task comments as a small owned subresource. Comments belong to one task, inherit ownership through that task, and expose only create, list, and delete operations. The nested route, service contract, validation rules, stable ordering, and cascade behavior are defined before implementation so each following lesson can make one focused change.

Exercise

Review the planned delete route and explain why matching only `commentId` would be unsafe. Then list every identity and relationship condition that must hold before deletion.

Next Lesson

Day 142 adds the `TaskComment` entity, configures its relationship to `JobTask`, and creates and inspects the EF Core migration.

Day 142 - Add the Task Comment Entity

Today's Goal

Add `TaskComment` to the domain, configure its relationship to `JobTask`, and generate an EF Core migration.

This lesson changes the persistence model only. The create-comment service and endpoint begin on Day 143.

Create the Entity

Add `TaskComment` to the Domain entities:

```
namespace JobTrackr.Domain.Entities
{
    public class TaskComment
    {
        public int Id { get; set; }

        public string Content { get; set; } = string.Empty;

        public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

        public int TaskId { get; set; }

        public JobTask? Task { get; set; }
    }
}
```

One entity represents one note attached to one task. `TaskId` is the required foreign key, while `Task` is the navigation property.

The nullable navigation permits a comment object to exist in memory without its task being loaded. The non-nullable integer foreign key and EF configuration still make the database relationship required.

Add the Task Navigation Collection

Add this property to `JobTask`:

```
public List<TaskComment> Comments { get; set; } = [];
```

The collection represents the many side of the relationship. Initializing it avoids a null collection in memory; it does not automatically load comments from SQL Server.

Configure AppDbContext

Expose comments through the context:

```
public DbSet<TaskComment> TaskComments { get; set; }
```

Configure content and the task relationship:

```
modelBuilder.Entity<TaskComment>()
    .Property(comment => comment.Content)
    .IsRequired()
    .HasMaxLength(1000);

modelBuilder.Entity<TaskComment>()
    .HasOne(comment => comment.Task)
    .WithMany(task => task.Comments)
    .HasForeignKey(comment => comment.TaskId)
    .IsRequired()
    .OnDelete(DeleteBehavior.Cascade);
```

This configuration establishes:

- required comment content
- a maximum database length of 1000 characters
- one required task for every comment
- cascade deletion when a task is removed

There is no separate comment-to-user foreign key. Upcoming services will authorize access through the related task's `UserId`, avoiding another cascade path.

`IsRequired()` prevents SQL `NULL` content. It does not reject an empty or whitespace-only string. Request and service validation handle those rules during the create-comment implementation.

Build Before Generating the Migration

Build the solution before asking EF Core to compare the model:

```
dotnet build JobTrackr.slnx --configuration Release
```

Resolve compilation problems before continuing. Use an EF command-line tool version compatible with the project's EF Core 8 packages.

Generate the Migration

Run the migration command once:

```
dotnet ef migrations add AddTaskComments --project src\JobTrackr.Infrastructure --startup-
project src\JobTrackr.Api --output-dir Persistence\Migrations -- --environment Development
```

The command:

- compares the current model with the model snapshot
- stores the migration beside `AppDbContext`
- uses the API startup configuration
- loads the Development environment for local configuration

Generating a migration does not apply it to SQL Server. Do not run the add command again after it succeeds.

Inspect the Generated Migration

The `Up` method should create a table shaped like this:

```
TaskComments
    Id          int, identity, primary key
    Content     nvarchar(1000), not null
    CreatedAtUtc datetime2, not null
    TaskId      int, not null

Foreign key: TaskComments.TaskId -> Tasks.Id
Delete behavior: Cascade
Index: IX_TaskComments_TaskId
```

The `Down` method should drop `TaskComments`. Also inspect the generated designer and updated model snapshot.

The migration must not drop or rebuild unrelated user or task tables. Stop and investigate if unrelated schema operations appear.

The C# `CreatedAtUtc` initializer supplies values for newly constructed entities. It does not create a SQL default constraint.

Apply the Migration Carefully

Confirm the Development connection string points to the intended local JobTrackr database and that SQL Server is running. Then apply pending migrations:

```
dotnet ef database update --project src\JobTrackr.Infrastructure --startup-project src\
JobTrackr.Api -- --environment Development
```

This command changes the configured database. Always verify the target first, and never place local credentials or signing keys in tracked configuration.

Verify SQL Server

After applying the migration, inspect the real relational schema:

```
SELECT TOP (10) Id, Content, CreatedAtUtc, TaskId
FROM dbo.TaskComments;

SELECT MigrationId
FROM dbo.__EFMigrationsHistory
ORDER BY MigrationId DESC;

SELECT name, delete_referential_action_desc
FROM sys.foreign_keys
WHERE parent_object_id = OBJECT_ID(N'dbo.TaskComments');
```

Verify that:

- `TaskComments` exists and is initially empty
- migration history includes `AddTaskComments`
- the task foreign key reports cascade deletion

EF Core InMemory tests do not prove relational foreign keys, maximum SQL column lengths, or SQL Server cascade behavior. Check those against SQL Server.

Build and Test

After the model and migration are ready, run:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

The expected existing baseline is 52 passing tests. This lesson introduces no comment endpoints or service tests yet.

Repository Status

The entity, relationship configuration, generated migration, and updated model snapshot were committed together in the Day 142 repository change. Keeping the migration and snapshot beside the model change allows another environment to reproduce the schema.

Applying a migration to a particular local SQL Server database remains an environment-specific action and must be verified separately.

Interview Explanation

I introduced task comments as a required one-to-many relationship from `JobTask` to `TaskComment`. The database enforces required content with a 1000-character maximum and cascades comments when their parent task is deleted. Ownership remains derived through the task, which avoids an unnecessary second user foreign key. I generated and inspected the migration before applying it to a local database.

Exercise

Inspect the generated migration and explain every column, index, foreign key, and delete behavior. Then explain why the C# timestamp initializer is not a SQL default constraint.

Next Lesson

Day 143 adds the create-comment request and response DTOs, service use case, and authenticated endpoint with task ownership validation.

Day 143 - Create a Comment on an Owned Task

Today's Goal

Implement one authenticated endpoint:

```
POST /api/tasks/{taskId}/comments
```

It creates a comment only for the authenticated task owner. Missing and unowned tasks return not found, invalid content returns a bad request, and success returns `201 Created` with a response DTO.

Day 142 already supplied the database entity and migration. This lesson adds the first comment use case.

Add Shared Validation Messages

Add messages shared by request and service validation:

```
public const string CommentContentRequired = "Comment content is required.";
public const string CommentContentTooLong =
    "Comment content cannot exceed 1000 characters.";
```

Create the Request DTO

The client supplies only comment content:

```
using JobTrackr.Application.Common;
using System.ComponentModel.DataAnnotations;

namespace JobTrackr.Application.Comments
{
    public class CreateTaskCommentRequest
    {
        [Required(ErrorMessage = ErrorMessages.CommentContentRequired)]
        [MaxLength(1000, ErrorMessage = ErrorMessages.CommentContentTooLong)]
        public string Content { get; set; } = string.Empty;
    }
}
```

`taskId` comes from the route, while the owner ID comes from authenticated JWT claims.

Create the Response DTO

Return only public comment fields:

```
namespace JobTrackr.Application.Comments
{
    public class TaskCommentResponse
    {
        public int Id { get; set; }
        public int TaskId { get; set; }
        public string Content { get; set; } = string.Empty;
        public DateTime CreatedAtUtc { get; set; }
    }
}
```

Returning a DTO prevents the API from exposing the entity graph or task navigation property.

Add the Service Contract

The first interface contains only comment creation:

```
public interface ITaskCommentService
```

```
{
    Task<TaskCommentResponse?> CreateAsync(
        int taskId,
        CreateTaskCommentRequest request,
        int userId);
}
```

A nullable response represents a task that is missing or not owned by the caller. List and delete methods arrive in later lessons.

Implement Comment Creation

The service first checks ownership with one query:

```
var ownsTask = await _dbContext.Tasks.AnyAsync(
    task => task.Id == taskId && task.UserId == userId);

if (!ownsTask)
{
    return null;
}
```

Combining task ID and user ID prevents another user from discovering whether a task exists.

Then validate direct service callers:

```
if (string.IsNullOrWhiteSpace(request.Content))
{
    throw new ArgumentException(ErrorMessages.CommentContentRequired);
}

if (request.Content.Length > 1000)
{
    throw new ArgumentException(ErrorMessages.CommentContentTooLong);
}
```

Check supplied length before trimming, as established in the plan. Create and save the entity:

```
var comment = new TaskComment
{
    TaskId = taskId,
    Content = request.Content.Trim(),
    CreatedAtUtc = DateTime.UtcNow
};

_dbContext.TaskComments.Add(comment);
await _dbContext.SaveChangesAsync();
```

Saving assigns the generated ID before response mapping. The service can use the valid foreign-key value without loading the full task or setting the navigation property.

Return an explicit `TaskCommentResponse` containing the generated ID, task ID, trimmed content, and UTC creation timestamp.

Register the Service

Register the interface and implementation as scoped:

```
builder.Services.AddScoped<ITaskCommentService, TaskCommentService>();
```

The controller and service then share the request-scoped `AppDbContext` through dependency injection.

Add the Controller

Use a nested, authenticated route:

```
[ApiController]
[Authorize]
[Route("api/tasks/{taskId:int}/comments")]
public class TaskCommentsController : ControllerBase
```

The create action should:

1. Read the current user ID from claims.
2. Return unauthorized when the identity is unusable.
3. Call `CreateAsync` with route task ID, request DTO, and user ID.
4. Return not found when the service returns `null`.
5. Return bad request for known content validation failures.
6. Return `201 Created` with the saved response.

The controller reads identity and chooses HTTP responses. It does not access `AppDbContext`.

Set the Location Header

The create action returns the comment collection URL:

```
var location = $"{Request.PathBase}/api/tasks/{taskId}/comments";

return Created(location, response);
```

The GET endpoint for this URL arrives on Day 144. Use `Created` instead of `CreatedAtAction` because there is not yet a matching GET action.

Verify the Endpoint

With a valid JWT for the task owner, submit:

```
{
  "content": "  Waiting for the recruiter to reply.  "
}
```

The successful response should contain trimmed content, a generated ID, the route task ID, and a UTC timestamp.

Verify these cases:

Scenario	Expected result
Valid comment on an owned task	`201`, DTO, and Location header
Empty or whitespace-only content	`400`
More than 1000 characters	`400`
Exactly 1000 valid characters	`201`
Missing task ID	`404` with `Task not found.`
Another user's task with valid content	`404`, no row saved
No bearer token	`401`

Automatic DTO validation may return `400` before ownership is checked. Use a valid body when specifically testing ownership.

Inspect SQL Server to confirm successful requests create rows and failed requests do not.

Build and Test Context

Build and run the existing suite after implementation:

```
dotnet build JobTrackr.slnx --configuration Release
```

```
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

The existing baseline contains 52 tests. Those tests do not yet provide dedicated comment endpoint coverage; comment service and integration tests are scheduled for Day 146.

Repository Status

The request and response DTOs, service contract and implementation, controller, shared messages, and dependency registration were committed together in the Day 143 repository change.

README endpoint documentation remains scheduled for Day 147.

Interview Explanation

I implemented task comments as an authenticated nested resource. The service verifies ownership using task ID and user ID in one query, validates direct callers, trims content, and returns a DTO. Missing and unowned tasks intentionally share the same not-found result. The controller owns claims and HTTP mapping while persistence remains in the service.

Exercise

Trace a valid comment request from JWT claim to SQL row and response DTO. Then explain why the ownership test must use a valid request body and a different user's token.

Next Lesson

Day 144 adds the endpoint that lists comments for an owned task and returns an empty list when the task has no comments.

Day 144 - List Comments for an Owned Task

Today's Goal

Add the authenticated comment collection endpoint:

```
GET /api/tasks/{taskId}/comments
```

The task owner receives `200 OK` with a list. An owned task without comments returns an empty list. Missing and unowned tasks return not found, while anonymous requests return unauthorized.

Reuse the DTO, controller route, authorization, and scoped service registration from Day 143. No new package, entity, or migration is required.

Extend the Service Contract

Add the list operation after comment creation:

```
public interface ITaskCommentService
{
    Task<TaskCommentResponse?> CreateAsync(
        int taskId,
        CreateTaskCommentRequest request,
        int userId);

    Task<List<TaskCommentResponse>?> GetAllAsync(int taskId, int userId);
}
```

The nullable list communicates two different outcomes:

```
null       -> the task is missing or unavailable to this user
empty list -> the owned task exists but has no comments
```

Only the first outcome maps to not found.

Implement the Query

First verify that the authenticated user owns the parent task:

```
var ownsTask = await _dbContext.Tasks.AnyAsync(
    task => task.Id == taskId && task.UserId == userId);

if (!ownsTask)
{
    return null;
}
```

Then query the comments with the task and ownership conditions retained in the data query:

```
return await _dbContext.TaskComments
    .Where(comment => comment.TaskId == taskId &&
        comment.Task!.UserId == userId)
    .OrderBy(comment => comment.CreatedAtUtc)
    .ThenBy(comment => comment.Id)
    .Select(comment => new TaskCommentResponse
    {
        Id = comment.Id,
        TaskId = comment.TaskId,
        Content = comment.Content,
        CreatedAtUtc = comment.CreatedAtUtc
    })
    .ToListAsync();
```

The first query establishes whether the parent resource is available. The comment query also carries the ownership restriction, keeping the data selection itself protected.

Understand Navigation Translation

`comment.Task!.UserId` tells the compiler that the required relationship is expected in this query. It does not load the task navigation into memory or add a runtime null check.

EF Core translates navigation access into SQL. An `Include` is unnecessary because the query projects comment fields directly into response DTOs.

Preserve Stable Ordering

Comments are ordered oldest first:

```
.OrderBy(comment => comment.CreatedAtUtc)
.ThenBy(comment => comment.Id)
```

The ID tie-breaker makes ordering predictable when two comments have the same timestamp.

`Select` chooses the public response fields. It does not limit the number of returned rows. Pagination can be added later if comment volume requires it.

Preserve the Empty Collection Contract

`ToListAsync()` returns an empty list when no rows match. Do not convert that list to `null`.

An owned task with no comments is a valid collection resource and should return:

```
[]
```

Not found is reserved for a missing task or a task the authenticated user does not own.

Add the Controller Action

Add a GET action under the existing nested route:

```
[HttpGet]
public async Task<IActionResult> GetAll(int taskId)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var response = await _taskCommentService.GetAllAsync(
        taskId,
        currentUserId.Value);

    if (response is null)
    {
        return Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Not Found",
            detail: ErrorMessages.TaskNotFound);
    }

    return Ok(response);
}
```

The controller's existing authorization attribute protects this action. There is deliberately no `Count == 0` not-found check.

Unexpected exceptions continue to reach the existing exception middleware.

Connect the POST Location Header

Day 143 returns the collection URL in the create response:

```
/api/tasks/12/comments
```

The new GET action makes that URL readable. The existing POST route and location construction need no changes.

Verify the Workflow

Use these checks:

1. Log in as User A and create a task.
2. List its comments and expect `200` with `[]`.
3. Add two comments and list again.
4. Confirm the comments appear oldest first.
5. Add a comment to another task and confirm it does not appear in the first list.
6. Log in as User B and request User A's comments; expect `404`.
7. Request comments for a missing task; expect the same not-found response.
8. Remove authorization and expect `401`.

Compare the API response with a SQL query ordered by `CreatedAtUtc` and `Id`. The GET endpoint must not insert, update, or delete any data.

Build and Test Context

Run the build and existing suite after implementation:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

The existing baseline contains 52 tests. Dedicated comment workflow tests remain scheduled for Day 146.

Repository Status

The service contract, EF Core list query, and controller GET action were committed together in the Day 144 repository change.

README comment documentation remains scheduled for Day 147.

Interview Explanation

I implemented comment listing as an authenticated nested collection. The service distinguishes an unavailable task from an owned task with no comments by returning `null` or an empty list. The query retains task and owner restrictions, projects directly to DTOs, and uses timestamp plus ID ordering for stable results.

Exercise

Explain why an empty collection is not a missing resource. Then modify sample timestamps so two comments match and verify that the ID tie-breaker still gives deterministic ordering.

Next Lesson

Day 145 adds comment deletion and verifies that the comment belongs to the route task and authenticated owner.

Day 145 - Delete a Comment from an Owned Task

Today's Goal

Add the authenticated comment deletion endpoint:

```
DELETE /api/tasks/{taskId}/comments/{commentId}
```

Only the authenticated task owner can delete a comment attached to that task. Success returns `204 No Content`; every unavailable target returns not found.

Reuse the current entity, DTOs, dependency registration, and nested controller route. No new package or migration is required.

Add a Safe Shared Message

Add one message for unavailable deletion targets:

```
public const string CommentNotFound = "Comment not found.";
```

Use this response for:

- a missing comment
- a comment attached to a different route task
- a comment belonging to another user's task

The shared message does not reveal whether another user's comment exists.

Extend the Service Contract

Add deletion after the create and list operations:

```
Task<bool> DeleteAsync(int taskId, int commentId, int userId);
```

The return values mean:

```
true  -> the owned matching comment was deleted and saved
false -> no permitted matching comment was available
```

Implement a Guarded Delete Query

Query all required conditions together:

```
public async Task<bool> DeleteAsync(int taskId, int commentId, int userId)
{
    var comment = await _dbContext.TaskComments.FirstOrDefaultAsync(
        comment => comment.Id == commentId &&
            comment.TaskId == taskId &&
            comment.Task!.UserId == userId);

    if (comment is null)
    {
        return false;
    }

    _dbContext.TaskComments.Remove(comment);
    await _dbContext.SaveChangesAsync();

    return true;
}
```

The conditions protect three separate boundaries:

Condition	Purpose
`comment.Id == commentId`	Identifies the requested comment
`comment.TaskId == taskId`	Confirms the comment belongs to the route task
`comment.Task!.UserId == userId`	Confirms the authenticated user owns that task

Looking up only `commentId` would ignore both the nested route relationship and ownership.

A separate parent-task query is unnecessary because every unavailable target intentionally maps to the same result.

Understand EF Navigation Translation

EF Core translates `comment.Task!.UserId` into the database query. An `Include` is unnecessary because the task entity does not need to be materialized.

The null-forgiving operator suppresses the compiler warning for the required relationship. It does not add a runtime null check.

Only the comment is removed. Its parent task and task owner remain. Unexpected database failures must continue to reach exception middleware rather than being converted into `false`.

Add the Controller Action

Add a DELETE action under the existing task-comment route:

```
[HttpDelete("{commentId:int}")]
public async Task<IActionResult> Delete(int taskId, int commentId)
{
    var currentUserId = GetCurrentUserId();

    if (currentUserId is null)
    {
        return Unauthorized();
    }

    var isDeleted = await _taskCommentService.DeleteAsync(
        taskId,
        commentId,
        currentUserId.Value);

    if (!isDeleted)
    {
        return Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Not Found",
            detail: ErrorMessages.CommentNotFound);
    }

    return NoContent();
}
```

The controller route supplies `taskId`, and the action route adds `commentId`. The existing authorization attribute protects the whole controller.

There is no request body. A successful response also has no body.

Verify the Workflow

Use disposable tasks and comments because successful checks permanently delete selected rows.

1. Log in as User A and create a task with two comments.

2. Delete the first comment with the matching task ID; expect `204`.
3. List comments and confirm only the second remains.
4. Retrieve the task and confirm it still exists.
5. Delete the first comment again; expect `404`.
6. Use a different owned task ID with the second comment ID; expect `404` and keep the comment.
7. Log in as User B and attempt deletion using User A's correct IDs; expect `404`.
8. Return to User A and confirm the comment remains.
9. Remove authorization and expect `401`.

Deleting an already deleted comment returns not found rather than no content in this API, even though the desired database state remains unchanged.

Verify SQL Server

Compare the comments and task rows after testing:

```
SELECT Id, TaskId, Content, CreatedAtUtc
FROM dbo.TaskComments
WHERE TaskId = 12
ORDER BY CreatedAtUtc, Id;

SELECT Id, UserId, Title
FROM dbo.Tasks
WHERE Id = 12;
```

Confirm that the selected comment is gone, retained comments remain, and the task still exists. Failed ownership and mismatched-route requests must remove nothing.

Build and Test Context

Run the solution build and existing suite after implementation:

```
dotnet build JobTrackr.slnx --configuration Release
dotnet test JobTrackr.slnx --configuration Release --no-build --no-restore
```

The existing baseline contains 52 tests. Dedicated comment deletion and workflow tests are added on Day 146.

Repository Status

The shared message, service contract, guarded deletion implementation, and controller DELETE action were committed together in the Day 145 repository change.

README comment documentation remains scheduled for Day 147.

Interview Explanation

I implemented comment deletion with one ownership-aware EF Core query. The query requires the comment ID, route task ID, and authenticated task owner to match. All missing, mismatched, and unowned targets return the same safe not-found response. The controller maps a successful deletion to 204 while unexpected persistence failures remain exceptions.

Exercise

Explain why deleting by comment ID alone is unsafe for a nested resource. Then write the three predicates required to prove the requested comment belongs to the route task and authenticated owner.

Next Lesson

Day 146 adds service and integration tests for comment creation, listing, deletion, validation, authentication, and ownership.

Day 146 - Test Task Comment Workflows and Ownership

Today's Goal

Add focused service tests and HTTP integration tests for task comments. The suite should verify persistence, validation, ordering, deletion, authentication, and ownership boundaries without changing application behavior.

Service Test Coverage

The new `TaskCommentServiceTests` class uses a fresh EF Core InMemory database for each test. A helper creates a real task and related user so ownership queries follow valid relationships.

The service tests verify that comment creation:

- trims surrounding whitespace before saving
- returns the persisted comment data
- rejects empty, whitespace-only, and oversized content
- saves nothing when validation fails
- returns no result for missing or unowned tasks

Selected tests clear the EF Core change tracker before reading data again. This ensures assertions inspect stored values rather than only checking the object already held in memory.

Verify Filtering and Stable Ordering

Comment listing must return only comments from the requested owned task. Results are ordered by creation time and then by ID:

```
OrderBy(comment => comment.CreatedAtUtc)
    .ThenBy(comment => comment.Id)
```

The test deliberately inserts a later comment first and gives two comments the same timestamp. This proves both parts of the ordering rule. An owned task with no comments returns an empty list, while a missing or unowned task remains unavailable.

Verify Safe Deletion

Deletion tests confirm that:

- the matching comment is removed
- other comments remain
- the parent task and user remain
- deleting the same comment again returns `false`
- unowned, mismatched-task, and missing targets are preserved

These cases protect the nested-resource boundary. A valid comment ID alone is not enough; the comment, route task, and authenticated owner must all match.

HTTP Integration Coverage

The new `TaskCommentEndpointTests` class starts the API in memory through the existing test factory. Authentication uses the real registration and login endpoints, producing separate JWT-backed clients for the owner and another user.

The owner workflow verifies:

1. Listing comments initially returns an empty collection.
2. Invalid content returns `400 Bad Request`.
3. Creating a trimmed comment returns `201 Created`.
4. Listing returns the new comment.
5. Deleting it returns `204 No Content`.
6. Listing afterward is empty.
7. Repeating the deletion returns the expected `404` problem response.
8. The parent task still exists.

A separate access test confirms that another authenticated user receives `404` for list, create, and delete operations. Anonymous requests receive `401 Unauthorized`. The owner's comment remains unchanged after every denied request.

Keep Test Identities Separate

Each integration-test user receives a unique email address. Separate HTTP clients keep owner, other-user, and anonymous authentication state isolated while the test factory uses its shared test database.

This arrangement checks the real ASP.NET Core pipeline, including routing, authorization, model binding, controller behavior, serialization, and middleware responses.

Verified Test Result

The Release test suite was run after the Day 146 commit:

```
Passed: 63
Failed: 0
Skipped: 0
```

The addition contains nine service cases and two integration tests, raising the previous 52-test baseline to 63 tests.

Understand the Limits

EF Core InMemory tests do not prove SQL Server migration application, foreign-key enforcement, cascade behavior, or SQL translation. Those database-specific checks remain separate from this fast service and HTTP test suite.

Useful future cases include the exact 1000-character boundary, more error-body assertions, and additional same-owner but wrong-task combinations.

Repository Status

The service and integration test files were committed together in the Day 146 repository change. No application behavior or migration was changed.

Interview Explanation

I tested task comments at two levels. Service tests isolate persistence, validation, ordering, ownership, and deletion behavior. Integration tests exercise the authenticated HTTP workflow through the real ASP.NET Core pipeline. Separate clients verify that owners succeed, other users receive a safe not-found response, and anonymous callers receive unauthorized responses. The complete Release suite passes all 63 tests.

Exercise

Explain why both service and integration tests are useful here. Then identify one behavior caught most directly by the service suite and one behavior caught most directly by the HTTP suite.

Next Lesson

Day 147 updates the README with the task-comment endpoints, request examples, ownership behavior, and current test totals.

Day 147 - Document Task Comments in the README

Today's Goal

Update the project README so developers can discover, understand, and test the task-comment feature built during Days 142-146.

This lesson changes documentation only. It does not modify application behavior, database migrations, dependencies, or tests.

Document the Feature Clearly

The README now identifies task comments as a current feature and summarizes the important guarantees:

- authenticated create, list, and delete endpoints
- ownership enforced through the parent task
- validated comment content
- stable oldest-first ordering

Documentation is part of the API contract. A feature is harder to adopt when readers must inspect controllers and services to discover its routes and behavior.

Publish the Comment Endpoints

The endpoint section now lists the three nested routes:

```
POST /api/tasks/{taskId}/comments
GET /api/tasks/{taskId}/comments
DELETE /api/tasks/{taskId}/comments/{commentId}
```

All three require a valid bearer token and are restricted to the parent task's owner.

The documentation explains the main responses:

Operation	Successful response	Unavailable target
Create	`201 Created`	`404 Task not found`
List	`200 OK`	`404 Task not found`
Delete	`204 No Content`	`404 Comment not found`

Missing or invalid authentication returns `401 Unauthorized`. An owned task without comments returns `200 OK` with an empty JSON array.

An invalid create request may return `400 Bad Request` before ownership is checked. This detail matters because request validation and resource lookup occur at different stages of the HTTP pipeline.

Explain Comment Behavior

The README now records the rules that callers need to know:

- the request body accepts only comment content

- the task ID comes from the route
- the user identity comes from the JWT
- content is required and cannot be whitespace-only
- content cannot exceed 1000 characters
- length is checked before trimming
- leading and trailing whitespace is removed before saving
- completed tasks may receive comments
- deleting a comment leaves its task and user intact
- deleting a task is configured to cascade-delete its comments

The response DTO exposes only the comment ID, task ID, content, and UTC creation time. Database navigation properties remain internal.

The README also states the current boundaries: editing, replies, attachments, and comment pagination are not implemented.

Add Request and Response Examples

Concrete HTTP and JSON examples demonstrate the create, list, and delete workflow. Placeholder IDs and tokens make the examples safe to publish while showing the exact request shape.

The create example includes a JSON body:

```
{
  "content": "Waiting for the recruiter to reply."
}
```

The documented response shows the generated comment ID, parent task ID, saved content, and UTC timestamp. The delete example explains that success has no response body and that repeating the deletion returns not found.

Add a Runnable PowerShell Workflow

The README now includes a PowerShell example that:

1. Reuses the authenticated base URL and headers.
2. Creates a disposable task.
3. Creates a comment for that task.
4. Lists the task's comments.
5. Deletes the new comment.
6. Reads the raw response afterward to make the empty JSON array visible.

Inspecting the raw response is useful because an empty PowerShell table alone does not prove whether the API returned an empty array or no body.

The example deletes only its comment. Its disposable task remains until the reader removes it separately.

Update Test Documentation

The README now reports the verified Day 146 totals:

```
63 total test cases
57 service cases
```

```
6 API integration tests
```

The coverage summary includes comment creation, trimming, validation, filtering, stable ordering, empty-list behavior, deletion, parent preservation, the authenticated HTTP workflow, and access restrictions.

Theory rows are counted as separately executed test cases rather than as one test method.

State the Test Limits

The documentation distinguishes fast automated coverage from database-specific verification. Service tests use EF Core InMemory, while integration tests run the API through the in-memory ASP.NET Core test host.

These tests do not require a running API, SQL Server, or local secrets. They also do not prove SQL Server foreign-key enforcement, cascade behavior, migration application, or SQL translation.

This distinction prevents configured database behavior from being presented as something the InMemory suite has verified.

Repository Status

The README documentation and corrected test totals were committed together in the Day 147 repository change. No controller, service, migration, package, or test behavior changed.

Interview Explanation

I documented the task-comment feature as a complete API workflow rather than only listing routes. The README now explains authentication, ownership, validation, response codes, request and response shapes, a runnable PowerShell example, automated test coverage, and the limits of InMemory testing. This makes the implemented feature discoverable and reduces the chance that consumers infer incorrect behavior from the code.

Exercise

Explain why API documentation should describe ownership failures and validation order, not only successful requests. Then identify which claims can be verified by integration tests and which still require SQL Server verification.

Next Lesson

Day 148 reviews the project for focused cleanup and runs the Release build and complete test suite.

Day 148 - Focused Cleanup and Release Verification

Today's Goal

Make small readability improvements to the task-comment code, search for unfinished starter code, and verify that the complete solution remains healthy.

The cleanup must preserve routes, queries, validation, database relationships, DTOs, migrations, packages, and runtime behavior.

Inspect Before Editing

Cleanup should begin with the repository status and current diff. This prevents unrelated local work from being mixed into a formatting change.

The intended scope for this lesson is limited to three files:

- the `TaskComment` entity
- the EF Core database context
- the task-comments controller

Keeping the scope narrow makes the review straightforward and reduces the chance of accidental behavioral changes.

Remove Unused Imports

The `TaskComment` entity still contained several unused namespace imports. Removing them makes the file easier to scan without changing any property or relationship.

The entity continues to contain:

- the comment identifier
- required content
- the UTC creation timestamp
- the parent task identifier
- the task navigation property

Implicit usings keep `DateTime` available, so the removed imports are not required for compilation.

Improve Database Context Spacing

The database context received two small readability adjustments:

- a blank line separates the `TaskComments` set from `OnModelCreating`
- the existing content-property configuration is aligned consistently

The configuration itself remains unchanged:

```
modelBuilder.Entity<TaskComment>()
    .Property(comment => comment.Content)
    .IsRequired()
    .HasMaxLength(1000);
```

No relationship, foreign key, maximum length, or delete behavior changed. Because this is formatting only, no migration is needed.

Separate Controller Methods

A blank line now separates the comment deletion action from the private helper that reads the authenticated user ID.

This does not change routing, authorization, status codes, or service calls. It simply makes the method boundary easier to identify during review.

Search for Leftover Code

The source and test projects were searched for common starter artifacts and unfinished implementation markers, including:

```
Class1
WeatherForecast
TODO
NotImplementedException
```

No matching unfinished starter code was found.

A text search is useful evidence, but it does not prove that every method is used. Controllers and dependency-injected services can be activated by the framework without a direct reference in application code.

Generated migrations and the model snapshot remain in place because they are database history, not dead code.

Verify the Release Build

The complete solution was built in Release configuration after the cleanup:

```
Build succeeded
Warnings: 0
Errors: 0
```

A successful build confirms that removing imports and adjusting formatting did not introduce compilation problems.

Run the Complete Test Suite

The existing automated suite was then run against the Release build:

```
Passed: 63
Failed: 0
Skipped: 0
```

The suite contains 57 service cases and six HTTP integration tests, including the task-comment workflow.

These tests use EF Core InMemory and do not verify the local SQL Server schema, relational constraints, migration application, or SQL translation.

Review the Final Diff

The committed change contains only the intended cleanup:

- unused imports removed from the entity
- spacing aligned in the database context

- a method boundary added in the controller

There are no route, query, validation, DTO, migration, package, or behavior changes.

Small cleanups should not trigger repository-wide formatting. A focused diff is easier to understand, verify, and reverse if necessary.

Repository Status

The three cleanup files were committed together in the Day 148 repository change. The repository was clean after the commit.

Interview Explanation

I performed a narrowly scoped cleanup around the task-comment feature. I removed unused imports, improved spacing in the EF Core configuration and controller, searched for starter and unfinished code, and reviewed the final diff for behavioral changes. I then verified the complete Release build with zero warnings or errors and ran all 63 automated tests successfully.

Exercise

Explain why a formatting-only database-context change does not require a migration. Then list the checks you would perform before committing a small cleanup in a shared repository.

Next Lesson

Day 149 reviews the final change list and confirms the intended work is committed and pushed.

Day 149 - Review Git and Confirm Publication

Today's Goal

Review the repository state, confirm that the intended cleanup is committed, and verify that the local `main` branch matches the version published on GitHub.

This is a verification lesson. No application, test, migration, or documentation change is required, and an empty commit should not be created merely to mark the day.

Start with Repository Status

The first check is the working tree:

```
git status
```

A clean result means there are no modified, staged, or untracked files waiting for review. It does not, by itself, prove that local commits have been pushed.

The Day 149 check found a clean working tree.

Review Unstaged and Staged Changes

Several Git commands answer different questions:

Command	Purpose
`git diff --stat`	Summarizes unstaged tracked changes
`git diff`	Shows the complete unstaged patch
`git diff --cached`	Shows the patch staged for the next commit
`git diff --check`	Detects whitespace errors in tracked changes

All of these checks were empty for the current repository state. There was nothing new to stage or commit.

Untracked file contents do not appear in a normal diff, which is why `git status` remains an important part of the review.

Inspect the Latest Commit

The latest commit is:

```
bc53a71 Day 148: Clean up comment code formatting
```

It contains only the three intended cleanup files:

- the task-comment entity
- the EF Core database context
- the task-comments controller

Earlier commits already contain the comment entity and migration, create/list/delete endpoints, automated tests, and README documentation. Recommitting those changes would add no value.

Confirm the Branch and Remote

The repository uses the `main` branch and this GitHub remote:

```
https://github.com/BackendSoftwareJourney/jobtrackr-backend.git
```

Two full hashes were compared:

```
Local HEAD:  bc53a7137f777f98f9dde96c407884156a36df6c
GitHub main: bc53a7137f777f98f9dde96c407884156a36df6c
```

The exact match confirms that the local commit is published as the current GitHub `main` commit at the time of the check.

Using `git ls-remote` matters because it contacts the remote repository. A local `origin/main` reference may be stale until Git communicates with GitHub.

Understand an Up-to-Date Push

When `git push origin main` reports that everything is up to date, the push has succeeded as a verification step. It means there are no local commits waiting to be published.

No empty commit is needed. Commits should represent real project changes, not calendar markers.

If a push is rejected because the remote has advanced, inspect the divergence before integrating it. Never force-push simply to bypass that review.

Check Continuous Integration Separately

The repository's public Actions page was checked for a workflow run associated with the Day 148 commit. No run for that commit was listed. The newest visible workflow run was associated with Day 116.

Therefore, Day 149 confirms Git publication but does not claim a successful GitHub Actions result for Day 148.

This distinction is important:

- matching local and remote hashes proves the commit is published
- a successful local build proves the code compiled locally
- a successful local test run proves the suite passed locally
- a green CI run proves the configured remote workflow completed successfully

These are related but separate signals.

Retain the Verified Local Result

Day 148 independently verified the current commit locally:

```
Release build: succeeded
Warnings: 0
Errors: 0
Tests passed: 63
Tests failed: 0
Tests skipped: 0
```

The Git-only review on Day 149 did not require another code change or test run.

Publication Checklist

The final repository review confirms:

- the working tree is clean
- there are no staged or unstaged changes
- the expected cleanup commit is at local `HEAD`
- the active branch is `main`
- the configured remote is correct
- local `HEAD` and GitHub `main` have identical hashes
- no additional commit is required
- no Day 148 GitHub Actions result is currently listed

Repository Status

Day 149 made no repository change. The Day 148 cleanup was already committed and published, so verification was the correct outcome.

Interview Explanation

I verified publication by checking the working tree, staged and unstaged diffs, latest commit contents, active branch, remote URL, and full local and remote commit hashes. The hashes matched, proving GitHub main contained the intended cleanup. I also treated CI as a separate signal: no workflow run was listed for that commit, so I did not describe it as green even though the local Release build and all 63 tests had passed.

Exercise

Explain the difference between `git status`, `git diff`, `git diff --cached`, and `git ls-remote`. Then explain why matching local and remote hashes does not prove that a CI workflow passed.

Next Lesson

Day 150 completes the Month 5 review of maintainability, task comments, testing, documentation, and repository health.

Day 150 - Month 5 Review

Today's Goal

Review the work completed during Days 121-150, verify the current backend, and identify what is ready for the next deployment-readiness phase.

This review covers validation, error handling, maintainability, task comments, automated testing, documentation, Git publication, and continuous integration.

What Changed This Month

The project improved across several related areas:

Area	Progress
Task behavior	Idempotent completion and reopening, future due-date rules, and supported priorities
User input	Email format, normalization, duplicate detection, and password validation
Error handling	Field errors, Problem Details responses, authentication failures, and safe server errors
Maintainability	Clearer controller and service responsibilities with unnecessary abstractions deferred
Task comments	Entity, EF Core relationship, create/list/delete endpoints, and owner-only access
Verification	Comment service tests, HTTP integration tests, README updates, and focused cleanup

The existing four-project architecture remains intact: API, Application, Domain, and Infrastructure.

Trace Comment Deletion End to End

Comment deletion follows this path:

```
TaskCommentsController.Delete
    -> ITaskCommentService.DeleteAsync
    -> TaskCommentService.DeleteAsync
    -> ownership-aware EF Core query
    -> SaveChangesAsync
    -> HTTP 204 or safe 404
```

The database query checks three values together:

- the requested comment ID
- the task ID from the nested route
- the authenticated owner's user ID

Checking both comment and task IDs proves that the comment belongs to the route's parent resource. Checking the owner through the task prevents one user from deleting another user's comment.

The authenticated user ID comes from the JWT claims read by the controller. An unavailable or unowned target returns the same safe `404` response so the API does not reveal whether another user's comment exists.

Deleting a comment removes only that comment. The task remains because the relationship is configured from task to comments, not from comment deletion back to the parent.

Confirm Maintainability

The current design keeps responsibilities reasonably focused:

- controllers receive HTTP requests, read claims, call service interfaces, and select responses
- services contain use-case rules and coordinate persistence
- response DTOs prevent EF navigation graphs and password hashes from leaking
- task comments use a small dedicated service within the existing architecture
- helper methods keep repeated query and validation behavior readable

A common result type and further service splitting remain deferred. That is an intentional decision: abstractions should solve demonstrated complexity rather than exist only for theoretical consistency.

Clean structure does not make every endpoint production-ready. Existing authorization limitations must remain visible during deployment planning.

Verified Release Build

The complete solution was built in Release configuration during this review:

```
Build succeeded
Warnings: 0
Errors: 0
```

All projects restored and compiled successfully.

Verified Automated Tests

The complete Release test suite was run after the successful build:

```
Passed: 63
Failed: 0
Skipped: 0
```

The suite contains 57 service cases and six HTTP integration tests. Theory input rows are counted as separate executed cases.

The comment coverage verifies persistence, trimming, invalid content, task filtering, stable ordering, empty lists, deletion, authentication, and ownership boundaries.

Understand the Verification Boundary

Service and HTTP integration tests use EF Core InMemory. They provide fast coverage of application behavior and the ASP.NET Core request pipeline, but they do not prove:

- SQL Server migration application
- relational foreign-key enforcement
- cascade deletion in the real database
- SQL query translation
- local secret or connection-string correctness

The SQL Server comment workflow remains a separate manual verification step. It should use disposable tasks and comments, confirm owner and non-owner behavior, inspect stored rows, and test cascade deletion without touching real user data.

Review Documentation

The README now documents:

- all three task-comment routes
- authentication and owner-access rules
- content validation and trimming
- empty-list and stable-ordering behavior
- HTTP and JSON examples
- a runnable PowerShell workflow
- the 63-test total
- the limitations of InMemory testing

Documentation also keeps known authorization gaps visible rather than presenting the API as ready for unrestricted public deployment.

Known Gaps

The next phase must continue tracking:

- unprotected legacy user routes
- the user-ID-based task-list route
- missing JWT revocation
- relational checks being separate from InMemory tests
- production configuration and secret handling

These gaps do not erase the month's progress, but they affect deployment readiness and should be addressed deliberately.

Verify Repository Health

The repository checks confirmed:

- the working tree is clean
- there are no whitespace errors in the current diff
- local `HEAD` is the expected Day 148 cleanup commit
- local `main` and GitHub `main` have the same full commit hash
- no additional commit is required for the review day

The published commit is:

```
bc53a71 Day 148: Clean up comment code formatting
```

Verify Continuous Integration

The GitHub `Build` workflow for the published commit completed successfully. This is recorded separately from the local Release build and test run because local verification, remote publication, and CI are different signals.

Together, the checks show that the reviewed commit is published, builds locally, passes all local tests, and has a successful remote workflow result.

Month 5 Outcome

Month 5 closes with a more maintainable backend, stronger input validation and error responses, an owner-protected task-comment feature, 63 automated test cases, current README documentation, a clean repository, and successful CI for the published commit.

The project is ready to begin deployment-readiness work, but it is not yet approved for public exposure. Authorization, configuration, secrets, containerization, and production operating habits remain part of the next phase.

Interview Explanation

During Month 5, I improved validation and error handling, reviewed controller and service responsibilities, added task comments as a nested owner-protected resource, and covered the workflow with service and HTTP integration tests. I documented the API, cleaned up the implementation, verified a warning-free Release build and 63 passing tests, confirmed GitHub publication, and checked the successful CI workflow. I also kept the remaining authorization and relational-verification gaps explicit before deployment work begins.

Exercise

Give a two-minute explanation of comment deletion from HTTP request to database save. Then explain which parts are proven by automated tests, which require SQL Server verification, and which remain deployment risks.

Next Lesson

Day 151 begins the deployment-readiness phase by reviewing application settings, connection strings, environment variables, and secret handling.